



UNIVERSIDAD DE GUAYAQUIL

FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS

CARRERA DE SOFTWARE

**PROTOTIPO DE APLICACIÓN MÓVIL PARA EL CONTROL DE PAGOS
DE ALÍCUOTAS, ACCESO CON QR Y RESERVAS DE ÁREAS
SOCIALES DE UNA URBANIZACIÓN PRIVADA**

PROYECTO DE INTEGRACIÓN CURRICULAR

Previa a la obtención del Título de:

INGENIERO DE SOFTWARE

AUTORES:

LÓPEZ PANTA JHONAS JEREMY

PIEDRA JIMÉNEZ BELKY XIOMARA

TUTOR(A):

ING. ANGEL CUENCA ORTEGA, M. Sc

GUAYAQUIL – ECUADOR

2026

  		
REPOSITORIO NACIONAL EN CIENCIAS Y TECNOLOGÍAS		
FICHA DE REGISTRO DE TRABAJO DE INTEGRACIÓN CURRICULAR		
TÍTULO: <i>“Prototipo de aplicación móvil para el control de pagos de alícuotas, acceso con QR y reservas de áreas sociales para una urbanización privada”</i>		
AUTOR(ES): Jhonas Jeremy López Panta Belky Xiomara Piedra Jiménez	REVISOR(A): Nombres y apellidos del (la) docente revisor(a)	
INSTITUCIÓN: Universidad de Guayaquil	FACULTAD: Ciencias Matemáticas y Físicas	
CARRERA: Software		
FECHA DE PUBLICACIÓN:	N° DE PÁGS.: 999	
ÁREA TEMÁTICA: Software		
PALABRAS CLAVES: aplicación móvil, urbanización, código QR, Scrum, control de pago, prototipo		
RESUMEN: (Colocar el mismo resumen y palabras clave colocados en la sección del trabajo de integración curricular que corresponde a “RESUMEN”)		
N° DE REGISTRO:	N° DE CLASIFICACIÓN:	
DIRECCIÓN URL: (PROYECTO DE INTEGRACIÓN CURRICULAR EN LA WEB)		
ADJUNTO PDF	SI ☒	NO ☐
CONTACTO CON AUTOR(ES):	Teléfono:	Email:
CONTACTO DE LA INSTITUCIÓN	Nombre: Ab. Juan Chávez Atocha	
	Teléfono: 2307729	
	Email: juan.chaveza@ug.edu.ec	

APROBACIÓN DEL TUTOR

En mi calidad de Tutora del Trabajo de Integración Curricular, **“PROTOTIPO DE APLICACIÓN MÓVIL PARA EL CONTROL DE PAGOS DE ALÍCUOTAS, ACCESO CON QR Y RESERVAS DE ÁREAS SOCIALES DE UNA URBANIZACIÓN PRIVADA”** elaborado por el Sr. **LÓPEZ PANTA JHONAS JEREMY** y la Srta. **PIEDRA JIMÉNEZ BELKY XIOMARA**, **estudiantes no titulados** de la Carrera de Software, Facultad de Ciencias Matemáticas y Físicas de la Universidad de Guayaquil, previo a la obtención del Título de Ingeniero de Software, me permito declarar que luego de haber orientado, estudiado y revisado, la **apruebo** en todas sus partes.

Atentamente,

Ing. Angel Cuenca Ortega, M. Sc

TUTOR

DEDICATORIA

Si desea dedicar su Proyecto de Integración Curricular a su familia, a sus padres, a sus hijos, o alguna institución, entre otros, redáctelo en 1 o 2 párrafos, de lo contrario omita esta página.

Jhonas Jeremy López Panta

Si desea dedicar su Proyecto de Integración Curricular a su familia, a sus padres, a sus hijos, o alguna institución, entre otros, redáctelo en 1 o 2 párrafos, de lo contrario omita esta página.

Belky Xiomara Piedra Jiménez

AGRADECIMIENTO

Si desea realizar algún reconocimiento a las personas o instituciones que le apoyaron o ayudaron a la realización de su Proyecto de Integración Curricular, redacte en 1 o 2 párrafos, de lo contrario omitan esta página.

Jhonas Jeremy López Panta

Si desea realizar algún reconocimiento a las personas o instituciones que le apoyaron o ayudaron a la realización de su Proyecto de Integración Curricular, redacte en 1 o 2 párrafos, de lo contrario omitan esta página.

Belky Xiomara Piedra Jiménez

TRIBUNAL PROYECTO DE INTEGRACIÓN CURRICULAR

Ing. Douglas Iturburu Salvador, M. Sc.
DECANO DE LA FACULTAD
CIENCIAS MATEMÁTICAS Y FÍSICAS

Ing. Leili Lopezdominguez Rivas, M. Sc.
DIRECTORA DE LA CARRERA DE
SOFTWARE

Ing. Angel Cuenca Ortega, M. Sc
DOCENTE TUTOR(A) DEL PROYECTO
DE INTEGRACIÓN CURRICULAR

Nombre y Apellidos
DOCENTE REVISOR(A) DEL
PROYECTO DE INTEGRACIÓN
CURRICULAR

Ab. Juan Chávez Atocha, Esp.
SECRETARIO

DECLARACIÓN EXPRESA

“La responsabilidad del contenido de este Proyecto de Integración Curricular, me corresponden exclusivamente; y el patrimonio intelectual de la misma a la UNIVERSIDAD DE GUAYAQUIL”.

JHONAS JEREMY LÓPEZ PANTA
DEL TRABAJO DE INTEGRACIÓN CURRICULAR

BELKY XIOMARA PIEDRA JIMÉNEZ
DEL TRABAJO DE INTEGRACIÓN CURRICULAR



CESIÓN DE DERECHOS DE AUTOR

Ingeniero

Douglas Iturburu Salvador, M. Sc.

DECANO DE LA FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS

Presente.

A través de este medio indico a usted que procedo a realizar la entrega de la cesión de derechos de autor en forma libre y voluntaria del trabajo de integración curricular “**Prototipo de aplicación móvil para el control de pagos de alcúotas, accesos con QR y reservas de áreas sociales de una urbanización privada**”, realizado como requisito previo para la obtención del Título de Ingeniero de Software de la Universidad de Guayaquil.

Guayaquil, _____ de _____.

Jhonas Jeremy López Panta
C.I. N° 0943592030

Belky Xiomara Piedra Jiménez
C.I. N° 0954485595



UNIVERSIDAD DE GUAYAQUIL

FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS

CARRERA DE SOFTWARE

**PROTOTIPO DE APLICACIÓN MÓVIL PARA EL CONTROL DE PAGOS
DE ALÍCUOTAS, ACCESO CON QR Y RESERVAS DE ÁREAS
SOCIALES DE UNA URBANIZACIÓN PRIVADA**

Proyecto de Integración Curricular que se presenta como requisito para optar por el título de
INGENIERO DE SOFTWARE

Autor(es): Jhonas Jeremy López Panta

C.I. N° 0943592030

Belky Xiomara Piedra Jiménez

C.I. N° 0954485595

Tutor(a): Ing. Angel Cuenca Ortega, M. Sc.

Guayaquil, -- de agosto de 2026

CERTIFICADO DE ACEPTACIÓN DEL TUTOR

En mi calidad de Tutor(a) del Proyecto de Integración Curricular, nombrado por el Consejo Directivo de la Facultad de Ciencias Matemáticas y Físicas de la Universidad de Guayaquil.

CERTIFICO:

Que he analizado el Proyecto de Integración Curricular presentado por el/la/los estudiantes(s) **JHONAS JEREMY LÓPEZ PANTA, BELKY XIOMARA PIEDRA JIMÉNEZ**, como requisito previo para optar por el Título de Ingeniero(a) de Software cuyo proyecto es:

**PROTOTIPO DE APLICACIÓN MÓVIL PARA EL CONTROL DE PAGOS DE
ALÍCUOTAS, ACCESO CON QR Y RESERVAS DE ÁREAS SOCIALES DE UNA
URBANIZACIÓN PRIVADA**

Considero aprobado el trabajo en su totalidad.

Presentado por:

López Panta Jhonas Jeremy

Cédula de identidad N° 0943592030

Piedra Jiménez Belky Xiomara

Cédula de identidad N° 0954485595

Tutor(a): _____

Firma

Guayaquil, -- agosto de 2026



UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE SOFTWARE

**AUTORIZACIÓN PARA PUBLICACIÓN DE PROYECTO DE INTEGRACIÓN CURRICULAR EN
 FORMATO DIGITAL**

1. Identificación del Proyecto de Integración Curricular

Nombre del Estudiante: Jhonas Jeremy López Panta	
Dirección: PENDIENTE	
Teléfono: 0978897935	Email: jhonas.lopezp@ug.edu.ec

Nombre del Estudiante: Belky Xiomara Piedra Jiménez	
Dirección: Villa España I, Mallorca Mz 67 V 20	
Teléfono: 0980634788	Email: belky.piedraj@ug.edu.ec

Facultad: Ciencias Matemáticas y Físicas
Carrera: Software
Proyecto de Integración Curricular al que opta: Ingeniería de Software
Docente Tutor: Ing. Angel Cuenca Ortega, M. Sc.

Título del Proyecto de Integración Curricular: Prototipo de aplicación móvil para el control de pagos de alícuotas, acceso con QR y reservas de áreas sociales de una urbanización privada.
--

Palabras Claves: : aplicación móvil, urbanización, código QR, Scrum, control de pago, prototipo
--

2. Autorización de Publicación de Versión Electrónica del Proyecto de Integración Curricular

A través de este medio autorizo a la Biblioteca de la Universidad de Guayaquil y a la Facultad de Ciencias Matemáticas y Físicas a publicar la versión electrónica de este Proyecto de Integración Curricular.

Publicación Electrónica:

Inmediata	☒	Después de 1 año	☐
-----------	-------------------------------------	------------------	--------------------------

Firma Estudiante:

 López Panta Jhonas Jeremy

 Piedra Jiménez Belky Xiomara

0943592030
 Cédula de identidad N°

0954485595
 Cédula de identidad N°

3. Forma de envío:

El texto del Proyecto de Integración Curricular debe ser enviado en formato Word, como archivo .docx, .RTF o. Puf para PC. Las imágenes que la acompañen pueden ser: .gif, .jpg o .TIFF.

DVDROM ☐

CDROM ☐

ÍNDICE GENERAL

FICHA DE REGISTRO DE TRABAJO DE INTEGRACIÓN CURRICULAR	2
APROBACIÓN DEL TUTOR.....	3
DEDICATORIA.....	4
AGRADECIMIENTO	5
TRIBUNAL PROYECTO DE INTEGRACIÓN CURRICULAR.....	6
DECLARACIÓN EXPRESA.....	7
CESIÓN DE DERECHOS DE AUTOR	8
CERTIFICADO DE ACEPTACIÓN DEL TUTOR	10
AUTORIZACIÓN PARA PUBLICACIÓN DE PROYECTO DE INTEGRACIÓN CURRICULAR EN FORMATO DIGITAL	11
ÍNDICE GENERAL	12
ÍNDICE DE TABLAS.....	19
ÍNDICE DE FIGURAS.....	22
ABREVIATURAS.....	30
RESUMEN.....	31
ABSTRACT.....	32
INTRODUCCIÓN	33
CAPÍTULO I.....	22
PLANTEAMIENTO DEL PROBLEMA	22

Descripción de la situación problemática	22
Ubicación del problema en un contexto.....	22
Situación conflicto nudos críticos	22
Delimitación del problema.....	23
Evaluación del problema.....	24
Causas y consecuencias del problema	26
Formulación del problema	27
Objetivos del proyecto	28
Objetivo general.....	28
Objetivos específicos	28
Alcance del Problema	29
Módulo de usuarios.....	30
Funciones Principales:.....	30
Módulo de control de pagos.....	31
Funciones Principales:.....	31
Módulo de control de acceso	31
Funciones Principales:.....	32
Módulo de reservas	32
Funciones Principales:.....	32
Módulo de reportes	33

Justificación e Importancia.....	34
Justificación técnica	35
Justificación académica	35
Limitaciones del Estudio	36
CAPÍTULO II	30
MARCO TEORICO	30
Antecedentes del Estudio.....	30
A nivel global.....	30
A nivel regional.....	30
A nivel local	30
Fundamentación Teórica.....	32
Sistemas de información para gestión residencial y automatización administrativa	32
Urbanizaciones y Privadas y Gestión Administrativa	32
Gestión de Alícuotas.....	33
Automatización de servicios.....	34
Digitalización de procesos residenciales	35
Arquitectura de software y desarrollo de aplicaciones móviles.....	36
Arquitectura de tres capas	36
Aplicaciones Móviles	37
Tipos de Aplicaciones Móviles	37

	15
Desarrollo Multiplataforma con Flutter.....	38
Lenguaje de Programación Dart.....	40
Arquitectura Backend.....	40
Framework NestJS.....	41
Seguridad informática y control de acceso mediante códigos qr.....	43
Control de acceso con código QR	43
Funcionamiento de los códigos QR.....	43
Propuesta científica a contestarse.....	44
Definiciones Conceptuales.....	45
CAPÍTULO III.....	30
PROPUESTA TECNOLÓGICA.....	30
Análisis de factibilidad	30
Factibilidad operacional.....	30
Factibilidad técnica	31
Factibilidad legal.....	33
Factibilidad económica	34
Metodologías del proyecto.....	37
Metodología de investigación	38
Metodología de desarrollo del proyecto.....	38
Roles Scrum	39

Requerimientos del sistema.....	40
Requerimientos funcionales.....	40
Requerimientos no funcionales.....	61
Criterios de validación	65
Definición de procesos	66
Proceso actual del control de pagos de alícuotas.....	67
Proceso propuesto para el control de pagos.	69
Proceso actual de acceso de visitantes.....	70
Proceso propuesto para el control de accesos con QR	71
Proceso actual de la gestión de reservas de áreas sociales	72
Proceso propuesto para la gestión de reserva de áreas sociales	74
Descripción de casos de uso.....	75
Caso de uso: Iniciar sesión	76
Caso de uso: Registrar comprobante de pago.....	76
Caso de uso: Validar pago de alícuota	77
Caso de uso: Registrar visitante	78
Caso de uso: Validar código QR	79
Casos de uso: Registrar reserva de áreas sociales	80
Caso de uso: Validar reserva de áreas sociales	81
Caso de uso Generar reporte de pagos	82

Desarrollo de historias de usuario	82
Product Backlog.....	88
Desarrollo de Sprints	90
Sprint 0. Preparación del entorno de desarrollo y arquitectura del proyecto	91
Retroalimentación de Sprint 0	121
Sprint 1. Implementación de la infraestructura funcional y autenticación del sistema.	123
Retroalimentación de Sprint 1	139
Sprint 2. Desarrollo del módulo de gestión de usuarios	140
Retroalimentación del Sprint 2	168
Sprint 3. Desarrollo del módulo de control de pagos de alícuotas.....	169
Retroalimentación del Sprint 3	178
Sprint 5. Desarrollo del módulo de reportes e integración del sistema.	199
Retroalimentación del Sprint 5	205
Beneficiarios Directos e Indirectos del Proyecto	206
Entregables del proyecto	208
Propuesta	209
Criterios de validación de la propuesta	211
Resultados	213
CAPÍTULO IV	30
CONCLUSIONES Y RECOMENDACIONES.....	30

Conclusiones	30
Recomendaciones	30
Trabajos Futuros	31
ANEXOS	30
Anexo 1. Planificación de actividades del proyecto	30
Anexo 2. Geolocalización del problema	31

ÍNDICE DE TABLAS

Tabla 1	Delimitación del problema.....	24
Tabla 2	Matriz de causas y consecuencias del problema.....	27
Tabla 3	Componentes de hardware utilizados para desarrollo	31
Tabla 4	Componentes de software utilizados para desarrollo.....	32
Tabla 5	Costos de inversión en software	34
Tabla 6	Costos referenciales por recursos humanos	35
Tabla 7	Costos de inversión en hardware	36
Tabla 8	Costos fijos involucrados en el proyecto	36
Tabla 9	Resumen de costos referenciales de inversión.....	37
Tabla 10	Roles definidos para el desarrollo.....	40
Tabla 11	RF01- Registro de usuarios.....	41
Tabla 12	RF02- Inicio de sesión	42
Tabla 13	RF03- Recuperación de contraseña	42
Tabla 14	RF04- Registro de comprobante de pago.....	43
Tabla 15	RF05- Consulta de valores pendientes.....	44
Tabla 16	RF06- Notificación de comprobante.....	44
Tabla 17	RF07- Validación de pagos.....	45
Tabla 18	RF08- Registro de observaciones	46
Tabla 19	RF09- Aprobación de pago	46
Tabla 20	RF10- Historial de pagos	47
Tabla 21	RF11- Registro de visitante	47
Tabla 22	RF12- Generación de código QR.....	48

Tabla 23 RF13- Escaneo de código QR.....	49
Tabla 24 RF14- Validación de código QR	50
Tabla 25 RF15- Registro manual de visitantes	50
Tabla 26 RF16- Registro de alertas y restricción de intentos	51
Tabla 27 RF17- Historial de visitas	52
Tabla 28 RF18- Consulta de áreas sociales y disponibilidad.....	53
Tabla 29 RF19- Validación de disponibilidad	53
Tabla 30 RF20 - Reserva de área social.....	54
Tabla 31 RF21- Bloqueo temporal de horario	55
Tabla 32 RF22- Validación y confirmación de reserva	56
Tabla 33 RF23- Cancelación de reserva	56
Tabla 34 RF24- Liberación automática de horario	57
Tabla 35 RF25- Historial de reservas	58
Tabla 36 RF26- Reporte de pagos.....	58
Tabla 37 RF27- Reporte de acceso de visitantes	59
Tabla 38 RF28- Reporte de reservas de áreas sociales	60
Tabla 39 RNF01- Seguridad	61
Tabla 40 RNF02- Usabilidad	62
Tabla 41 RNF03- Disponibilidad.....	62
Tabla 42 RNF04 - Rendimiento.....	62
Tabla 43 RNF05 - Compatibilidad.....	63
Tabla 44 RNF06 – Integridad de datos	63
Tabla 45 RNF07 – Escalabilidad	63

Tabla 46 RNF08 – Mantenibilidad	64
Tabla 47 RNF09 – Arquitectura.....	64
Tabla 48 Criterios de validación del prototipo móvil	65
Tabla 49 Caso de uso – Iniciar sesión.....	76
Tabla 50 Caso de uso – Registrar comprobante de pago	77
Tabla 51 Caso de uso – Validar pago de alícuota	78
Tabla 52 Caso de uso – Registrar visitante	78
Tabla 53 Caso de uso – Validar código QR.....	79
Tabla 54 Caso de uso – Registrar reserva de áreas sociales	80
Tabla 55 Caso de uso – Validar reserva de áreas sociales	81
Tabla 56 Caso de uso – Generar reporte de pagos	82
Tabla 57 Historias de usuario.....	83
Tabla 58 Product Backlog.....	89
Tabla 59 Sprint Planning 0.....	91
Tabla 60 Sprint Planning 1.....	123
Tabla 61 Sprint Planning 2.....	140
Tabla 62 Sprint Planning 3.....	169
Tabla 63 Sprint Planning 4.....	179
Tabla 64 Sprint Planning 5.....	199
Tabla 65 Profesionales que conforman el juicio de expertos.....	212
Tabla 66 Preguntas para los criterios de evaluación	212
Tabla 67 Tabulación de resultados del juicio de expertos	213

ÍNDICE DE FIGURAS

Figura 1	Análisis comparativo: Flutter vs React Native vs Ionic	39
Figura 2	Comparación de frameworks backend	42
Figura 3	Esquema de Proceso Scrum	39
Figura 4	Proceso de pagos de alícuotas	68
Figura 5	Proceso propuesto para el control de pagos	69
Figura 6	Proceso actual del acceso de visitantes	70
Figura 7	Proceso propuesto para el control de acceso con QR.....	72
Figura 8	Proceso actual de reserva de áreas sociales.....	73
Figura 9	Proceso propuesto para reservas de áreas sociales	74
Figura 10	Diseño de pantalla de Login	93
Figura 11	Diseño de pantalla de crear usuario Administrador	93
Figura 12	Diseño de pantalla de crear usuario Residente	94
Figura 13	Diseño de pantalla de crear usuario Guardia.....	94
Figura 14	Diseño de pantalla de recuperar contraseña	95
Figura 15	Diseño de pantalla de perfil de residente.....	95
Figura 16	Diseño de pantalla de perfil de administrador.....	96
Figura 17	Diseño de pantalla de perfil de guardia	96
Figura 18	Diseño de pantalla de registro de pago en perfil de residente	97
Figura 19	Diseño de pantalla de detalle de pago en perfil de residente.....	97
Figura 20	Diseño de pantalla de detalle de pago en perfil de residente.....	98
Figura 21	Diseño de pantalla de validación de pago en perfil de administrador.....	98
Figura 22	Diseño de pantalla de aprobación de pago	99

Figura 23	Diseño de pantalla de pago denegado	99
Figura 24	Diseño de pantalla de registro de visitante	100
Figura 25	Diseño de la pantalla generación de QR	100
Figura 26	Diseño de pantalla de acceso de visitante	101
Figura 27	Diseño de pantalla de escaneo de QR	101
Figura 28	Diseño de pantalla de código válido.....	102
Figura 29	Diseño de pantalla de código inválido	102
Figura 30	Diseño de pantalla de ingreso registrado.....	103
Figura 31	Diseño de pantalla de ingreso denegado	103
Figura 32	Diseño de pantalla de ingreso manual.....	104
Figura 33	Diseño de pantalla de historial de registros.....	104
Figura 34	Diseño de pantalla de bitácora de ingresos	105
Figura 35	Diseño de pantalla de áreas sociales.....	105
Figura 36	Diseño de pantalla de detalle de reserva	106
Figura 37	Diseño de pantalla de nueva reserva	106
Figura 38	Diseño de pantalla de detalle de pago de reserva.....	107
Figura 39	Diseño de pantalla de subir comprobante de pago	107
Figura 40	Diseño de pantalla de historial de reservas	108
Figura 41	Diseño de pantalla de validación de pago de reserva.....	108
Figura 42	Diseño de pantalla de inicio de reportes.....	109
Figura 43	Diseño de pantalla de reporte de pagos	109
Figura 44	Diseño de pantalla de reporte de accesos	110
Figura 45	Diseño de pantalla de reporte de reservas	110

Figura 46	Reporte de usuarios	111
Figura 47	Diagrama Entidad–Relación de la base de datos.....	112
Figura 48	Instalación de dependencias del frontend: Flutter	113
Figura 49	Estructura del proyecto en VS Code	114
Figura 50	Creación de la base de datos.....	115
Figura 51	Arquitectura de tres capas de la aplicación móvil propuesta	116
Figura 52	Estructura de carpetas del proyecto Flutter	117
Figura 53	Repositorio Git con el commit inicial	118
Figura 54	Repositorio del frontend.....	119
Figura 55	Repositorio del backend	119
Figura 56	Diagrama de comunicación entre la aplicación móvil, la API REST y la base de datos	120
Figura 57	Diagrama de flujo del proceso de solicitud y respuesta entre el frontend y el backend	121
Figura 58	Interfaz implementada en Flutter en el emulador.....	124
Figura 59	Código fuente del formulario de registro	125
Figura 60	Código del servicio de registro (NestJS Controller/Service)	125
Figura 61	Modelo de usuario en Prisma	126
Figura 62	Registros almacenados en PostgreSQL	127
Figura 63	Código de controladores y servicios CRUD en NestJS	128
Figura 64	Pruebas en Postman (GET, POST, PUT, DELETE).....	128
Figura 65	Código del formulario de login en Flutter.....	130
Figura 66	Validaciones en ejecución	130

Figura 67	Código de consumo del endpoint de login	131
Figura 68	Estructura de la petición de credenciales	132
Figura 69	Código de generación de JWT	133
Figura 70	Estructura del payload JWT en el código.....	133
Figura 71	Respuesta de la API con token JWT	134
Figura 72	Eliminación de token.....	135
Figura 73	Código de la función logout en Flutter.....	136
Figura 74	Ejecución del cierre de sesión en la aplicación móvil.....	136
Figura 75	Código fuente del servicio de recuperación y actualización de contraseña	136
Figura 76	Contraseña actualizada en la base de datos PostgreSQL	137
Figura 77	Código fuente del servicio de recuperación de contraseña en Flutter	138
Figura 78	Interfaz de recuperación de contraseña de la aplicación móvil.....	138
Figura 79	Prueba de recuperación de contraseña desde la aplicación móvil.....	139
Figura 80	Creación de la tabla de residente	142
Figura 81	Creación de la tabla de administrador	142
Figura 82	Creación de la tabla de guardia	143
Figura 83	Prueba endpoint registro de administrador.....	144
Figura 84	Código de servicio de usuarios.....	145
Figura 85	Registros de usuarios almacenados en PostgreSQL	145
Figura 86	Interfaz de registro de usuario en la aplicación móvil.....	146
Figura 87	Código fuente del formulario de registro desarrollado en Flutter	147
Figura 88	Prueba del endpoint desde la aplicación móvil	149
Figura 89	Respuesta generada por la API REST para el registro de usuarios.....	150

Figura 90	Pruebas de endpoints CRUD en Postman	150
Figura 91	Resultado final de validación del módulo	152
Figura 92	Prueba del endpoint GET de consulta de usuarios en Postman	152
Figura 93	Prueba del endpoint PUT/PATCH de actualización de usuario	154
Figura 94	Validación de roles y permisos	155
Figura 95	Validación de datos de usuarios	155
Figura 96	Pruebas de acceso según rol del usuario	156
Figura 97	Código fuente de las pantallas de registro de usuarios.....	157
Figura 98	Interfaz de actualización de datos de usuarios	158
Figura 99	Información de usuarios actualizada en la aplicación	159
Figura 100	Código del servicio de comunicación entre Flutter y la API REST.....	160
Figura 101	Código del servicio de actualización de usuarios en la API REST	161
Figura 102	Código de las validaciones aplicadas para la modificación de datos	161
Figura 103	Registro actualizado en la base de datos PostgreSQL.....	162
Figura 104	Respuesta de la API REST durante el proceso de actualización.....	163
Figura 105	Prueba de actualización de información desde la aplicación móvil	164
Figura 106	Código del servicio de eliminación o desactivación de usuarios en NestJS	165
Figura 107	Prueba del endpoint de desactivación de usuarios en Postman.....	166
Figura 108	Interfaz de eliminación o desactivación de usuarios	166
Figura 109	Código del servicio de comunicación con la API REST.....	167
Figura 110	Código del servicio de registro en NestJS.....	170
Figura 111	Código del controlador de registro del comprobante	170
Figura 112	Registro almacenado con el estado inicial Pendiente.....	171

Figura 113	Servicio de registro de comprobante en la API REST	172
Figura 114	Código de la interfaz de registro de comprobantes	172
Figura 115	Interfaz del formulario de registro de comprobantes	172
Figura 116	Pantalla de registro exitoso en la aplicación móvil	172
Figura 117	Código del servicio de validación de pagos en NestJS	173
Figura 118	Registro actualizado en PostgreSQL	174
Figura 119	Interfaz del administrador con el resultado de la validación.....	174
Figura 120	Interfaz de validación de pagos	175
Figura 121	Consumo de la API.....	175
Figura 122	Interfaz de la validación del pago: Estado pendiente	176
Figura 123	Interfaz de la validación del pago: Estado aprobado.....	176
Figura 124	Implementación del servicio de consulta del historial de pagos	176
Figura 125	Implementación del endpoint	177
Figura 126	Interfaz de historial de pagos.....	177
Figura 127	Consumo del servicio Rest	178
Figura 128	Implementación del servicio de generación QR.....	181
Figura 129	Prueba de respuesta de la API REST	181
Figura 130	Código QR generado correctamente	182
Figura 131	Interfaz de generación de código QR	182
Figura 132	Integración de Flutter con la API REST	183
Figura 133	Visualización del código QR generado	183
Figura 134	Implementación del servicio de validación QR	184
Figura 135	Implementación del endpoint de validación.....	185

Figura 136	Interfaz de Código inválido	185
Figura 137	Interfaz de Código válido	186
Figura 138	Interfaz de validación de QR.....	186
Figura 139	Interfaz de datos del visitante	187
Figura 140	Interfaz de acceso exitoso.....	187
Figura 141	Implementación del servicio de consulta de ingresos QR en NestJS.....	188
Figura 142	Implementación del endpoint REST de consulta	188
Figura 143	Consulta de registros almacenados en la base de datos.....	188
Figura 144	Interfaz del historial de ingresos	189
Figura 145	Implementación del servicio de la consulta de historial.....	190
Figura 146	Código del servicio de reservas.....	191
Figura 147	Código de las validaciones implementadas para el registro de reservas	191
Figura 148	Interfaz del registro de una reserva	191
Figura 149	Código de la interfaz de reservas	192
Figura 150	Servicio de comunicación con la API REST.....	192
Figura 151	Implementación del servicio de la aprobación/denegación de reservas.....	193
Figura 152	Respuesta de la API REST durante el proceso de aprobación/denegación.....	193
Figura 153	Código de la interfaz del historial de ingresos	194
Figura 154	Interfaz de consulta del historial de ingresos	194
Figura 155		195
Figura 156	Código de la interfaz de historial de reservas.....	196
Figura 157	Mensaje de consulta de historial de reservas.....	196
Figura 158	Interfaz de historial de reservas.....	196

Figura 159	Implementación del servicio del estado de la reserva	197
Figura 160	Código de integración entre el servicio y la interfaz.....	197
Figura 161	Código de interfaz de cargar comprobante de reserva	198
Figura 162	Interfaz de mensaje de error de carga.....	198
Figura 163	Código del controlador del módulo de reportes	199
Figura 164	Código del servicio.....	200
Figura 165	Definición de las rutas de las API REST	200
Figura 166	Código de la pantalla principal del módulo de reportes	201
Figura 167	Interfaz del Módulo de reportes	201
Figura 168	Resumen de reporte de usuarios registrados	202
Figura 169	Reporte de Usuarios registrados.....	202
Figura 170	Código de integración de servicios en Flutter	204
Figura 171	Flujo entre módulos - Registro de pago	204
Figura 172	Flujo entre módulos - Validación del pago	204
Figura 173	Flujo entre módulos - Reporte de pago	205

ABREVIATURAS

JWT	Jason Wen Token
QR	Quick Response (Respuesta Rápida)
CRUD	Create, Read, Update, Delete
HTTP	Protocolo de Transferencia de Hipertexto
API	Application Programming Interface
ORM	Object Relational Mapping



**UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE SOFTWARE**

**PROTOTIPO DE APLICACIÓN MÓVIL PARA EL CONTROL DE PAGOS
DE ALÍCUOTAS, ACCESO CON QR Y RESERVAS DE ÁREAS
SOCIALES DE UNA URBANIZACIÓN PRIVADA**

Autor(es): Jhonas Jeremy López Panta
C.I. N° 0943592030
Belky Xiomara Piedra Jiménez
C.I. N° 0954485595

Tutor: Ing. Angel Cuenca Ortega, M. Sc.

RESUMEN

El presente proyecto tiene como objetivo desarrollar un prototipo de aplicación móvil para el control de pagos de alícuotas, acceso mediante códigos QR y reservación de áreas sociales en una urbanización privada, utilizando una arquitectura de tres capas y tecnologías como Flutter, Dart y NestJS, con el fin de sistematizar procesos administrativos que actualmente se realizan de forma manual y generan retrasos y errores en la información. El desarrollo se basa en conceptos de aplicaciones móviles, bases de datos y automatización de procesos, empleando la metodología Scrum y el modelo incremental para construir el sistema por módulos funcionales. La aplicación integra los módulos de usuario, pagos, control de acceso y reservaciones, buscando modernizar la gestión administrativa, mejorar la seguridad, la eficiencia operativa y la calidad de los servicios, demostrando la viabilidad de una solución tecnológica para la gestión residencial.

Palabras clave: aplicación móvil, urbanización, código qr, scrum, control de pago, prototipo



**UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE SOFTWARE**

**PROTOTYPE OF MOBILE APPLICATION FOR THE CONTROL OF FREE
PAYMENTS, QR-BASE ACCESS AND RESERVATION OF SOCIAL
AREAS IN A PRIVATE RESIDENTIAL COMMUNITY**

Author(s): Jhonas Jeremy López Panta
C.I. N° 0943592030
Belky Xiomara Piedra Jiménez
C.I. N° 0954485595

Tutor: Ing. Angel Cuenca Ortega, M. Sc.

ABSTRACT

The present project aims to develop a prototype of a mobile application for the control of fee payments, access through QR codes, and reservation of social areas in a private residential community, using a three-tier architecture and technologies such as Flutter, Dart, and NestJS, in order to systematize administrative processes that are currently performed manually and generate delays and errors in information management. The development is based on concepts related to mobile applications, database design, and process automation, employing the Scrum methodology and the incremental development model to build the system through functional modules. The application integrates user, payment, access control, and reservation modules, seeking to modernize administrative management, improve security, operational efficiency, and service quality, while demonstrating the feasibility of a technological solution for residential management.

Key words: mobile application, residential community, QR code, scrum, payment control, prototype.

INTRODUCCIÓN

En la actualidad, el crecimiento de las urbanizaciones privadas en la ciudad de Guayaquil ha generado nuevas necesidades en la gestión administrativa y operativa de los servicios comunitarios. Entre estas necesidades se destacan el control eficiente de los pagos de alcuotas, la regulación del acceso de residentes y visitantes y la organización de la reserva de áreas sociales.

Sin embargo, en muchas urbanizaciones estos procesos aún se realizan mediante registros manuales o sistemas informales, lo que provoca retrasos en la atención, errores en la información y limitaciones en la seguridad y el control de los recursos comunes. Tal es el caso de la Urbanización Villa España I, etapa Mallorca, la cual realiza sus procesos de forma semimanual teniendo inconsistencias en los registros y aumentando la carga operativa.

Este escenario se ha visto significativamente afectado en procesos clave como la generación de reportes, la entrega de comprobantes de pago a los residentes y la ejecución de auditorías. Así mismo, las transiciones administrativas derivadas del cambio de directiva en la urbanización presentan dificultades adicionales en la transferencia de información y en la continuidad de la gestión operativa.

En este contexto, el presente proyecto tiene como propósito desarrollar un prototipo de aplicación móvil que permita centralizar y automatizar el control de pagos de alcuotas, el acceso mediante códigos QR y la reserva de áreas sociales en la Urbanización Villa España I, etapa Mallorca, agilizando el acceso de la información y mejorando la trazabilidad de sus procesos.

La importancia de este proyecto radica en su aporte al fortalecimiento de la gestión administrativa en la urbanización, ya que propone una solución tecnológica que integra funciones esenciales en una sola plataforma digital. Desde una perspectiva práctica, el desarrollo de este prototipo permitirá demostrar la viabilidad técnica de implementar una aplicación móvil que

mejore la organización de los procesos internos, incremente la seguridad en el control de accesos y facilite la administración de los recursos comunitarios. Además, contribuye al uso eficiente de la tecnología en el ámbito residencial, promoviendo la modernización de los servicios y la mejora continua en la gestión de la urbanización.

Finalmente, el presente proyecto se encuentra estructurado en varios capítulos que permiten organizar de manera lógica el desarrollo del proyecto.

- **Capítulo I**, presenta el planteamiento del problema, los objetivos, la justificación y el alcance de la investigación.
- **Capítulo II**, corresponde al marco teórico, donde se exponen los fundamentos conceptuales y antecedentes relacionados con el desarrollo de aplicaciones móviles y la gestión de sistemas de información.
- **Capítulo III**, describe la propuesta tecnológica, la metodología utilizada para el desarrollo del proyecto, incluyendo el modelo de desarrollo de software y las técnicas empleadas para la recolección y análisis de la información.
- **Capítulo IV**, presenta el desarrollo del prototipo, los resultados obtenidos y la validación del aplicativo móvil. Finalmente, se incluyen las conclusiones y recomendaciones derivadas del estudio realizado.

CAPÍTULO I

PLANTEAMIENTO DEL PROBLEMA

Descripción de la situación problemática

En el contexto actual, las urbanizaciones residenciales enfrentan desafíos operativos significativos derivados de la dependencia de procesos manuales y semimanuales para la gestión de actividades y la trazabilidad de la información. Esta situación se evidencia en la Urbanización Villa España I, etapa Mallorca, donde la limitada automatización de los procesos genera ineficiencias tales como una elevada carga operativa, duplicidad de registros, retrasos en el control de ingreso de visitantes y congestión en los accesos durante horarios de alta demanda.

La gestión manual de los procesos incrementa la carga laboral tanto del personal administrativo como del personal de seguridad, afectando la eficiencia y la calidad del servicio. En este contexto, la implementación de una solución tecnológica, específicamente un aplicativo móvil, orientado a la integración de procesos clave como pagos, control de accesos y reservas, se plantea como una alternativa viable para optimizar la gestión operativa. Se espera que dicha solución contribuya a la reducción de tiempos de atención, minimización de errores y mejora en la trazabilidad de la información. No obstante, su adopción estará condicionada por el nivel de aceptación y disposición al cambio por parte de los usuarios involucrados, considerando que la resistencia al cambio constituye una de las principales barreras en procesos de transformación

digital. En consecuencia, la implementación del aplicativo móvil se considera un factor estratégico para alcanzar mejoras sustanciales en la administración y operación de la urbanización.

Ubicación del problema en un contexto

La problemática identificada se sitúa en la Urbanización Villa España I, etapa Mallorca, un conjunto residencial que presenta limitaciones en la gestión de sus procesos operativos debido a la ausencia de herramientas tecnológicas integradas. En este contexto específico, las actividades relacionadas con el control de accesos, gestión de pagos y administración de reservas se ejecutan mediante mecanismos manuales o semimanuales, lo que afecta directamente la eficiencia operativa y la trazabilidad de la información.

Dicha situación se desarrolla principalmente en los puntos de control y en la administración de la urbanización, donde interactúan el personal de seguridad, el personal administrativo y los residentes. La alta afluencia de visitantes en horarios pico intensifica las deficiencias del sistema actual, evidenciando retrasos, acumulación de personas en los accesos y registros inconsistentes. Por tanto, el problema se delimita en el ámbito de la gestión operativa interna de la urbanización, específicamente en los procesos de control de pagos, accesos de visitas y reservaciones de las áreas sociales, los cuales carecen de automatización y de un sistema centralizado que garantice eficiencia y precisión de la información.

Situación conflicto nudos críticos

En primera instancia, se identifica la ausencia de un sistema tecnológico integrado, la cual constituye núcleo del problema, ya que impide la articulación de los procesos de control de pagos, accesos y reservas, generando segmentación de la información y dependencia de registros manuales.

Se evidencia la limitada disponibilidad y trazabilidad de la información en tiempo real, como consecuencia directa de la falta de digitalización, lo que dificulta la verificación oportuna de datos y la toma de decisiones por parte de la administración. Este aspecto deriva en inconsistencias en los registros, duplicidad de información y retrasos en los procesos operativos.

Otro punto relevante es la alta dependencia de procesos manuales y semimanuales, que incrementa la probabilidad de errores humanos y reduce la eficiencia operativa. Esta condición impacta directamente en la calidad del servicio, generando tiempos de espera prolongados, especialmente en el control de accesos durante horarios de alta demanda. Ante todos estos factores mencionados se identifica la limitada capacidad de respuesta operativa ante el incremento del flujo de residentes y visitantes, la cual es consecuencia de los factores anteriores. La falta de herramientas tecnológicas adecuadas impide escalar los procesos de manera eficiente, afectando el control, la seguridad y la organización interna de la urbanización.

Delimitación del problema

El presente proyecto se delimita al desarrollo de un prototipo de aplicación móvil y web orientado a la gestión administrativa de la Urbanización Villa España I, etapa Mallorca, específicamente en los procesos de control de pagos de alcótuas, registro y autorización de accesos mediante códigos QR, y reservación de áreas sociales como se observa en la *Tabla 1*. La solución estará dirigida a los actores principales del entorno residencial: administradores, personal de seguridad y residentes, quienes interactuarán con el sistema para optimizar la gestión, el control y la disponibilidad de la información.

Respecto a la delimitación espacial y contextual, el proyecto se circunscribe al entorno operativo de la Urbanización Villa España I, etapa Mallorca, considerando sus condiciones actuales en la gestión administrativa y control de accesos.

Tabla 1

Delimitación del problema

Delimitador	Descripción
Campo	Tecnologías de la Información y Comunicación (TIC) aplicadas a la gestión administrativa, control de accesos y seguridad en urbanizaciones privadas.
Área	Desarrollo de software y aplicaciones móviles orientadas a la automatización de pagos, control de accesos mediante códigos QR y gestión de reservas
Aspecto	Automatización de procesos administrativos, validación de accesos de residentes y visitantes, y organización del uso de áreas comunes mediante herramientas tecnológicas.
Tema	<i>“Prototipo de aplicación móvil para el control de pagos de alícuotas, acceso QR y reservas de áreas sociales de una urbanización privada”</i>

Nota: En esta tabla se plantean los términos de análisis aplicados para la delimitación del problema conforme al contexto en donde se desarrolla la problemática. **Fuente:** Elaboración propia producto de la investigación.

Evaluación del problema

El problema de investigación relacionado con el desarrollo de la aplicación móvil para la Urbanización Villa España I, etapa Mallorca, cumple con los siguientes aspectos generales de evaluación:

Los aspectos generales de evaluación son:

- **Delimitado:** El problema se encuentra claramente delimitado en términos de espacio, tiempo y población. Espacialmente, se ajusta a la Urbanización Villa España I, etapa Mallorca; temporalmente, a la situación actual de los procesos administrativos y operativos; y poblacionalmente, a los residentes, administración y personal de seguridad. La gestión de pagos, accesos y reservas se rige por reglamentos internos de

la urbanización, los cuales requieren indicadores de control como el registro de pagos, validación de accesos autorizados y asignación de áreas comunes, elementos que actualmente no cuentan con un sistema automatizado que facilite su medición y seguimiento.

- **Claro:** El problema está redactado de forma precisa y clara, identificando de manera directa la falta de automatización en los procesos de pagos, control de accesos y reserva de áreas comunes. Las ideas se presentan de forma concisa, permitiendo reconocer con facilidad la necesidad de una solución tecnológica mediante una aplicación móvil integrada.
- **Evidente:** La problemática presenta manifestaciones claras y observables, como el alto tiempo invertido en la verificación manual de pagos, el control de accesos dependiente del factor humano y los conflictos frecuentes por el uso de áreas comunes. Estos procesos manuales generan retrasos, errores en los registros y falta de información en tiempo real, evidenciando ineficiencias en la gestión actual.
- **Concreto:** El problema se formula de manera directa y precisa, enfocándose exclusivamente en la automatización de pagos, el uso de códigos QR para accesos y la reserva de áreas comunes, sin abarcar procesos ajenos al objetivo del proyecto. La problemática es relevante para la comunidad, ya que impacta directamente en la seguridad, la convivencia y la eficiencia administrativa de la urbanización. Su resolución mediante un enfoque científico y tecnológico contribuye a mejorar la calidad de vida de los residentes y optimizar la gestión comunitaria.
- **Relevante:** La problemática es relevante para la comunidad, ya que impacta directamente en la seguridad, la convivencia y la eficiencia administrativa de la

urbanización. Su resolución mediante un enfoque científico y tecnológico contribuye a mejorar la calidad de vida de los residentes y optimizar la gestión comunitaria.

- **Original:** El proyecto se distingue por su enfoque al proponer una solución tecnológica integrada que consolida, en una única aplicación móvil, la automatización del control de pagos de alcuotas, la gestión de accesos mediante códigos QR y la administración de reservas de áreas comunes, orientado a la digitalización y optimización de procesos, contribuyendo a la modernización de la gestión administrativa y operativa en entornos residenciales privados.

Causas y consecuencias del problema

La identificación de las causas y consecuencias del problema representadas en la *Tabla 2*, permite comprender de manera integral los factores que originan las deficiencias en la gestión administrativa, el control de accesos y la organización de las áreas comunes en la Urbanización Villa España I, etapa Mallorca. Este análisis facilita reconocer los elementos críticos que inciden directamente en la problemática, así como las posibles afectaciones que podrían intensificarse si la situación actual se mantiene.

Para ello, se ha considerado un enfoque de análisis basado en recursos humanos, métodos, procesos, tecnología e infraestructura, permitiendo establecer una relación clara entre las causas que generan el problema y las consecuencias que impactan en la seguridad, eficiencia operativa y convivencia de los residentes. A continuación, se presenta la tabla que detalla las principales causas identificadas y sus respectivas consecuencias.

Tabla 2*Matriz de causas y consecuencias del problema*

Causas	Consecuencias
C1. Registro de pagos en hojas de cálculo sin control de acceso.	E1. Pérdida de dinero y errores al registrar pago de residente.
C2. Gestión de la entrega de comprobantes de pago de manera física.	E2. Retrasos y pérdidas de comprobantes de pago por el personal administrativo.
C3. Falta de disponibilidad de información en tiempo real.	E3. Dificultades en la trazabilidad y control de la información.
C4. Protocolo de autorización de ingreso de visitas basadas en llamadas telefónicas y verificación manual.	E4. Largas fila, embotellamientos al ingreso de visitantes y dependencia total del personal de seguridad.
C5. Reservación del área social de manera presencial por parte del residente limitado por el horario administrativo.	E5. Solapamientos en reservas del área social.

Nota: Esta tabla refleja el análisis causal que se realizó en base a la recopilación inicial de información de la situación problemática que genera el proyecto mediante la aplicación de la Metodología de Marco Lógico, se considera por ello datos relevantes de la fase de investigación del proyecto. Elaboración propia.

Formulación del problema

Para expresar la problemática de forma explícita y concisa, se plantea a continuación la siguiente interrogante:

¿Cuál es el impacto de la implementación de una aplicación móvil para el control de pagos de alícuotas, accesos con códigos QR y la reserva de áreas sociales en la mejora de la gestión administrativa, la seguridad y la organización de procesos en la Urbanización Villa España I, etapa Mallorca?

Objetivos del proyecto

Objetivo general

Desarrollar un prototipo de aplicación móvil mediante una arquitectura de tres capas, tecnologías Flutter, Dart, Nest.JS para la sistematización el registro y control de pagos de alcúotas, acceso con códigos QR y reservación de áreas sociales en una urbanización privada.

Objetivos específicos

- Analizar los procesos actuales de control de pago de alcúotas, acceso y reservación de áreas en las urbanizaciones de Guayaquil, para la determinación de los requerimientos funcionales y no funcionales.
- Diseñar la arquitectura y los módulos de control de pagos de alcúotas, acceso con códigos QR y reservación de áreas sociales para la organización de la base de datos, la comunicación entre los componentes del prototipo de aplicación móvil.
- Construir los módulos del prototipo de aplicación móvil que permitan cumplir los requerimientos funcionales.
- Validar el diseño funcional de la aplicación con plan de pruebas y juicios de expertos, para la consolidación de la versión final del prototipo de aplicación móvil asegurando el funcionamiento y calidad.

Alcance del Problema

El presente proyecto tiene como alcance el desarrollo de un prototipo de aplicación móvil orientado a la optimización de la gestión administrativa en la Urbanización Villa España I, etapa Mallorca, abordando específicamente los procesos de control de pagos de alcúotas, gestión de accesos mediante códigos QR y reservación de áreas sociales.

En una primera fase, el proyecto contempla el análisis de los procesos actuales implementados en la Urbanización Villa España I, etapa Mallorca, con el propósito de identificar las limitaciones operativas existentes y a partir de ello, definir los requerimientos funcionales y no funcionales que orientarán el desarrollo de la solución tecnológica. Este análisis permitirá establecer una base sólida para la construcción del sistema, alineada con las necesidades reales del entorno.

Posteriormente, se desarrollará el diseño de la arquitectura del sistema y de los módulos del prototipo, considerando la estructuración de la base de datos, la definición de la lógica de negocio y la comunicación entre los componentes de la aplicación móvil. Este diseño se fundamentará en una arquitectura de tres capas, con el uso de herramientas y entornos de trabajo Flutter, Dart, Nest.JS, garantizando escalabilidad, mantenibilidad y adecuada organización del sistema.

En la fase de desarrollo, se procederá a la construcción de los módulos del prototipo, implementando las funcionalidades definidas para la gestión de pagos, control de accesos y reservación de áreas comunes, asegurando el cumplimiento de los requerimientos previamente establecidos.

Finalmente, el alcance incluye la validación del diseño funcional del prototipo mediante la ejecución de un plan de pruebas y la aplicación de juicios de expertos, con el objetivo de verificar

su funcionamiento, usabilidad y calidad. Esta etapa permitirá consolidar una versión final del prototipo que demuestre la viabilidad técnica y operativa de la solución propuesta. Cabe destacar que el proyecto se limita al desarrollo y validación de un prototipo funcional, sin contemplar su implementación en un entorno productivo ni la integración con sistemas externos, ni dispositivos de seguridad, ni plataformas financieras.

La aplicación móvil incluirá los siguientes módulos:

Módulo de usuarios

El módulo de usuario será responsable de la gestión de la información y autenticación de los usuarios del sistema, permitiendo el acceso seguro a la aplicación móvil según el rol asignado ya sea administrador o residente.

Este módulo permitirá el registro, actualización y consulta de los datos personales de los usuarios, así como la administración de credenciales de acceso y la validación de identidad mediante mecanismos de autenticación. Además, facilitará la gestión de perfiles y la asignación de permisos para el uso de las funcionalidades del sistema.

Funciones Principales:

- Registro de usuarios.
- Inicio y cierre de sesión seguro.
- Actualización de información personal.
- Gestión de roles y permisos.
- Visualización del perfil del usuario.

Módulo de control de pagos

El módulo de control de pagos permitirá la administración y seguimiento de los pagos de alícuotas realizados por los residentes de la urbanización, proporcionando un registro actualizado y confiable. Este módulo permitirá generar estados de cuenta, registrar pagos realizados, consultar historiales de pago y notificar a los usuarios sobre valores pendientes o fechas de vencimiento.

Funciones Principales:

- Registro de pagos de alícuotas
- Generación de estados de cuenta
- Consulta de historial de pagos
- Visualización de deudas pendientes
- Notificaciones de vencimiento de pagos
- Reportes de pagos realizados

Módulo de control de acceso

El módulo de control de acceso permitirá la gestión y verificación del ingreso de residentes y visitantes a la urbanización mediante el uso de códigos QR, mejorando la seguridad y el control de acceso vehicular y peatonal. Este módulo generará códigos QR únicos para cada usuario o visitante autorizado, los cuales serán escaneados por el personal de seguridad en los puntos de acceso. El sistema registrará automáticamente la fecha y hora de cada ingreso y salida, permitiendo la trazabilidad de los accesos y la generación de reportes de control.

Funciones Principales:

- Generación de códigos QR para residentes y visitantes
- Escaneo y validación de códigos QR
- Registro de entradas y salidas
- Consulta de historial de accesos

Módulo de reservas

El módulo de reservaciones permitirá a los residentes solicitar y gestionar la reserva de áreas sociales de la urbanización, tales como canchas deportivas, salones comunales o piscinas, optimizando el uso de los espacios comunes. Este módulo facilitará la visualización de la disponibilidad de las áreas sociales, la solicitud de reservas en fechas y horarios específicos, y la confirmación o cancelación de reservas. Además, permitirá al administrador supervisar el uso de los espacios y evitar conflictos de programación.

Funciones Principales:

- Consulta de disponibilidad de áreas sociales
- Registro de solicitudes de reserva
- Confirmación y cancelación de reservas
- Control de horarios y fechas
- Notificaciones de confirmación de reserva
- Historial de reservaciones realizadas

Módulo de reportes

Permitirá generar y visualizar información relevante sobre las actividades realizadas dentro de la aplicación, incluirá reportes de pagos de alcuotas, historial de accesos mediante códigos QR, reservas de áreas sociales y estadísticas generales del sistema, presentando la información de manera organizada y comprensible.

El proceso de desarrollo iniciará con la fase de Diseño UI/UX, donde se creará un prototipado de alta fidelidad en Figma. Este diseño estará centrado en la creación de componentes reutilizables y en la usabilidad, asegurando que cada pantalla y elemento de interacción sea validado visualmente antes de su codificación en Flutter. Este enfoque permitirá optimizar la experiencia del usuario y garantizar una navegación intuitiva en los módulos de pagos, accesos y reservaciones.

El backend será implementado con Node.js y el framework NestJS - TypeScript, el cual contempla una API REST para el procesamiento de la lógica del negocio, incluyendo la gestión de usuarios, validación de accesos y generación de códigos QR. Para garantizar la seguridad, se implementará un sistema de autenticación basada en JWT con Guards de NestJS, definiendo roles específicos para administradores, residentes y guardias de seguridad.

La persistencia de la información se gestionará en una base de datos relacional PostgreSQL, utilizando ORM Prisma para el modelado de entidades y gestión de migraciones. La aplicación contará con notificaciones en tiempo real mediante Fire Cloud Messaging para notificar la llegada de los visitantes o el vencimiento de los pagos.

La infraestructura tecnológica se desplegará en la nube utilizando Railway para el hosting del servidor NestJS y la base de datos PostgreSQL, asegurando la accesibilidad del sistema desde cualquier ubicación. El código fuente será administrado a través de GitHub, garantizando el control de versiones y la trazabilidad del proceso del desarrollo.

No Incluye:

- Desarrollo de módulos adicionales.
- Integración con APIs de instituciones financieras.
- La integración con sistemas externos o hardware de control de acceso.

Justificación e Importancia

La Urbanización Villa España I, etapa Mallorca, presenta actualmente limitaciones en la gestión de sus procesos administrativos y operativos, particularmente en el control de pagos de alcuotas, el registro y validación de accesos, y la reservación de áreas sociales. Estas actividades se realizan mediante mecanismos manuales y semimanuales, lo que genera ineficiencias reflejadas en la duplicidad de registros, demoras en la atención, dificultades en la trazabilidad de la información y una alta dependencia del recurso humano. Esta situación impacta directamente en la calidad del servicio, la seguridad y la capacidad de respuesta de la administración frente a las necesidades de los residentes.

En este contexto, el desarrollo de un prototipo de aplicación móvil constituye una alternativa tecnológica orientada a integrar y automatizar los procesos críticos de la urbanización. La propuesta busca centralizar la información, mejorar el control operativo y optimizar la gestión administrativa mediante funcionalidades como la automatización del registro de pagos, el control de accesos mediante códigos QR y la gestión de reservas de áreas comunes. De esta manera, se

pretende reducir los tiempos de atención, minimizar errores en los registros y fortalecer los mecanismos de control y seguridad.

Desde el punto de vista académico, el proyecto representa una aplicación práctica de conceptos de ingeniería de software, desarrollo móvil y experiencia de usuario, al abordar una necesidad real del entorno local. Además, sirve como referencia para futuros trabajos orientados a la digitalización de procesos en urbanizaciones y comunidades residenciales, fortaleciendo la formación profesional y demostrando la capacidad de proponer soluciones tecnológicas a problemáticas concretas.

Justificación técnica

La arquitectura de tres capas apoyada en servicios se adopta debido a que permite una adecuada separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y la gestión de datos. Esta estructura facilita el desarrollo de un sistema organizado y modular, mejorando su mantenimiento y comprensión.

La capa de presentación se encarga de la interacción con los usuarios mediante una aplicación móvil intuitiva, mientras que la capa de lógica de negocio centraliza las reglas del sistema, como la validación de pagos, la gestión de códigos QR y la disponibilidad de áreas sociales. La capa de datos permite estructurar la información de forma coherente, garantizando la trazabilidad de los procesos. Esta arquitectura se ajusta al alcance del proyecto, ya que permite simular los procesos principales sin necesidad de integraciones externas.

Justificación académica

La arquitectura de tres capas permite aplicar principios fundamentales del diseño de software, como la modularidad y la separación de responsabilidades. Su uso facilita la

comprensión del funcionamiento de una aplicación móvil y el diseño de soluciones basadas en servicios.

Esta arquitectura contribuye al desarrollo de competencias relacionadas con el análisis de requerimientos, el modelado de sistemas y la documentación técnica, permitiendo vincular los conocimientos teóricos con una problemática real. Su implementación a nivel de prototipo garantiza el cumplimiento de los objetivos del proyecto dentro de un marco académico y técnicamente viable.

Limitaciones del Estudio

El presente estudio presenta ciertas limitaciones que deben ser consideradas durante el desarrollo y evaluación del proyecto. En primer lugar, la investigación se limita espacialmente a la Urbanización Villa España I, etapa Mallorca, por lo que los resultados obtenidos no pueden generalizarse a otras urbanizaciones con características distintas.

Desde el punto de vista temporal, el desarrollo y la validación de la aplicación móvil se realizará dentro del período establecido para el proyecto académico, lo cual puede restringir la evaluación del desempeño del sistema en el largo plazo, especialmente en lo relacionado con la adopción por parte de los usuarios y la sostenibilidad del sistema.

En el ámbito tecnológico, la aplicación dependerá de la disponibilidad de dispositivos móviles y de conexión a internet por parte de los residentes y del personal administrativo, lo que podría limitar su uso en casos de fallas de conectividad o escasa alfabetización digital de algunos usuarios. Así mismo, el estudio no contempla la integración con plataformas externas como entidades bancarias o sistemas de pago avanzados, restringiéndose a modelos de pago definidos por la administración de la urbanización. De igual manera, el control de accesos mediante códigos

QR no se implementará a dispositivos o tecnología avanzada usadas en el ámbito de la seguridad para el control de accesos.

Finalmente, el estudio se enfoca exclusivamente en los procesos de pagos, accesos y reserva de áreas comunes, sin abordar otros aspectos de la gestión administrativa de la urbanización, lo que delimita el alcance de la solución propuesta.

CAPÍTULO II

MARCO TEORICO

Antecedentes del Estudio

La migración de procesos manuales a sistematizados ha tenido gran acogida por parte de las urbanizaciones privadas en los últimos años, esto con el fin brindar una mejora en los diferentes servicios ofrecidos, reducción de errores, sin embargo, ciertas urbanizaciones privadas, específicamente en la ciudad de Guayaquil, aun realizan sus procesos en documentos físicos lo que ocasiona retrasos, molestias para los residentes y sobrecarga al personal administrativo. En este contexto, la Urbanización privada Villa España I, etapa Mallorca forma parte de las urbanizaciones que llevan ciertos procesos de forma manual. A continuación, se detallan una serie de antecedentes relevantes que promueven el desarrollo de una aplicación móvil para digitalizar los procesos anteriormente descritos de la urbanización en mención.

A nivel global

En Indonesia, Febrianto et al. (2023) abordaron las dificultades existentes en la administración de las cuotas de residentes mediante el diseño del Sistema de Información para la Gestión de Cuotas de Residentes (SIUMAS). Esta solución tecnológica fue desarrollada con el propósito de optimizar los procesos relacionados con la recaudación y distribución de cuotas por parte del comité administrativo, así como de proporcionar a los residentes un mayor nivel de control, transparencia y seguimiento sobre la utilización de los fondo recaudados.

El sistema no abarca la gestión administrativa ni la reserva de áreas comunes, dirige su enfoque exclusivamente al proceso de pagos; fue implementado utilizando el lenguaje de programación PHP y una base de datos MySQL, bajo un enfoque de desarrollo orientado a objetos, lo que permitió una estructura modular, escalable y de fácil mantenimiento.

A nivel regional

En el caso de Perú, Condori (2021) desarrolló un sistema web para mejorar la administración del condominio Las Terrazas de Surco, utilizando la metodología Scrum con un enfoque cuantitativo y diseño experimental aplicado a una población de 50 propietarios. El estudio identificó siete sprints distribuidos en cinco fases de desarrollo, obteniendo un instrumento validado por juicio de expertos con un 97% de confiabilidad, los resultados evidenciaron que la implementación de un sistema web basado en metodología ágil mejora significativamente los procesos administrativos del condominio, reduciendo tiempos de gestión y aumentando la satisfacción de los propietarios.

A nivel local

En el contexto ecuatoriano, se han desarrollado diversas investigaciones orientadas a digitalizar los procesos administrativos de urbanizaciones privadas. Orellana & Ramírez (2025), desarrollaron un prototipo de aplicación móvil destinado al control de acceso vehicular en urbanizaciones privadas, incorporando tecnologías como reconocimiento facial, OCR y códigos QR con el fin de mejorar la seguridad y agilizar los procesos de ingreso que anteriormente se realizaban de forma manual. Cabe mencionar que este proyecto de la Universidad de Guayaquil se enfocó únicamente en el control de acceso, biometría facial, procesamiento de imágenes y arquitectura cliente-servidor, orientados a impulsar la transformación digital y garantizar un mejor

seguimiento de los eventos registrados. Se diseñó una arquitectura tecnológica utilizando Flutter, FastAPI y PostgreSQL, implementando el sistema de manera progresiva. Entre las funcionalidades desarrolladas destacan la validación mediante códigos QR, el registro de visitantes sin QR, la lectura automática de placas vehiculares mediante OCR, la autorización de accesos por llamada telefónica y la generación de registros digitales verificables.

Por otro lado, Loor & Betancourt (2024) demostraron que gestionar el ingreso de visitantes mediante registros físicos es propenso a errores de transcripción e impide la generación de reportes en tiempo real. Los autores desarrollaron en la ESPOL un sistema híbrido compuesto por una aplicación móvil y un módulo web, utilizando el código QR como mecanismo central de autenticación, donde cada visitante recibe un código único y dinámico validado por el guardia al momento del ingreso. El proyecto no contempla la gestión de los procesos administrativos, va dirigido solamente al control de accesos. El sistema desarrollado bajo metodología Scrum, logró un incremento del 74% en la eficiencia del proceso de verificación en los puntos de control.

Finalmente, Correa & Castro (2016) identificaron que la urbanización La Joya, etapa Murano, presentaba graves inconvenientes en su administración respecto al pago mensual de alcuotas, dado que el registro de pagos se realizaba de forma manual y la persona encargada frecuentemente omitía el ingreso de valores cancelados ante el alto volumen de transacciones diarias, generando desorden contable y conflictos con los residentes. Como solución, desarrollaron una aplicación web en PHP bajo el framework Laravel exclusivamente para gestionar el control de los pagos que permitió centralizar la administración del conjunto y automatización de este proceso, eliminando los registros en papel y proporcionando a la directiva información financiera confiable y actualizada en tiempo real.

Fundamentación Teórica

El desarrollo del presente marco teórico se articula como la base científica y conceptual que sustenta el diseño del prototipo móvil para la gestión residencial en la Urbanización Villa España I, etapa Mallorca.

En este sentido, la investigación trasciende la mera implementación técnica para situarse en la intersección de la sociología urbana y la transformación digital, abordando cómo la automatización de procesos administrativos y de seguridad influye en la cohesión comunitaria. A través de una revisión bibliográfica exhaustiva y actualizada, se examinan las dimensiones pedagógicas de la alfabetización digital, las teorías de aceptación tecnológica y los protocolos de seguridad informática que validan la viabilidad del control de alícuotas, la gestión de reservas y el acceso mediante tecnología QR. Bajo este enfoque multidimensional, los fundamentos aquí expuestos no solo pretenden describir el funcionamiento del software, sino fundamentar éticamente la transparencia y eficiencia que la modernización de servicios aporta al entorno habitacional privado contemporáneo.

Sistemas de información para gestión residencial y automatización administrativa

Urbanizaciones y Privadas y Gestión Administrativa

Se conoce como urbanización a el proceso mediante el cual la población de un país se concentra progresivamente en las zonas urbanas, dejando de residir principalmente en áreas rurales. Como consecuencia, las ciudades experimentan un aumento tanto en su población como en la extensión de su territorio, generando una expansión continua de los espacios urbanos (Raffino, Equipo editorial, Etecé, 2026).

En Guayaquil, el panorama de urbanizaciones privadas es amplio y diverso. Según García (2025), citado por Primicias, en base a las investigaciones que se enfocan en la realidad de la vía a la Costa, ubicada al oeste de Guayaquil, donde actualmente existen 114 urbanizaciones en las que habitan aproximadamente 100.000 personas, de acuerdo con la información proporcionada por la Alcaldía. Esto refleja la importancia y el crecimiento de este modelo de desarrollo urbano en la ciudad.

Los principales factores que motivan a los ciudadanos a elegir residir en urbanizaciones privadas son la gestión administrativa eficiente, la seguridad que ofrecen y la disponibilidad de áreas y servicios comunes. Sin embargo, algunas urbanizaciones, como Villa España I, etapa Mallorca donde aún mantienen procesos administrativos semi automáticos con un nivel limitado de modernización tecnológica, lo que evidencia la necesidad de implementar soluciones digitales que optimicen la gestión de pagos, control de accesos y reservas de áreas sociales.

Gestión de Alícuotas

Las alícuotas que los propietarios o inquilinos pagan mensualmente son fundamentales para cubrir los gastos necesarios y asegurar el correcto funcionamiento y mantenimiento de un condominio o conjunto residencial (El Comercio, 2011). Este valor es el ingreso neto para la urbanización, mismo que cubre el pago al personal de seguridad, recolección de basura, mantenimiento de infraestructura y áreas verdes y desde luego el personal administrativo. Los métodos de pago varían según la administración de cada urbanización, muchas de ellas manejan dispositivos administrados por las entidades financieras, mismos que se utilizan para pagos con tarjetas, también se frecuenta el pago con transferencia y el manejo de dinero en efectivo es también parte de la forma de los pagos.

Por estos factores es que es importante la implementación de un sistema que pueda gestionar o controlar estos pagos que son muy importantes para el bienestar de los residentes de la urbanización en general, ya que con el uso de una herramienta tecnológica enlazada a una base de datos se puede automatizar el proceso de control de pagos y tener una buena trazabilidad de los ingresos.

Automatización de servicios

El equipo de Comidor (2019) señala que los procesos manuales suelen requerir más tiempo y generar mayores costos en comparación con aquellos que han sido automatizados. Esto se debe a que las actividades manuales dependen directamente de la intervención humana para tareas como el registro, validación y verificación de datos, mientras que en los sistemas automatizados estas funciones son ejecutadas por herramientas tecnológicas o sistemas informáticos, permitiendo procesos como el escaneo, la clasificación y el procesamiento de información de manera más rápida y eficiente.

En este sentido, los procesos manuales implican una mayor cantidad de pasos intermedios para alcanzar un resultado final, lo que incrementa la probabilidad de errores humanos, retrasos en la ejecución y una menor consistencia en los resultados. Además, al depender de la disponibilidad y desempeño de las personas, estos procesos pueden verse afectados por factores como la carga laboral, la fatiga o la falta de estandarización en las actividades.

Por el contrario, la automatización de procesos permite mejorar significativamente la eficiencia operativa, ya que reduce la intervención humana en tareas repetitivas y minimiza los márgenes de error. Así mismo, los sistemas automatizados ofrecen mayor precisión, rapidez y

trazabilidad de la información, lo que facilita el control y la toma de decisiones basada en datos más confiables.

En consecuencia, la transición de procesos manuales o semi manuales hacia procesos automatizados representa una oportunidad de mejora para optimizar la gestión operativa dentro de la urbanización Villa España I, etapa Mallorca. Esta transformación permite incrementar la eficiencia, reducir tiempos de ejecución y mejorar la calidad de los resultados, contribuyendo así a una administración más moderna, organizada y orientada a la tecnología.

Digitalización de procesos residenciales

La resistencia al cambio es una reacción común que suele presentarse en las organizaciones cuando se enfrentan a procesos de innovación o transformación. Este comportamiento se vuelve más evidente en contextos complejos, como el actual escenario de la era digital. En este sentido, lograr superar esta resistencia se ha convertido en un aspecto clave para las empresas que buscan seguir siendo competitivas y adaptarse a un entorno que cambia de manera continua (Martínez, 2025).

La autora señala que cuando una organización comienza un proceso de transformación digital, es común que aparezcan obstáculos vinculados a su cultura interna y a la manera en que tanto los mandos intermedios como los colaboradores interpretan los cambios que se proponen. Identificar la resistencia al cambio en sus distintas formas, ya sea de manera activa mediante la oposición directa o de forma pasiva a través de la falta de compromiso, constituye el primer paso fundamental para poder enfrentarla y gestionarla adecuadamente.

Por ello, es fundamental desarrollar herramientas que sean intuitivas, accesibles y que faciliten la aceptación del cambio, reduciendo la resistencia de los usuarios frente a nuevas formas de trabajo. En este contexto, el proyecto busca contribuir a la mejora de la eficiencia operativa y la modernización de los procesos, promoviendo una gestión más ordenada, rápida y confiable, alineada con las exigencias actuales de la transformación digital.

Arquitectura de software y desarrollo de aplicaciones móviles

Arquitectura de tres capas

Según Bayarri (2024) la arquitectura de tres capas es un modelo de diseño de software que organiza una aplicación en tres niveles bien diferenciados; la capa de presentación, la capa de lógica de negocio y la capa de acceso a datos. Su propósito principal es separar las funciones del sistema en componentes independientes, lo que hace que el desarrollo sea más ordenado y simplifica el mantenimiento, la escalabilidad y la administración general de la aplicación.

Este enfoque permite una clara separación entre la capa de presentación, la lógica de negocio y la capa de acceso a datos, lo que facilita la modularidad, el mantenimiento y la escalabilidad del sistema. En particular, esta división es crítica en soluciones que integran múltiples funcionalidades transaccionales y de autenticación, como las requeridas en este proyecto, ya que reduce el acoplamiento entre componentes y mejora la capacidad de evolución del sistema ante cambios en los requisitos funcionales o tecnológicos.

De igual forma, la arquitectura de tres capas favorece la integración eficiente entre el frontend desarrollado en Flutter y el backend implementado en NestJS mediante APIs bien definidas, lo que incrementa la cohesión del sistema y mejora la seguridad al centralizar la lógica

de negocio y el control de acceso a los datos. En consecuencia, este estilo arquitectónico no solo responde a criterios técnicos de calidad de software, sino también a la necesidad de garantizar confiabilidad, mantenibilidad y escalabilidad en entornos residenciales digitalizados.

Aplicaciones Móviles

Las aplicaciones móviles son programas que se instalan y utilizan en dispositivos portátiles como teléfonos celulares y tabletas. Su uso y desarrollo son necesarios debido a nuestra creciente dependencia a estos dispositivos para realizar diversas actividades cotidianas. Estas incluyen desde aprendizaje, transferencias bancarias, comunicación, monitoreo de salud, hasta recreación, entre otras.

De acuerdo con Böhm (2023), las aplicaciones móviles son software de aplicación diseñados para ejecutarse en dispositivos móviles, como smartphones, cuya finalidad es aplicar la funcionalidad del hardware y el software del sistema para resolver problemas específicos del usuario. Estas aplicaciones permiten la personalización del dispositivo mediante programas y datos instalados por el usuario final.

Tipos de Aplicaciones Móviles

Existen tres tipos de aplicaciones móviles, entre ellas están las aplicaciones nativas, aplicaciones web y las aplicaciones híbridas.

- **Aplicación nativa.** Este tipo de aplicaciones son aquellas que se crean específicamente para un sistema operativo móvil bajo el lenguaje de programación nativo, cabe mencionar que, si se desea desarrollar una aplicación móvil

multiplataforma, el uso de una aplicación nativa multiplica las horas de trabajo ya que cada sistema operativo necesitara una aplicación diferente.

- **Aplicación Web.** Estas son desarrolladas principalmente con el lenguaje JavaScript en conjunto con CSS y HTML; en la comparativa con las aplicaciones nativas, estas son más fáciles y rápidas de desarrollar, pero el rendimiento es en menor. Regularmente son utilizadas para mostrar un versione optimizadas de un sitio web en un dispositivo móvil.
- **Aplicación Híbrida.** Permiten la ejecución de una aplicación web dentro de una nativa, en otras palabras, también son consideras aplicaciones web, que se convierte en una aplicación que puede instalarse en el dispositivo, un claro ejemplo es Uber que combina la tecnología de JavaScript, pero con componentes nativos.

Según un artículo de IBM indica que el desarrollo de aplicaciones móviles se ha ido incrementando en todos los ámbitos desde pequeños negocios hasta las grandes industrias, ya que el potencial que aportan en la administración de los procesos y la satisfacción de los usuarios.

Desarrollo Multiplataforma con Flutter

Flutter es un framework lanzado en 2017 por Google para el desarrollo de apps móviles multiplataforma nativas. Chávez (2023). Todo eso a partir de un solo código base, generando la aplicación; se implementa en diversos dispositivos tanto web, móviles, escritorio y dispositivos integrados.

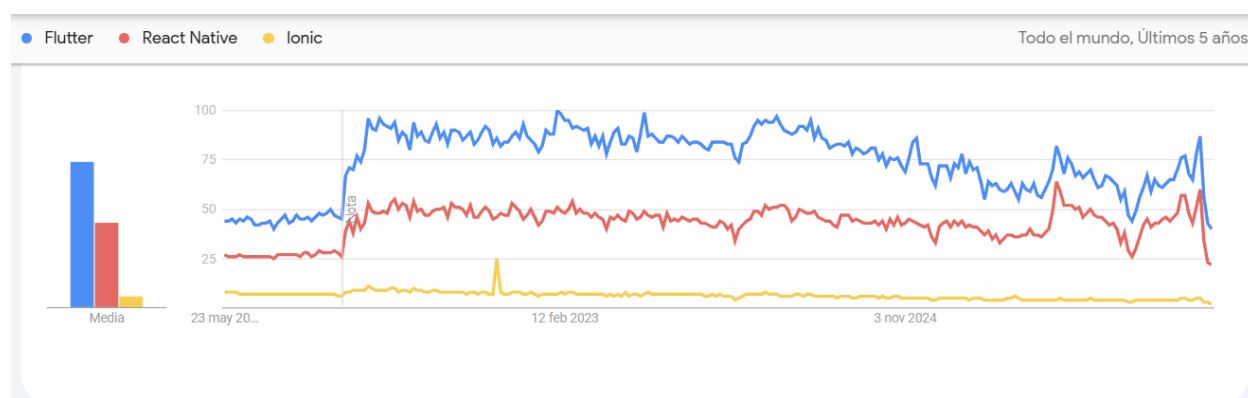
Flutter está basado en el lenguaje de programación Dart, cuyas características dotan a Flutter la capacidad de compilarse directamente para la arquitectura y el sistema operativo de destino, lo que permite que se puedan desarrollar aplicaciones multiplataforma.

Además, es necesario destacar que una gran ventaja que brinda Flutter al momento de desarrollar es el Hot Reload, que básicamente es que todo cambio que se realice se podrá visualizar de inmediato en la pantalla. Otra de las ventajas que ofrece son los widgets, que no son nada más que elementos de la interfaz que se reutilizables lo que facilita su movilidad como piezas a lo largo de la pantalla hasta obtener la interfaz deseada.

Una evaluación cuantitativa del interés de búsqueda global en Google Trends confirma que Flutter lidera significativamente el ecosistema de desarrollo móvil multiplataforma, alcanzando el pico máximo de popularidad en el periodo analizado. En segundo lugar, React Native sostiene un posicionamiento intermedio con una brecha clara respecto al líder, mientras que Ionic muestra una relevancia marginal cercana a cero. Como se puede observar en la *Figura 1*, las estadísticas demuestran de forma empírica que la tendencia actual de la industria converge con mayor fuerza hacia soluciones basadas en el framework de Google, validando su relevancia técnica y tracción en el mercado de software actual frente a sus competidores directos.

Figura 1

Análisis comparativo: Flutter vs React Native vs Ionic



Nota: Esta distribución estadística corrobora en los últimos cinco años, una marcada tendencia del sector hacia soluciones basadas en rendimiento nativo y renderizado propio, con Flutter a la vanguardia del ecosistema multiplataforma. Tomado de (Google Trends, 2026)

Lenguaje de Programación Dart

Es un lenguaje creado por Google y diseñado para el desarrollo de aplicaciones en cualquier plataforma, con el objetivo de ser un lenguaje de programación productivo en el desarrollo multiplataforma. Este lenguaje impulsa las aplicaciones de Flutter por medio del lenguaje y los entornos que provee.

Dart posee dos compiladores: AOT y JIT, donde el primero es para la compilación clásica lo que significa que la aplicación debe ser compilada antes de su ejecución y el segundo donde la compilación se realiza por partes durante la ejecución. Cuenta con amplio conjunto de bibliotecas que brindan elementos fundamentales para diversas tareas de programación, por otro lado, también provee muchas API por medio de un conjunto completo de paquetes.

Arquitectura Backend

Dentro del desarrollo web, la arquitectura backend juega un papel fundamental, ya que es la que se encarga de la construcción de la estructura y la lógica del negocio de un sitio, En otras palabras, el backend es la lógica detrás de lo que se puede ver en una aplicación. En el backend se escribe el código que permite que la aplicación funcione de manera correcta. El backend está compuesto por algunos componentes:

- **Servidores.** Recopila las interacciones que tiene el usuario con las aplicaciones para luego interpretarlo y enviarlos por una red.
- **Lógica.** La continuidad de las operaciones que los programadores codifican en el backend, esta se ejecuta únicamente en el servidor.
- **Marcos.** Guías usadas durante la estructura del código y todos los elementos relacionados con la arquitectura; trabajan a manera de plantillas para el backend.

- **API.** Fundamentales en el desarrollo del software, lo que permite que las distintas aplicaciones mantengan comunicación con otros servidores y base datos para el intercambio de información.

Framework NestJS

NestJS es un framework diseñado para desarrollar aplicaciones del lado del servidor en Node.js de manera eficiente y con capacidad de crecimiento. Está basado en TypeScript y ofrece compatibilidad total con este lenguaje, aunque también permite trabajar únicamente con JavaScript. Además, integra distintos paradigmas de programación, como la programación orientada a objetos, la funcional y la reactiva funcional.

En su funcionamiento interno, NestJS utiliza frameworks HTTP sólidos como Express, que viene configurado por defecto, aunque también puede emplearse Fastify como alternativa para mejorar el rendimiento.

Una de sus ventajas es que proporciona una capa de abstracción más avanzada que otros frameworks populares de Node.js, como Express o Fastify. Aun así, permite al desarrollador acceder directamente a sus API, ofreciendo mayor flexibilidad y la posibilidad de aprovechar numerosos módulos y bibliotecas de terceros disponibles en el ecosistema de Node.js.

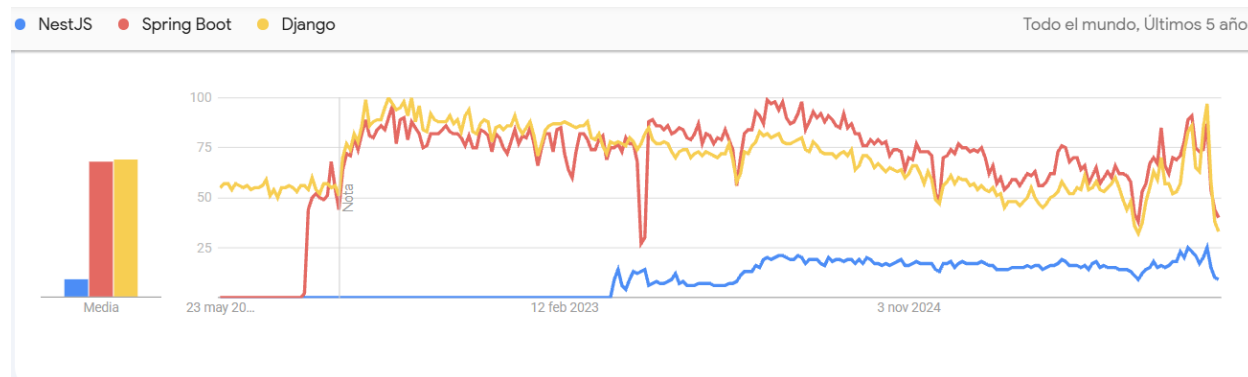
En los últimos años, Node.js ha permitido que JavaScript deje de utilizarse únicamente en el frontend y pase a emplearse también en el desarrollo backend, convirtiéndose en uno de los lenguajes más utilizados en la web. Gracias a esto surgieron tecnologías como Angular, React y Vue.js, las cuales ayudan a los desarrolladores a trabajar de manera más eficiente y facilitan la creación de aplicaciones modernas, rápidas y fáciles de ampliar.

A pesar de la gran cantidad de herramientas y librerías disponibles para Node.js, muchas de ellas no ofrecen una solución clara para estructurar correctamente la arquitectura de una aplicación. Frente a esto, NestJS propone una arquitectura ya organizada que facilita el desarrollo de aplicaciones escalables, fáciles de probar y mantener. Además, busca reducir el acoplamiento entre componentes, lo que mejora la organización del código y el trabajo en equipo. Su diseño y estructura están inspirados en Angular.

De acuerdo con la gráfica comparativa representada en la *Figura 2*, donde NestJS muestra un crecimiento sostenido en los últimos años frente a otros frameworks consolidados como Spring Boot y Django. Aunque su nivel de popularidad de NestJS aún es menor, la tendencia ascendente refleja una mayor adopción por parte de la comunidad de desarrollo, especialmente en proyectos modernos orientados a arquitecturas escalables y servicios en tiempo real. Este comportamiento evidencia que NestJS se ha convertido en una alternativa tecnológica robusta y ampliamente aceptada para el desarrollo de aplicaciones empresariales y móviles.

Figura 2

Comparación de frameworks backend



Nota: Comparación de popularidad de NestJS, Spring Boot y Django. Tomado de (Google Trends, 2026).

Seguridad informática y control de acceso mediante códigos qr

Control de acceso con código QR

El control de acceso se entiende como el conjunto de políticas, mecanismos y procedimientos orientados a regular y gestionar quién puede ingresar, visualizar o utilizar determinados recursos dentro de un entorno específico. En el ámbito de la seguridad física, este concepto se aplica a la autorización o restricción del ingreso de personas y vehículos a instalaciones determinadas, como edificios, puertas, estacionamientos o urbanizaciones. Dicho control se fundamenta en procesos de identificación y autenticación de los usuarios, con el fin de garantizar que solo las personas autorizadas puedan acceder a las áreas protegidas (Pazmiño, 2025)

Actualmente, estos sistemas se han fortalecido mediante el uso de tecnologías avanzadas, entre las que destacan los sistemas biométricos basados en huella digital o reconocimiento facial, así como soluciones de identificación por radiofrecuencia mediante tarjetas de proximidad, además de métodos tradicionales como el uso de claves o contraseñas, lo que permite incrementar la seguridad y eficiencia en la gestión del acceso.

Funcionamiento de los códigos QR

Los códigos QR se utilizan en diversos contextos para facilitar el acceso a información y servicios sin necesidad de contacto físico ni interacción directa entre personas. Por ejemplo, se emplean para compartir menús digitales en restaurantes, gestionar suscripciones a listas de correo, presentar información de productos o servicios como la venta de vehículos y viviendas, así como para registrar entradas y salidas en citas médicas o profesionales.

De manera similar a los códigos de barras tradicionales que se encuentran en los productos y que son leídos por escáneres en las cajas registradoras para identificar las compras, los códigos QR funcionan como una evolución de esta tecnología, permitiendo almacenar y acceder a una mayor cantidad de información de forma rápida y eficiente (Ruoti, 2022).

Con el desarrollo de la aplicación móvil propuesta, el control de acceso será gestionado mediante códigos QR, con el objetivo de modernizar y fortalecer el sistema de seguridad actual. Este mecanismo permitirá que cada residente disponga de un código QR único y dinámico, el cual será generado desde la aplicación y utilizado como credencial de ingreso para visitantes autorizados.

La aplicación contribuye directamente al fortalecimiento de la seguridad de los residentes, ya que permite un registro más preciso y trazable de cada acceso, incluyendo hora, fecha e identidad del usuario. Además, dificulta el ingreso de personas no autorizadas, debido a que los códigos QR pueden ser personalizados, temporales o desactivados en caso de pérdida o emergencia. En consecuencia, la urbanización Villa España I, etapa Mallorca contará con un sistema de control de acceso más eficiente, confiable y adaptado a las necesidades actuales de seguridad digital.

Propuesta científica a contestarse

¿En qué medida el desarrollo de un prototipo de aplicación móvil para el control de pagos de alícuotas, accesos con códigos QR y la reserva de áreas sociales, mejorará la gestión administrativa, agilizará los ingresos seguros de visitantes y optimizará la gestión de reservas de las áreas sociales en la Urbanización Villa España I, etapa Mallorca?

Definiciones Conceptuales

Automatización de Procesos: En el contexto del proyecto se busca eliminar las tareas manuales y repetitivas con el objetivo de mejorar la gestión. Esto se logra con la implementación de software y tecnologías donde se busca la mínima intervención humana. Permite reducir errores, acortar plazos y liberar tiempo de los equipos para tareas de mayor valor (Ayerdi, 2026).

API REST: Es una interfaz de programación de aplicaciones (API) que se ajusta a los principios de diseño del estilo arquitectónico de transferencia de estado representacional (REST), un estilo utilizado para conectar sistemas de hipermedia distribuidos (IBM, 2025).

Backend: Es la parte invisible pero esencial de un sitio, encargada de manejar la lógica y el procesamiento de datos necesarios para que todo funcione de manera correcta y segura. El backend se ocupa de tareas como almacenar y recuperar datos de una base de datos, procesar formularios, autenticar usuarios y gestionar la seguridad del sitio (Chapaval, 2018).

Figma: Es un interesante e innovador editor de gráficos vectoriales diseñado para páginas web y cuyo uso ha ido creciendo sin parar durante los últimos años. A día de hoy, es una de las plataformas más utilizadas por diseñadores UX/UI, agencias de comunicación y empresas de todo el mundo (Blandino, 2023).

Framework: Se refiere a un conjunto de herramientas y buenas prácticas utilizadas para crear aplicaciones de software. Proporciona código y componentes prescritos que agilizan el proceso de desarrollo, de modo que los desarrolladores no tengan que empezar desde cero cada vez que crean algo (Simonova, 2025).

Frontend: El frontend se refiere a la parte visible de un sitio web o aplicación con la que los usuarios pueden interactuar directamente. Es la cara del sitio, ubicada en el lado del cliente. En el contexto de diseño web y desarrollo web, se refiere a todas las tecnologías que corren en el

navegador y que se encargan de la interactividad con los usuarios. También se encarga de hacer que todo funcione correctamente y de responder a tus acciones, como hacer clic en un botón o desplazarte por la página (Chapaval, 2018)

GitHub: Es una plataforma de desarrollo colaborativo que aloja proyectos en la nube utilizando el sistema de control de versiones llamado Git. Ayuda a los desarrolladores a almacenar y administrar el código llevando un registro de cambios (Saavedra, 2023).

Postman: Es una herramienta que sirve de gran ayuda al equipo de desarrollo, permitiendo mantener las colecciones actualizadas, ahorrando los tiempos de respuesta al momento de realizar los test o las llamadas a los servicios. Es potente gracias a su fácil integración con APIs de terceros y cuenta con una gran comunidad (Muradas, 2019).

PostgreSQL: Es un potente sistema de bases de datos objeto-relacionales de código abierto que utiliza y amplía el lenguaje SQL, combinado con numerosas funcionalidades que permiten almacenar y escalar de forma segura las cargas de trabajo de datos más complejas. Los orígenes de PostgreSQL se remontan a 1986, como parte del proyecto POSTGRES de la Universidad de California en Berkeley, y cuenta con casi 40 años de desarrollo activo en su plataforma principal (PostgreSQL, 2026).

Prototipo: Es un modelo inicial creado para probar el diseño y su viabilidad antes del desarrollo completo. Un prototipo ayuda a visualizar ideas, recopilar comentarios de los usuarios y reducir riesgos. (Galen, 2025).

Railway: Es una plataforma de infraestructura que permite a los desarrolladores implementar aplicaciones de forma rápida y sencilla. Con railway, puedes gestionar tus servicios, bases de datos y almacenamiento de manera eficiente, todo desde una interfaz intuitiva y fácil de usar (Angéloz, 2024).

Scrum: Es un marco utilizado por los equipos para administrar el trabajo y resolver problemas de forma colaborativa en ciclos cortos. Scrum implementa los principios de Agile como un conjunto concreto de artefactos, prácticas y roles (Microsoft Build, 2025).

Trazabilidad de Información: Mide la capacidad de un conjunto de datos para ser rastreado, auditado y actualizado en el tiempo, garantizando la transparencia, la confianza institucional y la posibilidad de reutilización. (Ministerio TIC de Colombia, 2025).

CAPÍTULO III

PROPUESTA TECNOLÓGICA

Como se ha destacado en los capítulos anteriores, el presente trabajo de titulación propone el desarrollo de un prototipo de aplicación móvil para el control de pagos de alcuotas, acceso con QR y reservas de áreas sociales en una urbanización privada, con el objetivo de agilizar y centralizar los procesos, mejorar la trazabilidad de la información, mitigar los errores de registros, mejorar la experiencia de usuario y disminuir la carga operativa.

Esta solución se apoya en el uso de tecnologías móviles modernas y una arquitectura de software de tres capas, la cual permite separar de manera estructurada la lógica de presentación, la lógica de negocio y la gestión de datos. Este enfoque arquitectónico favorece la escalabilidad, mantenibilidad y reutilización de los componentes del sistema, facilitando futuras mejoras y adaptaciones de acuerdo con las necesidades de la urbanización. Además, la implementación de una arquitectura desacoplada contribuye a optimizar la comunicación entre los diferentes módulos funcionales, garantizando un procesamiento eficiente y seguro de la información.

Análisis de factibilidad

La viabilidad del proyecto propuesto se ha evaluado desde cuatro dimensiones fundamentales: operacional, técnica, legal y económica. Este análisis integral permite determinar las posibilidades reales de éxito de la aplicación móvil, considerando aspectos relacionados con la

aceptación de los usuarios, la disponibilidad tecnológica, el cumplimiento normativo y la justificación económica de la inversión.

Factibilidad operacional

La factibilidad operacional del presente proyecto se considera viable debido a que la solución propuesta responde directamente a las necesidades identificadas en la gestión administrativa de la urbanización Villa España I, etapa Mallorca. Actualmente, los procesos relacionados con el registro de pagos de alcuotas, el control de acceso de visitantes, así como la reservación de áreas sociales, se realizan de forma semi manual o mediante herramientas dispersas, lo que genera retrasos, duplicidad de información, dificultades de seguimiento y una mayor carga operativa para la administración.

La implementación del prototipo de aplicación móvil permitirá centralizar estos procesos en una única plataforma digital, facilitando el acceso a la información en tiempo real y mejorando la interacción entre los residentes y la administración. Los usuarios podrán realizar consultas sobre sus pagos, visualizar el estado de sus obligaciones, gestionar reservas de áreas sociales y generar códigos QR para el control de acceso, mediante una interfaz intuitiva y de fácil utilización. De igual manera, los administradores dispondrán de herramientas que les permitirán registrar información, realizar seguimientos y generar reportes de manera más eficiente.

Desde el punto de vista organizacional, la adopción de la solución no requiere modificaciones significativas en los procesos existentes, ya que el sistema ha sido diseñado considerando los procedimientos actuales de la urbanización. Además, la mayoría de los usuarios potenciales dispone de dispositivos móviles inteligentes con acceso a internet, lo que favorece la implementación y utilización de la aplicación sin necesidad de inversiones adicionales en equipamiento especializado. Por otra parte, la propuesta incorpora mecanismos de autenticación,

control de acceso y gestión de información que contribuyen a garantizar la seguridad, integridad y trazabilidad de los datos. En consecuencia, se concluye que el proyecto presenta una alta factibilidad operacional, dado que existe aceptación por parte de los usuarios involucrados, disponibilidad de los recursos necesarios para su utilización y beneficios evidentes en términos de eficiencia, control y calidad de los servicios ofrecidos por la urbanización.

Factibilidad técnica

Como se observa en la *Tabla 3*, se detallan los recursos tecnológicos requeridos para el desarrollo e implementación del prototipo, incluyendo los componentes de hardware y software necesarios para la construcción, ejecución y validación de la solución propuesta. Entre estos recursos se contemplan equipos de cómputo para las actividades de desarrollo, herramientas de software para la programación y pruebas, así como dispositivos móviles destinados a la ejecución de pruebas funcionales y a la validación del mecanismo de acceso mediante códigos QR. La disponibilidad de estos recursos tecnológicos permite garantizar que las actividades de desarrollo, integración, pruebas y validación se realicen en un entorno controlado y acorde con los requerimientos del proyecto.

Tabla 3

Componentes de hardware utilizados para desarrollo

Componentes	Especificación	Uso dentro del proyecto
Laptops de desarrollo	HP Pavilion 15-cs3; Windows 11 Home; Intel Core i7-1065G7 1.3 GHz; 12 GB RAM/ Asus Vivobook 15; Windows 11 Home; Intel Core i5-12Gen 1.3GHz; 16GB RAM	Desarrollo del frontend, backend, base de datos, pruebas locales y gestión de repositorio de versiones.
Dispositivos móviles	Honor Magic 5 Lite (Android 13)	Pruebas funcionales del prototipo en dispositivos móviles con sistema Android. Escaneo y validación del código QR para accesos.

Nota: En esta tabla se detallan los recursos de hardware empleados en el desarrollo del prototipo. **Fuente:** Elaboración propia.

En la *Tabla 4* se presentan las herramientas de software seleccionadas para el desarrollo del prototipo de aplicación móvil. Estas herramientas comprenden las tecnologías empleadas para la construcción de la capa de presentación, frontend, la implementación de la lógica de negocio y servicios del sistema, backend, así como la gestión y almacenamiento de datos mediante el sistema de base de datos.

Tabla 4

Componentes de software utilizados para desarrollo

Componente	Tecnología	Versión	Uso dentro del proyecto
Backend	NestJS	11.0.21	Desarrollo de la API REST, implementación de negocio, autenticación y control de acceso.
	Node.js	20.18.0	Entorno de ejecución para los servicios del backend.
Seguridad	TypeScript		Lenguaje de programación para el desarrollo del Backend.
	JSON Web Token (JWT)	-	Gestión de autenticación y autorización basada en tokens.
	NestJS Guards	-	Control de acceso y validación de permisos según los roles de usuario.
Frontend	Flutter	3.32.4	Desarrollo de la aplicación móvil.
	Dart	3.8.1	Lenguaje de programación utilizado para la implementación de la aplicación móvil.
Base de datos	PostgreSQL	17.10	Almacenamiento y gestión de la información del sistema.
Notificaciones	Firebase Cloud Messaging	-	Envío de notificación en tiempo real.
ORM	Prisma ORM		Modelado de entidades, acceso a datos y gestión de migraciones de la base de datos
Infraestructura Cloud	Railway	-	Despliegue y alojamiento del backend y la base de datos en la nube.
IDE	VS Code	1.122.1	Entorno de desarrollo para la implementación y depuración del sistema.

Repositorio de código	GitHub	-	Almacenamiento remoto del código fuente y trazabilidad del desarrollo.
Control de versiones	Git	2.49.0	Gestión y seguimiento de cambios en el código fuente.
Pruebas de API	Postman		Validación y pruebas de los servicios REST del backend.

Nota: En esta tabla se detallan las herramientas de software empleadas en el desarrollo del prototipo. **Fuente:** Elaboración propia.

En consecuencia, se determina que el proyecto presenta factibilidad técnica, ya que se dispone de la infraestructura, herramientas y recursos tecnológicos necesarios para desarrollar, probar y validar el prototipo de aplicación móvil. Esto asegura la viabilidad de la propuesta y respalda su correcta implementación dentro del contexto operativo de la urbanización privada.

Factibilidad legal

El análisis de factibilidad legal verifica que la aplicación móvil propuesta cumple con las leyes vigentes y normativas aplicables, particularmente en materia de protección de datos personales, propiedad intelectual y regulaciones del sector.

El proyecto debe cumplir con la Ley Orgánica de Protección de Datos Personales del Ecuador, publicada en 2021), que regula el tratamiento de datos personales y garantiza los derechos de los titulares. En este contexto, el sistema recuperará únicamente datos personales necesarios para su funcionamiento: nombres, números de identificación, teléfono, correo electrónico, información de pagos y registros de accesos.

Se implementarán medidas de seguridad apropiadas para proteger la información personal contra accesos no autorizados, pérdida o alteración. Los residentes serán informados sobre el tipo de datos recopilados, la finalidad de su tratamiento y sus derechos de acceso, rectificación y eliminación de datos. Por otra parte, la administración de la urbanización actuará como responsable del tratamiento de datos, asumiendo las obligaciones legales correspondiente.

El sistema propuesto se ajusta al reglamento interno de la urbanización Villa España I, etapa Mallorca, en cuanto a los procedimientos control de pago de alcúotas, autorización de visitantes y reserva de áreas sociales. No se proponen cambios que vulneren las normas establecidas en los reglamentos de la urbanización o en el régimen de propiedad horizontal.

Actualmente, no existen prohibiciones legales para el desarrollo e implementación de aplicaciones móviles destinadas a la gestión administrativa de urbanizaciones privadas en Ecuador tanto en el reglamento como en la ley de propiedad horizontal, publicada en el 2005. El proyecto no incurre en actividades reguladas que requieran permisos especiales o autorizaciones gubernamentales.

Factibilidad económica

El proyecto presenta factibilidad económica, debido a que se fundamenta en el uso de herramientas tecnológicas de código abierto y software libre como se observa en la *Tabla 5*, lo cual reduce significativamente los costos asociados a licencias de uso y adquisición de software propietario. En este sentido, la inversión requerida se centra principalmente en los recursos humanos involucrados en el análisis, diseño e implementación del sistema, así como en los equipos tecnológicos necesarios para el desarrollo, pruebas y validación del prototipo de aplicación móvil. En consecuencia, se evidencia que la propuesta es económicamente viable dentro del contexto del proyecto.

Tabla 5

Costos de inversión en software

Tecnología	Tipo de licencia	Costo estimado
Node.js 20.18.0	Open Source	\$0,00
NestJS 11.0.21	Open Source	\$0,00
TypeScript	Open Source	\$0,00
Flutter 3.32.4	Open Source	\$0,00

Dart 3.8.1	Open Source	\$0,00
PostgreSQL 17.10	Open Source	\$0,00
Prisma	Open Source	\$0,00
Git 2.49.0	Open Source	\$0,00
GitHub	Freemium	\$0,00
VS Code 1.122.1	Freeware	\$0,00
Postman	Freemium	\$0,00
Firebase Cloud Messagin	Freemium	\$0,00
Railway	Freemium	\$0,00
Railway PostgreSQL	Freemium	\$0,00
Total		\$0,00

Nota: En esta tabla se presenta la inversión a realizar en software que se ha considerado en el presente proyecto.

Fuente: Elaboración propia.

En la *Tabla 6* se detallan los costos asociados al recurso humano requerido para el desarrollo del proyecto, considerando su participación en las distintas fases del ciclo de vida del software, tales como análisis, diseño, implementación, pruebas y documentación. Estos costos constituyen uno de los principales componentes de la inversión del proyecto, dado que reflejan el esfuerzo técnico y profesional necesario para la correcta ejecución de la solución propuesta.

Tabla 6

Costos referenciales por recursos humanos

Cargo	Costo estimado	Cantidad	Total
Desarrollador	\$1.200,00	2	\$2.400,00
Total			\$2.400,00

Nota: En esta tabla se presentan los principales recursos humanos que se han considerado en el presente proyecto.

Fuente: Elaboración propia.

En la *Tabla 7* se detallan los costos de inversión en hardware necesarios para el desarrollo y validación del prototipo. Se incluyen los equipos de desarrollo utilizados en las fases de implementación del sistema, los dispositivos móviles destinados a la ejecución de pruebas funcionales y validación de la aplicación, así como aquellos empleados para el escaneo y verificación de códigos QR en el proceso de control de acceso.

Tabla 7*Costos de inversión en hardware*

Componente	Costo	Cantidad	Total
Laptop HP Pavilion 15-cs3	\$900,00	1	\$900,00
Laptop Asis Vivobook 15	\$700,00	1	\$700,00
Honor Magic 5 lite	\$300,00	1	\$300,00
iPhone 11	\$300,00	1	\$300,00
Total			\$2.200,00

Nota: En esta tabla se presenta la inversión a realizar en hardware que se ha considerado en el presente proyecto.

Fuente: Elaboración propia.

En la *Tabla 8* se detallan los gastos fijos asociados al consumo de servicios básicos necesarios para el desarrollo del proyecto, específicamente electricidad e internet. Estos recursos son indispensables para la ejecución continua de las actividades de análisis, diseño, desarrollo, pruebas y validación del prototipo de aplicación móvil.

Tabla 8*Costos fijos involucrados en el proyecto*

Detalle	Costo mensual	Cantidad	Total
Electricidad	\$20,00	3	\$60,00
Internet	\$32,00	3	\$96,00
Total			\$156,00

Nota: En esta tabla se presenta la inversión de los costos fijos que se ha considerado en el presente proyecto.

Fuente: Elaboración propia.

En la *Tabla 9*, se detallan los costos totales asociados al desarrollo del proyecto, considerando todos los recursos necesarios para su ejecución. Estos incluyen gastos de recurso humano, inversión en hardware, consumo de servicios básicos como electricidad e internet, así como el uso de herramientas tecnológicas y servicios de infraestructura requeridos para el desarrollo, pruebas y despliegue del prototipo de aplicación móvil.

Tabla 9*Resumen de costos referenciales de inversión*

Detalle	Total
Recurso humano	\$2800,00
Tecnologías por utilizarse	\$0,00
Hardware	\$2000,00
Costos Fijos	\$156,00
Total	\$4.756,00

Nota: En esta tabla se presenta el resumen de los costos referenciales de la inversión total del proyecto.

Fuente: Elaboración propia.

Esta inversión garantiza la disponibilidad de la infraestructura física requerida para la correcta ejecución, evaluación y pruebas del sistema, asegurando condiciones técnicas adecuadas para la implementación del proyecto.

Metodologías del proyecto

El desarrollo de una solución tecnológica orientada a optimizar los procesos administrativos y operativos de una urbanización no solo requiere una implementación técnica adecuada, sino también el respaldo de una base metodológica sólida que garantice la coherencia, viabilidad y pertinencia del proyecto. En este contexto, es indispensable definir con claridad tanto el enfoque de investigación, que permite comprender y analizar la problemática existente, como la metodología de desarrollo de software, que orienta la construcción estructurada de la solución propuesta.

La integración de una metodología de investigación formal con un enfoque ágil de desarrollo permite abordar el proyecto de manera integral, asegurando que las decisiones de diseño e implementación se fundamenten en información real obtenida de los usuarios y del contexto operativo. Así mismo, este enfoque facilita la evolución incremental del sistema, permitiendo su

adaptación continua a los requerimientos identificados durante el ciclo de desarrollo. A continuación, se describen las metodologías seleccionadas que guiarán el cumplimiento de los objetivos planteados en el presente proyecto.

Metodología de investigación

El presente proyecto adopta un enfoque aplicado, debido a que busca resolver una problemática específica mediante el desarrollo de un prototipo de aplicación móvil para la gestión de pagos de alcuotas, control de acceso con códigos QR y reserva de áreas sociales en la urbanización Villa España I, etapa Mallorca.

Para la obtención de información se emplea una investigación de campo, utilizando técnicas como la observación directa y las entrevistas a los actores involucrados en los procesos administrativos y operativos de la urbanización, que este caso son el personal administrativo y de seguridad. Estas técnicas permiten identificar las necesidades de los residentes, comprender los procesos actuales y levantar los requerimientos funcionales y no funcionales que servirán como base para el diseño y desarrollo de la solución tecnológica propuesta.

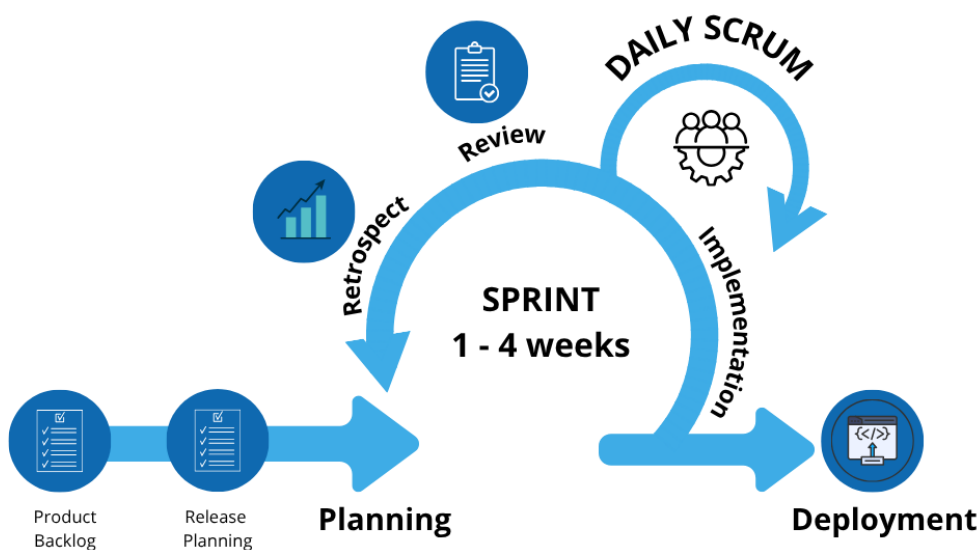
Metodología de desarrollo del proyecto

En el desarrollo del presente proyecto se adoptó la metodología ágil Scrum que es un marco de trabajo ligero que ayuda a personas, equipos y organizaciones a generar valor mediante soluciones adaptativas para problemas complejos (Schwaber & Sutherland, 2020), debido a su enfoque iterativo e incremental que permite una gestión flexible y eficiente del proceso de construcción del software. Esta metodología se caracteriza por organizar el desarrollo del proyecto en ciclos de trabajo cortos, conocidos como sprints, así como se muestra en la *Figura 3*, que

generalmente tienen una duración de una a cuatro semanas (Martin, 2026). Este enfoque facilita la construcción incremental del producto, permitiendo incorporar mejoras de manera continua y asegurar una mayor adaptación a los requerimientos del proyecto. De la misma manera, fomenta la colaboración entre los integrantes del equipo y la optimización constante de los procesos de desarrollo.

Figura 3

Esquema de Proceso Scrum



Autor: Belky Piedra J.

Nota: Esquema general de la metodología Scrum utilizada para el desarrollo del prototipo. Elaboración propia a partir de los lineamientos descritos en la Guía Scrum (Schwaber & Sutherland, 2020).

Roles Scrum

Con el propósito de organizar de manera eficiente el proceso de desarrollo del prototipo de aplicación móvil, se establecieron los roles que participarían durante las diferentes fases del proyecto. La definición de estos roles permitió asignar responsabilidades claras para la gestión de los requerimientos, la planificación de las iteraciones, el desarrollo de los módulos, la realización

de pruebas y la validación de los resultados obtenidos. Esta distribución de responsabilidades facilitó el seguimiento de las actividades y aseguró el cumplimiento de los objetivos establecidos en el proyecto. A continuación, se describen los roles considerados. la *Tabla 10*.

Tabla 10

Roles definidos para el desarrollo

Rol	Descripción
Product Owner	Jhonas López Panta; Belky Piedra Jiménez
Scrum Master	Jhonas López Panta; Belky Piedra Jiménez
Equipo de desarrollo	Jhonas López Panta; Belky Piedra Jiménez
Jueces expertos	Profesionales externos encargados de validar el prototipo

Nota: En esta tabla se presenta los roles definidos para el desarrollo del proyecto. **Fuente:** Elaboración propia.

Requerimientos del sistema

En este apartado se detallan los requerimientos del sistema para su diseño, desarrollo y validación del prototipo de aplicación móvil para el control de pagos, accesos y reservas de áreas sociales en una urbanización privada. Dichos requerimientos constituyen la base fundamental para el desarrollo de software, ya que permiten identificar, documentar y definir las necesidades que debe satisfacer una aplicación para cumplir con los objetivos planteados. Su correcta especificación garantiza que el producto final responda a las expectativas de los usuarios y a los procesos que se desean optimizar dentro de la urbanización.

Requerimientos funcionales

Los requerimientos funcionales corresponden a las características y funcionalidades que debe ofrecer el sistema para responder a las necesidades de sus usuarios. Estos definen las tareas y procesos que la aplicación debe ejecutar, permitiendo asegurar que la solución propuesta cumpla

con los objetivos planteados y brinde una experiencia adecuada a quienes la utilizan (Jonker & Gomstyn, 2015).

En esta sección se detallan los requisitos funcionales del prototipo de aplicación móvil, los cuales fueron definidos tomando como base los objetivos del proyecto y las necesidades que se identificaron previamente en el levantamiento de la información. Estos requerimientos describen las funcionalidades que la aplicación debe ofrecer, así mismo representan el fundamento del desarrollo del proyecto y cada uno de sus componentes.

Tabla 11

RF01- Registro de usuarios

ID	RF-01
Nombre	Registro de usuarios
Descripción	El sistema deberá permitir el registro de forma autónoma de residentes, administradores y guardias, por parte del administrador del sistema.
Actor(es)	Residente, Administrador
Entradas	Cédula, Nombres, Apellidos, Correo Electrónico, Manzana, Villa, Teléfono, Usuario, Contraseña, Rol de Usuario, Id de Administrador, Id de Guardia.
Proceso / Flujo principal	1. El residente/administrador accede al formulario de registro y completa la información requerida. 2. El sistema valida que el usuario no exista previamente. 3. El sistema registra la cuenta del residente, guardia o administrador. 4. El administrador podrá registrar cuentas de guardias y administradores y su vez generar credenciales de acceso.
Flujo alternativo	1. Si el usuario ya existe, el sistema mostrará un mensaje indicando duplicidad. 2. Si existen campos obligatorios vacíos, el sistema impedirá el registro.
Salidas	Usuario registrado correctamente o mensaje de error.
Condiciones y Reglas de negocio	1. El residente podrá registrarse únicamente como residente. 2. Solo el administrador podrá registrar cuentas de guardias y administradores. 3. El nombre de usuario deberá ser único.
Prioridad	Alta

Criterios de aceptación

1. El sistema deberá registrar correctamente residentes.
2. El administrador podrá crear cuentas de guardias y administradores.
3. El sistema no deberá permitir usuarios duplicados.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 12*RF02- Inicio de sesión*

ID	RF-02
Nombre	Inicio de sesión
Descripción	El sistema deberá permitir el inicio de sesión mediante usuario y contraseña.
Actor(es)	Residente, Administrador, Guardia de Seguridad
Entradas	Usuario, contraseña
Proceso / Flujo principal	1. El usuario accede al formulario de inicio de sesión. 2. Ingresa usuario y contraseña. 3. El sistema valida las credenciales. 4. El sistema identifica el rol del usuario. 5. El sistema redirige al módulo correspondiente.
Flujo alternativo	1. Si las credenciales son incorrectas, el sistema mostrará un mensaje de error. 2. Si el usuario está inactivo, el sistema impedirá el acceso.
Salidas	Acceso exitoso o mensaje de proceso fallido.
Condiciones y Reglas de negocio	1. Solo usuarios registrados podrán acceder al sistema. 2. El sistema deberá diferenciar permisos según el rol del usuario.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá permitir acceso únicamente con credenciales válidas. 2. El sistema deberá mostrar mensaje de error cuando las credenciales sean incorrectas.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 13*RF03- Recuperación de contraseña*

ID	RF-03
Nombre	Recuperación de contraseña
Descripción	El sistema deberá permitir al residente, guardia o administrador recuperar su contraseña mediante el correo electrónico registrado.
Actor(es)	Residente, Guardia de Seguridad, Administrador

Entradas	Cédula de Identidad, Correo electrónico registrado
Proceso / Flujo principal	1. El residente, guardia o administrador selecciona “Recuperar contraseña”. 2. Ingresa cédula y correo registrado. 3. El sistema valida los datos ingresados. 4. El sistema envía una contraseña restablecida al correo electrónico.
Flujo alternativo	Si el correo no se encuentra registrado, el sistema mostrará un mensaje de error.
Salidas	Correo de recuperación enviado o mensaje de error.
Condiciones y Reglas de negocio	1. Solo podrá recuperarse la contraseña usando el correo registrado. 2. El correo debe existir en la base de datos del sistema.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá enviar el mecanismo de recuperación al correo registrado. 2. El sistema deberá impedir recuperación con correos inexistentes.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 14

RF04- Registro de comprobante de pago

ID	RF-04
Nombre	Registro y reenvío de comprobante de pago
Descripción	El sistema deberá permitir al residente cargar los comprobantes de pago de alícuotas.
Actor(es)	Residente
Entradas	Archivo de comprobante (JPG o PDF), mes de pago, año de pago, método de pago, nombre del banco, observación de pago.
Proceso / Flujo principal	1. El residente accede al módulo de pagos. 2. Selecciona la opción “Subir comprobante”. 3. Adjunta el comprobante de pago correspondiente. 4. Ingresa la información del pago. 5. El sistema almacena el comprobante y lo envía a validación administrativa. 6. En caso de observación, el residente podrá cargar un nuevo comprobante corregido.
Flujo alternativo	1. Si el archivo no cumple con el formato permitido, el sistema impedirá el envío. 2. Si no se adjunta comprobante, el sistema mostrará un mensaje indicando que el archivo es obligatorio. 3. Si no se incluye la información del pago, el sistema mostrar un mensaje indicando que debe completar los campos obligatorios.
Salidas	Comprobante registrado correctamente o mensaje de error.

Condiciones y Reglas de negocio	1. El comprobante deberá corresponder al período pendiente de pago. 2. El sistema solo permitirá formatos PDF o JPG. 3. El residente podrá reenviar comprobantes únicamente cuando exista una observación registrada.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá permitir cargar comprobantes válidos. 2. El sistema deberá permitir reenviar comprobantes observados. 3. El sistema deberá impedir formatos no permitidos.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 15

RF05- Consulta de valores pendientes

ID	RF-05
Nombre	Consulta de valores pendientes
Descripción	El sistema deberá mostrar al residente los valores pendientes de pago de alícuotas.
Actor(es)	Residente
Entradas	Selección del módulo de pagos
Proceso / Flujo principal	1. El residente accede al módulo de pagos. 2. El sistema consulta los valores pendientes. 3. El sistema muestra el monto adeudado y estado del pago.
Flujo alternativo	Si no existen pagos pendientes, el sistema mostrará un mensaje informativo.
Salidas	Valores pendientes visualizados.
Condiciones y Reglas de negocio	El sistema deberá mostrar únicamente la información asociada al residente autenticado.
Prioridad	Alta
Criterios de aceptación	El sistema deberá mostrar correctamente el monto pendiente de alícuota del residente.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 16

RF06- Notificación de comprobante

ID	RF-06
Nombre	Notificación de comprobante
Descripción	El sistema deberá notificar automáticamente al administrador cuando un residente registre un comprobante de pago.
Actor(es)	Sistema, Administrador

Entradas	Registro de nuevo comprobante de pago
Proceso / Flujo principal	1. El residente registra un comprobante. 2. El sistema identifica un nuevo comprobante pendiente de validación. 3. El sistema envía una notificación al administrador indicando la solicitud pendiente.
Flujo alternativo	Si la notificación no puede enviarse, el sistema mantendrá el comprobante en estado pendiente de revisión.
Salidas	Notificación enviada al administrador.
Condiciones y Reglas de negocio	1. Toda carga de comprobante deberá generar una notificación automática. 2. Solo administradores autenticados podrán visualizar comprobantes pendientes.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá generar una notificación por cada comprobante registrado. 2. El administrador deberá visualizar el estado pendiente de validación.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 17

RF07- Validación de pagos

ID	RF-07
Nombre	Validación de pagos
Descripción	El administrador podrá aprobar o rechazar los pagos registrados por los residentes.
Actor(es)	Administrador
Entradas	Comprobante de pago, datos del residente, estado de pago
Proceso / Flujo principal	1. El administrador accede al módulo de control de pagos. 2. Selecciona un comprobante registrado. 3. Verifica la información del pago. 4. Aprueba o rechaza el pago según corresponda.
Flujo alternativo	Si el comprobante presenta inconsistencias, el administrador podrá rechazar el pago y registrar observaciones.
Salidas	Pago aprobado o rechazado.
Condiciones y Reglas de negocio	1. Solo administradores podrán validar pagos. 2. Todo pago deberá tener un estado: Pendiente, Aprobado o Rechazado.
Prioridad	Alta
Criterios de aceptación	1. El administrador deberá poder aprobar pagos válidos. 2. El administrador deberá poder rechazar pagos inconsistentes.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 18

RF08- Registro de observaciones

ID	RF-08
Nombre	Registro de observaciones
Descripción	El administrador podrá registrar observaciones cuando un comprobante de pago presente inconsistencias.
Actor(es)	Administrador
Entradas	Comentario u observación del administrador
Proceso / Flujo principal	1. El administrador rechaza un comprobante. 2. Registra una observación indicando el motivo del rechazo. 3. El sistema almacena la observación y notifica al residente.
Flujo alternativo	Si no existe observación registrada, el sistema impedirá completar el rechazo del pago.
Salidas	Observación registrada y enviada al residente.
Condiciones y Reglas de negocio	1. Todo pago rechazado deberá incluir una observación obligatoria. 2. Las observaciones deberán quedar almacenadas en el historial del pago.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá exigir observación para pagos rechazados. 2. El residente deberá visualizar la observación registrada.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 19

RF09- Aprobación de pago

ID	RF-09
Nombre	Pago aprobado o denegado
Descripción	El sistema deberá notificar al residente cuando el pago de alícuota haya sido aprobado o denegado por la administración.
Actor(es)	Sistema, Residente
Entradas	Estado de pago aprobado o denegado.
Proceso / Flujo principal	1. El administrador puede aprobar o denegar un pago. 2. El sistema actualiza el estado del pago. 3. El sistema envía una notificación al residente indicando si el pago fue aprobado o denegado.
Flujo alternativo	Si la notificación no puede enviarse, el estado del pago permanecerá según lo que haya seleccionado el administrador y visible en el historial.
Salidas	Notificación de pago aprobado o denegado.

Condiciones y Reglas de negocio	1. Los pagos aprobados o denegados podrán generar confirmación. 2. El residente deberá visualizar el nuevo estado en su historial de pagos.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá notificar pagos aprobados o denegados. 2. El residente deberá visualizar el estado actualizado del pago.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 20

RF10- Historial de pagos

ID	RF-10
Nombre	Historial de pagos
Descripción	El residente podrá consultar el historial de pagos realizados y su estado de validación.
Actor(es)	Residente
Entradas	Acceso al módulo de historial de pagos
Proceso / Flujo principal	1. El residente accede al historial de pagos. 2. El sistema consulta los pagos registrados. 3. El sistema muestra fecha, valor y estado de cada pago realizado.
Flujo alternativo	Si no existen pagos registrados, el sistema mostrará un mensaje informativo.
Salidas	Historial de pagos visualizado.
Condiciones y Reglas de negocio	1. El residente solo podrá visualizar sus propios pagos. 2. El historial deberá incluir pagos aprobados, denegados y pendientes.
Prioridad	Media
Criterios de aceptación	1. El sistema deberá mostrar el historial correctamente. 2. El residente deberá visualizar estados y observaciones asociadas.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 21

RF11- Registro de visitante

ID	RF-11
Nombre	Registro de visitante
Descripción	El sistema deberá permitir al residente registrar la información de un visitante para autorizar su ingreso a la urbanización.
Actor(es)	Residente

Entradas	Datos del visitantes: nombre, cédula, fecha de ingreso, hora estimada de ingreso, observación opcional.
Proceso / Flujo principal	1. El residente accede al módulo de control de acceso. 2. Selecciona la opción “Registrar visitante”. 3. Ingresa la información solicitada del visitante. 4. El sistema valida que los campos obligatorios estén completos. 5. El sistema almacena la información del visitante. 6. El residente podrá guardar el visitantes como frecuente para una futura generación de código QR
Flujo alternativo	1. Si existen campos obligatorios vacíos, el sistema impedirá el registro. 2. Si la fecha de visita ingresada es inválida, el sistema mostrará un mensaje de error.
Salidas	Visitante registrado correctamente.
Condiciones y Reglas de negocio	1. Solo residentes autenticados podrán registrar visitantes. 2. El registro del visitante deberá estar asociado al residente autenticado. 3. La información del visitante deberá quedar almacenada para procesos posteriores de validación.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá permitir registrar visitantes correctamente. 2. El sistema no deberá permitir registros incompletos. 3. El residente deberá visualizar el visitante registrado.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 22

RF12- Generación de código QR

ID	RF-12
Nombre	Generación de código QR
Descripción	El sistema deberá generar automáticamente un código QR único y temporal asociado al visitante registrado para validar su ingreso a la urbanización.
Actor(es)	Sistema, Residente
Entradas	Información del visitante registrada
Proceso / Flujo principal	1. El residente finaliza el registro del visitante. 2. El sistema recupera la información del visitante registrado. 3. El sistema genera automáticamente un código QR único. 4. El sistema asocia el QR al visitante y residente correspondiente. 5. El sistema muestra el código QR generado en pantalla.
Flujo alternativo	Si ocurre un error durante la generación del QR, el sistema mostrará un mensaje de error y permitirá reintentar el proceso con un número de tres intentos.
Salidas	Código QR generado correctamente.

Condiciones y Reglas de negocio	1. Cada visitante deberá tener un código QR único. 2. El QR tendrá validez únicamente para la fecha y hora configuradas en el registro. 3. El QR quedará vinculado al residente y visitante registrados.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá generar automáticamente un QR después del registro del visitante. 2. El residente deberá visualizar el QR generado. 3. El sistema no deberá generar códigos duplicados.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 23

RF13- Escaneo de código QR

ID	RF-13
Nombre	Escaneo de código QR
Descripción	El sistema deberá permitir al guardia escanear códigos QR desde la aplicación móvil para validar el ingreso de visitantes.
Actor(es)	Guardia de Seguridad
Entradas	Código QR presentado por el visitante
Proceso / Flujo principal	1. El guardia accede al módulo de control de acceso. 2. El visitante presenta el código QR. 3. El sistema activa la cámara del dispositivo. 4. El sistema escanea y procesa el código QR.
Flujo alternativo	1. Si el QR no puede leerse, el sistema solicitará un nuevo escaneo, tiene tres intentos. 2. Si el QR está dañado o ilegible, el sistema mostrará un mensaje de error.
Salidas	Resultado del escaneo QR.
Condiciones y Reglas de negocio	1. Solo guardias autenticados podrán acceder al módulo de control de acceso. 2. El sistema deberá utilizar la cámara del dispositivo móvil para el escaneo.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá permitir escanear códigos QR correctamente. 2. El sistema deberá mostrar el resultado del escaneo en tiempo real.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 24

RF14- Validación de código QR

ID	RF-14
Nombre	Validación de código QR
Descripción	El sistema deberá validar automáticamente la autenticidad, vigencia y estado del código QR presentado por el visitante.
Actor(es)	Sistema, Guardia de Seguridad
Entradas	Código QR escaneado
Proceso / Flujo principal	1. El sistema recibe el QR escaneado. 2. Verifica autenticidad y vigencia del QR. 3. Valida que no haya sido utilizado previamente. 4. El sistema determina si el ingreso es autorizado o denegado. 5. Si es el código es válido se podrá visualizar la información del visitante. 6. El guardia registra la placa para terminar con el proceso de validación.
Flujo alternativo	1. Si el QR expiró, el sistema denegará el ingreso. 2. Si el QR ya fue utilizado, el sistema denegará el ingreso.
Salidas	Resultado de validación del QR (válido o inválido).
Condiciones y Reglas de negocio	1. El QR deberá ser único y temporal. 2. El QR no podrá reutilizarse después de un ingreso exitoso. 3. El sistema validará fecha, hora y estado del QR.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá permitir ingreso únicamente con QR válidos. 2. El sistema deberá bloquear QR expirados o reutilizados.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 25

RF15- Registro manual de visitantes

ID	RF-15
Nombre	Registro manual de visitante
Descripción	El sistema permitirá al guardia registrar manualmente el ingreso de un visitante cuando el código QR no pueda ser validado o no esté disponible. El registro quedará asociado al residente que autoriza el ingreso y almacenará la información para futuras consultas.
Actor(es)	Guardia
Entradas	Cédula, nombre y apellido, placa del vehículo, manzana y villa a la que se dirige, fecha y hora de ingreso, observaciones.

Proceso / Flujo principal	1. El guardia selecciona la opción "Registro manual". 2. Ingresa los datos del visitante. 3. Selecciona la manzana y villa que visita 4. Confirma la información. 5. El sistema almacena el registro y genera la evidencia del ingreso.
Flujo alternativo	Si el QR es inválido o expiró, el sistema no registrará el ingreso y mostrará un mensaje de denegación.
Salidas	Registro exitoso del visitante con fecha, hora y datos del guardia responsable.
Condiciones y Reglas de negocio	1. Solo los guardias autenticados podrán realizar registros manuales. 2. El número de identificación del visitante es obligatorio. 3. Debe seleccionarse obligatoriamente la manzana y villa que visita asociadas a un residente. 4. El sistema registrará automáticamente la fecha y hora del ingreso. 5. Todo registro manual quedará identificado como "Ingreso Manual". 6. El administrador podrá consultar el historial de registros manuales.
Prioridad	Alta
Criterios de aceptación	1. El sistema permitirá registrar un visitante sin utilizar código QR. 2. No permitirá guardar el registro si existen campos obligatorios vacíos. 3. El registro quedará almacenado correctamente en la base de datos. 4. El historial mostrará que el ingreso fue realizado manualmente. 5. El sistema registrará el usuario (guardia) que realizó el registro.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 26

RF16- Registro de alertas y restricción de intentos

ID	RF-16
Nombre	Registro de alertas y restricción de intentos
Descripción	El sistema deberá registrar intentos fallidos de validación del código QR y restringir el ingreso después de tres intentos fallidos consecutivos.
Actor(es)	Sistema, Guardia de Seguridad
Entradas	Código QR inválido o intento fallido de escaneo
Proceso / Flujo principal	1. El guardia escanea un código QR. 2. El sistema verifica la validez del código. 3. Si el QR es inválido, el sistema registra el intento fallido. 4. El sistema contabiliza los intentos realizados. 5. Al tercer intento fallido, el sistema deniega el ingreso automáticamente y registra la alerta.

Flujo alternativo	1. Si el QR es válido antes del tercer intento, el contador de intentos se restablecerá. 2. Si ocurre un error de lectura, el sistema permitirá un nuevo escaneo sin contabilizarlo como intento fallido.
Salidas	Registro de alerta y acceso denegado tras tres intentos fallidos.
Condiciones y Reglas de negocio	1. El sistema permitirá un máximo de tres intentos de validación fallidos. 2. Al tercer intento fallido se bloqueará el acceso automáticamente. 3. Todos los intentos fallidos deberán almacenarse en el historial del sistema.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá contabilizar intentos fallidos de validación QR. 2. El sistema deberá denegar el ingreso al tercer intento fallido. 3. El sistema deberá registrar la alerta correspondiente.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 27

RF17- Historial de visitas

ID	RF-17
Nombre	Historial de visitas
Descripción	El sistema deberá permitir al guardia consultar el historial de ingresos registrados de visitantes.
Actor(es)	Guardia
Entradas	Selección del módulo de historial de visitas
Proceso / Flujo principal	1. El guardia accede al módulo de historial de visitas. 2. El sistema consulta los registros almacenados. 3. El sistema muestra información del visitante, residente asociado, fecha y hora de ingreso.
Flujo alternativo	Si no existen registros disponibles, el sistema mostrará un mensaje informativo.
Salidas	Historial de visitas visualizado.
Condiciones y Reglas de negocio	1. Solo guardias autenticados podrán acceder al historial. 2. El historial deberá mostrar registros válidos e inválidos de ingreso.
Prioridad	Media
Criterios de aceptación	1. El guardia deberá visualizar el historial de visitas registradas. 2. El sistema deberá mostrar información detallada de cada ingreso.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 28

RF18- Consulta de áreas sociales y disponibilidad

ID	RF-18
Nombre	Consulta de áreas sociales y disponibilidad
Descripción	El sistema deberá mostrar las áreas sociales disponibles junto con el calendario de fechas y horarios disponibles para reserva.
Actor(es)	Residente
Entradas	Selección del módulo de áreas sociales
Proceso / Flujo principal	1. El residente accede al módulo de reservas. 2. El sistema consulta las áreas disponibles. 3. El sistema muestra las áreas sociales registradas. 4. El residente selecciona un área. 5. El sistema muestra el calendario con horarios disponibles.
Flujo alternativo	Si no existen horarios disponibles, el sistema mostrará un mensaje indicando fecha y horario no disponible.
Salidas	Áreas sociales y disponibilidad visualizadas.
Condiciones y Reglas de negocio	1. El sistema deberá mostrar únicamente áreas habilitadas para reserva. 2. El calendario deberá actualizarse automáticamente según reservas existentes.
Prioridad	Alta
Criterios de aceptación	1. El residente deberá visualizar las áreas sociales disponibles. 2. El sistema deberá mostrar horarios actualizados en tiempo real.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 29

RF19- Validación de disponibilidad

ID	RF-19
Nombre	Validación de disponibilidad
Descripción	El sistema deberá validar automáticamente si la fecha y el horario seleccionado por el residente se encuentra disponible para reserva.
Actor(es)	Sistema, Residente
Entradas	Área social seleccionada, fecha y horario solicitado
Proceso / Flujo principal	1. El residente selecciona un área, fecha y horario. 2. El sistema consulta las reservas existentes. 3. El sistema verifica disponibilidad del horario. 4. El sistema muestra los horarios disponibles.
Flujo alternativo	Si el horario ya está reservado, el sistema mostrara horarios disponibles.
Salidas	Resultado de disponibilidad del horario.

Condiciones y Reglas de negocio	1. No podrá existir doble reserva para el mismo horario y área social. 2. El sistema deberá validar disponibilidad en tiempo real antes de confirmar la reserva.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá validar correctamente la disponibilidad. 2. El sistema deberá impedir reservas en horarios ocupados.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 30

RF20 - Reserva de área social

ID	RF-20
Nombre	Reserva de área social
Descripción	El sistema deberá permitir al residente realizar reservas de áreas sociales, integrando el subproceso de carga y validación de comprobante de pago proveniente del módulo de pagos.
Actor(es)	Residente
Entradas	Área social seleccionada, fecha, horario, comprobante de pago (JPG o PDF)
Proceso / Flujo principal	1. El residente accede al módulo de áreas sociales. 2. Selecciona el área social deseada. 3. El sistema muestra horarios disponibles. 4. El residente selecciona fecha y horario. 5. El sistema valida disponibilidad. 6. El residente registra la reserva. 7. El sistema habilita el subproceso de carga de comprobante reutilizando el módulo de pagos. 8. El residente adjunta el comprobante. 9. El sistema registra la reserva en estado pendiente de validación administrativa.
Flujo alternativo	1. Si el horario seleccionado ya no se encuentra disponible, el sistema impedirá continuar la reserva. 2. Si no se adjunta comprobante, el sistema no permitirá finalizar la solicitud.
Salidas	Reserva registrada en estado pendiente de validación.
Condiciones y Reglas de negocio	1. Toda reserva deberá estar asociada a un comprobante de pago. 2. El proceso de carga y validación de comprobante deberá reutilizar las reglas definidas en el módulo de pagos. 3. No podrán existir reservas duplicadas para el mismo horario y área social.
Prioridad	Alta

Criterios de aceptación

1. El sistema deberá permitir reservas únicamente en horarios disponibles.
2. El sistema deberá exigir un comprobante válido antes de registrar la reserva.
3. La reserva deberá quedar almacenada en estado pendiente de validación.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 31*RF21- Bloqueo temporal de horario*

ID	RF-21
Nombre	Bloqueo temporal de horario
Descripción	El sistema deberá bloquear temporalmente el horario seleccionado por el residente durante el proceso de reserva para evitar conflictos de coincidencia.
Actor(es)	Sistema, Residente
Entradas	Área social seleccionada, fecha y horario solicitado
Proceso / Flujo principal	1. El residente selecciona un horario disponible. 2. El sistema verifica la disponibilidad en tiempo real. 3. El sistema bloquea temporalmente el horario seleccionado. 4. El residente continúa con el proceso de carga del comprobante y confirmación de reserva.
Flujo alternativo	1. Si otro usuario intenta seleccionar el mismo horario bloqueado, el sistema notificará que no está disponible temporalmente. 2. Si el residente abandona el proceso de reserva, el sistema liberará automáticamente el horario.
Salidas	Horario bloqueado temporalmente para el residente.
Condiciones y Reglas de negocio	1. El bloqueo temporal solo aplicará al horario y área seleccionados. 2. El sistema deberá impedir reservas simultáneas sobre el mismo horario. 3. El bloqueo permanecerá activo hasta completar o cancelar el proceso de reserva.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá bloquear automáticamente el horario seleccionado. 2. El sistema deberá impedir conflictos de reservas simultáneas. 3. El horario deberá liberarse si la reserva no continúa.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 32

RF22- Validación y confirmación de reserva

ID	RF-22
Nombre	Validación y confirmación de reserva
Descripción	El administrador deberá validar el comprobante de pago de la reserva y el sistema notificará al residente el resultado de la validación.
Actor(es)	Administrador, Sistema
Entradas	Comprobante de pago de reserva, datos de la reserva
Proceso / Flujo principal	1. El administrador accede al módulo de reservas. 2. Selecciona la reserva registrada. 3. Verifica el comprobante de pago asociado. 4. El administrador aprueba o rechaza la reserva. 5. El sistema actualiza el estado de la reserva. 6. El sistema notifica al residente el resultado de la validación.
Flujo alternativo	1. Si el comprobante presenta inconsistencias, el administrador rechazará la reserva y registrará observaciones. 2. Si la notificación falla, el estado de la reserva permanecerá actualizado en el historial del residente.
Salidas	Reserva aprobada o rechazada y notificación enviada al residente.
Condiciones y Reglas de negocio	1. Toda reserva deberá pasar por validación administrativa. 2. Solo administradores autenticados podrán validar reservas. 3. El subproceso de validación seguirá las reglas del módulo de control de pagos.
Prioridad	Alta
Criterios de aceptación	1. El administrador deberá poder aprobar o rechazar reservas. 2. El residente deberá recibir notificación del estado de la reserva. 3. La reserva deberá actualizar su estado correctamente.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 33

RF23- Cancelación de reserva

ID	RF-23
Nombre	Cancelación de reserva
Descripción	El sistema deberá permitir al residente cancelar una reserva antes de la confirmación final del administrador.
Actor(es)	Residente
Entradas	Selección de reserva registrada

Proceso / Flujo principal	1. El residente accede al historial de reservas. 2. Selecciona la reserva pendiente de pago. 3. Elige la opción “Cancelar reserva”. 4. El sistema solicita confirmación. 5. El sistema cambia el estado de la reserva a cancelada y libera el horario.
Flujo alternativo	1. Si la reserva ya fue aprobada, el sistema impedirá la cancelación. 2. Si el residente cancela la operación, la reserva mantendrá su estado actual.
Salidas	Reserva cancelada y horario liberado.
Condiciones y Reglas de negocio	1. Solo podrán cancelarse reservas pendientes de validación. 2. La cancelación liberará automáticamente el horario reservado.
Prioridad	Media
Criterios de aceptación	1. El residente deberá poder cancelar reservas pendientes. 2. El sistema deberá liberar automáticamente el horario cancelado.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 34*RF24- Liberación automática de horario*

ID	RF-24
Nombre	Liberación automática de horario
Descripción	El sistema deberá liberar automáticamente el horario reservado si el comprobante de pago no es validado dentro del tiempo establecido.
Actor(es)	Sistema
Entradas	Tiempo límite de validación, estado de reserva pendiente
Proceso / Flujo principal	1. El sistema monitorea reservas pendientes de validación. 2. Verifica el tiempo transcurrido desde el registro de la reserva. 3. Si supera el tiempo límite establecido, el sistema libera automáticamente el horario. 4. El sistema actualiza el estado de la reserva como expirada o cancelada automáticamente.
Flujo alternativo	Si el administrador valida la reserva antes del tiempo límite, el sistema mantendrá el horario reservado.
Salidas	Horario liberado automáticamente y reserva actualizada.
Condiciones y Reglas de negocio	1. El tiempo límite para validación será de 10 minutos. 2. Las reservas no validadas dentro del tiempo establecido serán canceladas automáticamente. 3. El sistema deberá liberar el horario inmediatamente después del vencimiento.
Prioridad	Alta
Criterios de aceptación	1. El sistema deberá liberar horarios vencidos automáticamente.

2. El sistema deberá cancelar reservas no validadas en el tiempo definido.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 35

RF25- Historial de reservas

ID	RF-25
Nombre	Historial de reservas
Descripción	El sistema deberá permitir al residente consultar el historial de reservas realizadas y sus respectivos estados.
Actor(es)	Residente
Entradas	Acceso al módulo de historial de reservas
Proceso / Flujo principal	1. El residente accede al módulo de reservas en la opción de historial. 2. El sistema consulta las reservas registradas del residente. 3. El sistema muestra el listado de reservas realizadas con información de área social, fecha, horario y estado de la reserva.
Flujo alternativo	Si no existen reservas registradas, el sistema mostrará un mensaje informativo indicando que no existen datos disponibles.
Salidas	Historial de reservas visualizado.
Condiciones y Reglas de negocio	1. El residente solo podrá visualizar las reservas asociadas a su cuenta. 2. El historial deberá incluir reservas pendientes, confirmadas y canceladas.
Prioridad	Media
Criterios de aceptación	1. El sistema deberá mostrar correctamente el historial de reservas del residente. 2. El residente deberá visualizar el estado actualizado de cada reserva.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 36

RF26- Reporte de pagos

ID	RF-26
Nombre	Reporte mensual de pagos
Descripción	El sistema deberá mostrar un reporte mensual de residentes que realizaron pagos de alcúotas y residentes con pagos pendientes, incluyendo un cálculo simple de cartera vencida del mes.
Actor(es)	Administrador

Entradas	Selección del mes de consulta
Proceso / Flujo principal	1. El administrador accede al módulo de reportes en la opción de pagos. 2. Selecciona el mes de consulta. 3. El sistema consulta los pagos registrados del período seleccionado. 4. El sistema identifica residentes que pagaron y residentes con pagos pendientes. 5. El sistema muestra el reporte consolidado de residentes que refleje su estado de pago.
Flujo alternativo	Si no existen registros de pagos en el período seleccionado, el sistema mostrará un mensaje informativo.
Salidas	Reporte mensual de pagos generado y visualizado.
Condiciones y Reglas de negocio	1. Solo administradores autenticados podrán acceder al reporte. 2. Si un cliente posee al menos una obligación vencida pendiente de pago, será clasificado como <i>pago vencido</i>. 3. El reporte deberá mostrar a los residentes con estado de <i>al día o vencido</i>.
Prioridad	Media
Criterios de aceptación	1. El sistema deberá mostrar correctamente los residentes que realizaron pagos. 2. El sistema deberá identificar residentes con pagos pendientes. 3. El sistema deberá generar el reporte del estado de pago de los residente.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 37

RF27- Reporte de acceso de visitantes

ID	RF-27
Nombre	Reporte de acceso de visitantes
Descripción	El sistema deberá permitir al administrador visualizar un reporte histórico de ingresos de visitantes registrados en la urbanización.
Actor(es)	Administrador
Entradas	Selección del período de consulta
Proceso / Flujo principal	1. El administrador accede al módulo de reportes en la opción de visitantes. 2. Selecciona un período de consulta. 3. El sistema consulta los registros históricos de accesos. 4. El sistema muestra el listado de visitantes ingresados con residente asociado, fecha y hora de ingreso.
Flujo alternativo	Si no existen registros para el período seleccionado, el sistema mostrará un mensaje informativo.
Salidas	Reporte de accesos de visitantes visualizado.

Condiciones y Reglas de negocio	1. Solo administradores autenticados podrán acceder al reporte. 2. El reporte deberá mostrar únicamente ingresos válidos registrados mediante QR.
Prioridad	Media
Criterios de aceptación	1. El administrador deberá visualizar correctamente el historial de visitantes ingresados. 2. El sistema deberá mostrar residente asociado, fecha y hora del acceso.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 38

RF28- Reporte de reservas de áreas sociales

ID	RF-28
Nombre	Reporte de reservas de áreas sociales
Descripción	El sistema deberá permitir al administrador visualizar un reporte histórico de reservas realizadas sobre las áreas sociales de la urbanización.
Actor(es)	Administrador
Entradas	Selección del período de consulta
Proceso / Flujo principal	1. El administrador accede al módulo de reportes en la opción de áreas sociales. 2. Selecciona el período de consulta. 3. El sistema consulta las reservas registradas. 4. El sistema muestra información del área reservada, residente asociado, fecha, horario y estado de la reserva.
Flujo alternativo	Si no existen reservas registradas para el período seleccionado, el sistema mostrará un mensaje informativo.
Salidas	Reporte de reservas visualizado.
Condiciones y Reglas de negocio	1. Solo administradores autenticados podrán acceder al reporte. 2. El reporte deberá incluir reservas aprobadas, pendientes, rechazadas y canceladas.
Prioridad	Media
Criterios de aceptación	1. El administrador deberá visualizar correctamente el historial de reservas. 2. El sistema deberá mostrar información completa de cada reserva registrada.

Nota: La tabla presenta los requisitos funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Requerimientos no funcionales

Los requerimientos no funcionales definen los estándares y características que debe cumplir un sistema para operar de manera eficiente, enfocándose en la forma en que funciona el sistema más que en las funciones que realiza (Valera, 2026). Para el presente proyecto, orientado al desarrollo de un prototipo de aplicación móvil para el control de pagos de alícuotas, acceso mediante códigos QR y reservas de áreas sociales en una urbanización privada, se han identificado requerimientos relacionados con seguridad, disponibilidad, rendimiento, compatibilidad, usabilidad, integridad de la información, escalabilidad, mantenibilidad y arquitectura tecnológica. Estos requisitos contribuirán a asegurar una solución confiable, eficiente y alineada con las necesidades operativas de los residentes, administradores.

Tabla 39

RNF01- Seguridad

ID	RNF-01
Nombre	Seguridad de acceso e información
Descripción	El sistema deberá garantizar la protección de las credenciales de acceso y de la información almacenada y transmitida entre la aplicación móvil y el servidor.
Categoría	Seguridad
Prioridad	Alta
Criterios de aceptación	1. Las contraseñas no deberán almacenamiento en texto plano. 2. El sistema deberá requerir autenticación para acceder a funcionalidades protegidas. 3. Toda transmisión de información deberá realizarse mediante conexión segura.

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 40*RNF02- Usabilidad*

ID	RNF-02
Nombre	Facilidad de uso
Descripción	La aplicación móvil deberá proporcionar una interfaz intuitiva que permita a residentes, administradores y guardias realizar sus actividades de forma sencilla y eficiente.
Categoría	Usabilidad
Prioridad	Alta
Criterios de aceptación	1. Los usuarios podrán contemplar los procesos de pagos, acceso y reservas sin requerir capacitación técnica especializada.

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 41*RNF03- Disponibilidad*

ID	RNF-03
Nombre	Disponibilidad del sistema
Descripción	El sistema deberá encontrarse disponible para el acceso de los usuarios durante la mayor parte del tiempo operativo.
Categoría	Disponibilidad
Prioridad	Alta
Criterios de aceptación	1. Los servicios de autenticación, pagos, reservas y validación QR deberán encontrarse operativos al menos el 99% del tiempo

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 42*RNF04 - Rendimiento*

ID	RNF-04
Nombre	Tiempo de respuesta
Descripción	El sistema deberá responder oportunamente a las solicitudes realizadas por los usuarios.
Categoría	Rendimiento
Prioridad	Alta

Criterios de aceptación	1. El 95% de las solicitudes deberá cumplir los tiempos de respuesta establecidos.
--------------------------------	--

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 43

RNF05 - Compatibilidad

ID	RNF-05
Nombre	Compatibilidad móvil
Descripción	La aplicación deberá ejecutarse correctamente en dispositivos móviles Android e iOS utilizados por residentes, administradores y guardias.
Categoría	Compatibilidad
Prioridad	Media
Criterios de aceptación	1. Todas las funcionalidades deberán ejecutarse correctamente en los dispositivos compatibles.

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 44

RNF06 – Integridad de datos

ID	RNF-06
Nombre	Integridad de la información
Descripción	El sistema deberá garantizar la consistencia y exactitud de la información almacenada en la base de datos.
Categoría	Integridad de datos
Prioridad	Alta
Criterios de aceptación	1. La base de datos deberá impedir registros duplicados mediante validaciones y restricciones de integridad.

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 45

RNF07 – Escalabilidad

ID	RNF- 07
Nombre	Escalabilidad del sistema

Descripción	El sistema deberá permitir la incorporación futura de nuevas funcionalidades y el incremento de usuarios sin afectar su funcionamiento.
Categoría	Escalabilidad
Prioridad	Media
Criterios de aceptación	1. Nuevas funcionalidades podrán incorporarse sin afectar los módulos existentes.

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 46

RNF08 – Mantenibilidad

ID	RNF- 08
Nombre	Facilidad de mantenimiento
Descripción	El sistema deberá facilitar futuras correcciones, actualizaciones y mejoras mediante una estructura organizada y documentada.
Categoría	Mantenibilidad
Prioridad	Alta
Criterios de aceptación	1. El proyecto deberá disponer de documentación técnica actualizada para facilitar su mantenimiento.

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Tabla 47

RNF09 – Arquitectura

ID	RNF- 09
Nombre	Arquitectura de tres capas
Descripción	El sistema deberá implementarse utilizando una arquitectura de tres capas que permita la separación de responsabilidades entre presentación, lógica de negocio y datos.
Categoría	Arquitectura
Prioridad	Alta
Criterios de aceptación	1. Las tres capas deberán encontrarse desacopladas y comunicarse mediante interfaces definidas. 2. Las reglas de negocio deberán implementarse únicamente en la capa de negocio.

Nota: La tabla presenta los requisitos no funcionales establecidos para el desarrollo del prototipo de aplicación móvil. Elaboración propia.

Criterios de validación

Como se observa en la *Tabla 48*, los criterios de validación permiten verificar que las funcionalidades implementadas cumplen con los requerimientos funcionales y no funcionales definidos para la aplicación móvil. La validación se realizará mediante pruebas funcionales, de rendimiento, seguridad y usabilidad, comprobando que cada módulo opere conforme a las reglas de negocio establecidas.

Tabla 48

Criterios de validación del prototipo móvil

ID	Funcionalidad	Criterio de validación
CV-01	Registro de usuarios	El sistema registra correctamente residentes, administradores y guardias, evitando usuarios duplicados y validando los campos obligatorios.
CV-02	Inicio de sesión	El sistema permite el acceso únicamente con credenciales válidas y redirige al usuario según su rol.
CV-03	Recuperación de contraseña	El sistema envía correctamente el mecanismo de recuperación al correo registrado.
CV-04	Consulta de pagos pendientes	El residente visualiza únicamente sus valores pendientes de pago.
CV-05	Registro de comprobantes	El sistema permite cargar comprobantes en formato PDF o JPG y almacena la información correctamente.
CV-06	Validación de pagos	El administrador puede aprobar o rechazar pagos y registrar observaciones obligatorias cuando corresponda.
CV-07	Historial de pagos	El residente visualiza correctamente el historial y estado de sus pagos.
CV-08	Recordatorios automáticos	El sistema envía notificaciones de vencimiento únicamente a residentes con pagos pendientes.
CV-09	Registro de visitantes	El residente registra visitantes con los datos requeridos y el sistema almacena la información correctamente.
CV-10	Generación de QR	El sistema genera un código QR único para cada visitante registrado.
CV-11	Escaneo de QR	El guardia puede escanear correctamente códigos QR desde la aplicación móvil.

CV-12	Validación de QR	El sistema acepta únicamente códigos QR vigentes y no reutilizados.
CV-13	Registro de ingreso	El sistema registra automáticamente la fecha y hora del ingreso validado.
CV-14	Historial de visitas	El guardia visualiza correctamente el historial de ingresos registrados.
CV-15	Consulta de áreas sociales	El residente visualiza las áreas disponibles y sus horarios actualizados.
CV-16	Validación de disponibilidad	El sistema impide reservas sobre horarios ya ocupados.
CV-17	Reservas de áreas sociales	El sistema registra reservas únicamente cuando existe disponibilidad y comprobante adjunto.
CV-18	Bloqueo temporal de horario	El sistema bloquea temporalmente horarios seleccionados evitando conflictos simultáneos.
CV-19	Validación de reservas	El administrador aprueba o rechaza reservas y el residente recibe la notificación correspondiente.
CV-20	Cancelación de reservas	El residente puede cancelar reservas pendientes y el sistema libera el horario automáticamente.
CV-21	Reportes administrativos	El sistema genera correctamente reportes de pagos, accesos y reservas según el período seleccionado.

Nota: En esta tabla se presenta los criterios de validación del prototipo móvil. **Fuente:** Elaboración propia.

Definición de procesos

Una vez identificados los requerimientos funcionales y no funcionales, es necesario definir los procesos que conforman el funcionamiento de la aplicación móvil. La definición de procesos permite describir de manera estructurada las actividades, flujos de información, actores involucrados y reglas de negocio que intervienen en cada módulo del sistema. En el presente proyecto, los procesos de la urbanización Villa España I, etapa Mallorca se encuentran orientados a la gestión de pagos de alcuotas, control de acceso de visitantes, administración de reservas de áreas sociales. A continuación, se describen los procesos actuales de gestión de pagos de alcuotas, control de acceso de visitantes y reservas de áreas sociales, considerando las actividades realizadas por residentes, administradores y personal de seguridad antes de la implementación de la solución tecnológica propuesta.

Proceso actual del control de pagos de alcúotas

Como se muestra en la *Figura 4*, el proceso de pago de alcúotas inicia cuando el residente identifica la obligación de pago correspondiente a la alcúota mensual de la urbanización. Para cumplir con esta obligación, el residente realiza la transferencia o depósito mediante los canales financieros establecidos por la administración y obtiene el respectivo comprobante de pago como evidencia de la transacción realizada.

Una vez efectuado el pago, el residente envía el comprobante a la administración para su revisión y validación. Al recibir el documento, el personal administrativo procede a verificar la información proporcionada y consulta el sistema financiero de la urbanización con el propósito de confirmar que la transferencia efectivamente haya sido registrada en la cuenta correspondiente.

Durante esta etapa se realiza una validación inicial para determinar si la transferencia existe en el sistema financiero. Si no se encuentra evidencia del pago, la administración notifica al residente que el pago ha sido rechazado debido a la imposibilidad de verificar la transacción, finalizando así el proceso. En caso de que la transferencia sea encontrada, la administración continúa con la validación del comprobante de pago, verificando que los datos registrados coincidan con la transacción realizada.

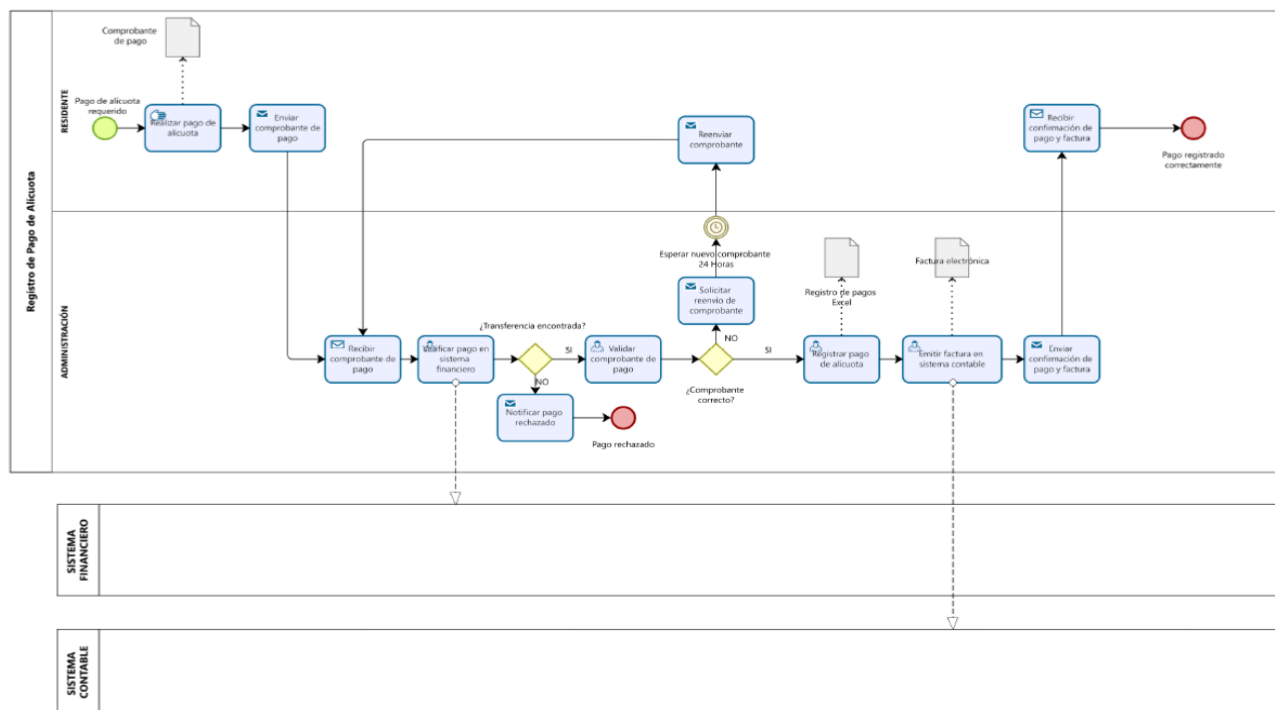
Esta revisión contempla aspectos como el valor pagado, la fecha de la transferencia, la entidad financiera y demás datos relevantes. Si el comprobante presenta inconsistencias, errores o información incorrecta, la administración solicita al residente el envío de un nuevo comprobante corregido. El residente dispone de un plazo de hasta 24 horas para reenviar la documentación requerida, tras lo cual el proceso de validación se repite hasta obtener un comprobante válido.

Cuando el comprobante es considerado correcto y coincide con la información registrada en el sistema financiero, la administración procede a registrar el pago de la alícuota en el control interno de pagos, generalmente mediante una hoja electrónica o registro manual. Posteriormente, la información es ingresada al sistema contable para la emisión de la correspondiente factura electrónica.

Una vez generada la factura, la administración envía al residente una confirmación formal del pago realizado junto con la factura electrónica emitida. Finalmente, el residente recibe la documentación y la confirmación correspondiente, concluyendo el proceso con el pago de la alícuota registrado correctamente en los sistemas administrativos y contables de la urbanización.

Figura 4

Proceso de pagos de alícuotas



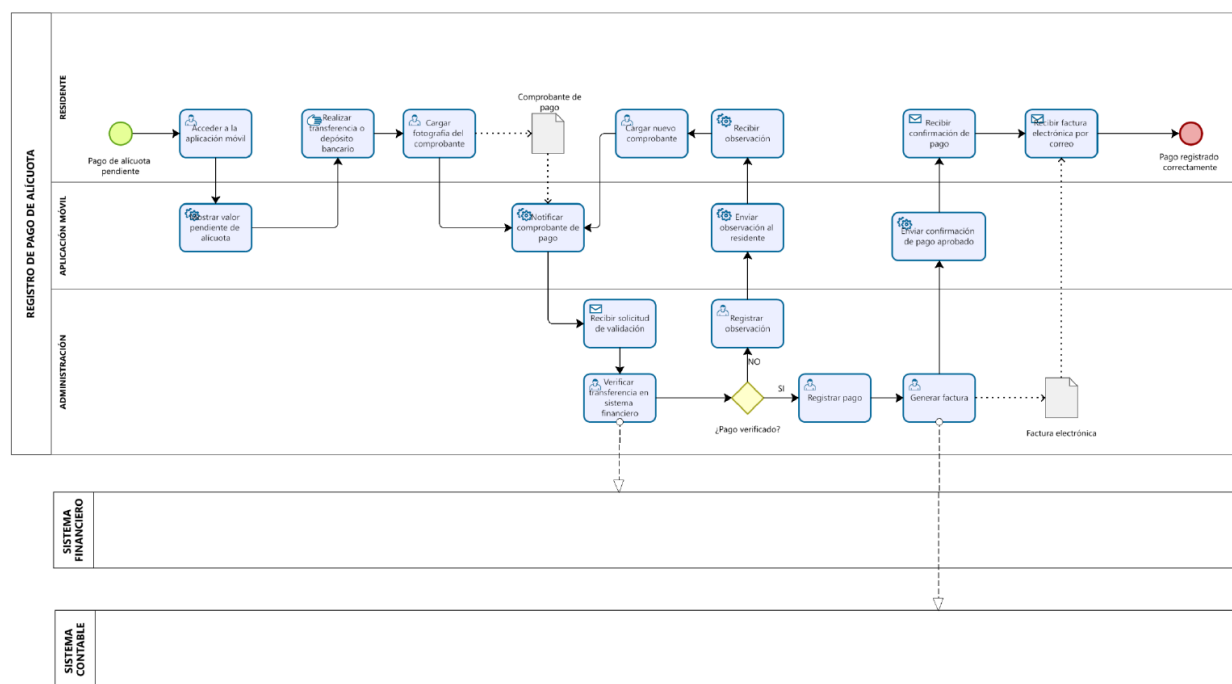
Nota: El diagrama representa como se gestiona el proceso actual del pago de alícuotas. Elaboración propia.

Proceso propuesto para el control de pagos.

El proceso propuesto inicia cuando el residente accede a la aplicación móvil y consulta el valor pendiente de la alícuota, como se observa en la *Figura 5*. Una vez realizado el pago mediante transferencia o depósito bancario, el residente carga un archivo JPG o PDF del comprobante directamente desde la aplicación. El sistema notifica automáticamente a la administración sobre la solicitud de validación, permitiendo verificar la transferencia en el sistema financiero.

Figura 5

Proceso propuesto para el control de pagos



Nota: El diagrama representa el proceso propuesto para el control de pago de alícuotas. Elaboración propia.

Si el pago presenta inconsistencias, la administración registra una observación y el sistema la envía al residente para que pueda cargar un nuevo comprobante corregido. Cuando el pago es validado exitosamente, la administración registra la transacción, genera la factura electrónica y el sistema notifica al residente la aprobación del pago.

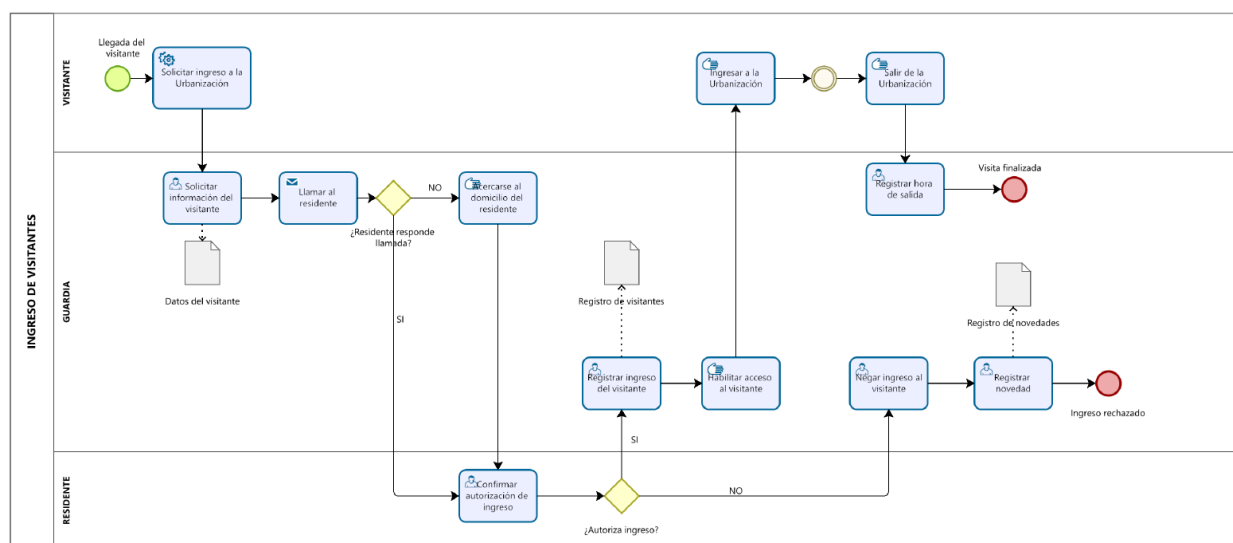
Finalmente, el residente recibe la confirmación y la factura electrónica por correo, quedando el pago registrado correctamente en el sistema. Este proceso centraliza y automatiza la gestión de pagos, reduciendo tiempos de validación, mejorando la trazabilidad de las solicitudes y disminuyendo la dependencia de registros manuales.

Proceso actual de acceso de visitantes

Como se representa en la *Figura 6*, el proceso actual de ingreso de visitantes inicia cuando una persona llega a la urbanización y solicita autorización para ingresar. El guardia de seguridad recibe al visitante y solicita sus datos personales, registrando la información necesaria para identificarlo. Posteriormente, el guardia intenta comunicarse con el residente, generalmente mediante una llamada telefónica, si no logra establecer comunicación, el guardia debe acercarse directamente al domicilio del residente para solicitar la autorización de ingreso. Una vez que el residente es contactado, este confirma si autoriza o no el acceso del visitante a la urbanización.

Figura 6

Proceso actual del acceso de visitantes



Nota: El diagrama representa el proceso actual del acceso de los visitantes a la urbanización. Elaboración propia.

Si el residente autoriza el ingreso, el guardia registra manualmente la entrada del visitante en el registro correspondiente y habilita el acceso. A continuación, el visitante ingresa a la urbanización. Al finalizar su visita y abandonar las instalaciones el guardia registra la hora de salida, concluyendo el proceso con la visita finalizada.

Por otro lado, si el residente no autoriza el ingreso, el guardia niega el acceso al visitante y registra la novedad correspondiente en el control de incidencias o novedades de seguridad. En este caso, el proceso concluye con el ingreso rechazado.

Proceso propuesto para el control de accesos con QR

Como se observa en la *Figura 7*, el proceso propuesto inicia cuando el residente necesita autorizar el ingreso de un visitante a la urbanización. Para ello, accede a la aplicación móvil y registra los datos del visitante. Una vez almacenada la información, la aplicación genera automáticamente un código QR único asociado a la autorización de ingreso, el cual es compartido con el visitante.

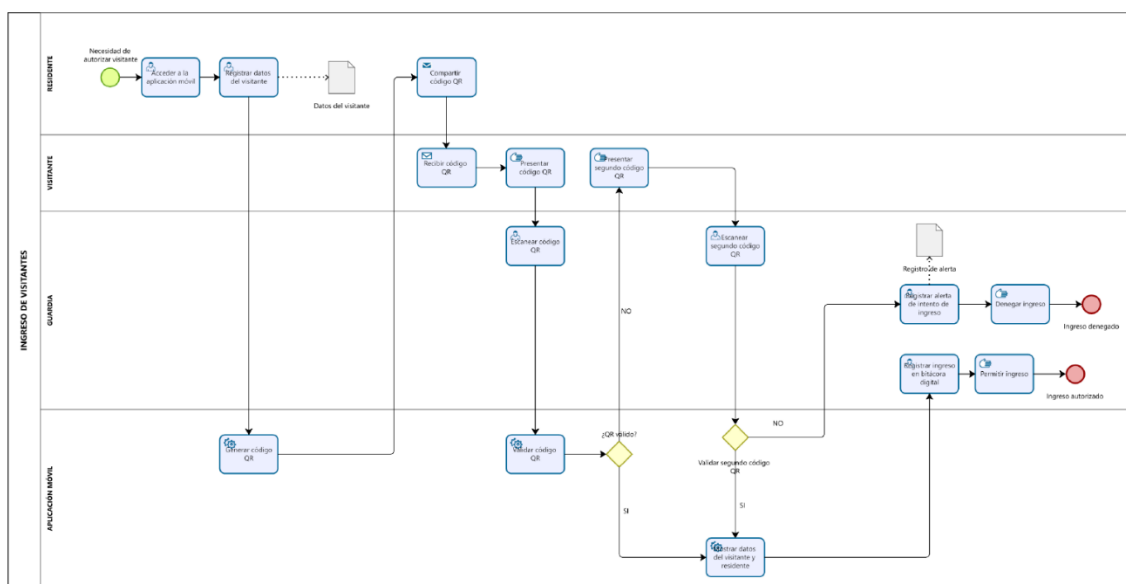
Al llegar a la urbanización, el visitante presenta el código QR al guardia de seguridad. El guardia procede a escanearlo utilizando la aplicación móvil, que valida automáticamente la autenticidad y vigencia del código. El guardia escanea el código QR y el sistema realiza la validación, la aplicación muestra la información del visitante y del residente asociado, permitiendo verificar la autorización registrada. Si el código es inválido, el residente deberá a generar un nuevo código para validar el ingreso.

Posteriormente, el sistema registra automáticamente el ingreso en una bitácora digital y el guardia habilita el acceso a la urbanización. En caso de que alguno de los códigos QR no sea

válido, haya expirado o presente inconsistencias, el sistema genera una alerta de intento de ingreso no autorizado, registra el evento en el historial correspondiente y el guardia deniega el acceso al visitante.

Figura 7

Proceso propuesto para el control de acceso con QR



Nota: El diagrama representa el proceso propuesto para el control de acceso con QR a la urbanización. Elaboración propia.

Proceso actual de la gestión de reservas de áreas sociales

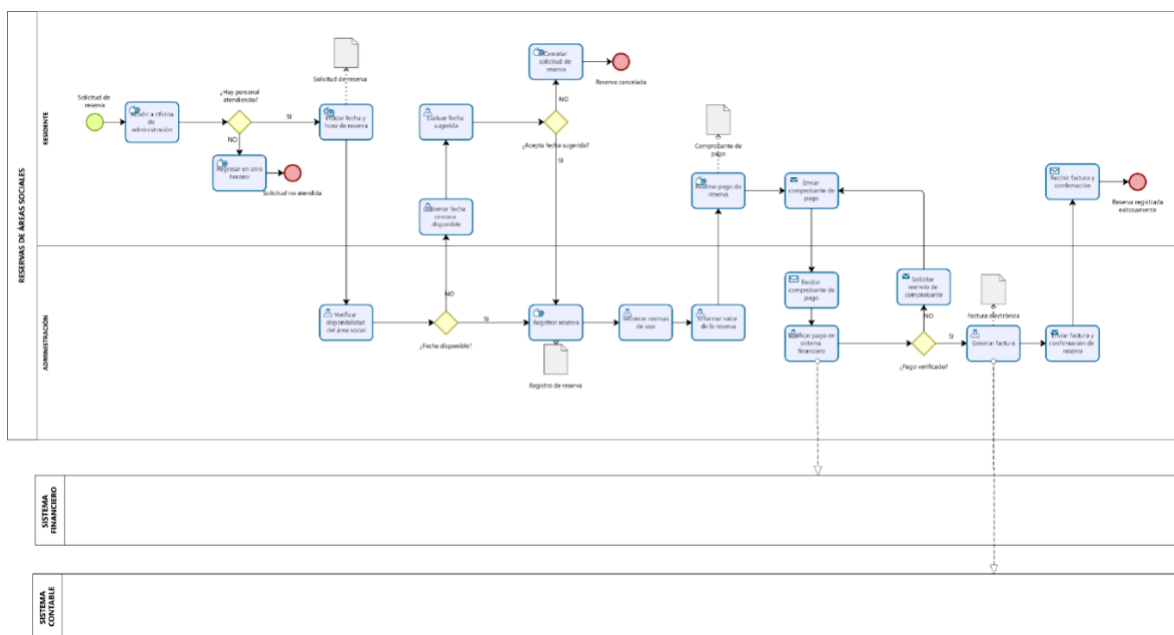
El proceso actual para la reserva de áreas sociales, inicia cuando el residente necesita reservar un área social y acude a la oficina de administración. Si hay personal atendiendo, el residente indica la fecha y hora deseadas para generar la solicitud; de lo contrario, deberá regresar en otro horario. Una vez receptada la información, la administración verifica la disponibilidad del espacio en el sistema. Si la fecha no está disponible, se le informa al residente una alternativa cercana para que la evalúe, permitiéndole aceptar el cambio o cancelar definitivamente la solicitud de reserva.

Cuando se confirma la disponibilidad de la fecha, la administración registra la reserva en el sistema y le notifica al residente tanto las normas de uso como el valor económico correspondiente. Con estos datos, el residente realiza el pago de la reserva y envía el comprobante digital a la administración. Al recibirlo, el personal administrativo procede a verificar el depósito en el sistema financiero. Si el pago no es verificado, el sistema genera una retroalimentación solicitando al residente el reenvío del comprobante para subsanar el error.

Posteriormente, una vez que el pago es validado con éxito, el sistema genera automáticamente la factura electrónica, registrando el asiento en el sistema contable. Finalmente, la administración envía la factura junto con la confirmación de la reserva al residente, quien recibe los documentos de soporte, concluyendo así de manera exitosa el proceso de registro de la reserva, detallado en la *Figura 8*.

Figura 8

Proceso actual de reserva de áreas sociales



Nota: El diagrama representa el proceso actual de la reservas de áreas sociales de la urbanización. Elaboración propia.

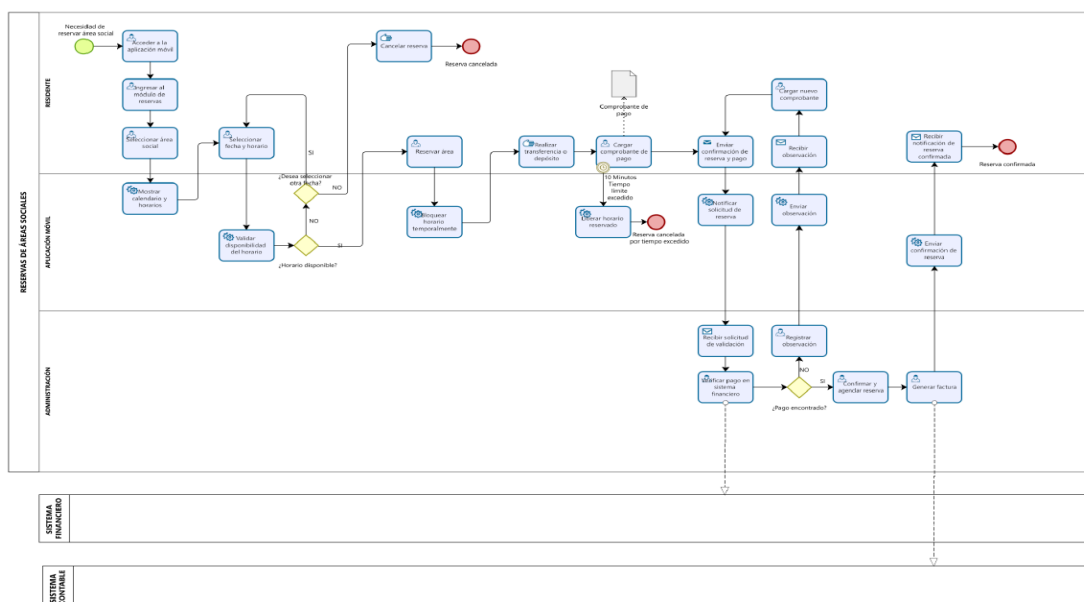
Proceso propuesto para la gestión de reserva de áreas sociales

Como se observa en la *Figura 9*, el proceso propuesto inicia cuando el residente necesita reservar un área social. Para ello, accede a la aplicación móvil, ingresa al módulo de reservas y selecciona el área social de su interés. La aplicación despliega automáticamente el calendario y los horarios disponibles, permitiendo al residente seleccionar la fecha y el horario deseados.

Posteriormente, la aplicación móvil valida la disponibilidad del horario; si el horario no está disponible, se consulta al usuario si desea seleccionar otra fecha, rediriéndolo al calendario de fechas disponibles o permitiéndole cancelar la reserva definitivamente.

Figura 9

Proceso propuesto para reservas de áreas sociales



Nota: El diagrama representa el proceso propuesto para las reservas de áreas sociales. Elaboración propia.

Si el horario está disponible, el residente procede a reservar el área y la aplicación móvil bloquea el horario temporalmente para asegurar el espacio. A partir de este momento, el usuario realiza la transferencia o depósito bancario y carga el comprobante de pago en la plataforma para

enviar la confirmación de reserva y pago. Cabe destacar que esta tarea cuenta con un evento de tiempo límite de 10 minutos; en caso de excederse este lapso, la aplicación móvil libera el horario reservado y el flujo finaliza automáticamente por tiempo excedido. Si el proceso continúa normalmente, la aplicación notifica la solicitud de reserva a la administración.

A continuación, la administración recibe la solicitud de validación y verifica el pago en el sistema financiero. Si el pago es encontrado y validado con éxito, la administración confirma y agenda la reserva, genera la factura correspondiente conectándose con el sistema contable y envía la confirmación de la reserva a través de la aplicación móvil, permitiendo que el residente reciba la notificación de reserva confirmada y culminando así el proceso de manera exitosa. Por el contrario, si el pago no es encontrado, la administración registra una observación y la envía al residente, quien recibe la notificación en la aplicación móvil y procede a cargar un nuevo comprobante para reiniciar el ciclo de validación.

Descripcion de casos de uso

Los casos de uso permiten representar de manera estructurada las funcionalidades que el sistema debe proporcionar para satisfacer las necesidades de los usuarios, definiendo los objetivos, flujos de interacción y resultados esperados de cada proceso. En el contexto de este proyecto, los casos de uso se derivan de los procesos de gestión de pagos de alícuotas, control de acceso de visitantes mediante códigos QR, reservas de áreas sociales y generación de reportes administrativos. Su definición constituye una herramienta fundamental para comprender el comportamiento esperado del sistema y sirve como base para las etapas de diseño, desarrollo, pruebas y validación del prototipo de aplicación móvil.

Caso de uso: Iniciar sesión

Como se muestra en la *Tabla 49*, el caso de uso permite a los residentes, administradores y guardias autenticarse en la aplicación móvil mediante el ingreso de sus credenciales de acceso. El sistema valida la información proporcionada y, una vez confirmada la identidad del usuario, habilita el acceso a las funcionalidades correspondientes según su rol, garantizando la seguridad y el control de acceso a la plataforma.

Tabla 49

Caso de uso – Iniciar sesión

ID	CU-01		
Nombre	Iniciar sesión		
Descripción	Permite a los usuarios autenticarse en la aplicación móvil para acceder a las funcionalidades correspondientes a su rol.		
Actores	Residente/ Administrador/ Guardia/ Sistema		
Precondiciones	El usuario se encuentra registrado en el sistema		
Condiciones	El servicio de autenticación se encuentra disponible.		
Post condiciones	El usuario accede al sistema y visualiza las opciones correspondientes a su perfil.		
Secuencia Normal	No.	Acción (actor)	Reacción (sistema)
	1	El usuario ingresa correo electrónico y contraseña.	El sistema valida los datos ingresados.
	2	El usuario selecciona "Iniciar Sesión".	El sistema verifica las credenciales.
	3	El sistema identifica el rol del usuario.	El sistema muestra el menú correspondiente al perfil autenticado.

Nota: En esta tabla se presenta el caso de uso de inicio de sesión en la aplicación móvil. **Fuente:** Elaboración propia.

Caso de uso: Registrar comprobante de pago

Este caso de uso permite al residente registrar el comprobante de pago de una alícuota pendiente mediante la carga de una fotografía o archivo digital desde la aplicación móvil. La

información es enviada a la administración para su posterior revisión y validación, facilitando el seguimiento y control de los pagos realizados por los residentes, como se muestra en la *Tabla 50*.

Tabla 50

Caso de uso – Registrar comprobante de pago

ID	CU-02		
Nombre	Registrar comprobante de pago		
Descripción	Permite al residente registrar el comprobante de pago de una alícuota para su validación por parte de la administración.		
Actores	Residente/ Sistema		
Precondiciones	El residente posee una alícuota pendiente de pago.		
Condiciones	La aplicación móvil y el servicio de almacenamiento se encuentran disponibles.		
Post condiciones	El comprobante queda registrado y pendiente de validación.		
Secuencia Normal	No.	Acción (actor)	Reacción (sistema)
	1	El residente consulta el valor pendiente.	El sistema muestra el detalle de la alícuota.
	2	El residente realiza el pago mediante transferencia o depósito.	N/A
	3	El residente selecciona la opción "Registrar Pago".	El sistema habilita la carga del comprobante.
	4	El residente adjunta la fotografía del comprobante	El sistema valida el formato del archivo.
	5	El residente confirma el envío.	El sistema registra la solicitud y notifica a la administración.

Nota: En esta tabla se presenta el caso de uso de registrar comprobante de pago en la aplicación móvil. **Fuente:** Elaboración propia.

Caso de uso: Validar pago de alícuota

Como se observa en la *Tabla 51*, este caso de uso permite al administrador verificar la autenticidad y existencia de los pagos reportados por los residentes. Para ello, revisa el comprobante registrado y consulta la información disponible en el sistema financiero. Como resultado de la validación, el pago puede ser aprobado o quedar sujeto a observaciones en caso de inconsistencias.

Tabla 51*Caso de uso – Validar pago de alícuota*

ID	CU-03		
Nombre	Validar pago de alícuota		
Descripción	Permite al administrador verificar la existencia y validez de un pago reportado por un residente.		
Actores	Administrador/ Sistema		
Precondiciones	Existe una solicitud de validación pendiente.		
Condiciones	El comprobante de pago es validado.		
Post condiciones	El pago queda aprobado o se registra una observación.		
Secuencia Normal	No.	Acción (actor)	Reacción (sistema)
	1	El administrador consulta las solicitudes pendientes.	El sistema muestra los pagos por validar.
	2	El administrador selecciona un comprobante.	El sistema presenta la información registrada.
	3	El administrador verifica la transferencia.	El sistema consulta la información financiera.
	4	El sistema verifica la vigencia y autenticidad del QR..	El sistema obtiene el resultado de validación.
	5	El administrador aprueba el pago.	El sistema registra el pago.

Nota: En esta tabla se presenta el caso de la validación de pago de alícuota en la aplicación móvil. **Fuente:** Elaboración propia.

Caso de uso: Registrar visitante

Este caso de uso permite al residente registrar previamente la información de una persona autorizada para ingresar a la urbanización. Los datos ingresados son almacenados en el sistema y utilizados para generar una autorización digital que facilitará el control y validación del acceso del visitante, detallado en la *Tabla 52*.

Tabla 52*Caso de uso – Registrar visitante*

ID	CU-04
Nombre	Registrar visitante
Descripción	Permite al residente registrar los datos de un visitante que ingresará a la urbanización.

Actores	Residente/ Sistema		
Precondiciones	El residente ha iniciado sesión.		
Condiciones	El módulo de visitantes se encuentra disponible.		
Post condiciones	El visitante queda registrado y se genera una autorización de ingreso.		
Secuencia Normal	No.	Acción (actor)	Reacción (sistema)
	1	El residente accede al módulo de visitantes.	El sistema muestra el formulario de registro.
	2	El residente ingresa los datos del visitante.	El sistema valida la información ingresada.
	3	El residente confirma el registro.	El sistema almacena la información del visitante.
	4	El sistema genera un código QR.	El sistema muestra el QR generado para compartirlo.

Nota: En esta tabla se presenta el caso de uso de registrar visitante en la aplicación móvil. **Fuente:** Elaboración propia.

Caso de uso: Validar código QR

Como se observa en la *Tabla 53*, este caso de uso permite al guardia verificar la autorización de ingreso de un visitante mediante el escaneo de un código QR generado previamente por el sistema. La aplicación valida la autenticidad, vigencia y relación del código con el visitante autorizado, permitiendo o rechazando el acceso según el resultado de la validación.

Tabla 53

Caso de uso – Validar código QR

ID	CU-05			
Nombre	Validar código QR			
Descripción	Permite al guardia validar la autorización de ingreso de un visitante mediante el escaneo del código QR generado por el sistema.			
Actores	Guardia/ Sistema			
Precondiciones	El visitante dispone de un código QR válido y vigente.			
Condiciones	El servicio de validación se encuentra disponible.			
Post condiciones	Se autoriza o deniega el ingreso del visitante y se registra el resultado en la base de datos.			
Secuencia Normal	<table><tr><th>No.</th><th>Acción (actor)</th><th>Reacción (sistema)</th></tr></table>	No.	Acción (actor)	Reacción (sistema)
No.	Acción (actor)	Reacción (sistema)		

1	El guardia selecciona la opción de escaneo de QR.	El sistema activa la cámara del dispositivo.
2	El visitante presenta el código QR.	El sistema captura y decodifica la información.
3	El guardia confirma el escaneo.	El sistema obtiene el resultado de validación.
4	El sistema verifica la vigencia y autenticidad del QR..	El sistema obtiene el resultado de validación.
5	El sistema registra el acceso autorizado.	El sistema muestra los datos del visitante y residente asociado.
6	El guardia permite el ingreso.	El sistema registra la fecha y hora de acceso.

Nota: En esta tabla se presenta el caso de la validación del código QR para ingreso en la aplicación móvil. **Fuente:** Elaboración propia.

Casos de uso: Registrar reserva de áreas sociales

Este caso de uso permite al residente solicitar la reserva de un área social disponible dentro de la urbanización. El usuario selecciona el espacio, la fecha y el horario deseado, adjuntando la documentación requerida para completar la solicitud. Posteriormente, la reserva queda registrada para su validación por parte de la administración, explicado en la *Tabla 54*.

Tabla 54

Caso de uso – Registrar reserva de áreas sociales

ID	CU-06		
Nombre	Registrar reserva de áreas sociales		
Descripción	Permite al residente solicitar la reserva de un área social disponible.		
Actores	Residente/ Sistema		
Precondiciones	El residente se encuentra autenticado.		
Condiciones	Existen horarios disponibles para reserva.		
Post condiciones	La reserva queda registrada y pendiente de validación.		
Secuencia Normal	No.	Acción (actor)	Reacción (sistema)
	1	El residente accede al módulo de áreas sociales.	El sistema muestra las áreas disponibles.
	2	El residente selecciona un área social.	El sistema muestra fechas y horarios disponibles.

3	El residente selecciona fecha y horario.	El sistema verifica disponibilidad.
4	El residente adjunta el comprobante requerido.	El sistema valida el archivo cargado.
5	El residente confirma la reserva.	El sistema registra la solicitud y bloquea temporalmente el horario.

Nota: En esta tabla se presenta el caso del registro de reservas de áreas sociales en la aplicación móvil. **Fuente:** Elaboración propia.

Caso de uso: Validar reserva de áreas sociales

Como se detalla en la *Tabla 55*, este caso de uso permite al administrador revisar las solicitudes de reserva registradas por los residentes y verificar el cumplimiento de las condiciones establecidas para su aprobación. Una vez analizada la información, el administrador puede aprobar o rechazar la reserva, notificando automáticamente el resultado al solicitante.

Tabla 55

Caso de uso – Validar reserva de áreas sociales

ID	CU-07		
Nombre	Validar reserva de área social		
Descripción	Permite al administrador aprobar o rechazar una solicitud de reserva.		
Actores	Administrador/ Sistema		
Precondiciones	Existe una reserva pendiente de validación.		
Condiciones	Existen horarios disponibles para reserva.		
Post condiciones	El módulo de reservas se encuentra operativo.		
Secuencia Normal	No.	Acción (actor)	Reacción (sistema)
	1	El administrador consulta las reservas pendientes.	El sistema muestra las solicitudes registradas.
	2	El administrador revisa la información.	El sistema presenta los detalles de la reserva.
	3	El administrador selecciona aprobar reserva.	El sistema actualiza el estado de la reserva.
	4	El sistema notifica al residente.	El sistema registra la aprobación.

Nota: En esta tabla se presenta el caso de la validación de reserva de área social en la aplicación móvil. **Fuente:** Elaboración propia.

Caso de uso Generar reporte de pagos

Este caso de uso como se observa en la *Tabla 57*, permite al administrador consultar información consolidada sobre los pagos de alcúotas registrados en un período determinado. El sistema procesa los registros almacenados y genera un reporte que identifica residentes al día, pagos pendientes y cartera vencida, facilitando el seguimiento financiero de la urbanización.

Tabla 56

Caso de uso – Generar reporte de pagos

ID	CU-08		
Nombre	Generar reporte de pagos		
Descripción	Permite al administrador consultar el estado mensual de pagos de los residentes.		
Actores	Administrador/ Sistema		
Precondiciones	Existen registros de pagos en el sistema.		
Condiciones	El módulo de reportes se encuentra disponible.		
Post condiciones	El reporte es generado y visualizado.		
Secuencia Normal	No.	Acción (actor)	Reacción (sistema)
	1	El administrador accede al módulo de reportes.	El sistema muestra las opciones disponibles.
	2	El administrador selecciona reporte de pagos.	El sistema solicita el período de consulta.
	3	El administrador selecciona el mes requerido.	El sistema procesa la información.
	4	El sistema genera el reporte.	El sistema muestra el estado de pago de los residentes y la cartera vencida.

Nota: En esta tabla se presenta el caso de reporte de pagos en la aplicación móvil. **Fuente:** Elaboración propia.

Desarrollo de historias de usuario

Las historias de usuario constituyen la base para la planificación y desarrollo de la aplicación móvil propuesta para la gestión de la urbanización. Esta serie de tareas reúne y organiza de manera priorizada las funcionalidades identificadas a partir de las necesidades de los principales

actores del sistema: residentes, administradores y personal de seguridad. Las historias de usuario definidas abarcan procesos esenciales como la autenticación y administración de usuarios, el control y seguimiento de pagos de alcuotas, y la gestión de accesos mediante códigos QR, permitiendo fortalecer la seguridad, optimizar los procesos administrativos y mejorar la experiencia de los usuarios. De la misma forma, cada historia incorpora criterios de aceptación que facilitan la validación de los requisitos y aseguran que las funcionalidades desarrolladas respondan efectivamente a los objetivos planteados para la solución tecnológica.

Tabla 57

Historias de usuario

Épica	Gestión de autenticación y usuarios
ID	HU-01
Descripción	
Como residente, guardia o administrador de la urbanización, quiero autenticarme en la aplicación mediante mis credenciales de acceso, para ingresar de forma segura al sistema y utilizar las funcionalidades autorizadas de acuerdo con mi rol.	
Sprint	1
Prioridad	Alta
% de aceptación objetivo	95%

Épica	Autenticación y Seguridad
ID	HU-02
Descripción	
Como residente, guardia o administrador de la urbanización, quiero recuperar el acceso a mi cuenta mediante un proceso seguro de restablecimiento de contraseña, para poder autenticarme nuevamente cuando haya olvidado mis credenciales de acceso.	
Sprint	1
Prioridad	Alta
% de aceptación objetivo	90%

Épica	Autenticación y Seguridad
ID	HU-03

Descripción	
Como administrador, residente o guardia, quiero cerrar sesión para proteger la información y evitar accesos no autorizados cuando deje de utilizar la aplicación.	
Sprint	1
Prioridad	Alta
% de aceptación objetivo	95%
Épica	Gestión de usuarios
ID	HU-04
Descripción	
Como administrador o residente, quiero registrar una cuenta de usuario para acceder a las funcionalidades del sistema según el rol correspondiente.	
Sprint	2
Prioridad	Alta
% de aceptación objetivo	96%
Épica	Gestión de usuarios
ID	HU-05
Descripción	
Como administrador o residente, quiero consultar la información disponible según mi rol para verificar que los datos registrados en el sistema sean correctos y estén actualizados.	
Sprint	2
Prioridad	Alta
% de aceptación objetivo	94%
Épica	Gestión de usuarios
ID	HU-06
Descripción	
Como administrador o residente, quiero actualizar la información de los usuarios autorizados según mi rol para mantener los datos correctos y actualizados en el sistema.	
Sprint	2
Prioridad	Alta
% de aceptación objetivo	9%
Épica	Gestión de usuarios
ID	HU-07

Descripción

Como administrador, quiero desactivar o eliminar cuentas de usuario según corresponda para controlar el acceso al sistema y mantener actualizado el registro de usuarios.

Sprint	2
Prioridad	Alta
% de aceptación objetivo	95%

Épica	Gestión y control de pagos de alcúotas
--------------	--

ID	HU-08
-----------	-------

Descripción

Como residente de la urbanización, quiero registrar el comprobante de pago de mi alcúota mediante la aplicación móvil para informar el pago realizado y facilitar su validación por parte del administrador.

Sprint	3
Prioridad	Alta
% de aceptación objetivo	97%

Épica	Gestión y control de pagos de alcúotas
--------------	--

ID	HU-09
-----------	-------

Descripción

Como administrador, quiero revisar y validar los comprobantes de pago enviados por los residentes para confirmar su autenticidad, actualizar el estado de las alcúotas y mantener un control confiable de los pagos registrados.

Sprint	3
Prioridad	Alta
% de aceptación objetivo	97%

Épica	Gestión y control de pagos de alcúotas
--------------	--

ID	HU-10
-----------	-------

Descripción

Como residente de la urbanización, quiero consultar el historial y estado de mis pagos de alcúotas para dar seguimiento a mis obligaciones y conocer si existen valores pendientes o pagos validados.

Sprint	3
Prioridad	Alta
% de aceptación objetivo	97%

Épica	Control de acceso mediante códigos QR
--------------	---------------------------------------

ID	HU-11
-----------	-------

Descripción

Como residente de la urbanización, quiero generar un código QR temporal para mis visitantes para autorizar su ingreso y facilitar su validación por parte del guardia.

Sprint	4
Prioridad	Alta
% de aceptación objetivo	97%

Épica	Control de acceso mediante códigos QR
--------------	---------------------------------------

ID	HU-12
-----------	-------

Descripción

Como guardia de seguridad de la urbanización, validar un código QR de acceso presentados por los visitantes, para validar la autorización de ingreso y registrar el acceso de manera segura y trazable.

Sprint	4
Prioridad	Alta
% de aceptación objetivo	98%

Épica	Control de acceso mediante códigos QR
--------------	---------------------------------------

ID	HU-13
-----------	-------

Descripción

Como administrador y residente, quiero consultar el historial de accesos registrados mediante códigos QR para supervisar los ingresos de residentes y visitantes y mantener un control de los accesos a la urbanización.

Sprint	4
Prioridad	Alta
% de aceptación objetivo	95%

Épica	Reservas de áreas sociales
--------------	----------------------------

ID	HU-14
-----------	-------

Descripción

Como residente de la urbanización, quiero reservar un área social disponible indicando la fecha y el horario de uso, para asegurar la disponibilidad del espacio para la actividad planificada.

Sprint	4
Prioridad	Alta
% de aceptación objetivo	92%

Épica	Reservas de áreas sociales
--------------	----------------------------

ID	HU-15
-----------	-------

Descripción

Como administrador de la urbanización, quiero aprobar o rechazar las reservas para administrar la disponibilidad de las áreas sociales.

Sprint	4
Prioridad	Alta
% de aceptación objetivo	96%

Épica	Reservas de áreas sociales
--------------	----------------------------

ID	HU-16
-----------	-------

Descripción

Como residente de la urbanización, quiero consultar el estado de mis reservas para conocer si fueron aprobadas o rechazadas.

Sprint	4
Prioridad	Alta
% de aceptación objetivo	96%

Épica	Reservas de áreas sociales
--------------	----------------------------

ID	HU-17
-----------	-------

Descripción

Como residente de la urbanización, quiero registrar el pago de mi reserva mediante la carga de un comprobante digital, para notificar el pago realizado y facilitar su validación por parte de la administración.

Sprint	4
Prioridad	Alta
% de aceptación objetivo	95%

Épica	Reportes y seguimientos
--------------	-------------------------

ID	HU-18
-----------	-------

Descripción

Como administrador, quiero generar reportes de pagos para dar seguimiento a la recaudación de alícuotas.

Sprint	5
Prioridad	Media
% de aceptación objetivo	88%

Épica	Reportes y seguimientos
--------------	-------------------------

ID	HU-19
-----------	-------

Descripción

Como administrador de la urbanización, quiero generar reportes de accesos mediante códigos QR para controlar el ingreso de visitantes.

Sprint	5
Prioridad	Media
% de aceptación objetivo	88%

Épica	Reportes y seguimientos
--------------	-------------------------

ID	HU-20
-----------	-------

Descripción

Como administrador, quiero generar reportes de reservas para supervisar el uso de las áreas sociales.

Sprint	5
Prioridad	Media
% de aceptación objetivo	88%

Épica	Reportes y seguimientos
--------------	-------------------------

ID	HU-21
-----------	-------

Descripción

Como administrador de la urbanización, quiero generar reportes de los registros de los usuarios en la aplicación móvil.

Sprint	5
Prioridad	Media
% de aceptación objetivo	88%

Product Backlog

A continuación, se presenta el Product Backlog del proyecto, elaborado a partir del levantamiento y análisis de los requerimientos funcionales y no funcionales identificados durante la etapa de análisis. Este artefacto reúne y organiza las funcionalidades que conformarán la aplicación móvil, estructuradas en historias de usuario y priorizadas según su valor para el usuario, su impacto en los objetivos del proyecto y su viabilidad de implementación.

El Product Backlog constituye la base para la planificación de los Sprints, permitiendo al equipo de desarrollo gestionar de manera ordenada las actividades, realizar entregas incrementales

de software funcional y adaptar el alcance del proyecto conforme evolucionen las necesidades de los interesados, en concordancia con los principios de la metodología Scrum.

Tabla 58

Product Backlog

ID	Historia de Usuario	Sprint	Objetivo
HU-AT-01	Diseño de interfaces de la aplicación móvil	0	Diseñar las interfaces de usuario de la aplicación móvil de acuerdo con los requerimientos funcionales y criterios de usabilidad, con el fin de proporcionar una experiencia de usuario intuitiva, accesible y consistente durante la interacción con el sistema
HU-AT-02	Diseño del modelo entidad - relación de la base de datos		Diseñar el modelo entidad-relación de la base de datos para definir una estructura de datos organizada, consistente y acorde con los requerimientos del sistema
HU-AT-03	Diseño de la arquitectura de aplicación móvil y elección de tecnologías		Diseñar la arquitectura de la aplicación móvil y seleccionar las tecnologías, herramientas y patrones de desarrollo más adecuados, con el fin de establecer una base técnica que garantice la escalabilidad, mantenibilidad, rendimiento e integración con el backend del sistema
HU-01	Crear cuenta de usuario	1	Registro seguro de usuarios implementado
HU-02	Iniciar sesión mediante credenciales		Autenticación con control de roles
HU-03	Recuperar contraseña		Restablecimiento seguro de credenciales
HU-04	Registro de usuarios	2	Permitir el registro de nuevos usuarios en el sistema
HU-05	Consultar información de usuarios		Facilitar la consulta de la información de los usuarios según el rol asignado
HU-06	Actualizar información de usuarios		Permitir la actualización de la información de los usuarios autorizados.
HU-07	Eliminar o desactivar usuarios		Permitir la eliminación o desactivación de los usuarios

HU-08	Registrar comprobante de alícuota		Permitir el registro de los pagos de la alícuotas en la aplicación
HU-09	Validar pagos de alícuotas	3	Permitir la validación de los pagos de la alícuotas en la aplicación al administrador
HU-10	Consultar historial de pagos de alícuotas		Facilitar la consulta de los pagos de alícuotas
HU-11	Generar QR de acceso		Permitir generar un QR para el ingreso de los visitantes
HU-12	Validar QR de acceso		Permitir validar un QR para el ingreso de los visitantes
HU-13	Consultar historial de ingresos		Facilitar la consulta de los ingresos a la urbanización
HU-14	Reserva un área social	4	Permitir reserva una áreas social disponible
HU-15	Aprobar o rechazar las reservas		Permitir rechazar o aprobar las reservas de las áreas sociales
HU-16	Consultar estado de reserva		Facilitar la consulta del estado de la reserva
HU-17	Registra comprobante de reserva		Permitir el registro de los pagos de la reservas de la área de eventos
HU-18	Desarrollo del módulo de reportes	5	Facilita la consulta de los reportes generados por cada proceso.
-	Integración de todos los módulos		Aplicación completamente integrada.

Nota: Esta tabla presenta el Product Backlog del proyecto, conformado por las historias de usuario priorizadas y organizadas según el Sprint en el que serán desarrolladas. Elaboración propia.

Desarrollo de Sprints

El desarrollo del prototipo se llevó a cabo mediante la metodología Scrum, empleando un enfoque iterativo e incremental que permitió organizar el trabajo en cinco sprints. En cada sprint se planificaron, desarrollaron y validaron un conjunto de funcionalidades priorizadas del Product Backlog, obteniendo incrementos funcionales del sistema al finalizar cada iteración. Esta estrategia facilitó el seguimiento continuo del avance del proyecto, la verificación del cumplimiento de los requisitos funcionales y la incorporación de mejoras derivadas de las revisiones realizadas durante el proceso. A continuación, se presenta el desarrollo de cada sprint, detallando su objetivo, las historias de usuario seleccionadas, las actividades ejecutadas, los entregables obtenidos y los resultados alcanzados en cada incremento del prototipo.

- **Sprint 0.** Preparación del entorno de desarrollo y arquitectura del proyecto
- **Sprint 1.** Implementación de la infraestructura funcional y autenticación del sistema.
- **Sprint 2.** Desarrollo del módulo de gestión de usuarios.
- **Sprint 3.** Desarrollo del módulo de control de pagos de alícuotas.
- **Sprint 4.** Desarrollo de los módulos de control de acceso mediante códigos QR y reservas de áreas sociales.
- **Sprint 5.** Desarrollo del módulo de reportes e integración del sistema.

Sprint 0. Preparación del entorno de desarrollo y arquitectura del proyecto

Tabla 59

Sprint Planning 0

SPRINT 0				
ID de Product Backlog	Historia de usuario	Tarea	Responsable	Estado
IPB01	Diseño de interfaces de la aplicación móvil	1. Diseñar prototipo en Figma con las interfaces del sistema según el alcance y levantamiento de información.	Jhonas López, Belky Piedra	Completado
IPB02	Diseño del modelo entidad - relación de la base de datos	2. Diseñar el DER detallando las entidades y relaciones de la aplicación móvil.	Jhonas López, Belky Piedra	Completado
IPB03	Diseño de la arquitectura de aplicación móvil y elección de tecnologías.	3. Seleccionar las tecnologías para el desarrollo del Frontend, backend y base de datos. 4. Diseñar la estructura base del proyecto. 5. Configurar las dependencias iniciales de la aplicación móvil.	Jhonas López, Belky Piedra	Completado

6. Configurar el entorno y repositorios para el frontend y backend.
7. Diseñar la estrategia de comunicación con la API del backend.

Nota: La tabla detalla las historias de usuario, tareas, responsables y estado del Sprint 0 para el progreso y desarrollo del proyecto. Elaboración propia.

Historia Técnica 1

IPB01 Diseño de interfaces de la aplicación móvil

Tarea 1. Diseñar prototipo en Figma con las interfaces del sistema según el alcance y levantamiento de información.

En esta tarea se desarrolló el prototipo de las interfaces de la aplicación móvil utilizando Figma, tomando como base los requerimientos previamente definidos y el alcance establecido para el proyecto. El diseño se elaboró siguiendo principios de usabilidad, accesibilidad y consistencia visual, con el propósito de validar la estructura, la navegación y la experiencia de usuario antes de iniciar la fase de desarrollo. A continuación, se presentan las principales interfaces de la aplicación móvil, las cuales representan los módulos funcionales contemplados dentro del alcance del proyecto.

- Prototipos del Módulo de usuario

Figura 10*Diseño de pantalla de Login*
Nota: Elaboración propia.**Figura 11***Diseño de pantalla de crear usuario Administrador*
Nota: Elaboración propia.

Figura 12*Diseño de pantalla de crear usuario Residente*

Crear Cuenta

Tipo de Usuario

Residente Admin

Cédula

Ej. 0999999999

Nombres

Ej. Juan Alberto

Apellidos

Ej. Pérez García

Correo

usuario@correo.com

Teléfono

+593 999 999 999

Siguiente

Volver al Inicio de sesión

Crear Cuenta

Tipo de Usuario

Residente Admin

Manzana

Ej. Mz. 2584

Villa

Ej. Villa 22

Usuario

Ej. Juan.perez

Contraseña

REQUISITOS DE SEGURIDAD

- ☒ Mínimo 8 caracteres
- ☐ Al menos un número (0-9)
- ☐ Un carácter especial (!@#%\$%)

☐ Acepto los [Términos y Condiciones](#) de uso de la aplicación.

☐ He leído y acepto la [Política de Privacidad](#) para el tratamiento de mis datos.

Registrarse

Nota: Elaboración propia.**Figura 13***Diseño de pantalla de crear usuario Guardia*

Crear Cuenta

Tipo de Usuario

Guardia

Cédula

Ej. 0999999999

Nombres

Ej. Juan Alberto

Apellidos

Ej. Pérez García

Correo

usuario@correo.com

Teléfono

+593 999 999 999

Siguiente

Registro de Guardia

Tipo de Usuario

Guardia

Usuario

Ej. Juan.perez

ID de Guardia

Ej. GUARD-12345

Contraseña

REQUISITOS DE SEGURIDAD

- ☒ Mínimo 8 caracteres
- ☐ Al menos un número (0-9)
- ☐ Un carácter especial (!@#%\$%)

☐ Acepto los [Términos y Condiciones](#) de uso de la aplicación.

☐ He leído y acepto la [Política de Privacidad](#) para el tratamiento de mis datos.

Registrarse

Nota: Elaboración propia.

Figura 14

Diseño de pantalla de recuperar contraseña

←


UrbanApp

Recuperar Contraseña
Introduce tu correo electrónico para recibir un enlace de recuperación

Cédula de Identidad

Ej: 1.234.567-8

Correo electrónico

ejemplo@correo.com

Enviar Instrucciones >

○

[Volver al inicio de sesión](#)

Nota: Elaboración propia.

Figura 15

Diseño de pantalla de perfil de residente

Urban App

 Bienvenido, Juan Pérez
Residente • Manzana 2584 • Villa 42

Resumen de Alicuota

SALDO PENDIENTE **\$20.00** VENCE EN 5 DÍAS

Subir Comprobante

Mis Accesos

Generar Nuevo Acceso QR
Crea accesos para visitantes

Mis Reservas

Generar Nueva Reserva
Crea nuevas reservas en nuestras áreas sociales

PAGOS QR INICIO RESERVAS PERFIL

Nota: Elaboración propia.

Figura 16

Diseño de pantalla de perfil de administrador



Nota: Elaboración propia.

Figura 17

Diseño de pantalla de perfil de guardia

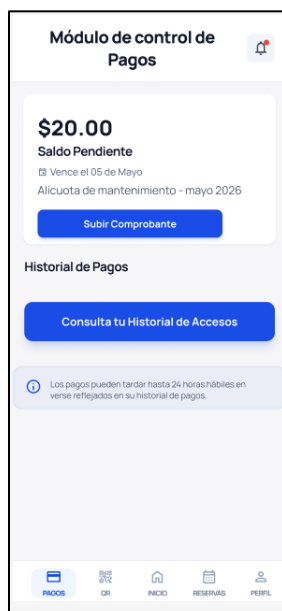


Nota: Elaboración propia.

- Prototipos del Módulo de Control de Pagos

Figura 18

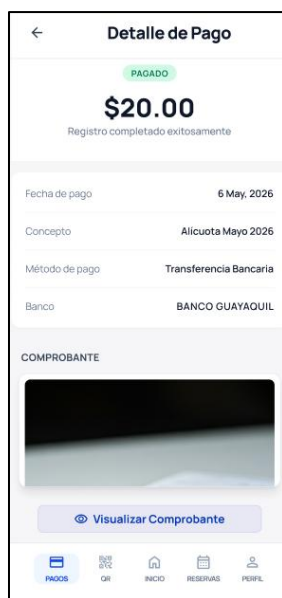
Diseño de pantalla de registro de pago en perfil de residente



Nota: Elaboración propia.

Figura 19

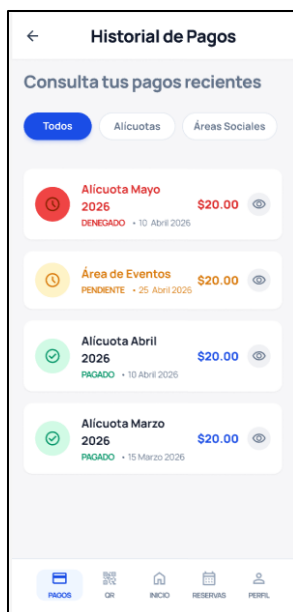
Diseño de pantalla de detalle de pago en perfil de residente



Nota: Elaboración propia.

Figura 20

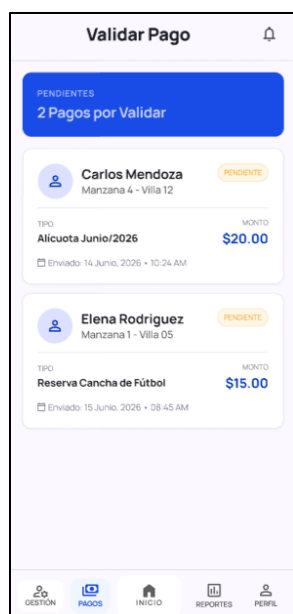
Diseño de pantalla de detalle de pago en perfil de residente



Nota: Elaboración propia.

Figura 21

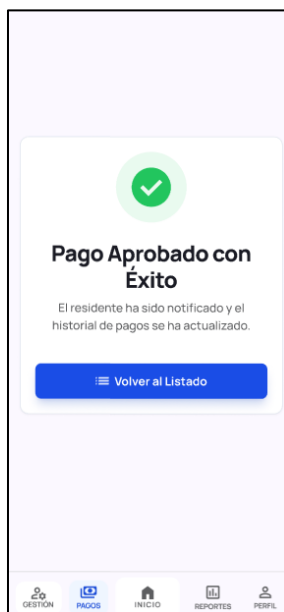
Diseño de pantalla de validación de pago en perfil de administrador



Nota: Elaboración propia.

Figura 22

Diseño de pantalla de aprobación de pago



Nota: Elaboración propia.

Figura 23

Diseño de pantalla de pago denegado



Nota: Elaboración propia.

- Prototipos del Módulo de Control de Accesos

Figura 24

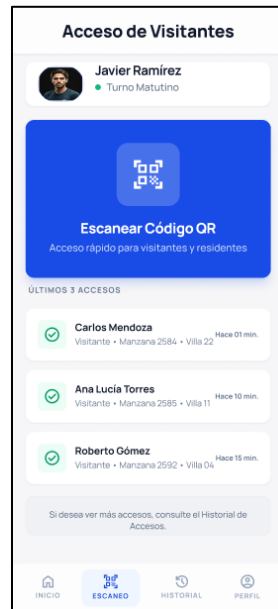
Diseño de pantalla de registro de visitante

Nota: Elaboración propia.

Figura 25

Diseño de la pantalla generación de QR

Nota: Elaboración propia.

Figura 26*Diseño de pantalla de acceso de visitante*

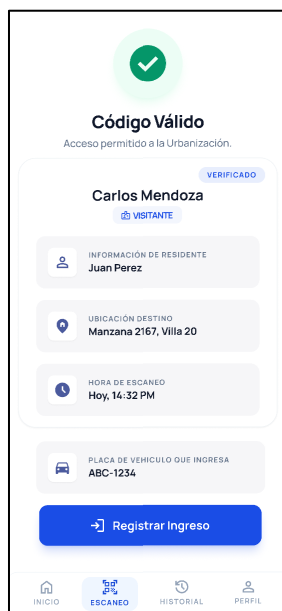
Nota: Elaboración propia.

Figura 27*Diseño de pantalla de escaneo de QR*

Nota: Elaboración propia.

Figura 27**Figura 28**

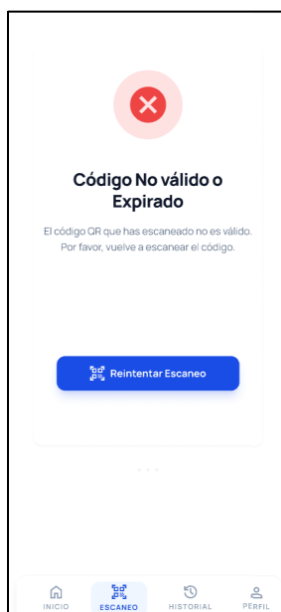
Diseño de pantalla de código válido



Nota: Elaboración propia.

Figura 29

Diseño de pantalla de código inválido



Nota: Elaboración propia.

Figura 30

Diseño de pantalla de ingreso registrado



Nota: Elaboración propia.

Figura 31

Diseño de pantalla de ingreso denegado



Nota: Elaboración propia.

Figura 32

Diseño de pantalla de ingreso manual

Ingreso Manual de Visitante

DATOS DEL VISITANTE

Cédula

Nombre

Apellido

Placa de Vehículo

DATOS DEL RESIDENTE QUE VA A VISITAR

Nombre del Residente

Manzana

Villa

Confirmar Registro

INICIO ESCANEO HISTORIAL PERFIL

Nota: Elaboración propia.

Figura 33

Diseño de pantalla de historial de registros

Historial de Accesos

Hoy Ayer Rango de fechas

REGISTROS RECIENTES 15 TOTAL DEL DIA

Juan Pérez
M2 2584 - Villa 22
HORA DE INGRESO
08:45 AM

Jose Delgado
M2 2587 - Villa 22
HORA DE INGRESO
08:45 AM

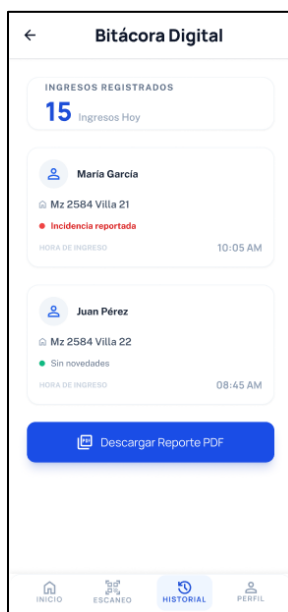
BITÁCORA DIGITAL

INICIO ESCANEO HISTORIAL PERFIL

Nota: Elaboración propia.

Figura 34

Diseño de pantalla de bitácora de ingresos

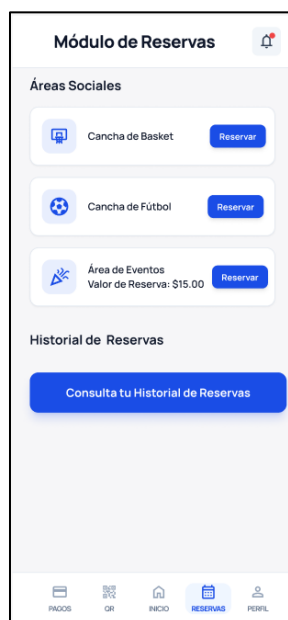


Nota: Elaboración propia.

- Prototipos del Módulo de Reservas

Figura 35

Diseño de pantalla de áreas sociales



Nota: Elaboración propia.

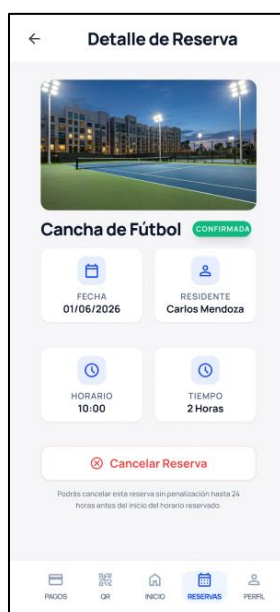
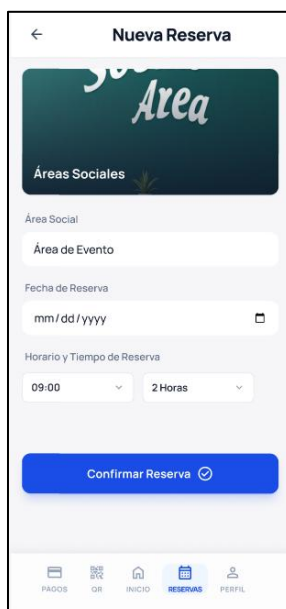
Figura 36*Diseño de pantalla de detalle de reserva***Nota:** Elaboración propia.**Figura 37***Diseño de pantalla de nueva reserva***Nota:** Elaboración propia.

Figura 38

Diseño de pantalla de detalle de pago de reserva

← Subir Comprobante de Pago

VALOR A PAGAR POR RESERVA
\$15.00

ÁREA SOCIAL
Área de Eventos

FECHA DE RESERVA
31/05/2026

HORA DE INICIO
09:00

TIEMPO DE RESERVA
4 Horas

SELECCIONAR MÉTODO DE PAGO

Método Banco

Siguiente

PAGOS QR INICIO **RESERVAS** PERFIL

Nota: Elaboración propia.

Figura 39

Diseño de pantalla de subir comprobante de pago

← Subir Comprobante de Pago

EVIDENCIA DE PAGO

Seleccionar Archivo

Sube una imagen o PDF de tu comprobante

Subir Comprobante

Observaciones (Opcional)

Ej: Pago realizado via transferencia Banco Pichincha...

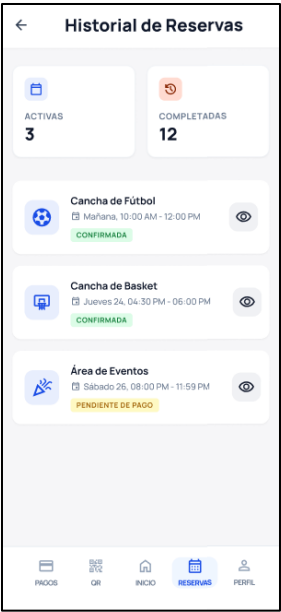
Enviar Comprobante

PAGOS QR INICIO **RESERVAS** PERFIL

Nota: Elaboración propia.

Figura 40

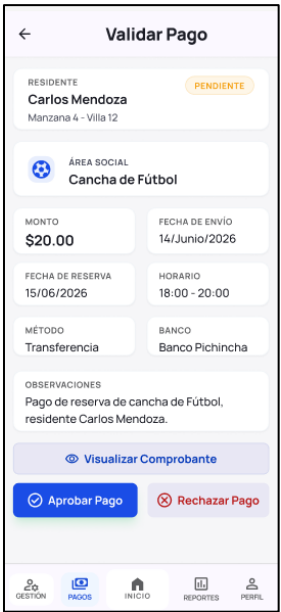
Diseño de pantalla de historial de reservas



Nota: Elaboración propia.

Figura 41

Diseño de pantalla de validación de pago de reserva

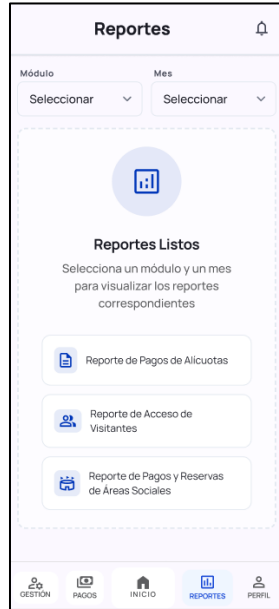


Nota: Elaboración propia.

- Prototipos del Módulo de Reportes

Figura 42

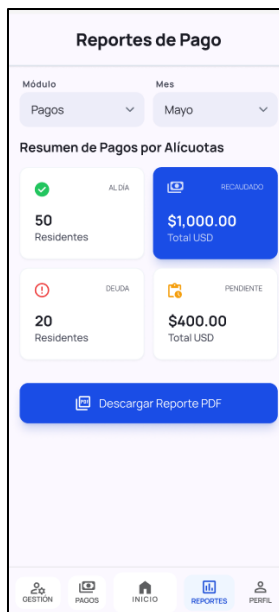
Diseño de pantalla de inicio de reportes



Nota: Elaboración propia.

Figura 43

Diseño de pantalla de reporte de pagos



Nota: Elaboración propia.

Figura 44

Diseño de pantalla de reporte de accesos



Nota: Elaboración propia.

Figura 45

Diseño de pantalla de reporte de reservas



Nota: Elaboración propia.

Figura 46

Reporte de usuarios

id	cedula	nombres	apellidos	correo	telefono	usuario	contraseña
1	0311e0f4e81-4a64-9561-ca929770e8...	Pedro	Ramirez	guardia@gmail.com	0985121325	guardia01	\$2b\$10\$K4c4Bk4t1Qd8IMpHQibui
2	03bdecf9-7edf-40a9-ab92-910ab8b51...	Jhonas	Lopez	vrjlonas@gmail.com	0909091020	maria011	\$2b\$10\$543z3e7EsefsaWQJ2EJK
3	08be77db-9830-42cb-9122-096e82955...	Carlos	Pérez	admin@urbanapp.com	0991234567	admin01	\$2b\$10\$8imylpLwGkPpEvs3pbgf
4	0fb9f85c-819f-49aa-6606-fbc0f51abec...	Gisel Maribel	Moreno Alvarado	gisell@gmail.com	0922334455	Gisel01	\$2b\$10\$mysa/G11VmrR8ZusTHZccf
5	125a1831-bf79-46f1-aaac-69c9c4c5c7...	Fernando	Salazar	fernando.residente@urbanapp...	0981234567	fernando_residente	\$2b\$10\$prQNOd.pFEMAHDeq/xWw
6	16c5ae69-8591-4c7d-9828-7fb36142f...	Luis	Castillo	luis.guardia@urbanapp.com	0976543210	luis_guardia	\$2b\$10\$QK40Fe21CddDgIme27W
7	1782d436-5210-44ad-833e-c3a6374d3...	Jhonas	Lopez	vrjlonas_lo@gmail.com	0978897931	Jhonaslp12	\$2b\$10\$1aAb5N89wmbu5AJddWl
8	283f071-d99f-4dee-bafe-50f662904e...	Beky	Piedra	bekypiedra@gmail.com	0978897928	Jhonaslp01	\$2b\$10\$WpghM60uhKQp.5a3eZzc
9	2e48894a-c726-4d3d-860b-80ed6d29f...	Jeremy	Panta	pruebapant@gmail.com	0920201010	guardia1205	\$2b\$10\$grgshQZW.JtuJ0tWGXK
10	2f182f53-913f-4255-a7be-14ba4ab923...	Ashley	Mero Villa	pruebapant@gmail.com	0978897935	Ashleym6	\$2b\$10\$JYvcC30.dHkK7B0u70ged
11	4bb8868c-0e29-4f6d-9209-045a4c4a6...	Roberto	Mendoza	roberto.admin@urbanapp.co...	0987654321	roberto_admin	\$2b\$10\$TL4tRBLD4BvC7ic1.DPug
12	4daa1ccc-36ca-47e6-9951-34a4f1956...	Jhonas Jeremy	Lopez Panta	jlonas@gmail.com	0978897935	admin02	\$2b\$10\$BfconV8BaFCQ8LM/aW
13	5670d6f5-b3ce-40b2-99ef-039ec778a...	Rafael Alonso	Lopez Mero	Rafa@gmail.com	0988698684	Rafael01	\$2b\$10\$ePBDWQDzSfKc8bLY2mf
14	65e9e9fd-671b-4166-afa3-ec1a8c70ff9f	Jhonas	Lopez	jlopez@gmail.com	0910201020	guardia0101	\$2b\$10\$FAFGQvchHhNp5/V7ZUC
15	6c766e6a-7683-4d46-50bf-5497a95c5...	Josimar	Lopez	josimar@gmail.com	0910159191	guardia0100	\$2b\$10\$8KXhR/ywV423SMItq1Iaq
16	739412aa-489d-49a6-b2da-fae4453be...	Valeria	Rios	valeria.residente@urbanapp...	0992345678	valeria_residente	\$2b\$10\$Hbtp0UJ2cMhTu5roAp.D

Nota: Elaboración propia.

Historia Técnica 2

IPB02 Diseño del modelo entidad - relación de la base de datos

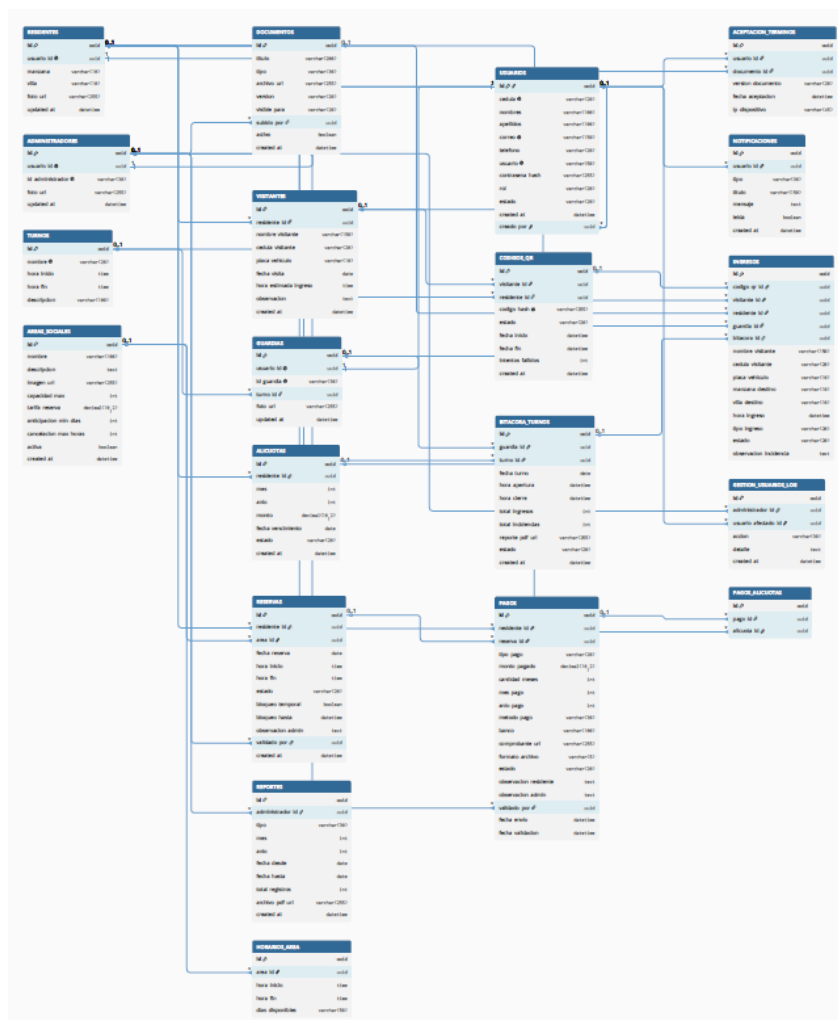
Tarea 2. Diseñar el DER detallando las entidades y relaciones de la aplicación móvil.

En esta tarea se diseñó el modelo entidad-relación (ER) de la base de datos, con el propósito de definir la estructura lógica que soportará el funcionamiento de la aplicación móvil. Para ello, se identificaron las entidades, sus atributos y las relaciones existentes entre ellas, garantizando una organización de la información acorde con los requerimientos funcionales previamente establecidos.

Para la elaboración del diagrama entidad-relación se utilizó dbdiagram.io, como se observa en la *Figura 47*, herramienta que permitió modelar visualmente la base de datos y facilitar posteriormente su implementación e integración con PostgreSQL.

Figura 47

Diagrama Entidad–Relación de la base de datos



Nota: La figura presenta el modelo de datos de la aplicación, conformado por las principales entidades del sistema y las relaciones existentes entre ellas. Elaboración propia.

Historia Técnica 3

IPB03 Diseño de la arquitectura de aplicación móvil y elección de tecnologías

Tarea 3. Seleccionar las tecnologías para el desarrollo del Frontend, backend y base de datos

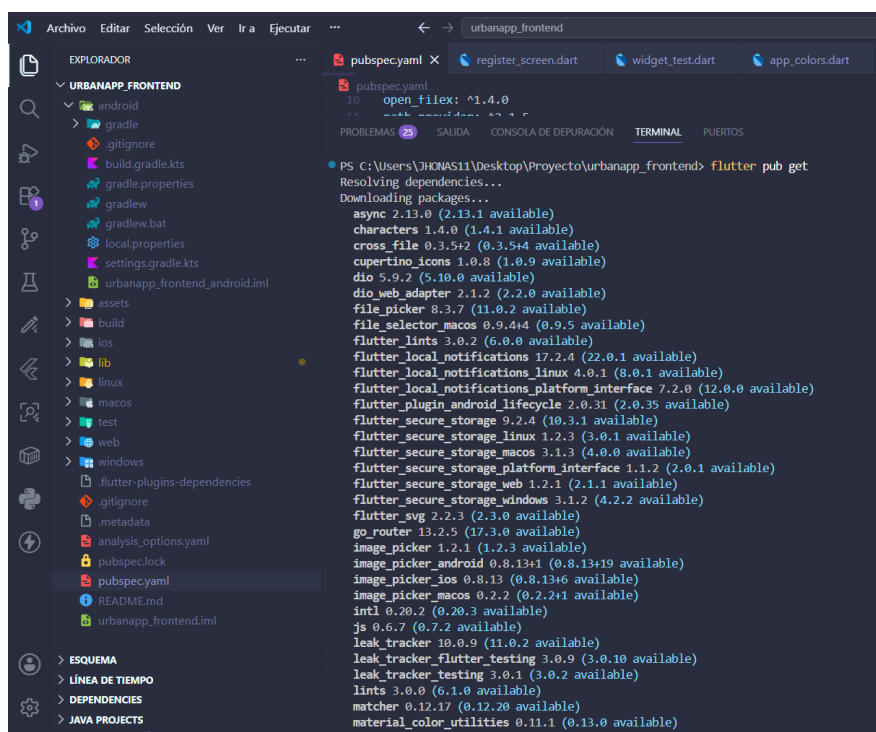
En esta tarea se seleccionaron las tecnologías que conforman la arquitectura tecnológica del sistema, considerando los requerimientos funcionales y no funcionales del proyecto. Se evaluaron alternativas para el desarrollo del frontend, backend y la base de datos, priorizando

aspectos como escalabilidad, rendimiento, mantenibilidad, compatibilidad e integración entre los diferentes componentes. Como resultado, se definió el uso de las tecnologías que servirán como base para la implementación de la solución y el desarrollo de las funcionalidades del sistema. A continuación, se mencionan las tecnologías seleccionadas:

- **Frontend:** El frontend de la aplicación móvil fue desarrollado con Flutter utilizando el lenguaje Dart, ilustrada en la *Figura 48*. Esta tecnología fue seleccionada por su capacidad de desarrollar aplicaciones multiplataforma a partir de un único código fuente, facilitando la integración con el backend mediante servicios REST y reduciendo los tiempos de desarrollo.

Figura 48

Instalación de dependencias del frontend: Flutter

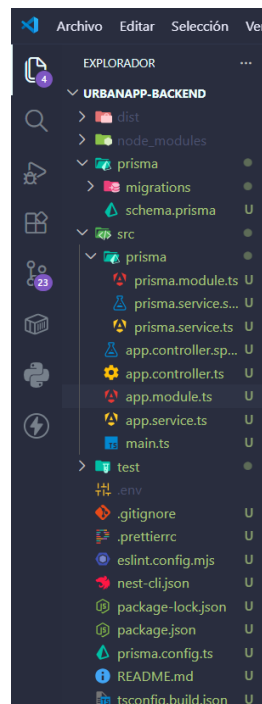


Nota: La figura presenta la instalación de las dependencias del frontend en Flutter del proyecto. Elaboración propia.

- **Backend:** El backend del sistema fue desarrollado utilizando Node.js como entorno de ejecución, mientras que la API REST se implementó con el framework NestJS, como se observa en la *Figura 49*, empleando el lenguaje de programación TypeScript. Esta combinación tecnológica fue seleccionada por ofrecer una arquitectura modular, escalable y de alto rendimiento, facilitando la organización del código mediante controladores, servicios y módulos. Además, el uso de TypeScript proporciona un sistema de tipo estático que mejora la calidad del código, reduce errores durante el desarrollo y simplifica el mantenimiento de la aplicación. La API REST actúa como intermediaria entre el frontend desarrollado en Flutter y la base de datos PostgreSQL, permitiendo una comunicación segura, eficiente y estructurada entre los diferentes componentes del sistema.

Figura 49

Estructura del proyecto en VS Code

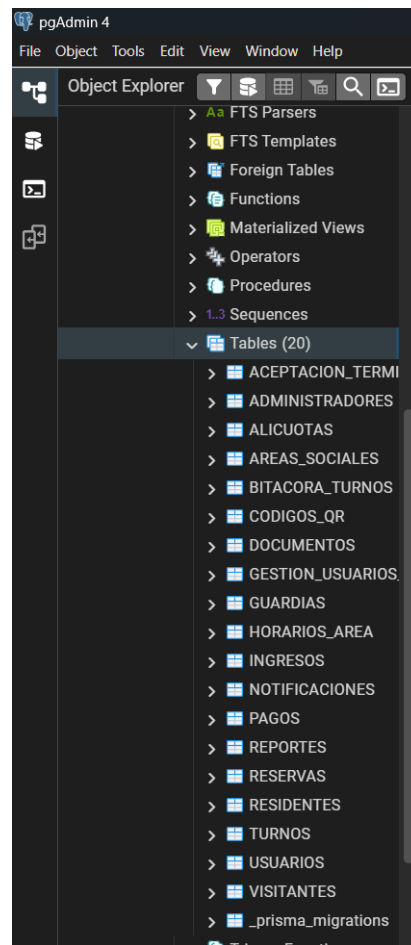


Nota: Inicialización del proyecto backend con NestJS, framework de Node.js basado en TypeScript. Elaboración propia.

- **Gestor de la base de datos:** Como sistema gestor de bases de datos se seleccionó PostgreSQL, como se observa en la *Figura 50*, debido a su robustez, alto rendimiento y capacidad para gestionar grandes volúmenes de información de manera eficiente.

Figura 50

Creación de la base de datos



Nota: Migración exitosa del esquema de base de datos mediante Prisma ORM. Elaboración propia.

Tarea 4. Diseñar la estructura base del proyecto.

En esta tarea se definió y organizó la estructura fundamental del proyecto, estableciendo la arquitectura inicial, representada en la *Figura 51*, que permitirá un desarrollo ordenado y escalable. Se creó la base del repositorio del sistema, definiendo la distribución de carpetas para el frontend,

backend y otros módulos complementarios, así como la separación de responsabilidades dentro de la aplicación.

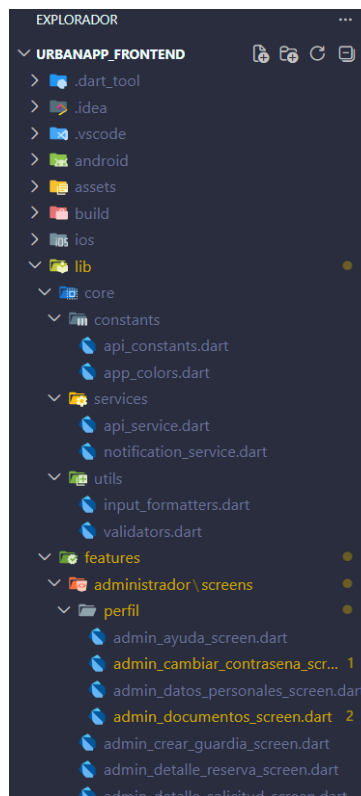
Figura 51

Arquitectura de tres capas de la aplicación móvil propuesta



Nota: La figura muestra la arquitectura de tres capas propuesta para el sistema, compuesta por la capa de presentación en Flutter, la capa de aplicación en NestJS y la capa de datos basada en PostgreSQL. Elaboración propia.

Además, se establecieron los patrones de arquitectura a utilizar, la configuración inicial del entorno de desarrollo y las convenciones de codificación que guiarán al equipo durante todo el ciclo del proyecto, ilustrada en la *Figura 52*.

Figura 52*Estructura de carpetas del proyecto Flutter*

Nota: Implementación del proyecto en Flutter. Elaboración propia

Tarea 5. Configurar las dependencias iniciales de la aplicación móvil.

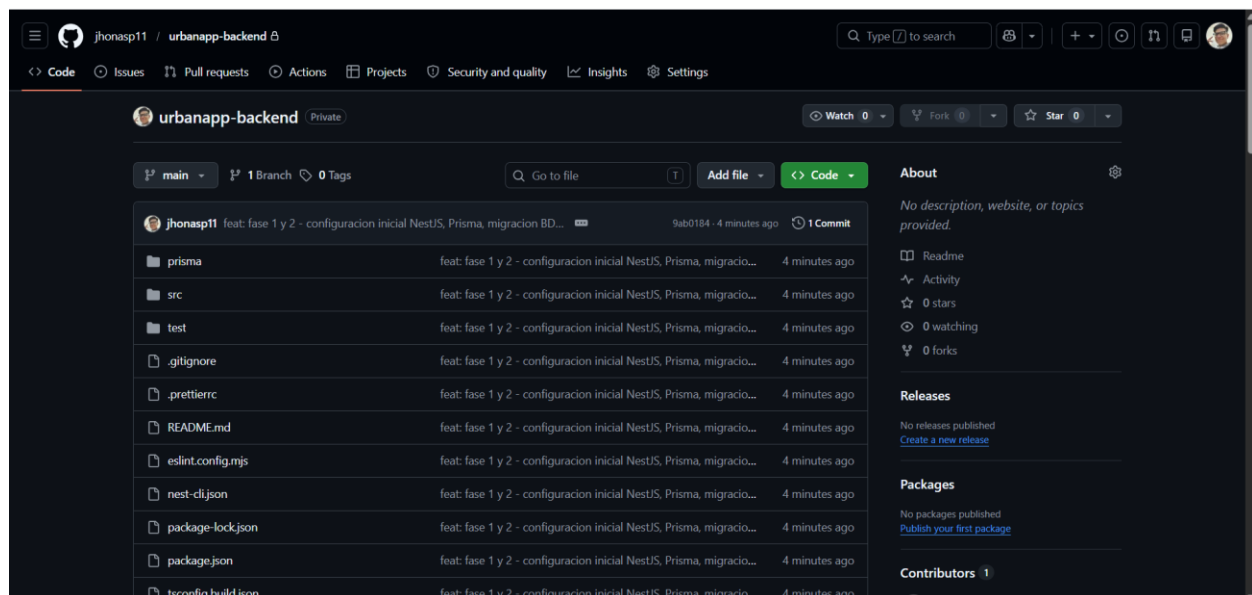
En esta tarea se configuraron las dependencias iniciales de la aplicación móvil para establecer el entorno de desarrollo necesario y garantizar el correcto funcionamiento del proyecto. Se incorporaron las librerías y paquetes esenciales requeridos por Flutter, incluyendo aquellas destinadas a la gestión de estado, navegación, consumo de servicios web, manejo de almacenamiento local y otras funcionalidades básicas de la aplicación. Además, se verificó la compatibilidad entre las dependencias, se configuró el archivo de gestión de paquetes y se preparó la base tecnológica que permitirá integrar nuevos módulos y funcionalidades durante las siguientes iteraciones del desarrollo.

Tarea 6. Configurar el entorno y repositorios para el frontend y backend.

En esta tarea se configuró el entorno de desarrollo para el frontend y el backend del sistema, preparando las herramientas y recursos necesarios para iniciar la implementación de la solución. Se crearon y configuraron los repositorios de control de versiones, como se observa en las Figuras 53, 54, 55, estableciendo la estructura inicial de cada proyecto y las ramas principales para la gestión del código fuente. También, se verificó la correcta instalación de las dependencias, los entornos de ejecución y las configuraciones necesarias para el desarrollo del frontend en Flutter y del backend en Node.js con NestJS, garantizando un flujo de trabajo organizado, colaborativo y orientado a las buenas prácticas de desarrollo de software.

Figura 53

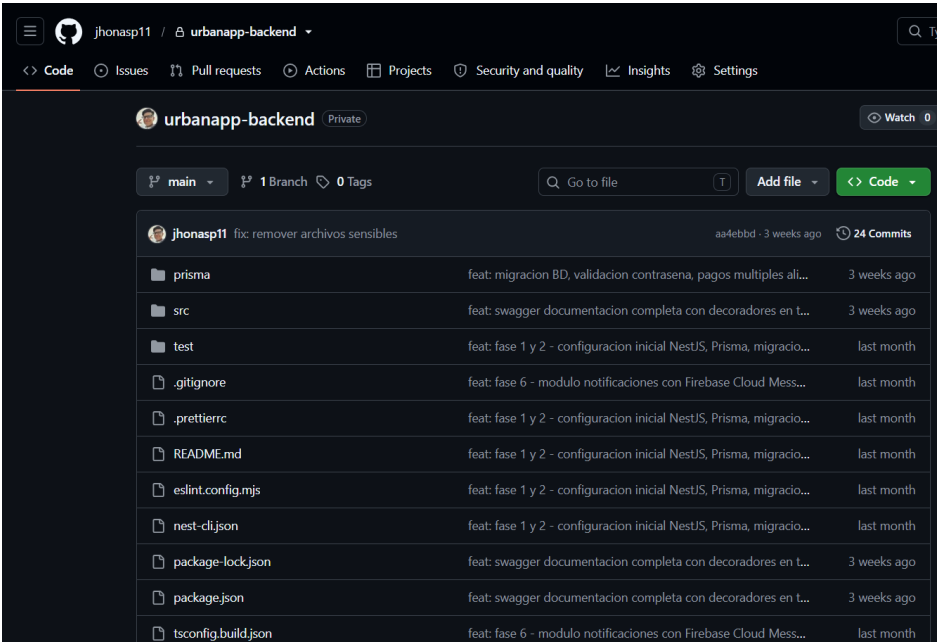
Repositorio Git con el commit inicial



Nota: En la Figura se presenta el primer commit consolida las fases 1 y 2 del desarrollo incluyendo la configuración del servidor, esquema de base de datos y módulo de autenticación. Elaboración propia.

Figura 54

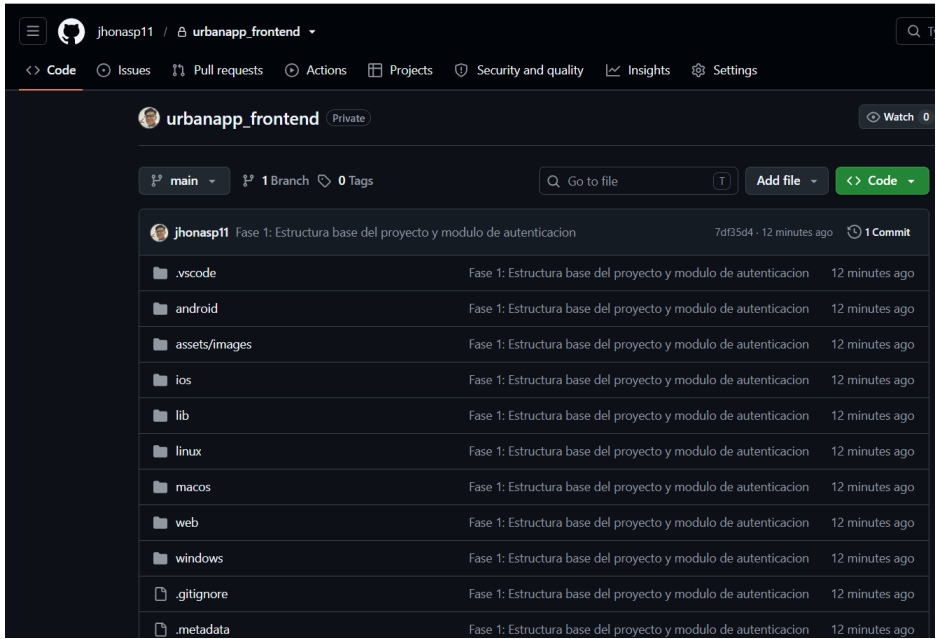
Repositorio del frontend



Nota: La figura muestra el repositorio del frontend del proyecto Elaboración propia.

Figura 55

Repositorio del backend



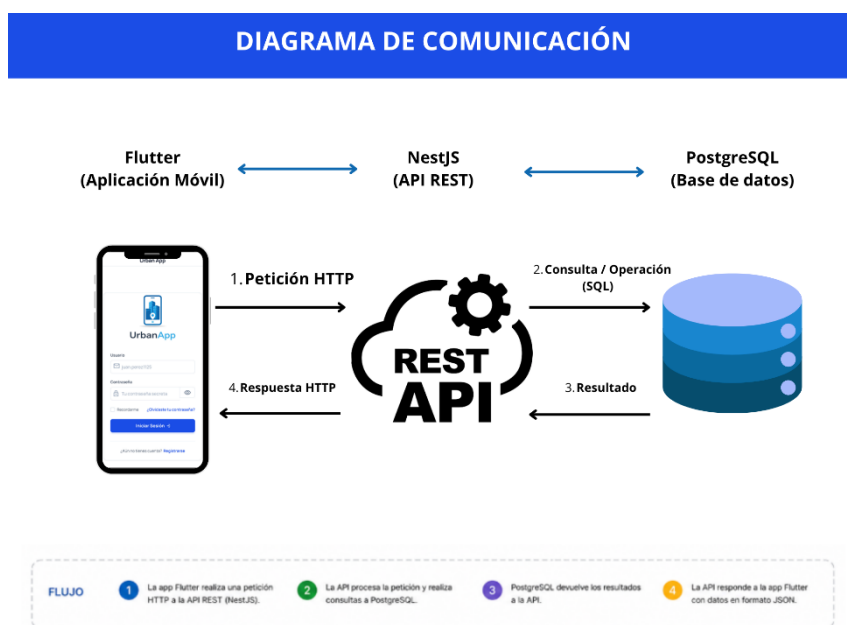
Nota: La figura muestra el repositorio del backend del proyecto Elaboración propia.

Tarea 7. Diseñar la estrategia de comunicación con la API del backend.

En esta tarea se diseñó la estrategia de comunicación entre la aplicación móvil y el backend, definiendo el mecanismo mediante el cual el frontend intercambiará información con la API REST, como se observa en la *Figura 56*.

Figura 56

Diagrama de comunicación entre la aplicación móvil, la API REST y la base de datos

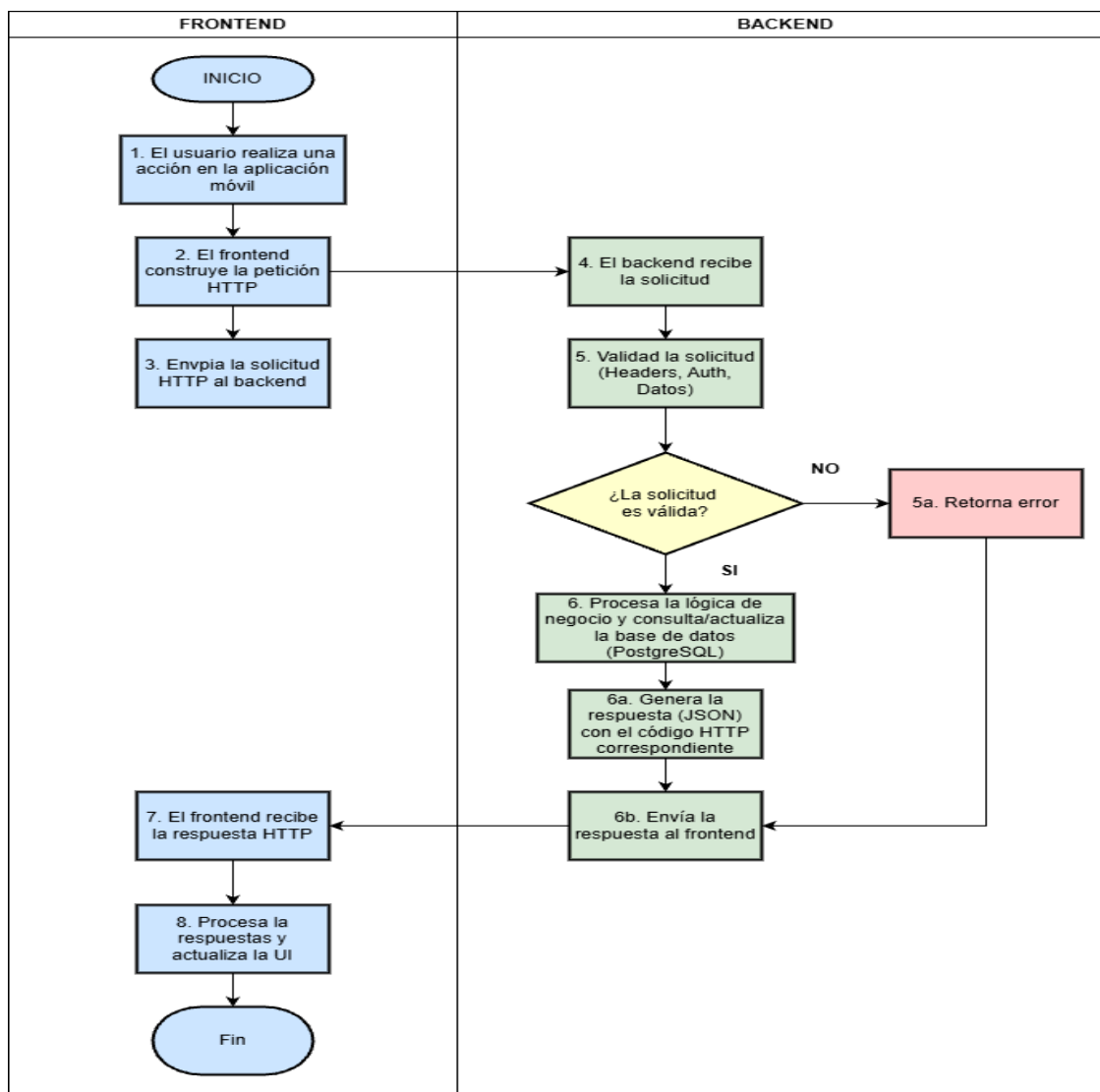


Nota: En la figura se representa la comunicación que hay entre la aplicación móvil, la API REST y la base de datos. Elaboración propia.

En la *Figura 57* se muestran los protocolos de comunicación que se establecieron, así como el formato de intercambio de datos mediante JSON, la estructura de los endpoints, los métodos HTTP a utilizar (GET, POST, PUT y DELETE) y el manejo de las respuestas y códigos de estado. Además, se definió la estrategia de autenticación y autorización para garantizar un acceso seguro a los recursos del sistema, así como los lineamientos para el manejo de errores y la integración de los diferentes módulos de la aplicación con los servicios del backend.

Figura 57

Diagrama de flujo del proceso de solicitud y respuesta entre el frontend y el backend



Nota: En la Figura se presenta diagrama de flujo del proceso de solicitud y respuesta entre el frontend y el backend. Elaboración propia.

Retroalimentación de Sprint 0

El Sprint 0 se ejecutó con el objetivo de establecer las bases funcionales y técnicas necesarias para el desarrollo de la aplicación móvil. Durante esta iteración se completaron

satisfactoriamente todas las actividades planificadas, permitiendo contar con una estructura sólida para iniciar el desarrollo de los módulos del sistema.

En primer lugar, se elaboró el prototipo de interfaces en Figma, lo que permitió definir la experiencia de usuario y validar la distribución de las pantallas conforme a los requerimientos obtenidos en el levantamiento de información. Por otro lado, se diseñó el Diagrama Entidad-Relación (DER), estableciendo la estructura lógica de la base de datos y las relaciones entre las entidades principales del sistema.

Posteriormente, se realizó la selección de las tecnologías que conforman la solución, definiendo Flutter para el desarrollo del frontend, Node.js con NestJS y TypeScript para el backend, Prisma ORM como herramienta de acceso a datos y PostgreSQL como sistema gestor de base de datos. De igual manera, se diseñó la estructura base del proyecto, organizando los componentes y módulos que facilitarán un desarrollo ordenado y escalable.

Finalmente, se configuraron las dependencias iniciales de la aplicación móvil, el entorno de desarrollo y los repositorios para el frontend y el backend, garantizando un flujo de trabajo colaborativo mediante el control de versiones. Además, se definió la estrategia de comunicación entre la aplicación móvil y la API REST, estableciendo los lineamientos para el intercambio seguro de información entre los diferentes componentes del sistema.

Como resultado, el Sprint 0 culminó con el 100 % de las tareas planificadas completadas, proporcionando una base tecnológica y metodológica sólida para el desarrollo de los siguientes sprints. La correcta planificación y configuración inicial redujo riesgos técnicos, facilitó la

organización del proyecto y permitió que el equipo iniciara la etapa de implementación con una arquitectura bien definida y un entorno de desarrollo completamente operativo.

Sprint 1. Implementación de la infraestructura funcional y autenticación del sistema.

Tabla 60

Sprint Planning 1

SPRINT 1				
ID de Product Backlog	Historia de usuario	Tarea	Responsable	Estado
IPB01	Crear cuenta de usuario	1. Diseñar la interfaz de registro de usuario e implementar el formulario de registro. 2. Desarrollar el servicio de registro en la API REST e implementar la persistencia de usuarios en la base de datos. 3. Desarrollo de componentes CRUD	Jhonas López, Belky Piedra	Completado
IPB02	Iniciar sesión mediante credenciales	4. Diseñar e implementar la interfaz y el formulario de inicio de sesión. 5. Consumo del endpoint del login 6. Implementar la generación de tokens 7. Implementar la eliminación del token de autenticación y el cierre de la sesión del usuario	Jhonas López, Belky Piedra	Completado
IPB03	Recuperar contraseña	8. Desarrollar los servicios de recuperación y actualización de contraseña en la API REST. 9. Integrar la aplicación móvil con la API para el proceso de recuperación de contraseña.	Jhonas López, Belky Piedra	Completado

Nota: La tabla detalla las historias de usuario, tareas, responsables y estado del Sprint 1 para el progreso y desarrollo del proyecto. Elaboración propia.

Historia de usuario 1

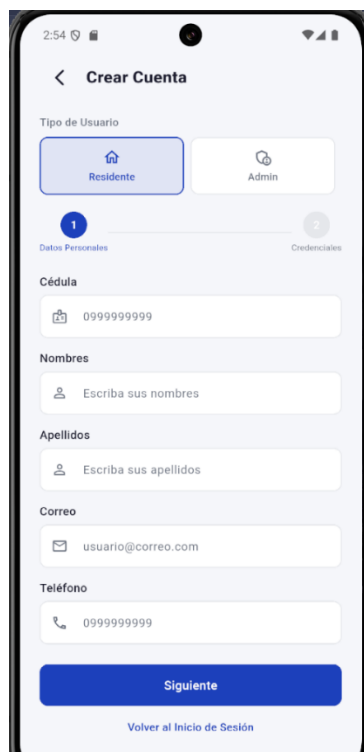
IPB01 Crear cuenta de usuario

Tarea 1. Diseñar la interfaz de registro de usuario e implementar el formulario de registro.

En esta tarea se diseñó e implementó la interfaz de registro de usuarios dentro de la aplicación móvil. Se definió la estructura de la pantalla conforme a los requerimientos del sistema y al prototipo establecido, garantizando una experiencia de usuario intuitiva y coherente con el diseño general de la aplicación.

Figura 58

Interfaz implementada en Flutter en el emulador



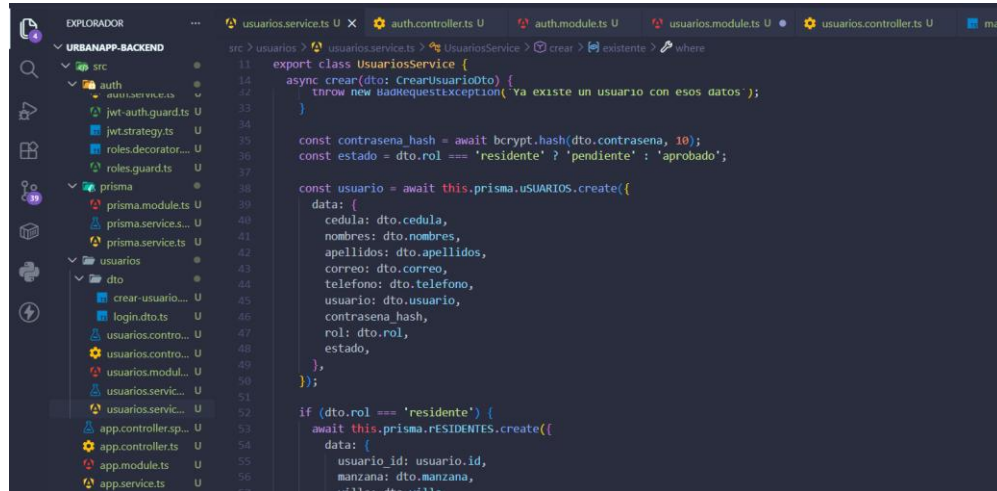
Nota: Interfaz de crear cuenta en la aplicación. Elaboración propia.

También, se desarrolló el formulario de registro en Flutter, como se observa en la *Figura 59*, incorporando los campos necesarios para el ingreso de información del usuario, como datos personales, credenciales de acceso y rol asignado. Finalmente, se integraron validaciones básicas

en el formulario para asegurar la correcta entrada de datos y mejorar la calidad de la información registrada en el sistema.

Figura 59

Código fuente del formulario de registro



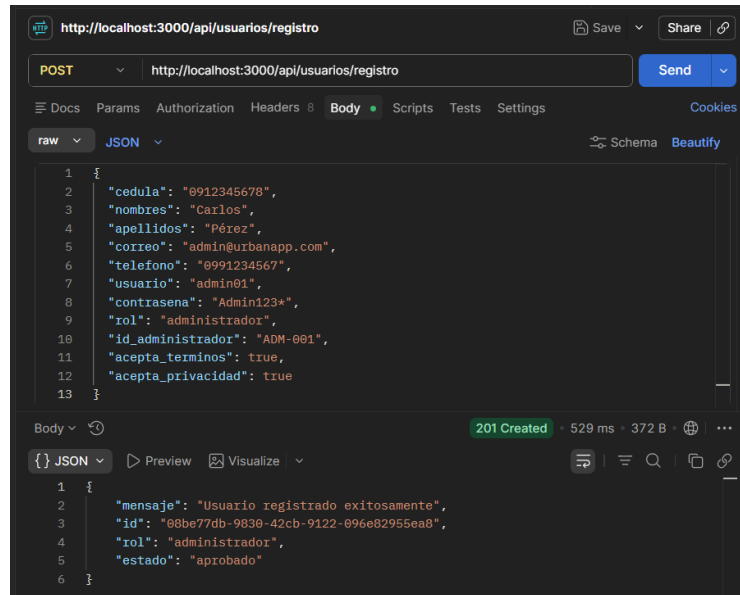
Nota: La figura presenta un fragmento de código fuente del formulario de registro de usuario implementado en la aplicación móvil. Elaboración propia.

Tarea 2. Desarrollar el servicio de registro en la API REST e implementar la persistencia de usuarios en la base de datos.

En esta tarea se desarrolló el servicio de registro de usuarios en la API REST utilizando NestJS, tal como se ilustra en la *Figura 60*, implementando el endpoint encargado de recibir y procesar la información enviada desde la aplicación móvil.

Figura 60

Código del servicio de registro (NestJS Controller/Service)

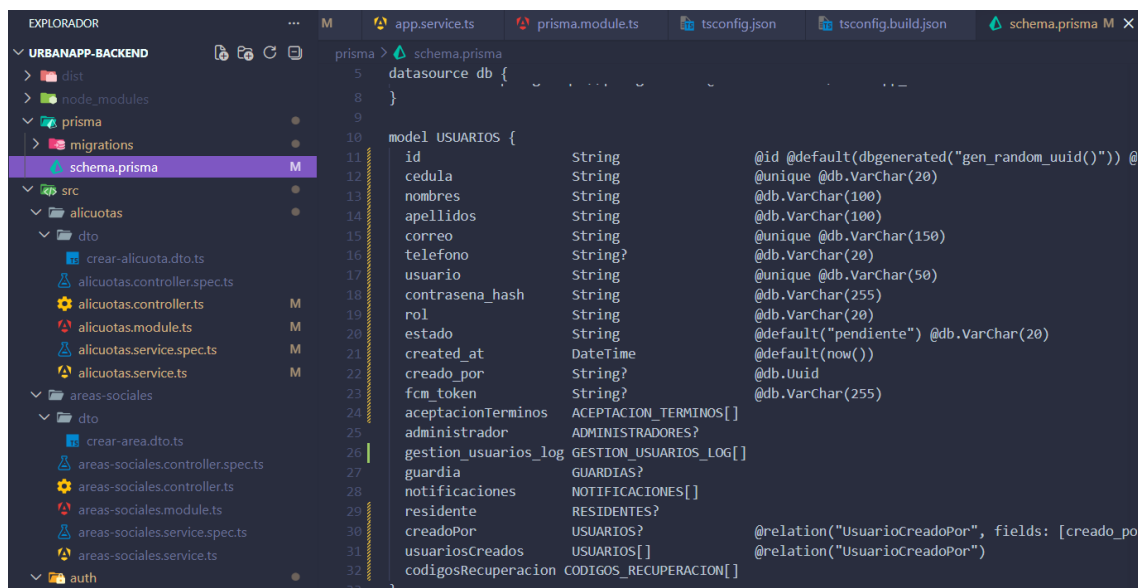


Nota: La figura representa la evidencia de la creación del endpoint de registro de usuarios en NestJS, encargado de procesar las solicitudes HTTP tipo POST. Elaboración propia.

Se definió la lógica de negocio para la creación de nuevos usuarios, aplicando validaciones necesarias para garantizar la integridad de los datos. Por otro lado, implementó la persistencia de usuarios en la base de datos mediante Prisma ORM y PostgreSQL, puede observarse en la *Figura 61, 62*, permitiendo el almacenamiento estructurado de la información registrada. Finalmente, se verificó la correcta comunicación entre la API y la base de datos, asegurando el registro exitoso y consistente de los usuarios.

Figura 61

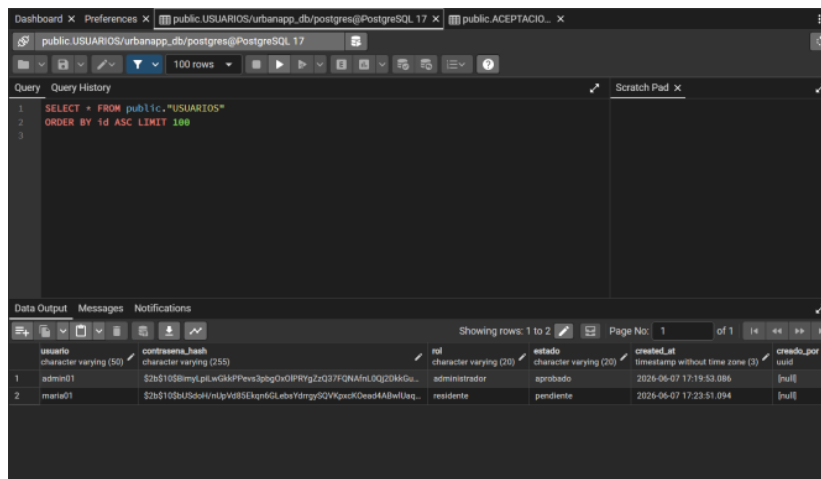
Modelo de usuario en Prisma



Nota: La figura muestra el modelo de usuario implementado en Prisma ORM. Elaboración propia.

Figura 62

Registros almacenados en PostgreSQL



Nota: La figura muestra los datos de usuarios almacenados en la base de datos PostgreSQL tras el proceso de registro en el sistema. Elaboración propia.

Tarea 3. Desarrollo de componentes CRUD

En esta tarea se desarrollaron los componentes CRUD (Crear, Leer, Actualizar y Eliminar) correspondientes a las entidades principales del sistema, ilustrado en la *Figura 63*. Se definieron

los servicios y controladores necesarios para permitir la manipulación de datos de forma estructurada y segura. También, se establecieron las reglas de validación para garantizar la integridad de la información almacenada en PostgreSQL.

Figura 63

Código de controladores y servicios CRUD en NestJS

```

src > usuarios > usuarios.controller.ts > UsuariosController > aprobarRechazar
22 import { RolesGuard } from '../auth/roles.guard';
23 import { Roles } from '../auth/roles.decorator';
24
25 @ApiTags('Usuarios')
26 @Controller('usuarios')
27 export class UsuariosController {
28   constructor(private readonly usuariosService: UsuariosService) {}
29
30   @ApiOperation({
31     summary: 'Registrar nuevo usuario (residente, admin o guardia)',
32   })
33   @ApiResponse({ status: 201, description: 'Usuario registrado exitosamente' })
34   @ApiResponse({
35     status: 400,
36     description: 'Datos invalidos o usuario ya existe',
37   })
38   @Post('registro')
39   registro(@Body() dto: CrearUsuarioDto) {
40     return this.usuariosService.crear(dto);
41   }
42
43   @ApiOperation({ summary: 'Listar residentes pendientes de aprobacion' })
44   @ApiResponse({ status: 200, description: 'Lista de residentes pendientes' })
45   @ApiBearerAuth('JWT-auth')
46   @UseGuards(JwtAuthGuard, RolesGuard)
47   @Roles('administrador')
48   @Get('residentes/pendientes')
49   listarPendientes() {
50     return this.usuariosService.listarResidentesPendientes();
51   }
52 }

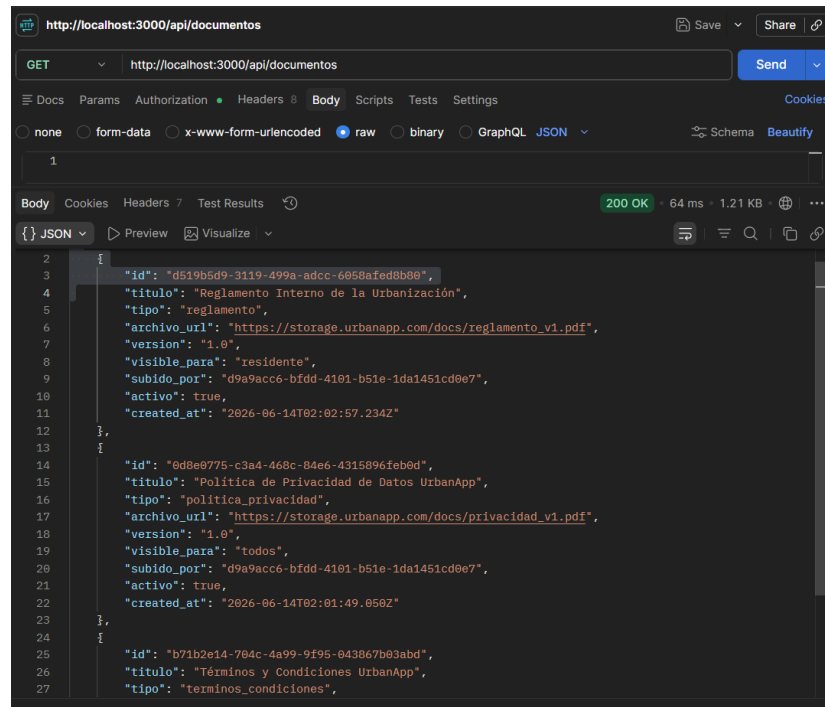
```

Nota: La figura muestra un fragmento del código fuente de los controladores y servicios. Elaboración propia.

Finalmente, se verificó el correcto funcionamiento de las operaciones CRUD mediante pruebas funcionales de cada uno de los endpoints implementados, como se observa en la Figura 64.

Figura 64

Pruebas en Postman (GET, POST, PUT, DELETE)



Nota: La figura muestra Pruebas get, post, put, delete en Postman. Elaboración propia.

Historia de usuario 2

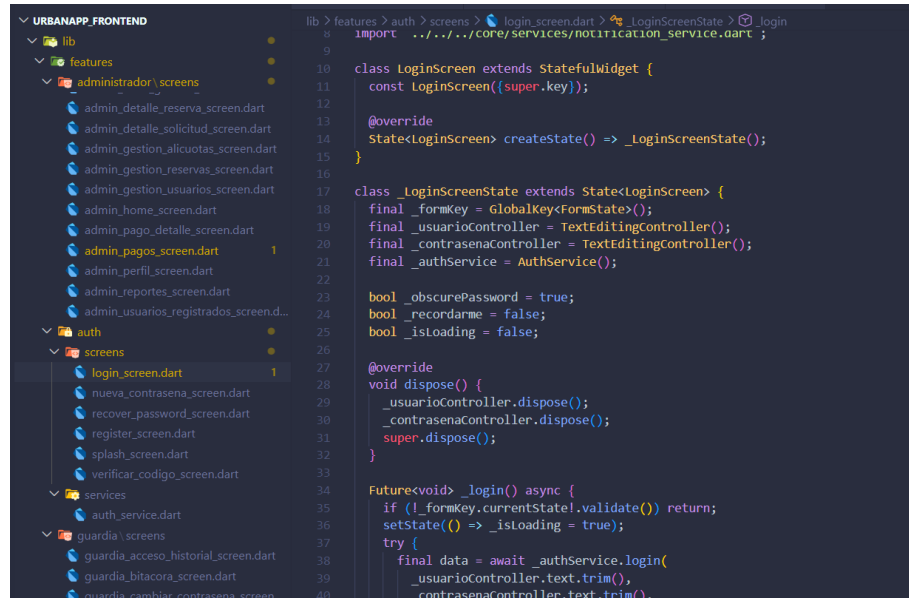
IPB02 Iniciar sesión mediante credenciales

Tarea 4. Diseñar e implementar la interfaz y el formulario de inicio de sesión

En esta tarea se diseñó e implementó la interfaz de inicio de sesión dentro de la aplicación móvil desarrollada en Flutter, como se aprecia en la *Figura 7*. Se estableció la estructura de la pantalla de autenticación de acuerdo con los requerimientos del sistema y el prototipo previamente definido, garantizando una experiencia de usuario intuitiva y coherente con el diseño general de la aplicación. De igual forma, se desarrolló el formulario de inicio de sesión, incorporando los campos necesarios para el ingreso de credenciales del usuario, como correo electrónico y contraseña. Finalmente, se implementaron validaciones básicas en el formulario para asegurar la correcta entrada de datos y mejorar la seguridad y calidad de la información ingresada.

Figura 65

Código del formulario de login en Flutter



Nota: La figura muestra un fragmento de código fuente del formulario del Login. Elaboración propia.

La tarea fue ejecutada satisfactoriamente, logrando el diseño e implementación de la interfaz y el formulario de inicio de sesión en la aplicación móvil, como se aprecia en la *Figura 66*. Esto permitió establecer el punto de acceso principal al sistema, garantizando una experiencia de usuario adecuada y el ingreso seguro de credenciales dentro de la aplicación.

Figura 66

Validaciones en ejecución

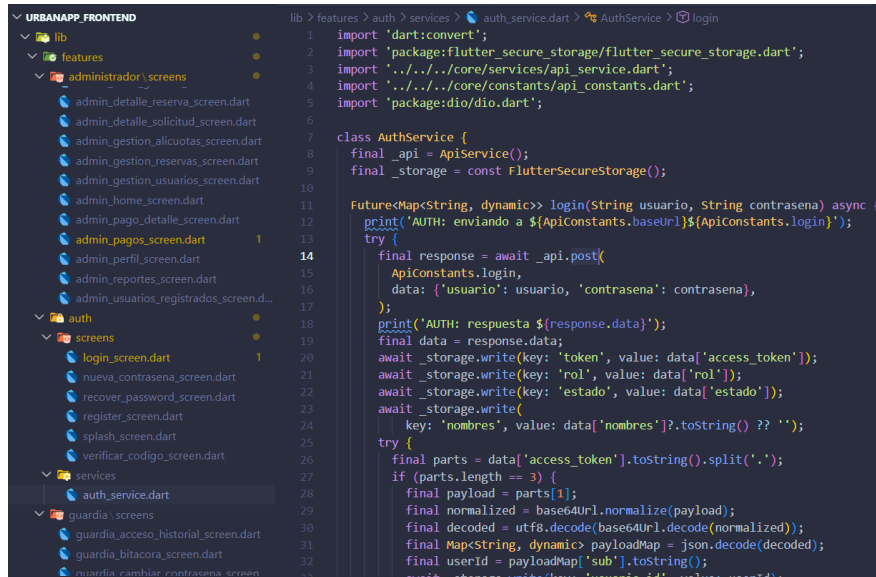
Nota: La figura muestra la validación de ingresar un correo y teléfono válido. Elaboración propia.

Tarea 5. Consumo del endpoint del login

En esta tarea se implementó el consumo del endpoint de inicio de sesión en la aplicación móvil. Se estableció la comunicación entre el frontend y la API REST, como se observa en la *Figura 67*, permitiendo el envío de las credenciales del usuario hacia el servicio de autenticación desarrollado en NestJS.

Figura 67

Código de consumo del endpoint de login



Nota: La figura evidencia la integración del frontend en Flutter con el endpoint de autenticación del backend. Elaboración propia.

Así mismo, se gestionó la recepción de la respuesta del servidor, la cual incluye el resultado de la autenticación y el token de acceso en caso de credenciales válidas. Finalmente, se integró la lógica necesaria para el manejo de sesiones dentro de la aplicación, garantizando una autenticación segura y funcional.

Figura 68

Estructura de la petición de credenciales

Nota: La figura muestra que se realizó el envío de las credenciales del usuario hacia la API REST mediante solicitudes HTTP. Elaboración propia.

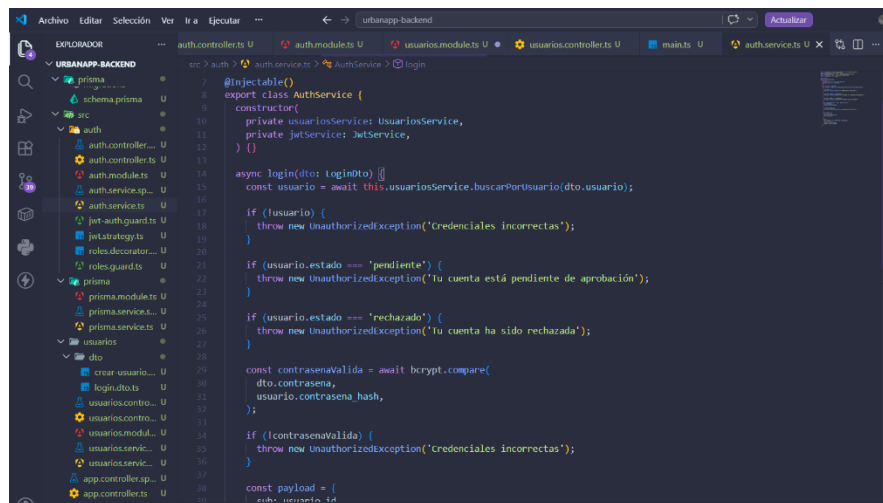
La tarea fue ejecutada satisfactoriamente, logrando la integración entre la aplicación móvil y el servicio de autenticación del backend. Esto permitió el envío de credenciales, la validación de usuarios y la recepción del token de acceso, asegurando un proceso de inicio de sesión funcional y seguro dentro del sistema.

Tarea 6. Implementar la generación de tokens

En esta tarea se implementó la generación de tokens de autenticación dentro del sistema, ilustrado en la *Figura 69*, utilizando JSON Web Token (JWT) en la API REST. Se configuró el proceso de generación de tokens como parte del flujo de inicio de sesión, permitiendo que los usuarios autenticados reciban un token seguro para el acceso a los recursos protegidos del sistema.

Figura 69

Código de generación de JWT

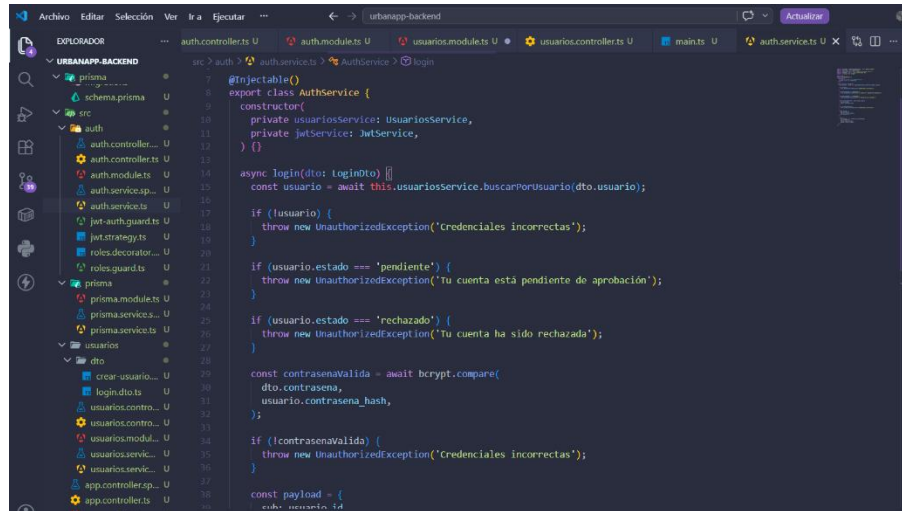


Nota: La figura evidencia la configuración del módulo de autenticación y la generación de tokens en NestJS. Elaboración propia.

En este mismo sentido, se definieron los datos incluidos en el payload del token, como el identificador del usuario y su rol, garantizando un control de acceso basado en permisos, como se observa en la *Figura 70*.

Figura 70

Estructura del payload JWT en el código

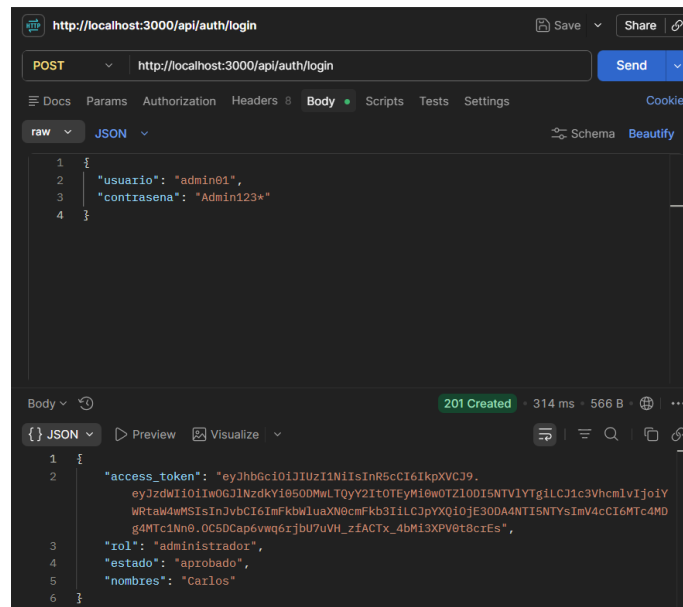


Nota: La figura muestra la definición los datos incluidos en el token, como ID del usuario y rol asignado. Elaboración propia.

Finalmente, se estableció el tiempo de expiración del token para mejorar la seguridad de la aplicación. La tarea fue ejecutada con éxito, logrando la implementación de la generación de tokens de autenticación mediante JWT, representado en la *Figura 71*. Esto permitió asegurar el acceso a los recursos protegidos del sistema, estableciendo un mecanismo de autenticación seguro y basado en roles dentro de la aplicación.

Figura 71

Respuesta de la API con token JWT



Nota: La figura muestra la correcta generación del token al iniciar sesión en el sistema. Elaboración propia.

Tarea 7. Implementar la eliminación del token de autenticación y el cierre de la sesión del usuario

En esta tarea se implementó el proceso de cierre de sesión dentro de la aplicación móvil desarrollada en Flutter, el cual consiste en la eliminación del token de autenticación almacenado en el dispositivo del usuario, como se observa en la *Figura 72*.

Figura 72

Eliminación de token

Nota: La figura muestra la implementación de la lógica para eliminar el token JWT almacenado en el dispositivo del usuario. Elaboración propia.

Se estableció la lógica necesaria para finalizar la sesión activa de manera segura, garantizando que el usuario pierda el acceso a las funcionalidades protegidas del sistema al cerrar sesión, representada en la *Figura 73*. Además, se implementó la redirección del usuario hacia la pantalla de inicio de sesión una vez ejecutado el proceso de logout, asegurando un flujo correcto dentro de la aplicación.

Figura 73

Código de la función logout en Flutter

Nota: La figura muestra el desarrollo de la funcionalidad que permite finalizar la sesión activa del usuario dentro de la aplicación. Elaboración propia.

La tarea fue ejecutada favorablemente, logrando la implementación del cierre de sesión mediante la eliminación del token de autenticación del dispositivo, como se observa en la *Figura 74*. Esto permitió garantizar la seguridad del sistema, evitando accesos no autorizados y asegurando un flujo adecuado de autenticación dentro de la aplicación.

Figura 74

Ejecución del cierre de sesión en la aplicación móvil

Nota: La figura la verificación del correcto funcionamiento del proceso logout, validando la eliminación del acceso a rutas protegidas. Elaboración propia.

Historia de usuario 3***IPB03 Recuperar contraseña***

Tarea 8. Desarrollar los servicios de recuperación y actualización de contraseña en la API REST

Durante esta tarea se desarrollaron e integraron los servicios de recuperación y actualización de contraseñas en la API REST, como se muestra en la *Figura 75*, fortaleciendo el módulo de autenticación para residentes y administradores.

El proceso se alineó con la arquitectura de tres capas del proyecto, distribuyendo las responsabilidades entre controladores, servicios y repositorios en la base de datos PostgreSQL.

Las acciones clave ejecutadas incluyeron:

Figura 75

Código fuente del servicio de recuperación y actualización de contraseña

Nota: La figura muestra la evidencia de la implementación de la lógica del proceso de recuperación y actualización de contraseña. Elaboración propia.

- **Lógica de validación y flujo:** El servicio verificó la existencia del usuario en el sistema para autorizar el restablecimiento de la credencial, permitiendo que el endpoint de actualización procesara el cambio de manera controlada.
- **Seguridad y cifrado:** Se aplicó una función criptográfica para cifrar la nueva contraseña antes de almacenarla, evitando el registro de datos sensibles en texto plano dentro de la base de datos.
- **Control de errores:** Se configuró el manejo de excepciones nativo de NestJS para emitir respuestas HTTP consistentes ante datos incorrectos o usuarios inexistentes, asegurando una integración fluida con el frontend en Flutter.
- **Pruebas funcionales:** Se evaluaron los endpoints mediante herramientas de consumo REST, validando el comportamiento correcto del sistema tanto en escenarios de éxito como en casos de error.

Esta implementación garantizó un mecanismo seguro y autónomo para la gestión de accesos, preservando la separación de responsabilidades arquitectónicas para futuros mantenimientos, como se muestra en la *Figura 76*.

Figura 76

Contraseña actualizada en la base de datos PostgreSQL

Nota: La figura demuestra que el cambio se reflejó correctamente en la base de datos y que la contraseña se almacena cifrada. Elaboración propia.

Tarea 9. Integrar la aplicación móvil con la API para el proceso de recuperación de contraseña

En esta tarea se integró esta funcionalidad con la aplicación móvil, como se muestra en la *Figura 77*. Para ello, se configuró el consumo de los endpoints mediante solicitudes HTTP, permitiendo la comunicación entre el frontend y el backend desarrollado con NestJS.

Figura 77

Código fuente del servicio de recuperación de contraseña en Flutter

Nota: La figura demuestra la integración con la API. Elaboración propia.

Posteriormente, se implementó la interfaz de recuperación de contraseña, incorporando un formulario con validaciones básicas para verificar la información ingresada antes de enviarla al servidor, como se muestra en la *Figura 78*. Una vez procesada la solicitud, la aplicación interpreta la respuesta de la API y muestra al usuario el resultado de la operación mediante mensajes de confirmación o de error, según corresponda.

Figura 78

Interfaz de recuperación de contraseña de la aplicación móvil

Nota: La figura muestra el formulario desde el cual el usuario inicia el proceso. Elaboración propia

Del mismo modo, se incorporó el manejo de excepciones relacionadas con la comunicación entre la aplicación móvil y la API REST, garantizando una respuesta adecuada ante errores de conexión o solicitudes no válidas. Finalmente, se realizaron pruebas funcionales, como se muestra en la *Figura 79*, para verificar el correcto intercambio de información entre ambas capas, confirmando el funcionamiento del proceso de recuperación de contraseña y la correcta integración de la aplicación móvil con los servicios del backend.

Figura 79*Prueba de recuperación de contraseña desde la aplicación móvil*

Nota: La figura evidencia el funcionamiento completo del flujo entre Flutter y la API REST. Elaboración propia

Retroalimentación de Sprint 1

Al finalizar el Sprint 1 se realizó la reunión de retrospectiva con el propósito de evaluar el trabajo desarrollado y analizar el cumplimiento de los objetivos establecidos para esta iteración. Durante esta actividad se revisó el avance de las tareas relacionadas con la configuración de la arquitectura del proyecto, la implementación del módulo de autenticación y la integración de la aplicación móvil con la API REST.

Como resultado de la revisión, se verificó el cumplimiento de las historias de usuario planificadas para el sprint 1, logrando disponer de una base funcional sobre la cual continuar el desarrollo de los siguientes módulos del sistema. La implementación de los procesos de autenticación, registro de usuarios y recuperación de contraseña permitió validar la comunicación entre la aplicación móvil desarrollada en Flutter y la API REST implementada con NestJS.

De igual forma, se identificó la importancia de mantener una comunicación constante entre las actividades del frontend y el backend para facilitar la integración de las funcionalidades y detectar oportunamente posibles inconsistencias durante las pruebas. Las validaciones realizadas en cada entrega contribuyeron a corregir incidencias de manera temprana, reduciendo retrabajos en las siguientes etapas del proyecto.

Finalmente, el equipo concluyó que los resultados obtenidos durante el Sprint 1 constituyen una base sólida para continuar con el desarrollo de los módulos funcionales del sistema, manteniendo las buenas prácticas de desarrollo, la organización del Product Backlog y la planificación de las actividades previstas para el siguiente sprint.

Sprint 2. Desarrollo del módulo de gestión de usuarios

Tabla 61

Sprint Planning 2

SPRINT 2				
ID de Product Backlog	Historia de usuario	Tarea	Responsable	Estado
IPB01	Registro de usuarios	1. Diseñar la estructura de la base de datos para el módulo de usuario. 2. Desarrollar los servicios de registro, consulta, actualización y eliminación/desactivación de usuarios en la API REST. 3. Diseñar e implementar la interfaz de registro de usuarios en la aplicación móvil. 4. Integrar la aplicación móvil con la API REST para el módulo de gestión de usuario. 5. Realizar pruebas funcionales de las operaciones CRUD del módulo de usuarios.	Jhonas López, Belky Piedra	Completado
IPB02	Consultar información de usuarios	6. Desarrollar los servicios de consulta y actualización de usuarios en la API REST, incluyendo la lógica de negocio necesaria para la gestión de información. 7. Implementar las validaciones de acceso y datos según el rol del usuario para garantizar la seguridad e integridad de la información. 8. Diseñar e implementar las interfaces de consulta y edición de información de usuarios en la aplicación móvil Flutter.	Jhonas López, Belky Piedra	Completado

IPB03	Actualizar información de usuarios	9. Integrar la aplicación móvil con la API REST y realizar pruebas funcionales de los procesos de consulta y actualización de usuarios.		
		10. Desarrollar el servicio de actualización de usuarios en la API REST, incluyendo las validaciones necesarias para garantizar la correcta modificación de la información.		
IPB04	Eliminar o desactivar usuarios	11. Implementar e integrar la interfaz de actualización de usuarios en Flutter con la API REST para permitir la edición y actualización de datos desde la aplicación móvil.	Jhonas López, Belky Piedra	Completado
		12. Desarrollar el servicio de eliminación o desactivación de usuarios en la API REST, incluyendo las validaciones de permisos y reglas de negocio.		
		13. Implementar e integrar la funcionalidad de eliminación o desactivación de usuarios en la aplicación móvil y realizar las pruebas de funcionamiento.		

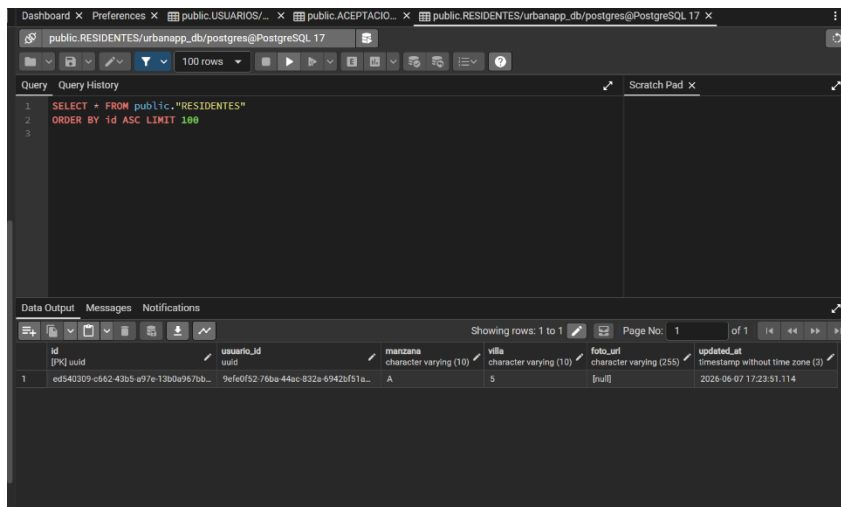
Nota: La tabla detalla las historias de usuario, tareas, responsables y estado del Sprint 2 para el progreso y desarrollo del proyecto. Elaboración propia.

Historia de usuario 4

IPB01 Registro de usuario

Tarea 1. Diseñar la estructura de la base de datos para el módulo de usuario

Como se observa en la *Figura 80, 81, 82*, en esta tarea se inició con el diseño de la estructura de la base de datos para el módulo de usuario: Administrador, Residente y Guardia; aquí se crearon las tablas correspondientes para cada uno de los roles, con sus respectivas relaciones.

Figura 80*Creación de la tabla de residente*


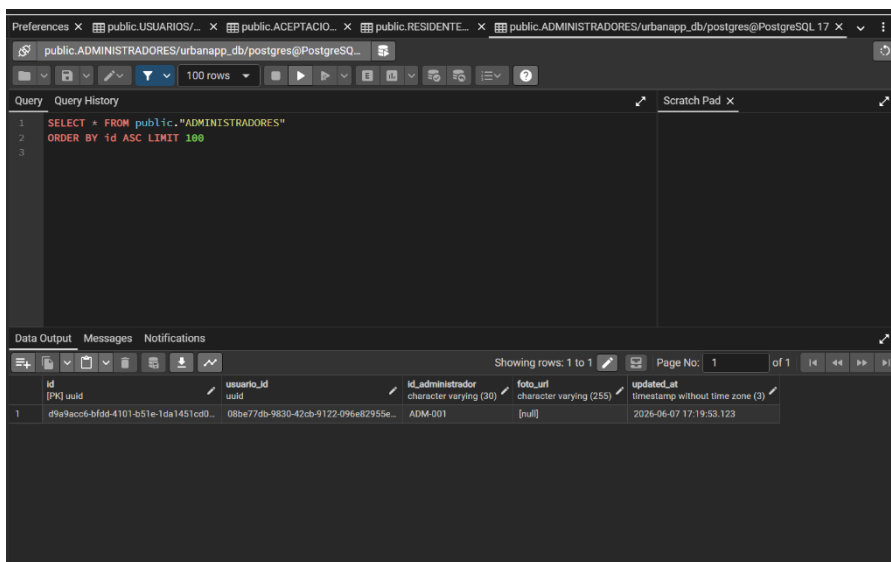
Query

```
1 SELECT * FROM public.\"RESIDENTES\"
2 ORDER BY id ASC LIMIT 100
3
```

Data Output

Showing rows: 1 to 1 | Page No: 1 | of 1

id	usuario_id	manzana	villa	foto_url	updated_at
1	ed540309-c662-43b5-a97e-13b0e967db...	9efcd052-78ba-44ac-832a-a942bf31a...	A	5	[null]

Nota: La figura evidencia la tabla para los residentes. Elaboración propia.**Figura 81***Creación de la tabla de administrador*


Query

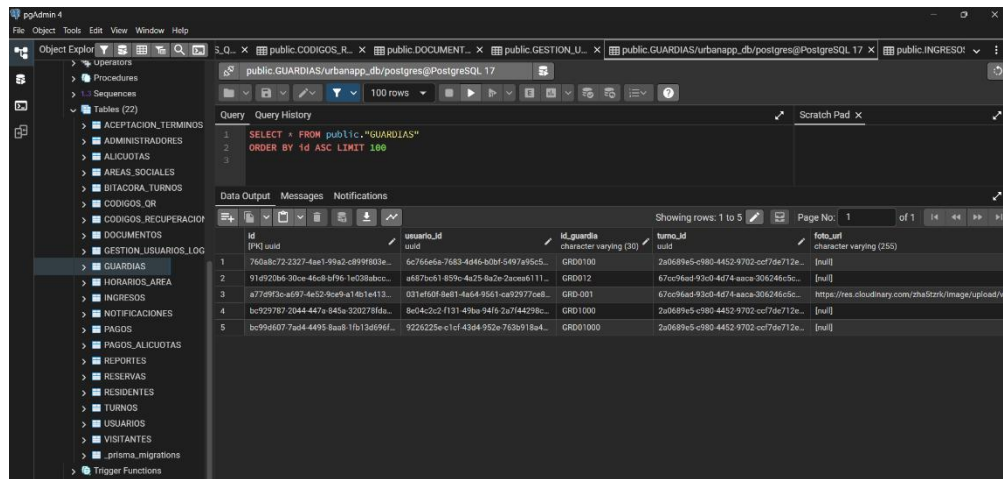
```
1 SELECT * FROM public.\"ADMINISTRADORES\"
2 ORDER BY id ASC LIMIT 100
3
```

Data Output

Showing rows: 1 to 1 | Page No: 1 | of 1

id	usuario_id	id_administrador	foto_url	updated_at
1	d9a9acc6-bfd4-4101-b51e-1da1451cd0...	08ba77db-9830-42cb-9122-096ae82955e...	ADM-001	[null]

Nota: La figura evidencia la tabla para el administrador. Elaboración propia

Figura 82*Creación de la tabla de guardia*


The screenshot shows a PostgreSQL database interface with the 'GUARDIAS' table selected. The table structure and data are as follows:

id	usuario_id	id_guardia	turno_id	foto_url
1	76da8c72-2327-4aa1-99a2-c899f903e...	6c764e5a-7683-4a46-b0bf-5497a95c5...	GRD0100	2a0689e5-c180-4452-9702-c077de712e...
2	91e920b6-30ca-46a8-b96e-1c038abcc...	e187aeb1-859e-4a25-ba2e-2a0e0111...	GRD012	67cc9ead-930d-4074-aaca-306246cde...
3	a7709f3c-a097-4e12-9ca9-a14b1e113...	031ef109-8a81-4a64-9561-ca92977ce8...	GRD081	67cc9ead-930d-4074-aaca-306246cde...
4	bc9297b7-2044-447a-845a-320278da...	8e042c22-1131-40ba-9416-2a7144298c...	GRD1000	2a0689e5-c180-4452-9702-c077de712e...
5	bc99d607-7ad4-4495-8a8f-1fb13d696f...	925b225e-c1ef-43d4-952e-763b918a4...	GRD01000	2a0689e5-c180-4452-9702-c077de712e...

Nota: La figura evidencia la tabla para el guardia. Elaboración propia

Posteriormente, se definieron las relaciones y restricciones necesarias dentro del modelo de datos, permitiendo mantener la integridad de la información y facilitar las operaciones realizadas desde la API REST desarrollada con NestJS. La estructura diseñada fue adaptada al gestor de base de datos PostgreSQL utilizado en el proyecto, siguiendo una organización que permita el crecimiento futuro del sistema.

Al finalizar la tarea, se determinó que la estructura diseñada constituyó la base para la implementación de los servicios correspondientes al módulo de usuarios, permitiendo una comunicación organizada y eficiente entre la capa de datos, la lógica de negocio del backend y la aplicación móvil desarrollada en Flutter. En este sentido, la arquitectura establecida facilitó la integración de los componentes del sistema y proporcionó una base sólida para la gestión eficiente de la información de los usuarios dentro del prototipo de aplicación móvil.

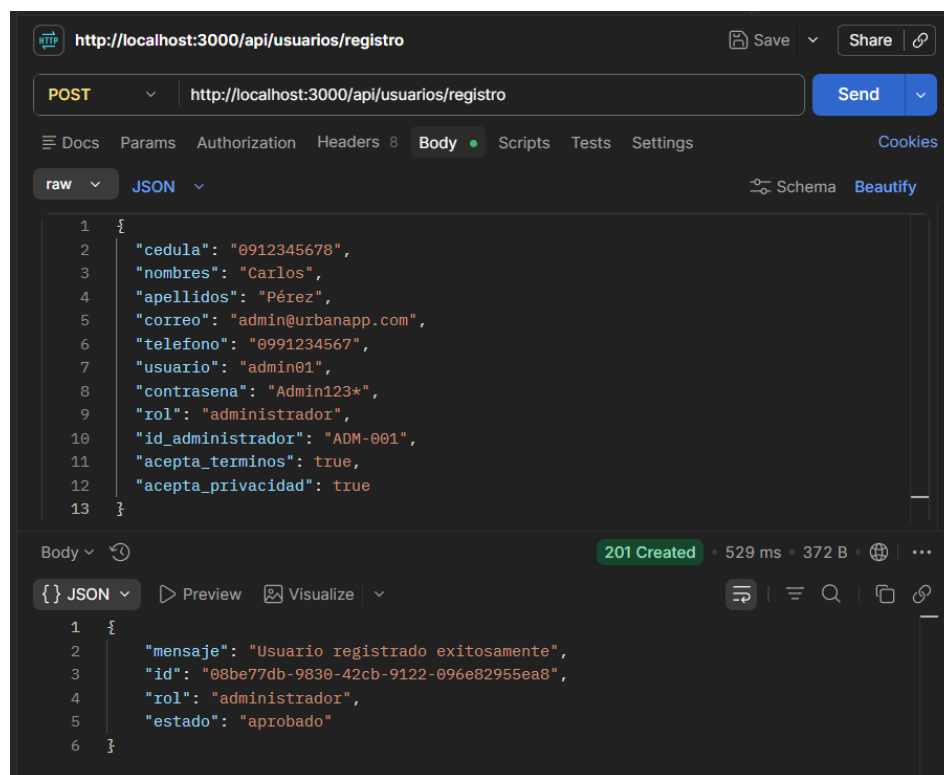
Tarea 2. Desarrollar los servicios de registro, consulta, actualización y eliminación/desactivación de usuarios en la API REST

Se implementó los servicios correspondientes a las operaciones principales de la administración de los usuarios por medio de la API REST.

Para la implementación se desarrolló la lógica de negocio en la capa de servicios del backend, siguiendo la arquitectura de tres capas definida para el proyecto. El servicio de registro permite almacenar nuevos usuarios en la base de datos PostgreSQL, como se muestra en la *Figura 83*, considerando la información requerida para su identificación y asignación del rol correspondiente dentro de la aplicación.

Figura 83

Prueba endpoint registro de administrador



Nota: La figura muestra la validación del servicio de registro de administrador al endpoint correspondiente de la API REST. Elaboración propia

De igual forma, se implementó el servicio de consulta de usuarios, como se muestra en la *Figura 84*, encargado de recuperar la información almacenada y proporcionar los datos necesarios para su visualización desde la aplicación móvil. Para la actualización de información, se desarrolló el proceso que permite modificar los datos de los usuarios registrados, aplicando validaciones para mantener la consistencia de la información.

Figura 84

Código de servicio de usuarios

Nota: La figura evidencia la lógica de registro, consulta, actualización y desactivación de usuarios. Elaboración propia

Mientras que, para la eliminación de usuarios, se consideró la opción de desactivar cuentas como mecanismo de control, permitiendo restringir el acceso al sistema sin afectar la trazabilidad de los registros existentes, como se muestra en la *Figura 85*. La implementación de estos servicios permitió establecer la funcionalidad principal del módulo de usuarios, proporcionando una base para la integración con la aplicación móvil.

Figura 85

Registros de usuarios almacenados en PostgreSQL

id	cedula	nombres	apellidos	correo	telefono	usuario	contraseña
1	03bdecf9-7edf-40a9-ab92-910ab8b51...	Jhonas	Lopez	vrjhoas@gmail.com	0969091020	maria011	\$2b\$10\$4Sx3z7xESaeFSaV
2	08be77db-9830-423b-9122-09e82955...	Carlos	Perez	admin@urbanapp.com	0991234455	admin01	\$2b\$10\$BmmyLPLwGkAP9
3	0fb9f85c-8197-49aa-9606-fbc0f51abec...	Gaeli Maribel	Moreno Alvarado	gaeli@gmail.com	0922234455	Gaeli01	\$2b\$10\$myG7I1VmrR8zn
4	125a1831-bf79-46f1-aaac-69c9c4c5c7...	Fernando	Salazar	fernando.residente@urbanapp.c...	0981234567	fernando_residente	\$2b\$10\$QON0dPfEMAH
5	16c5ae69-8591-4d7d-9828-7b361421...	Luis	Castillo	luis.guardia@urbanapp.com	0976543210	luis_guardia	\$2b\$10\$QKw0Fc21AdnDc
6	1782d336-5210-4ad8-833e-c3a6374d3...	Jhonas	Lopez	vrjhoas.lo@gmail.com	0978897931	jhonaslp12	\$2b\$10\$1a6AB5N9wmhz
7	283f5071-d99f-4dee-bafc-50fd62904e...	Belky	Piedra	belkypiedra@gmail.com	0978897938	jhonaslp01	\$2b\$10\$WpghEOhuQqp
8	2a48894a-c726-463d-860b-80ed6d23f...	Jeremy	Panta	pruebapant@gmail.com	0920201010	guardia1205	\$2b\$10\$gjeHqZwJhuJJK
9	2f182f53-913f-4255-47be-14ba4ab923...	Ashley	Mero Villa	pruebapant123@gmail.com	0978897935	Ashleym6	\$2b\$10\$JJYcC30.dHk47B
10	4eb8858c-0e29-4f6d-9209-045a4c4a6...	Roberto	Mendoza	roberto.admin@urbanapp.com	0967654321	roberto_admin	\$2b\$10\$TL4aRBL.Da8Yc7
11	4daa10c0-36ca-47e6-995f-34a4f1956...	Jhonas Jeremy	Lopez Panta	jhonas@gmail.com	0978897935	admin02	\$2b\$10\$BFconV8BaFCOL
12	567006f5-b3ca-40b2-99ef-039ec778a...	Rafael Alonso	Lopez Mero	Rafa@gmail.com	0988698684	Rafael01	\$2b\$10\$3ePBOWX0uSFnXz
13	656c9e1d-671b-4166-afa3-cc1a8c70ff9f	Jhonas	Lopez	jlopez@gmail.com	0910201020	guardia0101	\$2b\$10\$AFKQqvCHHvNp
14	6c76a6fa-7683-4a46-b0bf-5497a95c5...	Joimar	Lopez	joimar@gmail.com	0910159191	guardia0100	\$2b\$10\$RkRjyWw42359
15	739412aa-489d-49a6-b2da-fae4463be...	Valeria	Rico	valeria.residente@urbanapp.com	0992345678	valeria_residente	\$2b\$10\$Hbq0L2cMH7s
16	85976683-d6c0-43ae-9690-53aabacbe...	Iker	Lopez	iker@gmail.com	0920201010	Guardia25	\$2b\$10\$P4gmSc1ySH7Is

Nota: La figura muestra la información de los usuarios registrada y almacenada en la base de datos PostgreSQL. Elaboración propia

Tarea 3. Diseñar e implementar la interfaz de registro de usuarios en la aplicación móvil

En esta tarea se diseñó e implementó la interfaz del registro de usuarios en la aplicación móvil, así como también la integración de los servicios disponibles en la API REST, como se muestra en la *Figura 86*. De igual forma, se definieron los componentes visuales previamente diseñado en el prototipo visual, con el fin de obtener la información necesaria del usuario, desde luego cuidado la organización de los elementos de la interfaz que así mismo permita una interacción apropiada.

Figura 86

Interfaz de registro de usuario en la aplicación móvil

2:54

< Crear Cuenta

Tipo de Usuario

Residente Admin

1 2

Datos Personales Credenciales

Cédula

0999999999

Nombres

Escriba sus nombres

Apellidos

Escriba sus apellidos

Correo

usuario@correo.com

Teléfono

0999999999

Siguiente

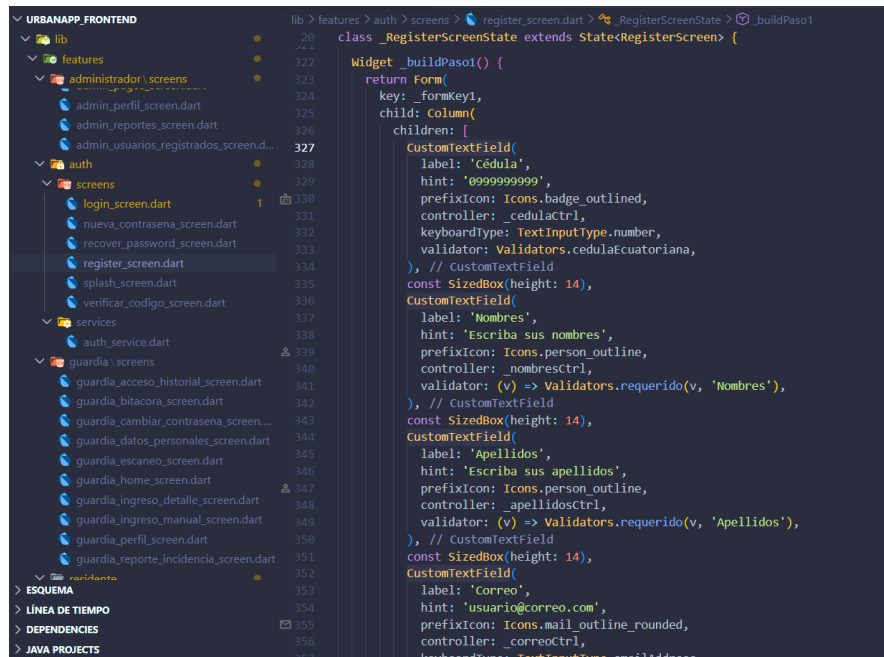
Volver al Inicio de Sesión

Nota: La figura presenta la interfaz desarrollada para el registro de usuarios en la aplicación móvil. Elaboración propia

En relación a la idea anterior, se implementaron los campos requeridos para el registro y se incorporaron validaciones básicas en la aplicación móvil para verificar que la información ingresada cumpla con las condiciones necesarias antes de ser enviada al backend, como se muestra en la *Figura 87*.

Figura 87

Código fuente del formulario de registro desarrollado en Flutter



Nota: La figura muestra el código fuente del formulario para el registro de usuarios. Elaboración propia.

La interfaz fue desarrollada siguiendo la estructura definida para la aplicación móvil, manteniendo una separación adecuada entre la capa de presentación y la lógica de comunicación con los servicios externos. Esto permite que la pantalla de registro pueda interactuar posteriormente con la API REST desarrollada en NestJS mediante el consumo de los endpoints correspondientes.

Tarea 4. Integrar la aplicación móvil con la API REST para el módulo de gestión de usuario

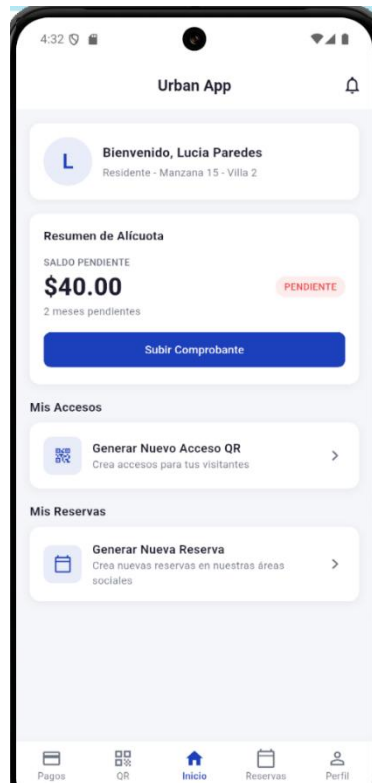
Para el desarrollo de esta tarea se realizó la integración entre la aplicación móvil y la API REST, con el objetivo de permitir la comunicación entre la capa de presentación y la lógica de negocio del sistema. Para ello, se configuraron los servicios de consumo de endpoints mediante solicitudes HTTP, permitiendo a la aplicación móvil enviar y recibir información relacionada con la gestión de usuarios.

Por otra parte, se implementó la conexión con los servicios de registro, consulta, actualización y eliminación/desactivación de usuarios, tal como se muestra en la *Figura 88*, asegurando el intercambio correcto de datos en formato JSON entre el frontend móvil y el backend.

Además, se establecieron modelos de datos en Flutter para representar la información de los usuarios y se configuraron métodos encargados de realizar las peticiones hacia la API REST.

Figura 88

Prueba del endpoint desde la aplicación móvil



Nota: Pantalla de Home del residente mostrando los datos del residente y de valores de alícuotas. Elaboración propia

En función a la de integración, se realizaron pruebas de comunicación para verificar la correcta respuesta de los servicios, validando el envío de información desde los formularios de la aplicación móvil y comprobando que los datos fueran procesados y almacenados adecuadamente en la base de datos PostgreSQL. También se verificó el manejo de respuestas exitosas y errores generados por el servidor, garantizando una interacción estable entre los componentes del sistema.

Al finalizar la tarea, como se muestra en la *Figura 89*, se logró establecer una comunicación funcional entre la aplicación móvil y la API REST, permitiendo la gestión de usuarios desde la

interfaz desarrollada. Esta integración facilitó la centralización de la información y proporcionó una base sólida para la implementación de los demás módulos del sistema.

Figura 89

Respuesta generada por la API REST para el registro de usuarios

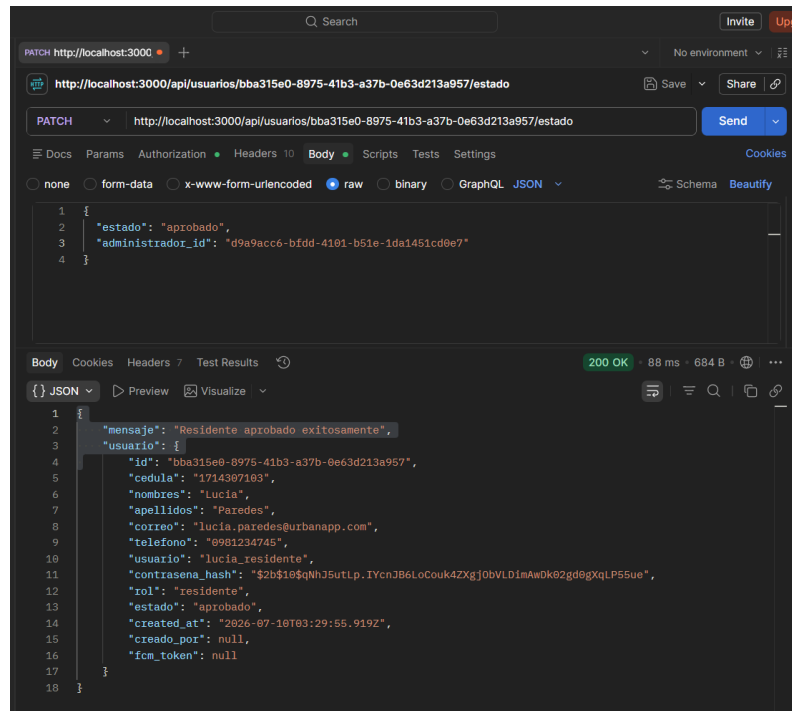
Nota: Respuesta generada por la API REST al ejecutar el endpoint de registro de usuarios y validar la comunicación con el backend. Elaboración propia.

Tarea 5. Realizar pruebas funcionales de las operaciones CRUD del módulo de usuarios

Se ejecutaron las pruebas funcionales de las operaciones CRUD (crear, consultar, actualizar y eliminar) de acuerdo con el módulo de usuarios con el fin de validar el correcto funcionamiento de los servicios implementados en la API REST y una integración adecuada, como se muestra en la *Figura 90*. Las pruebas permitieron validar el comportamiento de cada operación mediante diferentes contextos, comprobando que el sistema permita registrar nuevos usuarios, consultar la información almacenada, actualizar los datos existentes y realizar la eliminación o desactivación de registros según las reglas definidas.

Figura 90

Pruebas de endpoints CRUD en Postman



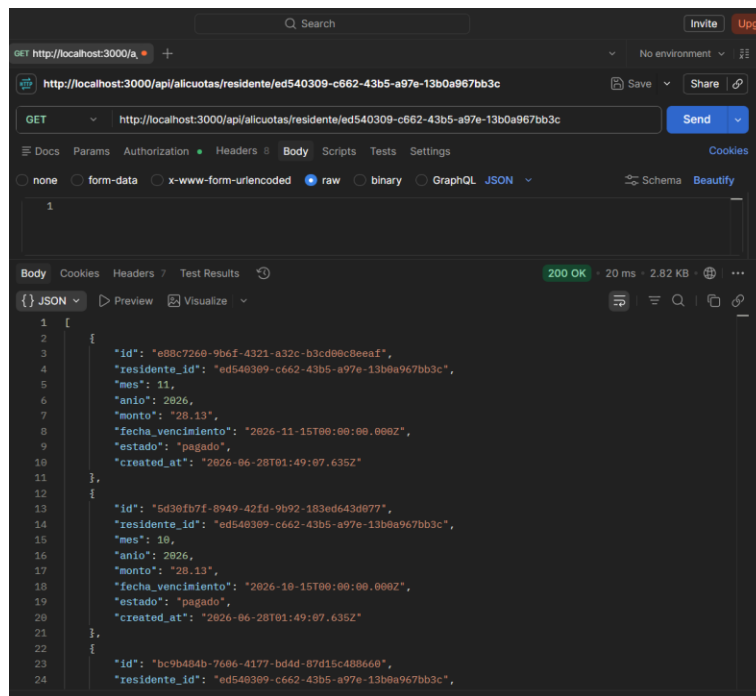
Nota: La figura evidencia la ejecución de pruebas de los servicios CRUD mediante la herramienta Postman. Elaboración propia

Para ello, se realizaron solicitudes a los endpoints establecidos en el backend desarrollado con NestJS, verificando los códigos de respuesta HTTP, los mensajes generados por el servidor y la correcta manipulación de la información en la base de datos PostgreSQL.

Al finalizar la tarea, se comprobó que las operaciones CRUD del módulo de usuarios funcionaban correctamente, asegurando la comunicación efectiva entre la aplicación móvil, la API REST y la base de datos, como se muestra en la *Figura 91*. Los resultados obtenidos permitieron validar la estabilidad del módulo y confirmar que cumple con los requerimientos funcionales establecidos para la gestión de residentes dentro del prototipo de aplicación móvil.

Figura 91

Resultado final de validación del módulo



Nota: La figura evidencia las pruebas funcionales ejecutadas con éxito de los servicios CRUD. Elaboración propia.

Historia de usuario 5

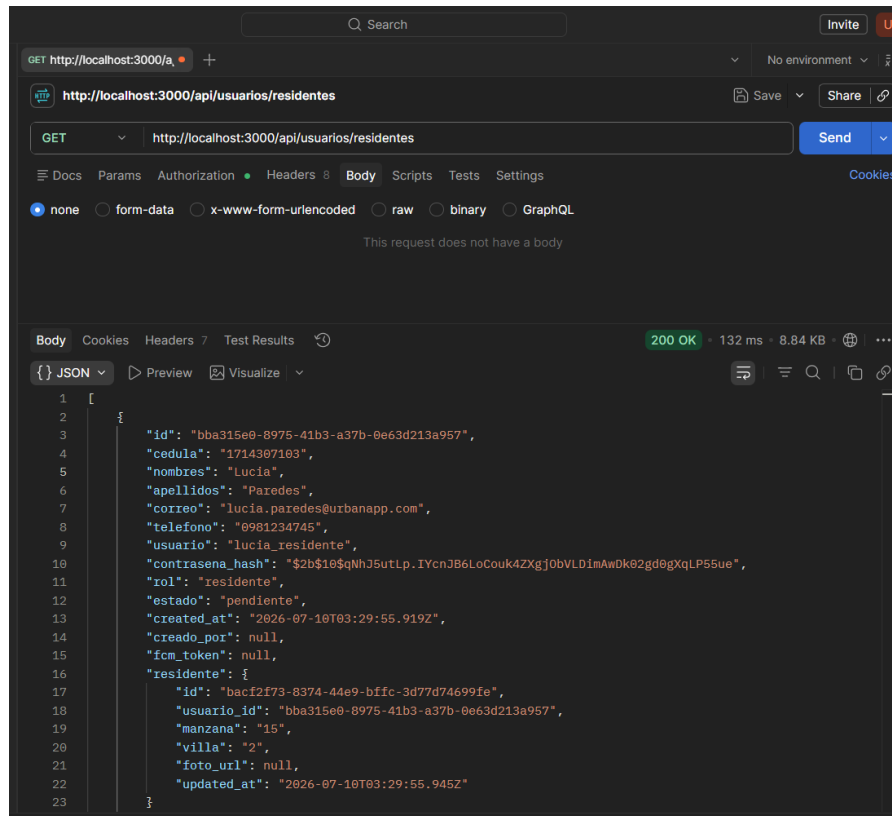
IPB02 Consultar información de usuarios

Tarea 6. Desarrollar los servicios de consulta y actualización de usuarios en la API REST y la lógica de negocio necesaria para la gestión de información.

Como se muestra en la *Figura 92*, se implementaron los servicios de consulta y actualización de usuarios dentro de la API REST, con el fin de permitir la gestión dinámica de la información almacenada en el sistema. También se desarrollaron los endpoints encargados de recuperar la información registrada, permitiendo obtener datos específicos de un usuario mediante su identificador, así como consultar el listado general de usuarios disponibles en el sistema.

Figura 92

Prueba del endpoint GET de consulta de usuarios en Postman



Nota: La figura evidencia la ejecución del endpoint GET mediante la herramienta Postman para validar la consulta y recuperación de información de usuarios almacenados en la base de datos. Elaboración propia.

De igual forma, se implementó el servicio de actualización de usuarios, incorporando la lógica necesaria para modificar datos existentes de manera controlada. Este proceso incluyó la validación de la información recibida, la actualización parcial o completa de los atributos permitidos y la confirmación de los cambios realizados en la base de datos. Además, se aplicaron reglas de negocio para preservar la integridad de los datos y evitar modificaciones inconsistentes dentro del sistema.

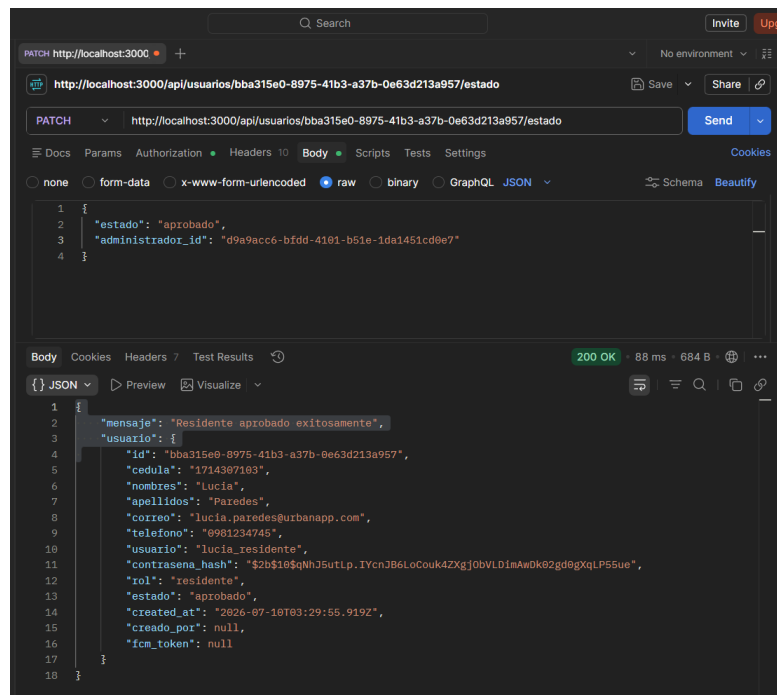
Para la validación se realizaron pruebas de los endpoints mediante solicitudes HTTP, verificando la correcta comunicación entre la aplicación móvil, la API REST y la base de datos.

Como se muestra en la *Figura 93*, la tarea se finalizó satisfactoriamente con la implementación de la capa funcional de consulta y actualización de usuarios, fortaleciendo el módulo de gestión de usuarios del prototipo de aplicación móvil y proporcionando los servicios

necesarios para administrar la información de los residentes de forma eficiente, segura y organizada.

Figura 93

Prueba del endpoint PUT/PATCH de actualización de usuario



Nota: La figura evidencia la ejecución del endpoint PUT/PATCH mediante la herramienta Postman para validar la actualización de la información de usuarios registrada en la base de datos. Elaboración propia

Tarea 7. Implementar las validaciones de acceso y datos según el rol del usuario para garantizar la seguridad e integridad de la información.

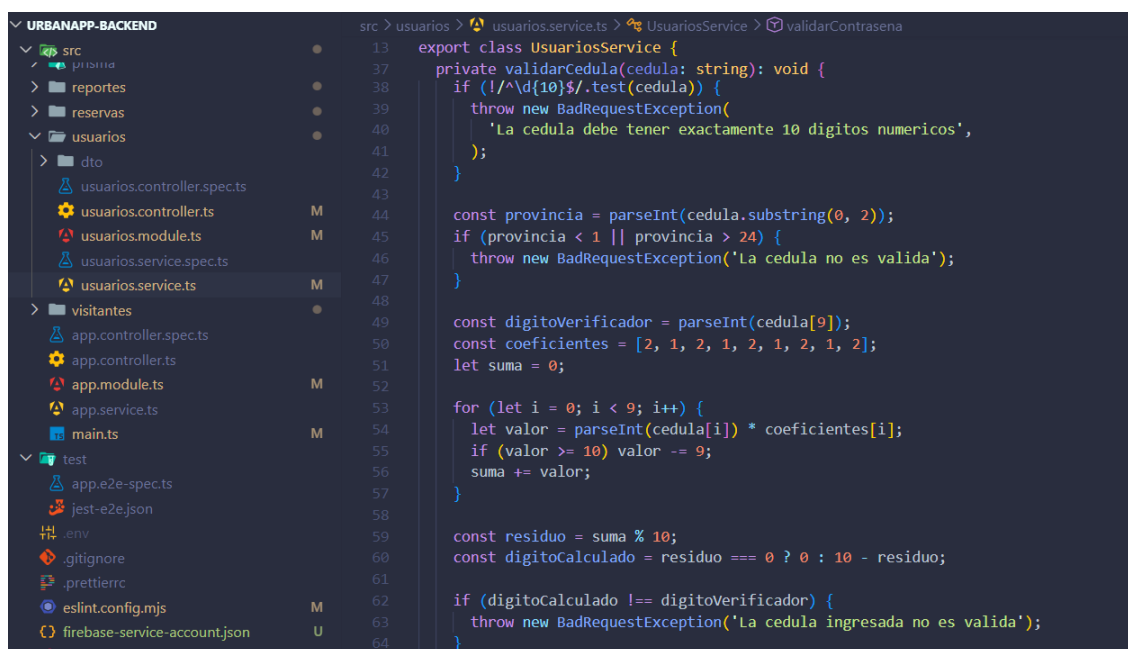
En esta tarea se implementaron validaciones de acceso y control de información considerando los roles definidos dentro del sistema. Esta tarea tuvo como finalidad garantizar que cada usuario pueda acceder únicamente a las funcionalidades e información correspondientes según sus permisos. Para ello, se incorporaron validaciones en la API REST, permitiendo controlar las operaciones realizadas sobre los datos de usuarios y verificar que la información ingresada cumpla con las condiciones requeridas antes de ser procesada, como se muestra en la *Figura 94*.

Figura 94*Validación de roles y permisos*

Nota: La figura muestra la validación de los roles y sus respectivos servicios en la API REST. Elaboración propia.

Además, se realizaron pruebas funcionales para comprobar el correcto comportamiento de las restricciones implementadas, verificando el acceso según el rol del usuario y la integridad de la información gestionada por el sistema, como se observa en la *Figura 95* y *96*. Esta tarea permitió fortalecer la seguridad del módulo de usuarios y establecer una administración adecuada de los permisos dentro de la aplicación.

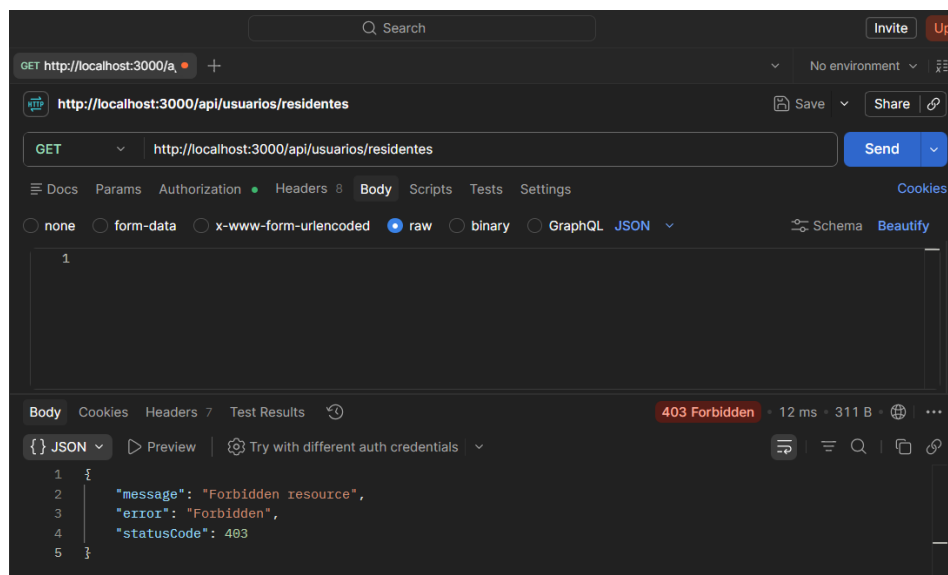
Figura 95*Validación de datos de usuarios*



Nota: La figura muestra la validación de los datos de los usuarios registrados. Elaboración propia.

Figura 96

Pruebas de acceso según rol del usuario



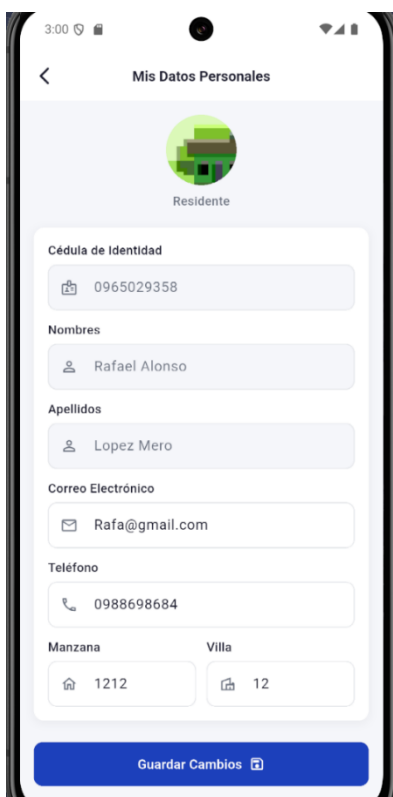
Nota: La figura evidencia la validación del control de acceso basado en roles de usuario, verificando que los permisos y funcionalidades disponibles correspondan al perfil asignado dentro del sistema. Elaboración propia

Tarea 8. Diseñar e implementar las interfaces de consulta y edición de información de usuarios.

En esta tarea se diseñaron e implementaron las interfaces para la consulta y modificación de la información de los usuarios registrado en la aplicación móvil, ilustrado en la *Figura 97*.

Figura 97

Código fuente de las pantallas de registro de usuarios



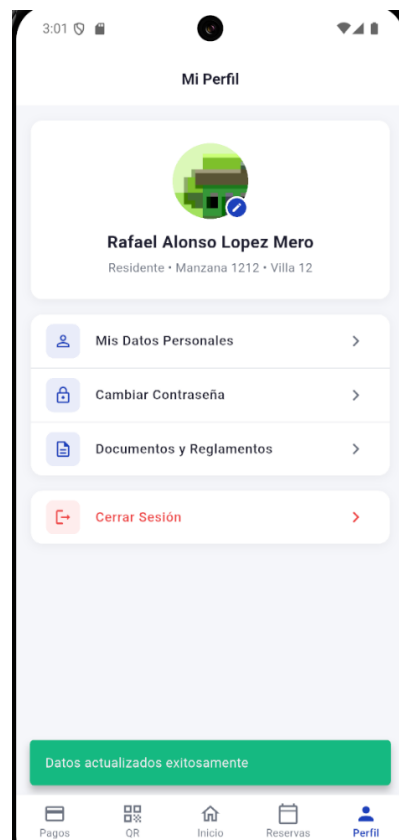
Nota: La figura muestra un fragmento del código fuente del desarrollo de las interfaces del registro de usuarios. Elaboración propia

Así mismo se desarrollaron las pantallas necesarias para visualizar la información registradas e igual forma la modificación de los datos respectivamente. Además, se consideraron validaciones básicas para asegurar que la información ingresada sea correcta antes de ser enviada al backend.

La tarea se culminó con éxito, como se observa en la *Figura 98*, validado el funcionamiento de las interfaces verificando la correcta visualización de los datos y la interacción del usuario con los formularios de actualización dentro de la aplicación móvil.

Figura 98

Interfaz de actualización de datos de usuarios



Nota: La figura muestra la pantalla de la actualización de usuarios en el módulo de usuarios. Elaboración propia

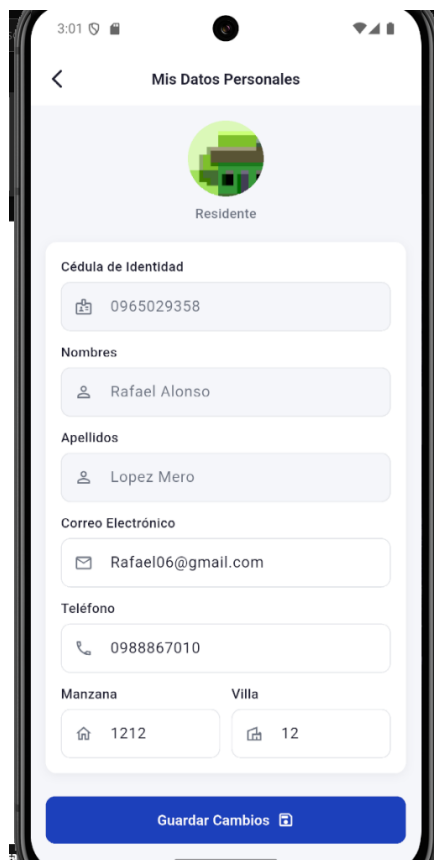
Tarea 9. Integrar la aplicación móvil con la API REST y realizar pruebas funcionales de los procesos de consulta y actualización de usuarios.

En el desarrollo de esta tarea se realizó la integración entre la aplicación móvil y la API REST, como se muestra en la *Figura 99*, permitiendo el consumo de los servicios necesarios para la consulta y actualización de información de usuarios. También se verificó la comunicación entre

ambas capas mediante el envío y recepción de datos, validando que la información obtenida desde la API pueda visualizarse correctamente en la aplicación móvil y que los cambios realizados sean procesados adecuadamente.

Figura 99

Información de usuarios actualizada en la aplicación

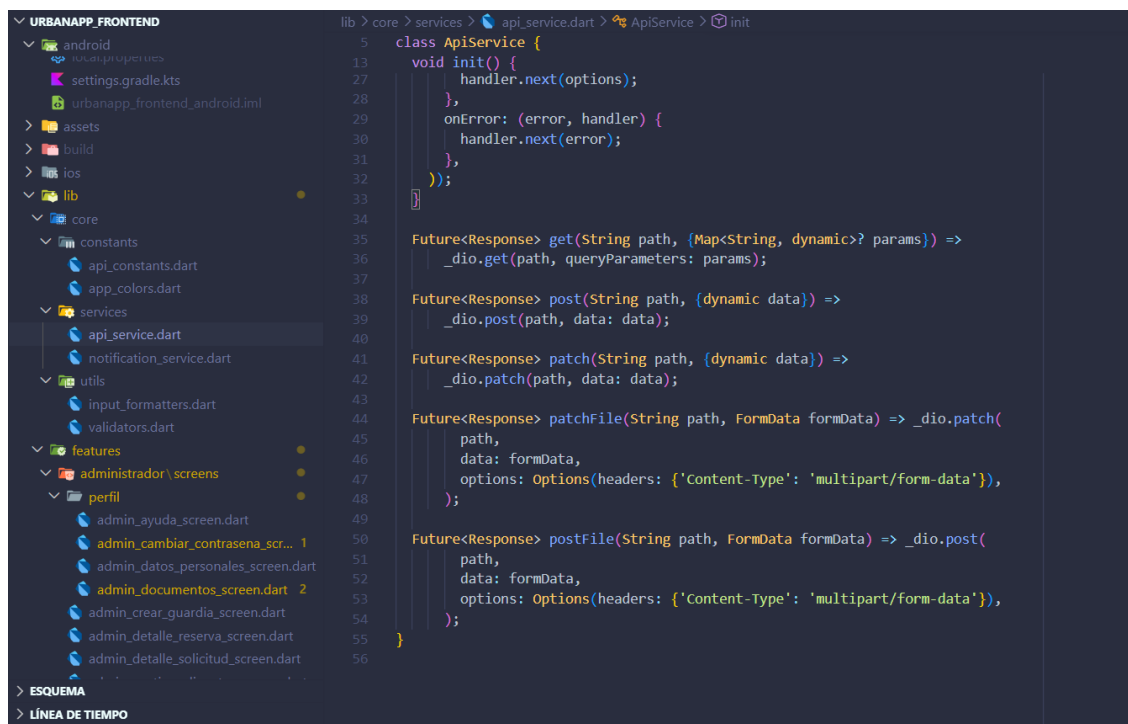


Nota: La figura muestra la actualización y visualización de la información de usuarios en la aplicación móvil, posterior al consumo del servicio de actualización proporcionado por la API REST. Elaboración propia

Finalmente, se ejecutaron pruebas funcionales para comprobar el correcto funcionamiento de los procesos de consulta y actualización de usuarios, identificando posibles errores y verificando la estabilidad de la integración implementada, como se muestra en la *Figura 100*.

Figura 100

Código del servicio de comunicación entre Flutter y la API REST



Nota: La figura muestra el código para establecer la comunicación con la API REST mediante solicitudes HTTP, permitiendo el intercambio de información entre la aplicación móvil y el backend. Elaboración propia

Historia de usuario 6

IPB03 Actualizar información de usuarios

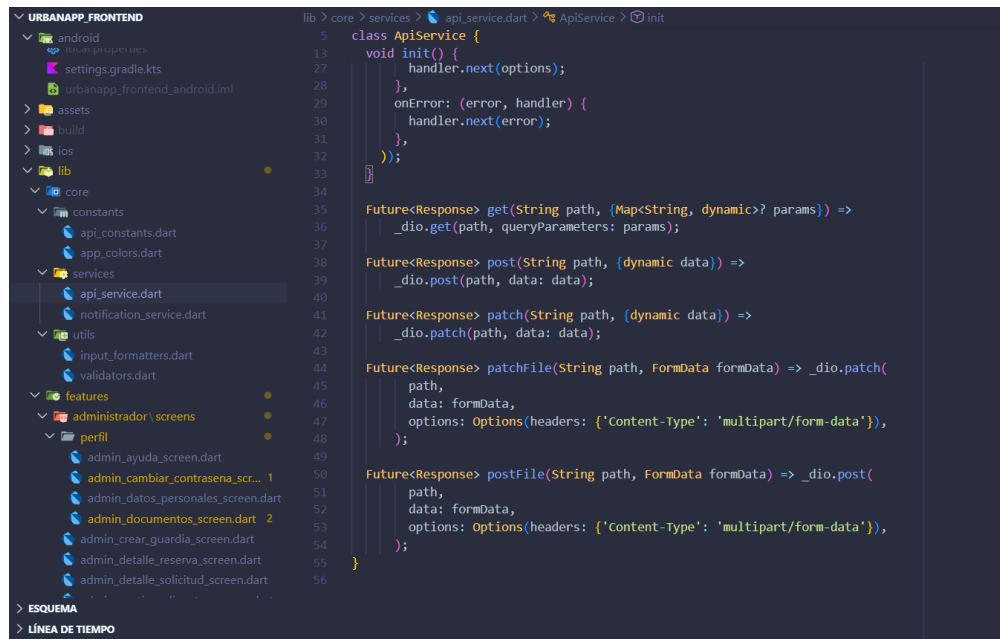
Tarea 10. Desarrollar el servicio de actualización de usuarios en la API REST y las validaciones necesarias para garantizar la correcta modificación de la información.

En esta tarea se desarrolló el servicio de actualización de información de usuarios en la API REST, como se muestra en la *Figura 101*. Esta tarea tuvo como objetivo permitir la modificación de los datos registrados, manteniendo la consistencia y confiabilidad de la información almacenada en el sistema. Durante la implementación se incorporó la lógica necesaria para procesar las solicitudes de actualización y se aplicaron validaciones sobre los datos recibidos, con el propósito

de verificar que la información ingresada cumpla con las condiciones establecidas antes de realizar los cambios correspondientes.

Figura 101

Código del servicio de actualización de usuarios en la API REST

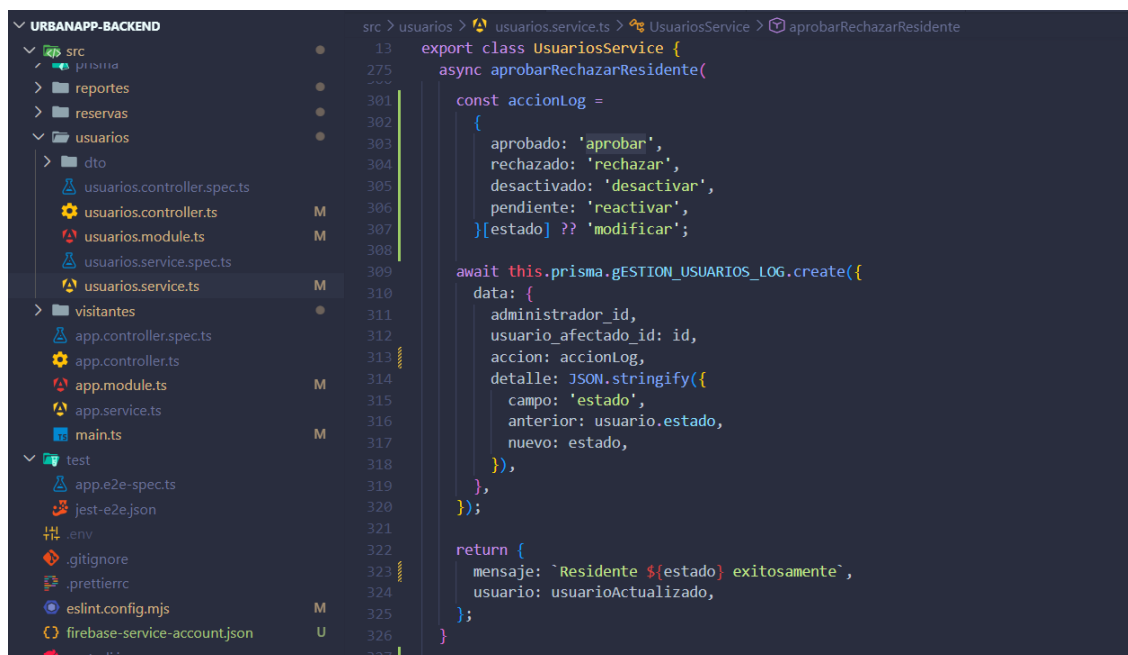


Nota: La figura muestra el código implementado en la API REST para gestionar la actualización de información de usuarios, aplicando la lógica de negocio y la comunicación con la base de datos. Elaboración propia

Como se observa en la *Figura 102*, se realizaron pruebas funcionales del servicio para comprobar el correcto procesamiento de las solicitudes de actualización y validar que los datos modificados se reflejen adecuadamente en el sistema.

Figura 102

Código de las validaciones aplicadas para la modificación de datos



Nota: La figura muestra el código implementado para validar la información ingresada durante el proceso de modificación de datos de usuarios. Elaboración propia.

Finalmente, se muestra en la *Figura 103*, los registros reales y actualizados de los usuarios en la base de datos en el gestor de la base de datos.

Figura 103

Registro actualizado en la base de datos PostgreSQL

id	cedula	nombres	apellidos	correo	telefono	usuario
7	1782d336-5210-44ad-833e-e3a6374d3...	Jhonas	Lopez	vrjohas.1@gmail.com	0978897931	Jhonas12
8	283f5071-d99f-4dee-baf0-50fd62904e...	Bely	Piedra	belkypiedra@gmail.com	0978897938	Jhonas101
9	2e48894a-c726-4d3d-860b-80ed6d29f...	Jeremy	Panta	pruebapanta@gmail.com	0920201010	guardia1205
10	2f182f53-913f-4255-a7be-14baf4b923...	Ashley	Mero Villa	pruebapanta123@gmail.com	0978897935	Ashley6
11	4b68868c-0e29-4f6d-9209-0454d4a6...	Roberto	Mendoza	roberto.admin@urbanapp.com	0987654321	roberto_admin
12	4daa10ee-36ca-476c-9951-34af1956...	Jhonas Jeremy	Lopez Panta	johonas@gmail.com	0978897935	admin02
13	567006f5-b3ca-40b2-99ef-039ec77ba...	Rafael Alonso	Lopez Mero	Rafa@gmail.com	0988698684	Rafael01
14	656d9efd-671b-4166-afa3-c01abc70ff9f	Jhonas	Lopez	jlopez@gmail.com	0910201020	guardia0101
15	6c7666ea-7683-4d46-b0bf-5497a95c5...	Josimar	Lopez	josimar@gmail.com	0910159191	guardia0100
16	739412aa-489d-49a6-b2da-fae4463be...	Valeria	Rioe	valeria.residente@urbanapp.com	0992345678	valeria_residente
17	85976683-d5cd-43ae-9698-53aabcac...	Iker	Lopez	iker@gmail.com	0920201010	guardia25
18	875c104f-c489-4073-8f8e-c914d7b2...	Jhonas Jeremy	Lopez Panta	vrjohas@gmail.com	0988121545	Jhonas10
19	89c9ea18-f5dd-43b8-91fe-b2584e352...	Matias Locuro	Lopez Mero	matias01@gmail.com	0988121540	Matias01
20	8e042c2c-1311-49be-94f6-2a7f4428bc...	Juan	Perez	juanito@gmail.com	0978785454	guardia00
21	9226225e-c1df-43d4-982a-763b918a4...	Jhonas	Lopez	jlope1@gmail.com	0910201010	guardia01000
22	9a55ca7c-1c3f-4c3a-9c7a-7f8b1a1a3e...	Iessania	Morann Alvarado	iessania@gmail.com	0976867945	Residente11

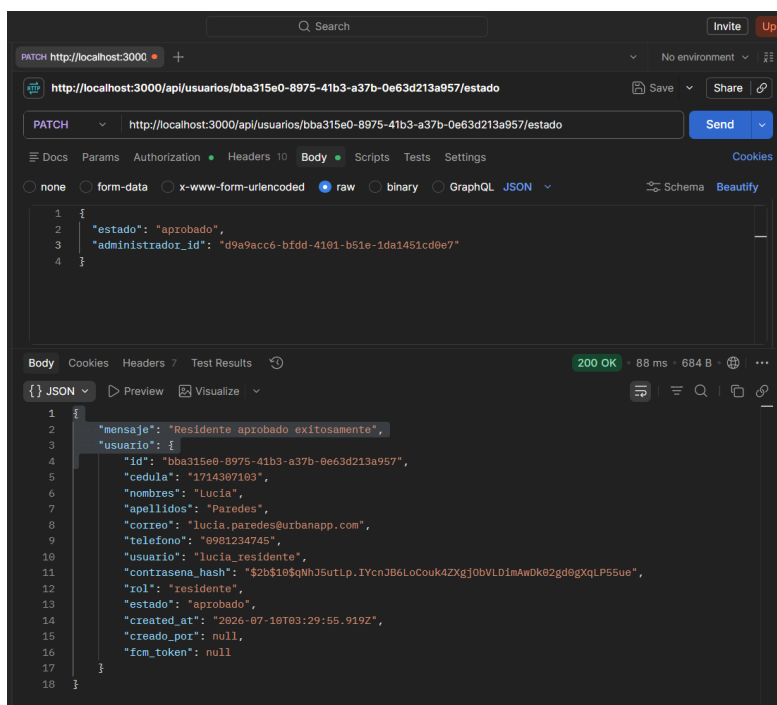
Nota: La figura muestra la actualización del registro de usuario almacenado en la base de datos PostgreSQL, verificando la persistencia correcta de los datos modificados mediante la API REST. Elaboración propia

Tarea 11. Implementar e integrar la interfaz de actualización de usuarios en Flutter con la API REST para permitir la edición y actualización de datos desde la aplicación móvil.

Se implementó la interfaz de actualización de información de usuarios en la aplicación móvil. Esta tarea se desarrolló con el fin de permitir que los usuarios autorizados puedan modificar sus datos registrados mediante un formulario dentro de la aplicación. De igual manera, se incorporaron los componentes necesarios para la edición de información y se realizó la integración con la API REST, como se muestra en la *Figura 104*, permitiendo enviar los datos modificados y procesar la respuesta generada por el servidor.

Figura 104

Respuesta de la API REST durante el proceso de actualización

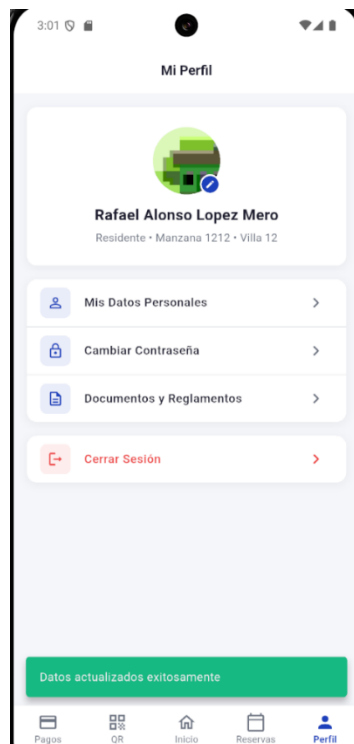


Nota: La figura muestra la respuesta generada por la API REST durante la actualización de información de usuarios, verificando la correcta ejecución de la solicitud y el procesamiento exitoso de los datos modificados. Elaboración propia

Finalmente, como se observa en la *Figura 105*, se realizaron pruebas funcionales para verificar el correcto funcionamiento del proceso de actualización, comprobando que la información ingresada desde la aplicación móvil sea enviada correctamente y que los cambios realizados se reflejen adecuadamente en el sistema.

Figura 105

Prueba de actualización de información desde la aplicación móvil



Nota: La figura muestra la ejecución del proceso de actualización de información de usuarios desde la aplicación móvil, validando la correcta comunicación con la API REST y la persistencia de los datos modificados. Elaboración propia.

Historia de usuario 7

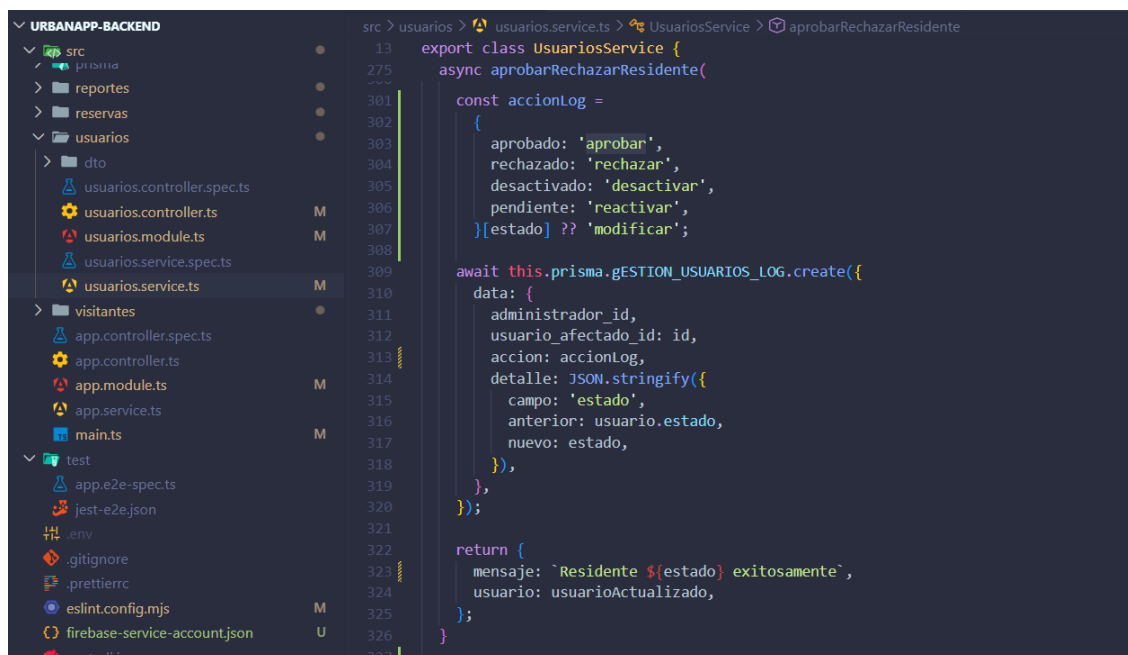
IPB04 Eliminar o desactivar usuarios

Tarea 12. Desarrollar el servicio de eliminación o desactivación de usuarios en la API REST, las validaciones de permisos y reglas de negocio.

En esta tarea se desarrolló el servicio de eliminación o desactivación de usuarios en la API REST con el objetivo de controlar el estado de las cuentas de usuario y administrar el acceso al sistema de acuerdo con las reglas definidas, ilustrada en la *Figura 106*.

Figura 106

Código del servicio de eliminación o desactivación de usuarios en NestJS



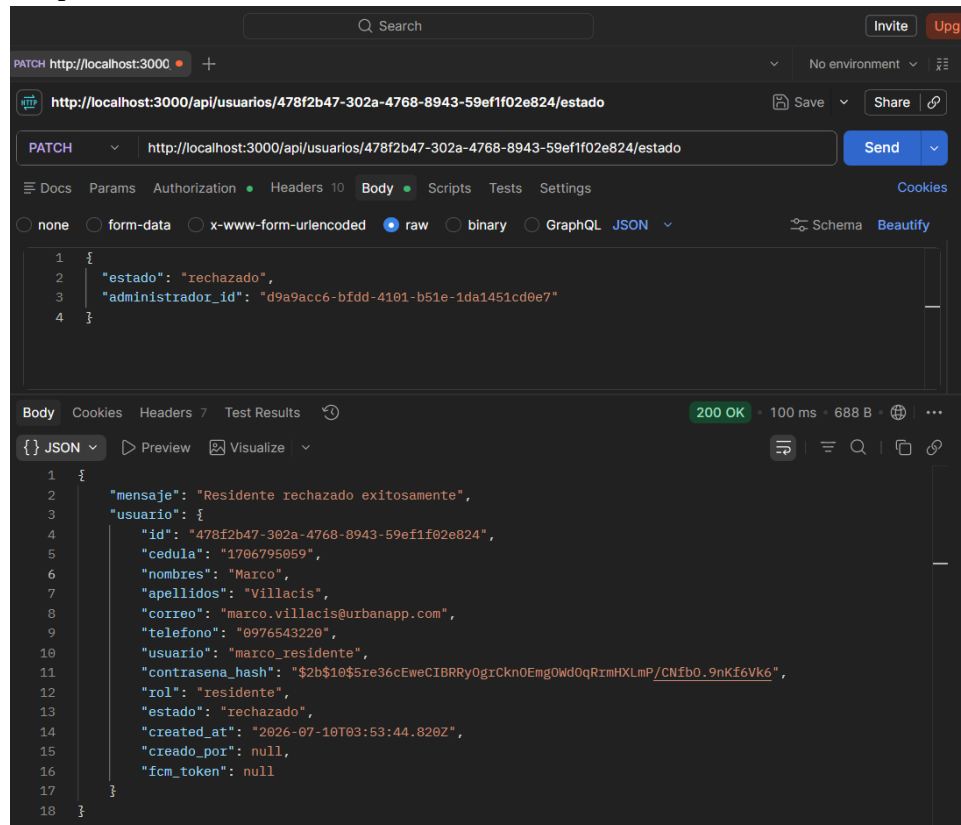
Nota: La figura muestra el código para gestionar la eliminación o desactivación de usuarios, aplicando la lógica necesaria para controlar el estado de los registros almacenados en la base de datos. Elaboración propia.

También se incorporó la lógica necesaria para procesar las solicitudes de desactivación o eliminación de usuarios, considerando las validaciones de permisos correspondientes para restringir esta operación a los usuarios autorizados. Además, se aplicaron reglas de negocio para garantizar que la gestión de usuarios mantenga la integridad de la información almacenada.

Finalmente, como se observa en la *Figura 107*, se realizaron pruebas funcionales del servicio para verificar que las operaciones ejecutadas respondan correctamente y que los cambios realizados en el estado de los usuarios sean procesados adecuadamente.

Figura 107

Prueba del endpoint de desactivación de usuarios en Postman



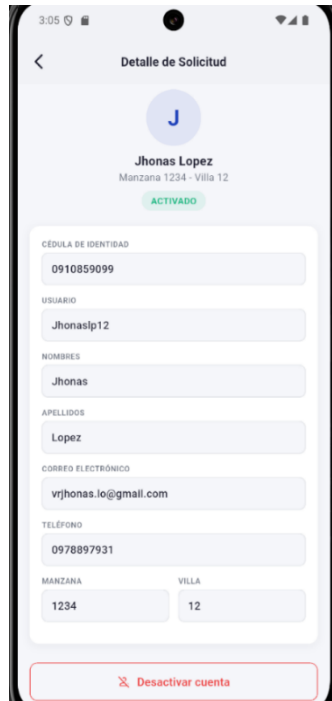
Nota: La figura muestra la ejecución del endpoint de desactivación de usuarios, verificando el correcto funcionamiento del servicio API REST y la actualización del estado del usuario en la base de datos. Elaboración propia.

Tarea 13. Implementar e integrar la funcionalidad de eliminación o desactivación de usuarios en la aplicación móvil y realizar las pruebas de funcionamiento.

Durante el desarrollo de esta tarea se implementó la funcionalidad de eliminación o desactivación de usuarios en la aplicación móvil con el fin de permitir la gestión del estado de los usuarios desde la interfaz móvil, mediante la comunicación con los servicios proporcionados por la API REST, como se observa en la *Figura 108*.

Figura 108

Interfaz de eliminación o desactivación de usuarios

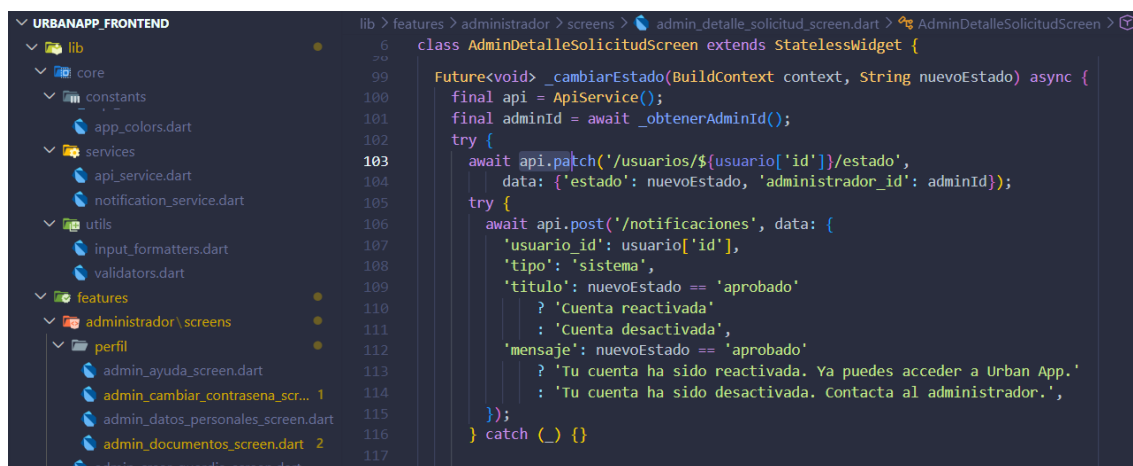


Nota: La figura muestra la interfaz implementada en la aplicación móvil para gestionar la eliminación o desactivación de usuarios. Elaboración propia.

Del mismo modo, se incorporaron las opciones necesarias para ejecutar la acción de desactivación o eliminación y se realizó la integración con el backend para enviar las solicitudes correspondientes y recibir la respuesta del servidor, como se observa en la *Figura 109*.

Figura 109

Código del servicio de comunicación con la API REST



Nota: Fragmento del código para realizar la comunicación con la API REST durante el proceso de eliminación o desactivación de usuarios, mediante el envío de solicitudes HTTP al servicio backend. Elaboración propia.

Se culminó la tarea, realizando pruebas funcionales para verificar el correcto funcionamiento de la interacción entre Flutter y la API REST, comprobando que la operación solicitada se ejecute correctamente y que el sistema responda de manera adecuada ante los diferentes escenarios.

Retroalimentación del Sprint 2

Al finalizar el Sprint 2 se realizó la retroalimentación del trabajo desarrollado con el propósito de evaluar el cumplimiento de las tareas planificadas y analizar los resultados obtenidos durante la implementación del módulo de gestión de usuarios.

Durante este sprint se logró diseñar la estructura de la base de datos para la administración de usuarios y desarrollar los servicios principales en la API REST utilizando NestJS, permitiendo gestionar las operaciones de registro, consulta, actualización y desactivación de usuarios. Además, se implementaron las interfaces correspondientes en la aplicación móvil, integrando la comunicación con el backend para el manejo de la información.

Como parte del desarrollo se incorporaron validaciones de acceso y datos según los roles definidos en el sistema, con el objetivo de controlar las acciones disponibles para cada usuario y mantener la integridad de la información almacenada. De igual forma, se realizaron pruebas funcionales de los procesos implementados, verificando el correcto funcionamiento de las operaciones del módulo y la comunicación entre la aplicación móvil, la API REST y la base de datos.

Durante la revisión del sprint se identificó que la integración entre el frontend y backend requirió validar continuamente la estructura de los datos enviados y recibidos, lo que permitió realizar ajustes en los servicios y mejorar la comunicación entre las capas del sistema. También se

evidenció la importancia de aplicar validaciones desde el desarrollo inicial para reducir errores durante las pruebas funcionales.

Finalmente, el Sprint 2 permitió completar la base funcional del módulo de gestión de usuarios, proporcionando las herramientas necesarias para administrar la información de los usuarios dentro del prototipo de aplicación móvil y dejando preparado el sistema para la incorporación de los siguientes módulos definidos en el proyecto.

Sprint 3. Desarrollo del módulo de control de pagos de alícuotas

Tabla 62

Sprint Planning 3

SPRINT 3				
ID de Product Backlog	Historia de usuario	Tarea	Responsable	Estado
IPB01	Registrar comprobante de alícuota	1. Desarrollar el servicio de registro de comprobantes de alícuotas en la API REST y la validación de la información ingresada. 2. Implementar e integrar la interfaz de registro de comprobantes con la API REST.	Jhonas López, Belky Piedra	Completado
IPB02	Validar pago de alícuota	3. Desarrollar el servicio de validación de pagos de alícuotas en la API REST, incorporando la lógica de aprobación o rechazo del comprobante. 4. Implementar e integrar la interfaz de validación de pagos en la aplicación móvil para la gestión del administrador.	Jhonas López, Belky Piedra	Completado
IPB03	Consultar historial de pagos de alícuotas	5. Desarrollar el servicio de consulta del historial de		

pagos de alícuotas en la API REST.

6. Implementar e integrar la interfaz de consulta de pago.

Nota: La tabla detalla las historias de usuario, tareas, responsables y estado del Sprint 3 para el progreso y desarrollo del proyecto. Elaboración propia.

Historia de usuario 8

IPB01 Registrar comprobante de alícuota

Tarea 1. Desarrollar el servicio de registro de comprobantes de alícuotas en la API REST y la validación de la información ingresada.

Se inició con el desarrollo del servicio de registro de comprobantes de pago de alícuotas en la API REST, como se ve en la *Figura 110*, este servicio permitió que los residentes pudieran enviar el comprobante de un pago realizado para que posteriormente fuera revisado y validado por el administrador de la urbanización.

Figura 110

Código del servicio de registro en NestJS

Nota: La figura muestra fragmento del código servicio de registro de comprobantes de pagos en la API REST. Elaboración propia.

Se implementó el endpoint encargado de recibir las solicitudes provenientes de la aplicación. El controlador capturó la información enviada por el residente y la transfirió al servicio correspondiente, donde se ejecutó toda la lógica de negocio relacionada con el registro del comprobante, representado en la *Figura 111*.

Figura 111

Código del controlador de registro del comprobante

Nota: La figura muestra fragmento del código del controlador del registro del comprobante del pago de alícuotas. Elaboración propia

De igual forma se realizaron validaciones para garantizar la calidad y consistencia de los datos recibidos, para ello se comprobó que los datos obligatorios estuvieran completos, que el identificador del residente existiera en el sistema, que el período correspondiente al pago fuera válido y que el comprobante incluyera la información necesaria para su posterior revisión.

Durante este proceso, la *Figura 112* muestra que el sistema asignó automáticamente un estado inicial de Pendiente, indicando que el comprobante aún debía ser revisado por el administrador antes de considerarse un pago aprobado. Adicionalmente, se implementó el manejo de excepciones para controlar situaciones como registros duplicados, referencias inexistentes o errores durante la persistencia de la información.

Figura 112

Registro almacenado con el estado inicial Pendiente

Nota: La figura muestra el registro almacenado con el estado inicial Pendiente, antes de la aprobación. Elaboración propia.

Con la implementación de esta funcionalidad se estableció la base del proceso de gestión de pagos de alícuotas dentro del prototipo, permitiendo que los comprobantes fueran registrados de forma estructurada y quedaran disponibles para su posterior revisión y aprobación por parte del administrador, contribuyendo así a un control más eficiente y confiable de los pagos realizados por los residentes.

Tarea 2. Implementar e integrar la interfaz de registro de comprobantes con la API REST.

En esta tarea se implementó e integró la interfaz de registro de comprobantes de pago de alícuotas con la API REST, permitiendo la comunicación entre la aplicación móvil y el servidor mediante el consumo del endpoint correspondiente. Se diseñó un formulario intuitivo en Flutter, como se muestra en la *Figura 113*, para el ingreso de la información del comprobante y se

incorporaron validaciones en el cliente para verificar que los campos obligatorios estuvieran completos antes de enviar la solicitud.

Figura 113

Servicio de registro de comprobante en la API REST

Nota: La figura muestra el servicio del registro del comprobante de alícuotas en la API REST. Elaboración propia

Posteriormente, se configuró el consumo de la API utilizando peticiones HTTP, procesando adecuadamente las respuestas del servidor para informar al usuario sobre el registro exitoso o los posibles errores detectados, representado en la *Figura 114*.

Figura 114

Código de la interfaz de registro de comprobantes

Nota: La figura muestra fragmento del código de la interfaz del registro de comprobantes. Elaboración propia.

Finalmente, se realizaron pruebas de integración que confirmaron el envío correcto de la información, el almacenamiento del comprobante en PostgreSQL con estado Pendiente y la correcta sincronización entre el frontend y el backend, como se observa en las *Figuras 115 y 116*.

Figura 115

Interfaz del formulario de registro de comprobantes

Nota: La figura muestra la interfaz del registro de comprobantes en la aplicación. Elaboración propia.

Figura 116

Pantalla de registro exitoso en la aplicación móvil

Nota: La figura el mensaje cuando el registro del pago es exitoso dentro de la aplicación móvil. Elaboración propia.

Historia de usuario 9

IPB02 Validar pago de alícuota

Tarea 3. Desarrollar el servicio de validación de pagos de alícuotas en la API REST, incorporando la lógica de aprobación o rechazo del comprobante.

En esta tarea se desarrolló el servicio encargado de validar los comprobantes de pago de alícuotas registrados por los residentes desde la aplicación móvil, como se observa en la *Figura 117*, con el fin de proporcionar al administrador un componente que permitiera verificar la información enviada por el usuario y de esta manera determinar si el pago es aprobado o rechazado, respaldando la confiabilidad de los registros financieros de la urbanización.

Figura 117

Código del servicio de validación de pagos en NestJS

Nota: La figura muestra un fragmento del código donde se implementa la lógica de negocio para aprobar o rechazar un comprobante de pago. Elaboración propia.

Por otro lado, se realizó la implementación en la API REST siguiendo la arquitectura establecida, separando la lógica del negocio, esta estructura facilitó el mantenimiento del código y favoreció la escalabilidad del sistema para futuras funcionalidades.

En primera instancia, se diseñó el servicio responsable de recibir la solicitud de validación enviada por el administrador. Una vez recibida la petición, el sistema verificó que el comprobante existiera en la base de datos y que se encontrara en estado pendiente de revisión. Esta validación evitó que un mismo comprobante pudiera ser procesado más de una vez o que se intentara validar un registro inexistente. Así mismo, se implementó la lógica de negocio encargada de evaluar la decisión del administrado; cuando el comprobante cumplía con los criterios establecidos, el sistema actualizaba automáticamente su estado Aprobado, registrando además la fecha de

validación y conservando la trazabilidad del proceso. Caso contrario, el comprobante era marcado como Rechazado, permitiendo registrar una observación que justificara la decisión tomada.

Una vez que la solicitud fue aprobada o rechazada, el servicio actualizó el registro correspondiente en PostgreSQL, representado en la *Figura 118*. Luego, la API REST generó una respuesta estructurada con el resultado de la operación, incluyendo un mensaje informativo y el estado actualizado del comprobante, lo que permitió a la aplicación móvil reflejar inmediatamente la decisión del administrador.

Figura 118

Registro actualizado en PostgreSQL

Nota: La figura muestra la base de datos donde se observa el cambio del estado del comprobante, junto con la fecha de validación y demás campos actualizados. Elaboración propia.

Al finalizar la tarea se realizaron las respectivas pruebas funcionales del endpoint para comprobar el comportamiento esperado del servicio en los distintos escenarios. Los resultados obtenidos como se observa en la *Figura 119*, confirmaron que el servicio cumplía con los criterios funcionales definidos para el Sprint y contribuía al objetivo de garantizar un proceso seguro, controlado y transparente en la gestión de pagos de alcúotas dentro de la urbanización.

Figura 119

Interfaz del administrador con el resultado de la validación

Nota: La figura muestra la pantalla de la aplicación donde el administrador aprueba o rechaza un comprobante y se refleja el estado actualizado. Elaboración propia.

Tarea 4. Implementar e integrar la interfaz de validación de pagos en la aplicación móvil para la gestión del administrador.

En el desarrollo de la tarea se implementó e integró el servicio encargado de validar los comprobantes de pago de alcúotas para la gestión por parte del administrador desde la aplicación

móvil, con el propósito de facilitar el proceso de revisión y aprobación de los pagos de alcúotas realizados por los residentes de la urbanización.

En la *Figura 120*, se muestra como se diseñó una interfaz intuitiva, siguiendo la estructura visual definida para el proyecto, en la que el administrador puede acceder a una lista de pagos pendientes, visualizando información relevante como el nombre del residente, el valor cancelado, la fecha del pago, comprobante del pago y el estado actual de la transacción.

Figura 120

Interfaz de validación de pagos

Nota: Pantalla de validación de pagos donde el administrador visualiza la lista de pagos pendientes. Elaboración propia.

Luego se integró esta interfaz con la capa de lógica de negocio para gestionar la obtención y actualización de los datos, estableciendo la comunicación con los servicios REST implementados en NestJS mediante solicitudes HTTP. De igual manera, se incorporó la funcionalidad que permite al administrador validar el comprobante de pago seleccionado, como se observa en la *Figura 121*, enviando la solicitud correspondiente al backend para actualizar el estado del registro y reflejar inmediatamente el cambio en la interfaz sin necesidad de reiniciar la aplicación.

Figura 121

Consumo de la API

Nota: La aplicación realiza correctamente la solicitud HTTP al backend para obtener los pagos pendientes. Elaboración propia.

Durante el desarrollo de la tarea también se implementaron validaciones para garantizar que únicamente los pagos con estado pendiente puedan ser aprobados. Todo el proceso fue desarrollado respetando la arquitectura de tres capas propuesta para el proyecto, separando la

interfaz de usuario, la lógica de negocio y el acceso a los datos, lo que favorece la mantenibilidad, escalabilidad e integración de la aplicación, resultados representados en las *Figuras 122 y 123*.

Figura 122

Interfaz de la validación del pago: Estado pendiente

Nota: Interfaz donde se observa el estado del pago como pendiente antes de la validación. Elaboración propia.

Figura 123

Interfaz de la validación del pago: Estado aprobado

Nota: Después de validar un pago, la lista se actualiza sin reiniciar la aplicación y el estado del pago pasa de pendiente a aprobado, según la lógica implementada. Elaboración propia.

Historia de usuario 10

IPB03 Consultar historial de pagos de alícuotas

Tarea 5. Desarrollar el servicio de consulta del historial de pagos de alícuotas en la API REST.

El desarrollo de la tarea consistió en implementar el servicio de consulta del historial de pagos de alícuotas dentro de la API REST, como se observa en la *Figura 124*, con el objetivo de proporcionar un mecanismo que permita recuperar de forma organizada el registro histórico de los pagos realizados por los residentes de la urbanización.

Figura 124

Implementación del servicio de consulta del historial de pagos

Nota: Implementación de la clase de servicio encargada de consultar el historial de pagos desde la base de datos. Elaboración propia.

Así mismo, se desarrolló el endpoint correspondiente para atender las solicitudes de consulta provenientes de la aplicación móvil. En la *Figura 125*, se muestra que el endpoint procesa

la petición, consulta la información almacenada y devuelve una respuesta estructurada en formato JSON. La respuesta contiene información importante como el período de la alícuota, la fecha de pago, el monto cancelado y el estado de la transacción.

Figura 125

Implementación del endpoint

Nota: Creación del endpoint que expone el servicio para ser consumido por la aplicación móvil. Elaboración propia.

También se realizaron validaciones, para garantizar un funcionamiento confiable del servicio; se verificó la existencia de registros antes de generar la respuesta y se implementó el tratamiento de excepciones para responder adecuadamente cuando no existieran datos disponibles o se produjera algún inconveniente durante la consulta.

Con esta implementación se fortaleció el módulo de control de pagos, permitiendo que la aplicación móvil consulte de manera eficiente el historial de pagos de alícuotas mediante la API REST, manteniendo la organización y escalabilidad definidas en la arquitectura del sistema.

Tarea 6. Implementar e integrar la interfaz de consulta de pago.

Se diseñó la interfaz, siguiendo los lineamientos gráficos y de navegación definidos para el prototipo, con una estructura que facilita la visualización del historial de pagos mediante una lista organizada que presenta información relevante como el período correspondiente, la fecha de pago, el monto cancelado, el comprobante y el estado de cada transacción, ilustrada en la *Figura 126*.

Figura 126

Interfaz de historial de pagos

Nota: Pantalla donde el usuario visualiza el historial de pagos de alícuotas con información organizada por registros. Elaboración propia.

Posteriormente, la interfaz fue integrada con la capa de lógica de negocio para gestionar la comunicación con la API REST, como se ve en la *Figura 127*. Para ello, se implementó el consumo del servicio encargado de consultar el historial de pagos, enviando las solicitudes HTTP correspondientes y procesando las respuestas recibidas del servidor. Una vez obtenidos los datos, la aplicación actualiza dinámicamente la interfaz, mostrando la información recuperada de forma estructurada y garantizando que el usuario visualice el estado más reciente de sus pagos de alícuotas.

Figura 127

Consumo del servicio Rest

Nota: Verificación de que la aplicación móvil consulta correctamente la API REST para obtener el historial de pagos. Elaboración propia.

Finalmente, se realizaron pruebas funcionales para verificar la correcta integración entre la interfaz y los servicios REST, comprobando que la información mostrada coincide con los datos almacenados en la base de datos y que las consultas se realizan de manera eficiente. Con esta funcionalidad se fortalece el módulo de control de pagos, permitiendo a los residentes consultar su historial de pagos de alícuotas desde la aplicación móvil.

Retroalimentación del Sprint 3

Durante el desarrollo del sprint se cumplieron las tareas planificadas para el módulo de control de pagos, alcanzando la integración entre la aplicación móvil y la API REST. Se desarrollaron los servicios necesarios para el registro de comprobantes, la validación de pagos y la consulta del historial de pagos, incorporando las validaciones requeridas para garantizar la consistencia de la información procesada.

De igual manera, se implementaron las interfaces móviles correspondientes, permitiendo el registro de comprobantes por parte de los residentes, la validación de pagos por el administrador y la consulta del historial de pagos mediante una interacción fluida con los servicios del backend.

Las pruebas funcionales realizadas durante el sprint permitieron verificar que la comunicación entre la aplicación móvil y la API REST se ejecuta correctamente, comprobando el registro de comprobantes, la actualización del estado de los pagos tras su aprobación o rechazo y la recuperación del historial de pagos de forma satisfactoria. Como consecuencia del desarrollo del sprint, se obtuvo un módulo funcional para el control de pagos, que cubre el flujo principal del proceso, desde el registro del comprobante hasta su validación y posterior consulta por parte de los usuarios.

Sprint 4. Desarrollo de los módulos de control de accesos y reservas de áreas sociales

Tabla 63

Sprint Planning 4

SPRINT 4				
ID de Product Backlog	Historia de usuario	Tarea	Responsable	Estado
IPB01	Generar QR de acceso	1. Desarrollar el servicio de generación de códigos QR en la API REST. 2. Implementar e integrar la interfaz de generación y visualización del código QR.	Jhonas López, Belky Piedra	Completado
IPB02	Validar QR de acceso	3. Desarrollar el servicio de validación de códigos QR en la API REST. 4. Implementar e integrar la interfaz de validación de códigos QR.	Jhonas López, Belky Piedra	Completado

IPB03	Consultar historial de ingresos	5. Desarrollar el servicio de consultar historial de ingresos QR en la API REST. 6. Implementar e integrar la interfaz de consulta del historial de ingresos.	Jhonas López, Belky Piedra	Completado
IPB04	Reserva un área social	7. Desarrollar el servicio de reservas de áreas sociales en la API REST. 8. Implementar e integrar la interfaz de reservas de áreas sociales.	Jhonas López, Belky Piedra	Completado
IPB05	Aprobar o rechazar las reservas	9. Desarrollar el servicio de aprobar o denegar reservas en la API REST. 10. Implementar e integrar la interfaz de consulta del historial de ingresos.	Jhonas López, Belky Piedra	Completado
IPB06	Consultar estado de reserva	11. Desarrollar el servicio de consultar el estado de reservas en la API REST. 12. Implementar e integrar la interfaz consulta de reservas.	Jhonas López, Belky Piedra	Completado
IPB07	Cargar comprobante de reserva	13. Desarrollar el servicio de registro de comprobantes de reservas en la API REST. 14. Implementar e integrar la interfaz de cargar de comprobantes de reserva.	Jhonas López, Belky Piedra	Completado

Nota: La tabla detalla las historias de usuario, tareas, responsables y estado del Sprint 4 para el progreso y desarrollo del proyecto. Elaboración propia.

Historia de usuario 11

IPB01 Generar QR de acceso

Tarea 1. Desarrollar el servicio de generación de códigos QR en la API REST.

En esta tarea se desarrolló el servicio de generación de códigos QR dentro de la API REST, como se muestra en la *Figura 128*, con el objetivo de implementar un mecanismo digital que permita gestionar el acceso de los residentes mediante códigos QR asociados a la información registrada en el sistema. Para ello, se implementó la lógica del servicio, estableciendo la estructura

necesaria para recibir las solicitudes, procesar la información del usuario y generar la respuesta correspondiente con el código QR creado.

Figura 128

Implementación del servicio de generación QR

Nota: La figura muestra un fragmento del código fuente del servicio encargado de ejecutar la lógica de generación del código QR dentro de la API REST. Elaboración propia.

Posteriormente, el servicio fue integrado con la capa de lógica de negocio para gestionar la comunicación entre la aplicación móvil y la API REST, ilustrado en la *Figura 129*. Para ello, se implementaron los endpoints necesarios para atender las solicitudes de generación del código QR, validando previamente la información recibida y verificando que los datos del residente sean correctos antes de ejecutar el proceso. Una vez generada la información, la API retorna una respuesta estructurada que permite a la aplicación móvil obtener y utilizar el código QR dentro del flujo de control de acceso.

Figura 129

Prueba de respuesta de la API REST

Nota: La figura muestra la validación del funcionamiento del servicio mediante Postman. Elaboración propia.

Durante el desarrollo también se incorporaron validaciones para garantizar la correcta generación del código, controlando escenarios como datos incompletos, usuarios no registrados o errores durante la comunicación con el servidor. Estas validaciones permiten mejorar la confiabilidad del servicio y asegurar que únicamente los residentes con información válida puedan disponer de un código QR generado por el sistema.

Finalmente, se realizaron pruebas funcionales para verificar la correcta operación del servicio y su integración con los componentes del prototipo, comprobando que las solicitudes sean

procesadas adecuadamente, que la API REST genere respuestas correctas y que la información obtenida pueda ser utilizada posteriormente por la aplicación móvil. Con esta funcionalidad se fortalece el módulo de control de accesos, representado en la *Figura 130*, permitiendo establecer una base tecnológica para la generación y posterior validación de códigos QR como mecanismo de ingreso.

Figura 130

Código QR generado correctamente

Nota: Generación de código QR por parte del residente para compartir al visitante. Elaboración propia.

Tarea 2. Implementar e integrar la interfaz de generación y visualización del código QR.

En esta tarea se desarrolló la interfaz móvil para la generación y visualización del código QR de acceso, representada en la *Figura 131*. Para ello, se desarrolló una pantalla que permite al residente acceder a la funcionalidad de generación de su código QR, presentando una estructura sencilla e intuitiva donde se visualiza el código generado junto con la información necesaria del visitante para su manejo como medio de identificación dentro de la aplicación.

Figura 131

Interfaz de generación de código QR

Nota: La figura muestra la interfaz de la generación de código QR en el módulo de accesos. Elaboración propia.

Posteriormente, la interfaz fue integrada con la capa de lógica de negocio para establecer la comunicación con la API REST. Sumado a ello, se implementó el consumo del servicio de generación de códigos QR mediante solicitudes HTTP, como se observa en la *Figura 132*, permitiendo que la aplicación móvil envíe la petición correspondiente y reciba la información

generada por el servidor. Una vez obtenida la respuesta, la aplicación procesa los datos recibidos y actualiza dinámicamente la interfaz para mostrar el código QR asociado al residente.

Figura 132

Integración de Flutter con la API REST

(Captura del código donde se realiza la petición HTTP desde Flutter.)

Nota: La figura evidencia del consumo del servicio backend encargado de generar el código QR. Elaboración propia.

Durante el desarrollo se incorporaron controles para gestionar correctamente los diferentes estados de la interfaz, considerando el proceso de carga de información, la generación exitosa del código y posibles errores durante la comunicación con el backend. Esto permite brindar una respuesta adecuada al usuario y evitar inconsistencias en la visualización del código QR cuando la información requerida no se encuentre disponible o exista algún problema con el servicio.

Finalmente, como muestra en la *Figura 133*, se realizaron pruebas funcionales para verificar la correcta integración entre la aplicación móvil y la API REST, comprobando que la solicitud de generación sea procesada correctamente, que el código QR recibido pueda visualizarse dentro de la aplicación y que se pueda compartir. Con esta implementación se fortalece el módulo de control de accesos, permitiendo a los residentes disponer de un código QR generado desde la aplicación móvil para su posterior validación.

Figura 133

Visualización del código QR generado



Nota: La figura muestra el resultado final dentro de la aplicación móvil mostrando el código QR correspondiente al residente. Elaboración propia.

Historia de usuario 12

IPB02 Validar QR de acceso

Tarea 3. Desarrollar el servicio de validación de códigos QR en la API REST.

En esta tarea, como se observa en la *Figura 134*, se desarrolló el servicio de validación de códigos QR dentro de la API REST, como parte del módulo de control de accesos, el fin de esta función fue permitir la verificación de los códigos QR generados previamente, comprobando que la información asociada corresponda a un residente registrado y autorizado dentro del sistema.

Figura 134

Implementación del servicio de validación QR

Nota: La figura muestra el código fuente donde se desarrolla la lógica encargada de validar los códigos QR recibidos por la API REST. Elaboración propia.

También, se implementó el servicio encargado de recibir la información del código QR, enviado desde la aplicación móvil, procesando los datos contenidos en el código y realizando las comprobaciones correspondientes dentro del sistema. La API REST fue configurada mediante un endpoint específico para atender las solicitudes de validación, ilustrado en la *Figura 135*, permitiendo verificar la existencia del registro asociado, la relación con el residente correspondiente y la disponibilidad del código para ser utilizado en el proceso de acceso.

Figura 135

Implementación del endpoint de validación

Nota: La figura muestra el controlador desarrollado para exponer el servicio de validación mediante la API REST. Elaboración propia.

De igual forma, se incorporaron validaciones para garantizar la confiabilidad del proceso, considerando escenarios como códigos QR inválidos, información inexistente o datos que no correspondan con un usuario registrado, ilustrado en la *Figura 136*. En estos casos, el servicio genera respuestas controladas que permiten identificar si el acceso puede ser autorizado o si debe ser rechazado, proporcionando información clara para que la aplicación móvil pueda mostrar el resultado de la validación al usuario encargado del control de ingreso.

Figura 136

Interfaz de Código inválido

Nota: La figura muestra cuando el código es inválido por varios motivos, como expiración o ya fue usado. Elaboración propia.

Además, se estableció la estructura de respuesta del servicio para facilitar la integración con los demás componentes. La API REST devuelve información procesada sobre el resultado de la validación, permitiendo diferenciar entre una solicitud exitosa y una validación no aprobada.

Finalmente, se realizaron pruebas funcionales del servicio mediante solicitudes a la API REST, verificando el comportamiento del sistema ante códigos QR válidos e inválidos, comprobando que la respuesta generada sea correcta y que la lógica de validación cumpla con el objetivo planteado. En la *Figura 137* se muestra que esta funcionalidad se logró implementar la base del proceso de control de acceso mediante códigos QR, permitiendo integrar posteriormente la interfaz móvil de escaneo y registro de ingresos dentro del prototipo de aplicación.

Figura 137

Interfaz de Código válido

Nota: La figura muestra cuando el código es válido para permitir el ingreso de visitantes. Elaboración propia.

Tarea 4. Implementar e integrar la interfaz de validación de códigos QR.

Durante esta tarea se implementó la interfaz de validación de códigos QR destinada al personal de seguridad, con el objetivo de agilizar el proceso de verificación de visitantes en los accesos de la urbanización. Como se muestra en la *Figura 138*, se desarrolló la pantalla siguiendo el diseño definido para el prototipo e incorporó un lector de códigos QR mediante la cámara del dispositivo móvil, además de una vista donde se presentan los datos obtenidos del código escaneado, como el nombre del visitante, cédula, el residente asociado, la fecha y hora de vigencia y el estado de la autorización. También se añadieron indicadores visuales que permiten identificar de forma inmediata si el código es válido, ha expirado o ya fue utilizado.

Figura 138

Interfaz de validación de QR

Nota: Pantalla que muestra la validación del código QR para el ingreso. Elaboración propia.

De la misma forma, se integró la interfaz con el servicio de validación de la API REST, permitiendo que cada código escaneado sea enviado automáticamente al servidor para verificar su autenticidad y vigencia. De acuerdo con la respuesta obtenida, la aplicación muestra al guardia el resultado de la validación y la información correspondiente del visitante en tiempo real, como se muestra en la *Figura 139*, posteriormente el guardia registra la placa del vehículo para culminar con la validación del ingreso.

Figura 139

Interfaz de datos del visitante

Nota: Pantalla que muestra los datos del visitante cuando se valida el código QR. Elaboración propia.

Finalmente, se realizaron pruebas funcionales para comprobar el correcto funcionamiento del lector QR, la comunicación con el backend y la visualización de los distintos escenarios de validación, como se muestra en la *Figura 140*, asegurando un proceso de acceso ágil, confiable y acorde con los requisitos establecidos para el prototipo.

Figura 140

Interfaz de acceso exitoso

Nota: Pantalla que muestra el acceso exitoso. Elaboración propia.

Historia de usuario 13

IPB03 Consultar historial de ingresos

Tarea 5. Desarrollar el servicio de consultar historial de ingresos QR en la API REST.

Se desarrolló el servicio de consulta del historial de ingresos mediante códigos QR dentro de la API REST. El objetivo principal de esta funcionalidad fue permitir la recuperación y consulta de los registros de accesos generados mediante la validación de códigos QR, proporcionando

información organizada sobre los ingresos realizados dentro del sistema. Por otra parte, se implementó la estructura necesaria para gestionar las solicitudes de consulta, separando la lógica de negocio encargada de procesar la información de accesos y la capa de datos responsable de obtener los registros almacenados, como se observa en la *Figura 141*.

Figura 141

Implementación del servicio de consulta de ingresos QR en NestJS

(Método encargado de consultar el historial de ingresos)

Nota: La figura muestra el código fuente donde se desarrolla la lógica encargada de consultar los registros de ingresos generados mediante códigos QR. Elaboración propia.

Posteriormente, se desarrolló el endpoint encargado de consultar el historial de ingresos registrados mediante códigos QR, ilustrado en la *Figura 142*. Este servicio permite recibir las solicitudes provenientes de la aplicación móvil, ejecutar la consulta correspondiente y retornar la información almacenada en la base de datos.

Figura 142

Implementación del endpoint REST de consulta

Nota: La figura muestra el controlador desarrollado para exponer el servicio de consulta del historial de ingresos mediante la API REST. Elaboración propia.

Estos datos contienen la información necesaria para identificar los accesos registrados, como la fecha y hora del ingreso, el usuario o residente asociado y el resultado de la validación del código QR, permitiendo presentar un historial organizado de los registros generados, ilustrado en la *Figura 143*.

Figura 143

Consulta de registros almacenados en la base de datos

Nota: La figura muestra la validación de que los datos entregados por la API corresponden a los registros almacenados después del proceso de validación QR. Elaboración propia.

Finalmente, se realizaron pruebas funcionales del servicio mediante solicitudes a la API REST, verificando que los registros de ingreso sean recuperados correctamente y que la información retornada coincida con los datos almacenados en el sistema. Se comprobó la correcta comunicación entre las capas del backend y la disponibilidad del servicio para su posterior integración con la interfaz móvil.

Tarea 6. Implementar e integrar la interfaz de consulta del historial de ingresos.

En esta tarea se implementó la interfaz móvil para la consulta del historial de ingresos registrados mediante códigos QR, como se observa en la *Figura 144*. El fin de esta funcionalidad fue proporcionar una pantalla que permita al administrador consultar de manera organizada los registros de accesos realizados, facilitando el seguimiento de los ingresos generados mediante la validación de códigos QR y contribuyendo al control y monitoreo de la seguridad dentro de la urbanización.

Figura 144

Interfaz del historial de ingresos

Nota: La figura muestra la consulta del historial de ingresos en funcionamiento. Elaboración propia.

Se diseñó una pantalla con una estructura orientada a la visualización de información histórica, donde los registros de ingreso se presentan mediante una lista organizada que muestra datos relevantes como el residente asociado, la fecha y hora del acceso, el tipo de ingreso realizado mediante código QR y el estado de la validación.

Posteriormente, se implementó el consumo del servicio encargado de consultar el historial de ingresos QR, enviando las solicitudes HTTP y procesando la información recibida desde el

servidor. Los registros obtenidos se muestran dinámicamente en la interfaz, manteniendo sincronizada la información con el backend, ilustrado en la *Figura 145*.

Figura 145

Implementación del servicio de la consulta de historial

(captura de que la interfaz consume correctamente el servicio o API y muestra la información esperada)

Nota: La figura muestra la evidencia de que la interfaz consume correctamente el servicio o API y muestra la información esperada. Elaboración propia.

Además, se incorporó el manejo de estados de la interfaz para controlar la carga de información, la existencia o ausencia de registros y los posibles errores durante la comunicación con la API REST, mejorando la experiencia de navegación del usuario.

Finalmente, se realizaron pruebas funcionales para verificar la integración entre la aplicación móvil y el servicio, validando la recuperación y visualización del historial de accesos, así como el comportamiento del sistema en diferentes escenarios. Con esta implementación se fortalece el módulo de control de accesos, permitiendo consultar y supervisar los ingresos realizados mediante códigos QR dentro de la urbanización privada.

Historia de usuario 14

IPB04 Reserva un área social

Tarea 7. Desarrollar el servicio de reservas de áreas sociales en la API REST.

En esta tarea se desarrolló el servicio de reservas de áreas sociales en la API REST. Se implementó la estructura necesaria para gestionar las solicitudes de reserva realizadas por los residentes desde la aplicación móvil, como se muestra *Figura 146*, estableciendo una comunicación segura entre el frontend y el backend mediante un endpoint REST. Además, se

definieron los modelos de datos y los mecanismos de persistencia que permitieron almacenar la información de cada reserva de forma organizada y consistente.

Figura 146

Código del servicio de reservas

Nota: La figura muestra un fragmento de código fuente de servicio en NestJS de las reservas de áreas sociales. Elaboración propia.

Posteriormente, se incorporó la lógica de negocio encargada de validar cada solicitud antes de su registro, como se muestra *Figura 147*. Para esta funcionalidad, se verificó que el área social estuviera disponible en la fecha y horario seleccionados, evitando solapamientos de reservas. De este modo, se validó que la información recibida cumpliera con los requisitos establecidos, garantizando que únicamente se procesaran solicitudes con datos completos y válidos. Estas validaciones contribuyeron a mantener la integridad de la información y el correcto funcionamiento del módulo de reservas.

Figura 147

Código de las validaciones implementadas para el registro de reservas

Nota: La figura muestra un fragmento de las validaciones en las reservas. Elaboración propia.

Finalmente, como se muestra *Figura 148*, se obtuvieron resultados que evidenciaron que el servicio registró las reservas correctamente, respondió con mensajes apropiados ante operaciones exitosas o errores de validación y almacenó la información en la base de datos sin inconsistencias.

Figura 148

Interfaz del registro de una reserva

Nota: Pantalla del registro de las reservas de áreas sociales. Elaboración propia.

Tarea 8. Implementar e integrar la interfaz de reservas de áreas sociales.

En el desarrollo de esta tarea se implementó e integró la interfaz de reservas de áreas sociales en la aplicación móvil, como se muestra *Figura 149*. Se diseñó una pantalla intuitiva que permitió a los residentes seleccionar el área social, la fecha, el horario de la reserva, subir el comprobante de pago, además de visualizar la información necesaria para completar la solicitud.

Figura 149

Código de la interfaz de reservas

Nota: Pantalla del registro de las reservas de áreas sociales. Elaboración propia.

Posteriormente, la interfaz se integró con la API REST mediante el consumo del servicio de registro de reservas, como se muestra *Figura 150*. Para ello, se implementó la comunicación entre la aplicación móvil y el backend utilizando solicitudes HTTP, enviando la información ingresada por el usuario y procesando las respuestas emitidas por la API. Además, se incorporaron validaciones en la interfaz para verificar el ingreso de los datos obligatorios y mostrar mensajes informativos en caso de errores o cuando la reserva se registrará correctamente.

Figura 150

Servicio de comunicación con la API REST

Nota: La figura muestra el servicio de comunicación del registro de reservas con la API REST. Elaboración propia.

Finalmente, se realizaron pruebas funcionales para verificar el correcto funcionamiento de la integración entre la aplicación móvil y la API REST. Los resultados evidenciaron que la información de las reservas se transmitió correctamente al servidor, fue procesada sin inconvenientes y quedó almacenada en la base de datos. Con esta implementación se logró

completar el flujo de reserva de áreas sociales, permitiendo que los residentes gestionen sus solicitudes desde la aplicación móvil de manera eficiente y confiable.

Historia de usuario 15

IPB05 Aprobar o rechazar las reservas

Tarea 9. Desarrollar el servicio de aprobar o denegar reservas en la API REST.

Se desarrolló el servicio de aprobación o denegación de reservas de áreas sociales en la API REST, como se muestra *Figura 151*. Luego se implementó un endpoint que permitió al administrador gestionar las solicitudes de reserva enviadas por los residentes, modificando el estado de cada registro según la decisión tomada. Además, se definieron las reglas necesarias para garantizar que únicamente los usuarios con los permisos correspondientes pudieran ejecutar esta funcionalidad.

Figura 151

Implementación del servicio de la aprobación/denegación de reservas

Nota: Fragmento del código fuente del servicio de aprobación o denegación de las reservas. Elaboración propia.

Se incorporó la lógica de negocio para validar el estado actual de cada reserva antes de actualizarla. El servicio verificó la existencia de la reserva, comprobó que esta se encontrara pendiente de revisión y evitó que una misma solicitud fuera aprobada o denegada en más de una ocasión, como se observa en la *Figura 152*. De igual forma, se registró el nuevo estado de la reserva en la base de datos, manteniendo la consistencia e integridad de la información.

Figura 152

Respuesta de la API REST durante el proceso de aprobación/denegación

Nota: La figura muestra la respuesta de la API REST en el proceso de aprobación o denegación de las reservas. Elaboración propia.

Finalmente, se realizaron pruebas funcionales para verificar el correcto funcionamiento del servicio. Los resultados evidenciaron que la API REST procesó correctamente las solicitudes de aprobación y denegación, actualizó el estado de las reservas en PostgreSQL y devolvió respuestas adecuadas para cada operación realizada.

Tarea 10. Implementar e integrar la interfaz de consulta del historial de ingresos.

En esta tarea se desarrolló una pantalla que permitió a los usuarios visualizar el registro de ingresos realizados a la urbanización, presentando información como la fecha de acceso, el visitante, el residente relacionado y el estado del ingreso, como se muestra en la *Figura 153*. La interfaz fue diseñada con una distribución clara y organizada para facilitar la consulta de la información almacenada en el sistema.

Figura 153

Código de la interfaz del historial de ingresos

Nota: La figura fragmento de código fuente de la interfaz del historial de ingresos. Elaboración propia.

Se integró con la API REST mediante el consumo del servicio de consulta del historial de ingresos. Para ello, se implementó la comunicación entre la aplicación móvil y el backend por medio de solicitudes HTTP, obteniendo la información registrada en la base de datos y mostrándola de forma dinámica en la aplicación, como se muestra en la *Figura 154*. También, se incorporó el manejo de respuestas y mensajes informativos para los casos en que no existieran registros o se presentaran errores durante la consulta.

Figura 154

Interfaz de consulta del historial de ingresos

Nota: Pantalla de historial de ingreso. Elaboración propia.

Finalmente, se realizaron pruebas funcionales para verificar el correcto funcionamiento de la integración entre la aplicación móvil y la API REST. Los resultados demostraron que el historial de ingresos se recuperó correctamente desde PostgreSQL y se visualizó de manera adecuada en la interfaz, permitiendo a los usuarios consultar los registros de acceso de forma rápida y confiable.

Historia de usuario 16

IPB06 Consultar estado de reserva

Tarea 11. Desarrollar el servicio de consultar el estado de reservas en la API REST.

Se desarrolló el servicio de consulta del estado de reservas en la API REST, así mismo se implementó un endpoint que permitió obtener la información de las reservas registradas, mostrando su estado actual, ya sea pendiente, aprobada o denegada, como se muestra en la *Figura 155*. Este servicio facilitó la consulta de la información desde la aplicación móvil, permitiendo a los residentes conocer el estado de sus solicitudes de reserva.

Figura 155

Prueba del endpoint de consulta del estado de la reserva

Nota: La figura muestra la prueba del endpoint de la consulta del estado de la reserva. Elaboración propia.

Tarea 12. Implementar e integrar la interfaz consulta de reservas.

Durante esta tarea se desarrolló una pantalla que permitió a los residentes visualizar el listado de sus reservas de áreas sociales, mostrando información relevante como el área reservada, la fecha, el horario y el estado de cada solicitud, como se muestra en la *Figura 156*. La interfaz fue diseñada con una estructura clara y de fácil navegación, facilitando la consulta de la información desde dispositivos móviles.

Figura 156*Código de la interfaz de historial de reservas*

Nota: Fragmento de código fuentes del historial de reservas. Elaboración propia.

Así mismo, la interfaz se integró con la API REST mediante el consumo del servicio de consulta de reservas. Para ello, se implementó la comunicación entre la aplicación móvil y el backend utilizando solicitudes HTTP, obteniendo la información almacenada en PostgreSQL y presentándola de forma dinámica en la interfaz. De igual forma, se incorporó el manejo de mensajes informativos para notificar al usuario cuando no existieran reservas registradas o se produjera algún error durante la consulta, como se muestra en la *Figura 157*.

Figura 157*Mensaje de consulta de historial de reservas*

Nota: Mensaje de notificación cuando se consulta el historial de reservas. Elaboración propia.

Se culminó la tarea exitosamente y los resultados evidenciaron que las reservas se recuperaron correctamente desde la base de datos y se visualizaron de forma adecuada en la interfaz, como se muestra en la *Figura 158*. Con esta implementación se completó el proceso de consulta de reservas, permitiendo a los residentes realizar el seguimiento de sus solicitudes de manera rápida, sencilla y confiable.

Figura 158*Interfaz de historial de reservas*

Nota: Pantalla de historial de reservas. Elaboración propia.

Historia de usuario 17

IPB07 Consultar estado de reserva

Tarea 13. Desarrollar el servicio de registro de comprobantes de reservas en la API REST.

Se desarrolló el servicio de registro de comprobantes de reservas en la API REST, con el propósito de permitir que los residentes envíen el comprobante de pago correspondiente a la reserva de un área social para su posterior validación por parte del administrador, como se muestra en la *Figura 59*. Para ello, se implementó un endpoint encargado de recibir la información de la reserva junto con el archivo del comprobante, validando la integridad de los datos antes de almacenarlos en la base de datos.

Figura 159

Implementación del servicio del estado de la reserva

Nota: Fragmento del código fuente del servicio de consulta de estado de las reservas. Elaboración propia.

Posteriormente, el servicio fue integrado con la base de datos y con la interfaz de la aplicación móvil, permitiendo que los comprobantes enviados por los usuarios sean registrados de forma segura y queden disponibles para su revisión, como se muestra en la *Figura 160*. Además, se realizaron pruebas funcionales para verificar el correcto almacenamiento de la información, la gestión de posibles errores durante el envío del archivo y la respuesta del servicio ante diferentes escenarios de uso, garantizando un proceso confiable y consistente dentro del módulo de reservas.

Figura 160

Código de integración entre el servicio y la interfaz

Nota: Fragmento del código fuente de la integración del servicio de la consulta de estado de reserva con la base de datos y la interfaz de la aplicación móvil. Elaboración propia.

Tare 14. Implementar e integrar la interfaz de cargar de comprobantes de reserva.

En esta tarea se implementó la interfaz de carga de comprobantes de reserva, con el objetivo de permitir que los residentes adjunten el comprobante de pago correspondiente a la reserva de un área social de manera sencilla e intuitiva. Como se observa en la *Figura 161*, se desarrolló la pantalla e incorporó opciones para seleccionar una imagen o documento desde el dispositivo. Además, se incluyeron validaciones para comprobar que el archivo seleccionado cumpliera con los formatos permitidos ya sea JPG o PDF.

Figura 161

Código de interfaz de cargar comprobante de reserva

Nota: Fragmento del código fuente la interfaz donde se carga el comprobante de pago de una reserva. Elaboración propia.

Así mismo, se integró la interfaz con el servicio de registro de comprobantes de la API REST, permitiendo que la información y el archivo sean enviados de forma segura al servidor para su almacenamiento. También, se implementó la visualización de mensajes de confirmación y notificaciones de error durante el proceso de carga, mejorando la experiencia del usuario, como se observa en la *Figura 162*.

Figura 162

Interfaz de mensaje de error de carga

Nota: Muestra la notificación de error en el proceso de carga del comprobante. Elaboración propia.

Finalmente, se realizaron pruebas funcionales para verificar el correcto envío del comprobante, la comunicación con el backend y el manejo de distintos escenarios de uso, asegurando un funcionamiento estable y acorde con los requisitos del prototipo.

Sprint 5. Desarrollo del módulo de reportes e integración del sistema.

Tabla 64

Sprint Planning 5

SPRINT 5				
ID de Product Backlog	Historia de usuario	Tarea	Responsable	Estado
IPB01	Desarrollo del módulo de reportes	1. Desarrollar el servicio de generación de reportes en la API REST. 2. Implementar e integrar la interfaz de consulta de reportes.	Jhonas López, Belky Piedra	Completado
IPB02	Integración de todos los módulos	3. Integrar los módulos funcionales con la API REST.	Jhonas López, Belky Piedra	Completado

Nota: La tabla detalla las historias de usuario, tareas, responsables y estado del Sprint 5 para el progreso y desarrollo del proyecto. Elaboración propia.

Historia de usuario 18

IPB01 Desarrollo del módulo de reportes

Tarea 1. Desarrollar el servicio de generación de reportes en la API REST.

Durante esta tarea se desarrolló el servicio de generación de reportes en la API REST, con el propósito de consolidar la información registrada por los diferentes módulos del sistema y ponerla a disposición del administrador para su consulta, como se muestra en la *Figura 163*. Para ello, se implementaron los endpoints encargados de obtener los datos almacenados en la base de datos, aplicando filtros por períodos para los reportes de pagos, accesos y reservas, de acuerdo con los requerimientos funcionales definidos para el módulo de reportes.

Figura 163

Código del controlador del módulo de reportes

Nota: Fragmento del código fuente del controlador del módulo de reportes. Elaboración propia.

De igual forma, se estructuró la respuesta del servicio para facilitar su consumo desde la aplicación móvil, como se muestra en la *Figura 164*. Posteriormente, el servicio fue integrado con la base de datos PostgreSQL, garantizando consultas eficientes y la consistencia de la información recuperada.

Figura 164

Código del servicio

Nota: Fragmento del código fuente del servicio del módulo de reportes. Elaboración propia.

Finalmente, se realizaron pruebas funcionales para verificar el correcto funcionamiento de los endpoints, como se muestra en la *Figura 165*. La precisión de los datos obtenidos y la respuesta del servicio ante diferentes criterios de búsqueda, asegurando que la información presentada corresponda con los registros almacenados en el sistema.

Figura 165

Definición de las rutas de las API REST

Nota: La figura muestra las rutas endpoints de las API REST. Elaboración propia.

Tarea 2. Implementar e integrar la interfaz de consulta de reportes.

En esta tarea se implementó la interfaz de consulta de reportes en la aplicación móvil, orientada a proporcionar al administrador una visualización clara y organizada de la información generada por el sistema. De igual forma, se desarrolló la pantalla siguiendo el diseño establecido, como se observa en la *Figura 166*, también se incorporó opciones para consultar los registros mediante filtros de búsqueda permitiendo visualizar el historial de pagos de alícuotas, accesos y las reservas de forma rápida e intuitiva.

Figura 166*Código de la pantalla principal del módulo de reportes*

Nota: Fragmento del código fuente de la pantalla principal del módulo de reportes. Elaboración propia.

Una vez finalizada la interfaz, se realizó su integración con el servicio de generación de reportes de la API REST, permitiendo recuperar y mostrar la información en tiempo real. Además, se implementó el manejo de estados de carga, mensajes de error y notificaciones cuando no existían registros para los criterios seleccionados, como se observa en la *Figura 167*.

Figura 167*Interfaz del Módulo de reportes*

Nota: Interfaz del módulo de reportes. Elaboración propia.

Finalmente, como se muestra en la *Figura 168* y *169*, se ejecutaron pruebas funcionales para comprobar la correcta comunicación entre el frontend y el backend, así como la visualización adecuada de los datos obtenidos.

Figura 168

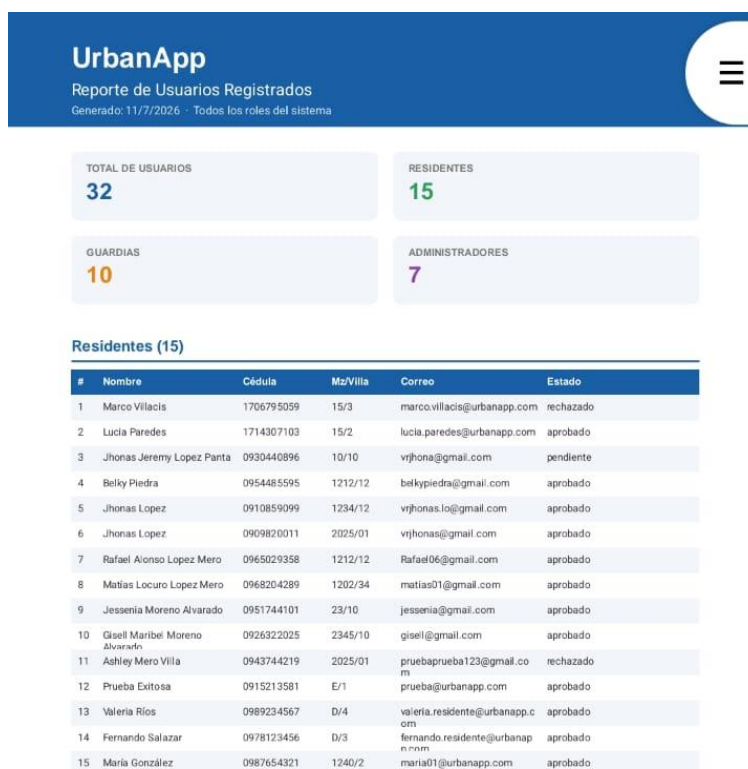
Resumen de reporte de usuarios registrados



Nota: Interfaz del resumen de los usuarios registrados. Elaboración propia.

Figura 169

Reporte de Usuarios registrados



Nota: Interfaz del reporte de los usuarios registrados. Elaboración propia.

IPB02 Integración de todos los módulos

Tarea 3. Integrar los módulos funcionales con la API REST.

Durante esta tarea se realizó la integración de los módulos funcionales desarrollados en los sprints anteriores, permitiendo la comunicación entre la aplicación móvil y la API REST. Se unificaron los módulos de autenticación, gestión de usuarios, control de pagos, control de accesos, reservas y reportes, verificando que el intercambio de información se efectuara de forma correcta a través de los servicios implementados, como se muestra en la *Figura 170*. De igual forma, se validó que cada módulo mantuviera la coherencia de los datos y respetara el flujo de navegación definido para el prototipo.

Figura 170*Código de integración de servicios en Flutter*

Nota: Fragmento del código de fuente de la integración de servicios en Flutter. Elaboración propia.

Así mismo, se verificó la interoperabilidad entre el frontend desarrollado en Flutter y el backend implementado en NestJS, comprobando el consumo de los diferentes endpoints y la persistencia de la información en PostgreSQL. Como resultado, se obtuvo un sistema integrado en el que las funcionalidades interactúan de manera coordinada, garantizando el correcto funcionamiento de los procesos principales de la aplicación.

Se presenta el flujo completo de interacción entre los módulos del sistema, iniciando con el registro del comprobante de pago por parte del residente, como se muestra en la *Figura 171*.

Figura 171*Flujo entre módulos - Registro de pago*

Nota: Flujo entre módulos, registro de pago. Elaboración propia.

Como se observa en la *Figura 172* se realizó la validación realizada por el administrador, como resultado se muestra la validación del pago de la alícuota.

Figura 172*Flujo entre módulos - Validación del pago*

Nota: Flujo entre módulos, validación del pago. Elaboración propia.

Finalmente, en la *Figura 173* se muestra la generación y consulta del reporte de pagos, evidenciando la integración y el funcionamiento coordinado de estas funcionalidades.

Figura 173*Flujo entre módulos - Reporte de pago*

Nota: Flujo entre módulos, reporte de pagos. Elaboración propia.

Retroalimentación del Sprint 5

Al finalizar el sprint, se verificó el cumplimiento de los objetivos planteados mediante la implementación del módulo de reportes y la integración de las funcionalidades desarrolladas en los sprints anteriores. Las pruebas realizadas evidenciaron una comunicación adecuada entre la aplicación móvil, la API REST y la base de datos, permitiendo que los procesos de autenticación, gestión de usuarios, control de pagos, control de accesos, reservas y generación de reportes funcionaran de manera integrada y consistente.

Durante la revisión del sprint se identificaron oportunidades de mejora relacionadas con la optimización de algunas consultas, el manejo de mensajes de error y la experiencia de usuario en determinadas interfaces. Estas observaciones fueron atendidas mediante ajustes en el prototipo, incrementando la estabilidad, el rendimiento y la usabilidad de la aplicación.

Finalmente, el equipo concluyó que los objetivos del sprint fueron alcanzados satisfactoriamente, ya que las funcionalidades planificadas fueron implementadas e integradas de acuerdo con los criterios de aceptación establecidos. Como resultado, se obtuvo un prototipo funcional con un alto nivel de integración entre sus módulos, preparado para la ejecución de las pruebas funcionales y la posterior validación mediante juicio de expertos, etapas que permitirán evaluar la calidad de la solución propuesta y su cumplimiento respecto a los objetivos del proyecto de investigación.

Beneficiarios Directos e Indirectos del Proyecto

Los beneficiarios del presente proyecto están conformados por los distintos actores que, de forma directa o indirecta, obtendrán beneficios como resultado de la implementación del prototipo de aplicación móvil para el control de pagos de alícuotas, el acceso mediante códigos QR y la gestión de reservas de áreas sociales en una urbanización privada. La solución propuesta contribuirá a optimizar los procesos administrativos, fortalecer el control de acceso, mejorar la organización de las reservas y facilitar la consulta de información en tiempo real. A continuación, se detallan los beneficiarios según su tipo:

Beneficiarios directos

Son aquellas personas y organizaciones que estarán enteramente involucradas con el uso y manejos del prototipo propuesto, y que verán los cambios inmediatos en el desarrollo de sus procesos. En este contexto se identifican como beneficiarios directos a los siguientes actores:

- **Residentes.** Quienes podrán contar con una aplicación móvil que les permitirá realizar de manera ágil y segura diversas gestiones relacionadas con la administración de la urbanización. A través de la plataforma podrán consultar el estado de sus pagos de alícuotas, registrar comprobantes de pago, generar códigos QR para el ingreso de visitantes, realizar reservas de áreas sociales y conocer el estado de sus solicitudes en tiempo real, sin necesidad de acudir personalmente a la administración. Esto favorecerá una comunicación más eficiente con la administración, incrementará la transparencia en la gestión de los procesos y ofrecerá un servicio más moderno, organizado y confiable dentro de la urbanización.

- **Personal de seguridad.** De igual forma podrán contar con una herramienta que facilitará el control de acceso a la urbanización mediante la validación de códigos QR y el registro de visitantes. La aplicación permitirá verificar de forma inmediata la autenticidad y vigencia de los códigos generados por los residentes, reduciendo el tiempo requerido para autorizar el ingreso y disminuyendo la dependencia de registros manuales, lo que contribuirá a un proceso de acceso más ágil y ordenado.
- **Personal administrativo.** Contará con una herramienta tecnológica que facilitará la gestión de los procesos relacionados con el control de pagos de alícuotas, la validación de comprobantes, la administración de reservas de áreas sociales y la generación de reportes. La automatización de estas actividades permitirá reducir el tiempo dedicado a tareas manuales, minimizar errores en el registro de la información, mantener un mayor control sobre las operaciones y disponer de reportes actualizados para apoyar la toma de decisiones.

Beneficiarios indirectos

Los beneficiarios indirectos del proyecto corresponden a los actores relacionados con la dinámica administrativa y operativa de la urbanización, quienes, aunque no interactúan directamente con todas las funcionalidades del prototipo, reciben beneficios derivados de la optimización de los procesos internos. Entre ellos se encuentra los visitantes ya que obtienen beneficios indirectos debido a la optimización del proceso de ingreso mediante códigos QR, reduciendo tiempos de espera en garita y mejorando la experiencia de acceso a la urbanización.

Por otro lado, también se contempla a la directiva que se beneficia indirectamente al contar con una gestión más organizada y transparente de la información relacionada con pagos, accesos

y uso de áreas comunes. Esto facilita la toma de decisiones basadas en datos actualizados y mejora la planificación de actividades internas.

Así como también las empresas de mantenimiento, servicios técnicos, delivery u otros proveedores autorizados que se benefician de procesos de acceso más organizados, disminuyendo inconvenientes durante los controles de ingreso y mejorando la coordinación con los residentes.

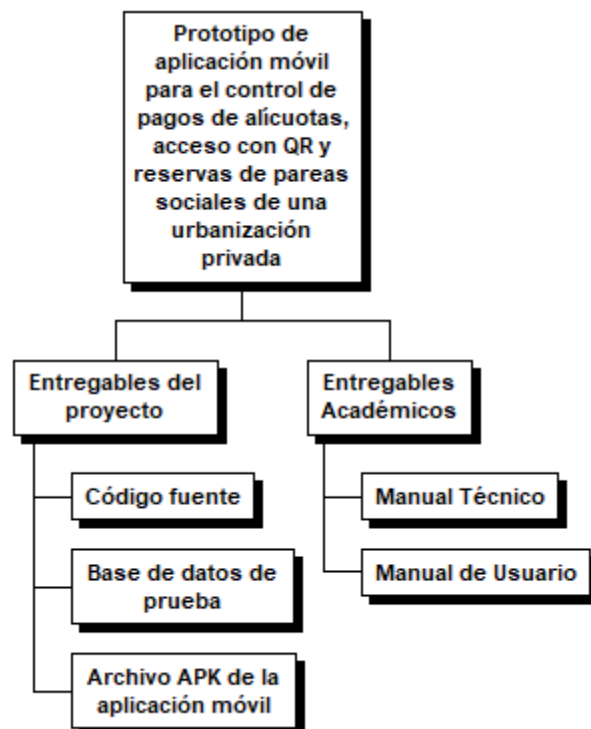
Entregables del proyecto

De acuerdo con la metodología ágil Scrum adoptada para el desarrollo del prototipo de aplicación móvil, los entregables del proyecto se desarrollarán de manera iterativa e incremental, permitiendo la incorporación progresiva de funcionalidades y la validación continua del producto durante cada sprint. Este enfoque facilitará la adaptación a los requerimientos definidos y garantizará la entrega gradual de valor a los principales involucrados, como residentes, personal administrativo y personal de seguridad de la urbanización privada. Al finalizar el proyecto, se obtendrán los siguientes entregables principales:

- Código fuente
- Archivo APK de la aplicación
- Base de datos de prueba
- Manual Técnico
- Manual de Usuario

Figura 174

Estructura del Desglose del Trabajo (EDT)



Nota: La EDT representa de forma jerárquica los entregables del proyecto. Elaboración propia en la herramienta WBS Chart Pro.

Propuesta

La propuesta consiste en el desarrollo de un prototipo de aplicación móvil para el control de pagos de alcuotas, accesos con QR y reservas de áreas sociales de una urbanización privada. La solución busca reemplazar los procedimientos manuales y semimanuales actualmente utilizados por mecanismos automatizados, centralizados y accesibles, permitiendo mejorar la organización de la información, reducir errores en los registros y disminuir la carga operativa del personal encargado.

El prototipo está desarrollado aplicando una metodología ágil basada en Scrum, mediante entregas iterativas e incrementales que permitan validar progresivamente las funcionalidades implementadas. La aplicación permite a los residentes realizar consultas y gestionar sus solicitudes de manera más eficiente, mientras que el personal administrativo y de seguridad contará con

herramientas digitales para facilitar el control de pagos, la verificación de ingresos y la administración de espacios comunes. La propuesta está basada en una arquitectura de tres capas, integrada por una aplicación móvil desarrollada en Flutter, un backend implementado mediante una API REST con NestJS y una base de datos PostgreSQL encargada de la persistencia de la información. Esta arquitectura permitirá centralizar y gestionar los registros de forma segura y eficiente, garantizando la trazabilidad de la información y facilitando los procesos de consulta, auditoría y control, además de favorecer la escalabilidad, el mantenimiento y la integración entre los diferentes componentes del sistema.

La propuesta contempla cuatro módulos funcionales principales que permitirán gestionar de manera integral los procesos administrativos y operativos de la urbanización privada.

- El primer módulo corresponde a la gestión de usuarios, mediante el cual se administrarán los perfiles de residentes, administradores y personal de seguridad, asignando los permisos y funcionalidades correspondientes a cada rol.
- El segundo módulo corresponde al control de pagos de alícuotas, donde los residentes podrán consultar sus alícuotas pendientes, registrar el pago mediante la carga de comprobantes y revisar el historial de transacciones realizadas. Por su parte, el administrador tendrá la capacidad de generar las alícuotas periódicas, validar los comprobantes recibidos y aprobar o denegar los pagos según corresponda.
- El tercer módulo está orientado al control de accesos mediante códigos QR. En este proceso, el residente podrá registrar a sus visitantes y generar un código QR con fecha y hora de vigencia para autorizar su ingreso. El personal de seguridad será responsable de validar los códigos QR al momento del acceso, consultar la información del visitante

vinculado al residente que autorizó su ingreso y realizar registros manuales en situaciones excepcionales cuando no sea posible utilizar el código QR.

- El cuarto módulo corresponde a la gestión de reservas de áreas sociales, permitiendo a los residentes realizar solicitudes de reserva, cancelar reservas, registrar el comprobante de pago correspondiente, consultar el estado de sus solicitudes y revisar el historial de reservas efectuadas.
- Finalmente, la propuesta incorpora un módulo de reportes, destinado al administrador, desde el cual podrá consultar y descargar informes relacionados con los pagos de alcuotas, los registros de acceso mediante códigos QR, las reservas de áreas sociales y la información de los usuarios registrados. Estos reportes facilitarán el seguimiento de las operaciones, fortalecerán la transparencia de la gestión y servirán como apoyo para la toma de decisiones administrativas y sus respectivas auditorías.

Adicionalmente, la propuesta incluye un manual de usuario, en el *Anexo x*, destinado a los beneficiarios directos, con el propósito de orientar el uso adecuado de la aplicación. Este documento describirá el funcionamiento de cada módulo y las principales funcionalidades del sistema, facilitando su comprensión, implementación y utilización por parte de residentes, administradores y personal de seguridad.

Criterios de validación de la propuesta

La validación de la propuesta se realizará mediante la técnica de juicio de expertos, un método ampliamente utilizado en investigaciones de desarrollo tecnológico para determinar la calidad, pertinencia y viabilidad de una solución antes de su implementación. Este proceso

permitirá obtener una valoración objetiva del prototipo a partir de la experiencia y conocimientos especializados de profesionales del área.

El juicio de expertos estará conformado por tres profesionales con experiencia en el desarrollo de aplicaciones móviles e ingeniería de software, quienes evaluarán la propuesta desde una perspectiva técnica y funcional, ilustrados en la *Tabla 65*. La selección de los expertos se fundamentará en su formación académica, experiencia profesional y conocimientos en el diseño, desarrollo e implementación de aplicaciones móviles, garantizando así la confiabilidad y objetividad de los resultados obtenidos.

Tabla 65

Profesionales que conforman el juicio de expertos

Nombres y Apellidos	Título Profesional	Institución	Años de experiencia
----------------------------	---------------------------	--------------------	----------------------------

Nota: La tabla presenta los expertos que realizaron la evaluación y validación del prototipo. Elaboración propia.

Para llevar a cabo la validación, se utilizará un instrumento de evaluación estructurado, el cual permitirá valorar aspectos fundamentales del prototipo, entre ellos la funcionalidad, usabilidad, seguridad, diseño de la interfaz, consistencia de los procesos, cumplimiento de los requerimientos funcionales y no funcionales, viabilidad técnica, escalabilidad y pertinencia de la solución frente a la problemática identificada, como se muestra en la *Tabla 66*. Cada criterio será calificado por los expertos y complementado con observaciones y recomendaciones orientadas al fortalecimiento de la propuesta.

Tabla 66

Preguntas para los criterios de evaluación

Criterio	Pregunta de evaluación
Claridad	¿La propuesta presenta una redacción clara, comprensible y organizada, facilitando la interpretación de sus objetivos, funcionalidades y alcance?
Objetividad	¿La propuesta se fundamenta en criterios técnicos y metodológicos objetivos, evitando ambigüedades y subjetividades en el planteamiento de la solución?
Actualidad	¿Las tecnologías, herramientas y metodologías empleadas en la propuesta son actuales y pertinentes para el desarrollo de aplicaciones móviles?
Suficiencia	¿La propuesta contempla las funcionalidades necesarias para atender de manera integral los procesos de control de pagos de alcúotas, acceso mediante códigos QR y reservas de áreas sociales?
Intencionalidad	¿La propuesta responde de manera adecuada al problema identificado y contribuye al cumplimiento de los objetivos planteados en el proyecto?
Consistencia	¿Existe coherencia entre el problema de investigación, los objetivos, los requerimientos del sistema, la arquitectura propuesta y las funcionalidades implementadas en el prototipo?
Metodología	¿La metodología Scrum y el proceso de desarrollo propuesto son adecuados para la planificación, construcción y validación del prototipo de aplicación móvil?
Aplicabilidad	¿Considera que el prototipo puede implementarse de manera factible en una urbanización privada para mejorar la gestión de pagos, accesos y reservas?

Nota: La tabla presenta las preguntas de los criterios de evaluación para el juicio de expertos. Elaboración propia.

Resultados

Los resultados obtenidos mediante el juicio de expertos fueron clasificados dentro de un rango de 0 a 100 que permitió determinar la calidad y funcionalidad del prototipo. Como se observa en la *Tabla 67*, la tabulación de las puntuaciones asignadas por los tres especialistas en cada uno de los criterios de evaluación. A partir de estas calificaciones se obtuvo el promedio correspondiente a cada criterio, lo que permitió consolidar la valoración global del instrumento y verificar la pertinencia, calidad y viabilidad de la propuesta desarrollada.

Tabla 67

Tabulación de resultados del juicio de expertos

Indicador	E1	E2	E3	Promedio
Claridad				
Objetividad				
Actualidad				

Suficiencia
Intencionalidad
Consistencia
Metodología
Aplicabilidad

Nota: La tabla presenta los resultados y promedio obtenidos por el juicio de expertos. Elaboración propia.

Redactar los resultados obtenidos

CAPÍTULO IV

CONCLUSIONES Y RECOMENDACIONES

Conclusiones

Una vez finalizado el desarrollo del prototipo, y con base en los resultados obtenidos durante las pruebas funcionales y la validación realizada mediante juicio de expertos, se exponen las conclusiones del proyecto, las cuales reflejan el cumplimiento de los objetivos específicos establecidos en la investigación. Así como también, permiten evidenciar los logros alcanzados en cada uno de los sprints desarrollados, demostrando que la propuesta implementa de manera satisfactoria las funcionalidades definidas para la gestión de pagos de alícuotas, el control de accesos mediante códigos QR y la administración de reservas de áreas sociales, contribuyendo a la optimización de los procesos administrativos y operativos de una urbanización privada.

- Se alcanzó el objetivo de analizar los procesos actuales de control de pagos de alícuotas, acceso de visitantes y reservas de áreas sociales, identificando que estas actividades dependen en gran medida de procedimientos manuales y semi manuales realizados por el personal administrativo y de seguridad. Este diagnóstico permitió fundamentar la necesidad de desarrollar una solución tecnológica que centralice la información y automatice los procesos, contribuyendo a reducir la carga operativa, minimizar errores, fortalecer el control de las operaciones y mejorar la eficiencia en la gestión de la urbanización privada.

- Se alcanzó el objetivo de diseñar la arquitectura y los módulos del prototipo, estableciendo una arquitectura de tres capas que permitió organizar la comunicación entre la aplicación móvil, la API REST y la base de datos. Además, se definieron los módulos correspondientes al control de pagos de alícuotas, acceso con QR, reservas y reportes, asegurando una adecuada organización de la información, la integración entre los componentes del sistema y una estructura tecnológica que facilita la escalabilidad, el mantenimiento y la eficiencia del prototipo desarrollado.
- Se alcanzó el objetivo de construir los módulos del prototipo de aplicación móvil, desarrollando e integrando cada una de las funcionalidades definidas en los requerimientos funcionales. Como resultado, se implementaron los módulos de gestión de usuarios, control de pagos de alícuotas, acceso con QR, reservas de áreas sociales y reportes, garantizando la interacción entre la aplicación móvil desarrollada en Flutter, la API REST implementada en NestJs y la base de datos PostgreSQL. Las pruebas funcionales realizadas confirmaron el correcto comportamiento de los módulos y demostraron que el prototipo satisface los requerimientos planteados para optimizar los procesos administrativos y operativos de la urbanización privada.
- Validación con JE.

Recomendaciones

Con base en los resultados alcanzados durante el desarrollo y validación del prototipo, se presentan las siguientes recomendaciones, orientadas a fortalecer la solución propuesta, facilitar su implementación y favorecer su mejora continua en futuras etapas de desarrollo.

- Incorporar una pasarela de pago electrónico que permita a los residentes cancelar las alícuotas y las reservas de áreas sociales directamente desde la aplicación, facilitando el proceso de pago y reduciendo la necesidad de registrar comprobantes.
- Incorporar nuevas funcionalidades que complementen la gestión de la urbanización, tales como registro de incidencias, emisión de comunicados, directorio de residentes, calendario de eventos y encuestas comunitarias.
- Integrar la aplicación con dispositivos físicos de control de acceso, como lectores de códigos QR, torniquetes, barreras vehiculares, permitiendo automatizar completamente el proceso de ingreso de visitantes y residentes.

Trabajos Futuros

El desarrollo del presente prototipo constituye una base para la implementación de futuras mejoras y la incorporación de nuevas funcionalidades que permitan ampliar su alcance y fortalecer su desempeño. En este sentido, se identifican diversas oportunidades de evolución tecnológica que contribuirán a optimizar la gestión administrativa y operativa de las urbanizaciones privadas, favoreciendo la escalabilidad, la integración con otros sistemas y la incorporación de herramientas innovadoras que respondan a las necesidades emergentes de los usuarios.

- **Implementación para dispositivos iOS.** Desarrollar una versión del prototipo compatible con el sistema operativo iOS, permitiendo su ejecución en dispositivos iPhone y iPad. Esta ampliación incrementará la cobertura de la aplicación, brindando acceso a un mayor número de usuarios y garantizando una experiencia multiplataforma. De igual manera, facilitará la adopción de la solución en urbanizaciones donde los residentes y el personal

administrativo utilicen dispositivos con diferentes sistemas operativos, fortaleciendo la accesibilidad y el alcance del sistema.

REFERENCIAS BIBLIOGRÁFICAS

- Angéloz, S. (2024). ¿Qué es Railway App? Hoy quiero compartir una herramienta que tiene... | por Serge Angéloz | Medium. <https://medium.com/@sangeloz69/what-is-railway-app-5b1bd17aa8f0>
- Ayerdi, A. (2026, enero 28). ¿Qué es la automatización de procesos? | DocuWare. <https://start.docuware.com/es/blog/automatizacion-de-procesos>
- Bayarri, C. (2024, febrero 29). Arquitectura de tres capas con generación automática de API. <https://carlosbayarricebrecos.medium.com/arquitectura-de-3-capas-con-generaci%C3%B3n-autom%C3%A1tica-de-api-78c0e62b81b3>
- Blandino, G. (2023). Figma: qué es y cómo funciona |. <https://www.pixartprinting.es/blog/figma-que-es/>
- Böhm, S., & Graser, S. (2023). AI-based Mobile App Prototyping: Status Quo, Perspectives and Preliminary Insights from Experimental Case Studies.
- Chapaval, N. (2018). Qué es Frontend y Backend: principal diferencia y ejemplos. <https://platzi.com/blog/que-es-frontend-y-backend/>
- Chávez, A. (2023). ¿Qué es Flutter? La tecnología que cambió el desarrollo multiplataforma | EDteam | EDteam. <https://ed.team/blog/que-es-flutter-la-tecnologia-que-cambio-el-desarrollo-multiplataforma>
- Correa, K., & Castro, J. (2016). Desarrollo e implementación de una aplicación para la administración de la urbanización La Joya, etapa Murano y control de alícuotas. <http://dspace.ups.edu.ec/handle/123456789/12306>
- Cuadrado, J., & Zambrano, I. (2022). ANÁLISIS, DISEÑO E IMPLEMENTACIÓN DE UNA APLICACIÓN MÓVIL Y UN SISTEMA WEB PARA GESTIONAR EL INGRESO DE

VISITAS A LA URBANIZACIÓN “LA GEMA”. *Repositorio de la Universidad Estatal de Milagro*. <https://repositorio.unemi.edu.ec/xmlui/handle/123456789/6360>

Dart. (s/f). Descripción general de los dardos. Recuperado el 16 de mayo de 2026, de https://dart-dev.translate.goog/overview?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

el Comercio. (s/f). Las alícuotas cubren mantenimiento y servicios - El Comercio. Recuperado el 19 de mayo de 2026, de <https://www.elcomercio.com/tendencias/construir/alicuotas-cubren-mantenimiento-y-servicios/>

Equipo Comidor. (2019). Proceso manual frente a proceso automatizado: una guía comparativa. Comidor. https://www-comidor-com.translate.goog/blog/business-process-management/a-comparison-of-manual-and-automated-workflow-processes/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

Febrianto, F., Aziz, I., Kosim, M., Darwis, M., & Hendrowati, R. (2023). Implementation of The Resident’s Dues Applications (SIUMAS) Using Waterfall Method in RT X Cinere Village. <https://doi.org/10.31326/jisa.v6i2.1766>

García, A. (2025, agosto 10). Un observatorio urbano repiensa el futuro de las urbanizaciones privadas de Guayaquil. Diario Primicias. <https://www.primicias.ec/guayaquil/observatorio-urbano-universidad-catolica-modelo-urbanizaciones-privadas-102585/>

IBM. (s/f). ¿Qué es el desarrollo de aplicaciones móviles? | IBM. Recuperado el 16 de mayo de 2026, de https://www.ibm.com/think/topics/mobile-application-development?utm_source=chatgpt.com

Indeed. (s/f). Qué es backend: diferencias con frontend, tipos y beneficios | Indeed.com.

Recuperado el 17 de mayo de 2026, de <https://www.indeed.com/orientacion-profesional/como-encontrar-empleo/que-es-backend>

International Organization for Standardization [ISO]. (2015). Principios de trazabilidad.

https://www.keyence.com.mx/ss/products/marketing/traceability/law_basic.jsp#sect_02

Jonker, A., & Gomstyn, A. (2024, enero 14). ¿Qué es la gestión de requisitos? | IBM.

<https://www.ibm.com/mx-es/think/topics/what-is-requirements-management>

José, S. (2023). Qué es Github: para qué sirve, cómo funciona y cómo usarlo.

<https://ebac.mx/blog/que-es-github>

Kendall, K. E., & Kendall, J. E. (2011). Análisis y Diseño de Sistemas / Pearson Educación.

[https://www.ebooks7-24.com/visorBook.aspx?i=3285&t=B11A18BD-1CBA-49BC-](https://www.ebooks7-24.com/visorBook.aspx?i=3285&t=B11A18BD-1CBA-49BC-9032-4A1F955D1890)

[9032-4A1F955D1890](https://www.ebooks7-24.com/visorBook.aspx?i=3285&t=B11A18BD-1CBA-49BC-9032-4A1F955D1890)

Leonardo Cobos Prada, D., Mauricio Yosa Velásquez, E., Andrés Nova Aguilar, S., Prada, C.,

Leonardo Yosa Velásquez, D., Mauricio Nova Aguilar, E., & Andrés Modelo, S. (2025).

Modelo de desarrollo de software aplicado al diseño de un prototipo de aplicación web para la gestión de correspondencia en conjuntos residenciales. Tecnología Investigación y

Academia, 12(1), 139–149.

<https://revistas.udistrital.edu.co/index.php/tia/article/view/22230>

Loor Macías, B. S., Betancourt Ramirez, B., & ESPOL.FIEC. (2024). Desarrollo de una aplicación

móvil para la gestión eficiente del acceso de visitantes a una entidad [ESPOL.FIEC].

<http://www.dspace.espol.edu.ec/handle/123456789/62321>

Martinez, A. (s/f). Tipos de aplicaciones móviles: Nativas, web e híbridas – Future Space S.A.

Recuperado el 16 de mayo de 2026, de <https://www.futurespace.es/tipos-de-aplicaciones-moviles/>

Martínez, J. (2025, noviembre 13). Cómo vencer la resistencia al cambio organizacional en la transformación digital. ENAE. <https://www.enaes.es/blog/como-vencer-la-resistencia-al-cambio-organizacional-en-la-transformacion-digital>

Martins, J. (2026, abril 9). Qué es Scrum: marco ágil, sprints, roles y artefactos. <https://asana.com/es/resources/what-is-scrum>

Matango, W. (2022). Desarrollo de una aplicación móvil para el control de acceso a conjuntos residenciales mediante bluetooth utilizando Framework Xamarin Forms. <https://repositorio.utn.edu.ec/handle/123456789/12873>

Microsoft Build. (s/f). ¿Qué es Scrum? - Azure DevOps | Microsoft Learn. Recuperado el 17 de mayo de 2026, de <https://learn.microsoft.com/es-es/devops/plan/what-is-scrum>

Muradas, Y. (2019). Qué es Postman y primeros pasos | OpenWebinars. <https://openwebinars.net/blog/que-es-postman/>

Orellana, P., & Ramírez, E. (2025). PROTOTIPO DE APLICACIÓN MÓVIL PARA EL CONTROL DE ACCESO VEHICULAR PARA VISITAS EN ENTORNOS RESIDENCIALES CERRADOS CON VALIDACIÓN DE IDENTIDAD, OCR Y CÓDIGOS QR.

Pazmino, A. (2025, mayo 3). ¿QUÉ ES EL CONTROL DE ACCESO? A Tiempo. <https://atiempo.com.ec/que-es-el-control-de-acceso/>

- PostgreSQL. (s/f). PostgreSQL: Acerca de. PostgreSQL. Recuperado el 17 de mayo de 2026, de https://www-postgresql-org.translate.google/about/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
- Raffino, & Equipo editorial, E. (2026). Urbanización. Editorial Etecé. <https://concepto.de/urbanizacion/>
- Reinoso, L. (2021). Desarrollo de sistema informático para la gestión de pagos de cuotas de los residentes de la Urbanización Belo Horizonte. <http://dspace.ups.edu.ec/handle/123456789/20332>
- Roca Avila, Y. C., & Revollo Linares, R. A. (2022). Desarrollo de un sistema web para mejorar la administración del Condominio Las Terrazas de Surco utilizando el marco SCRUM, 2021. <https://repositorio.utp.edu.pe/handle/20.500.12867/5431>
- Ruoti, S. (2022, mayo 22). Cómo funcionan los códigos QR y qué peligros suponen para tu celular. BBC New Mundo. <https://www.bbc.com/mundo/noticias-61084121>
- Sánchez, M. (2019a). Desarrollo e implementación de sistema web de registro de pagos de alícuotas para automatización de control de acceso de vehículos en urbanización privada. [Universidad Católica de Santiago de Guayaquil]. <http://repositorio.ucsg.edu.ec/handle/3317/13009>
- Sánchez, M. (2019b). Desarrollo e Implementación de Sistema Web de Registro de Pagos de Alícuotas para Automatización de Control de Acceso de Vehículos en Urbanización Privada. <http://repositorio.ucsg.edu.ec/bitstream/3317/13009/1/T-UCSG-PRE-ING-CIS-230.pdf>
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide The Definitive Guide to Scrum: The Rules of the Game.

Silvera, K. (2016, agosto 26). La mediación, mecanismo para cobrar las alícuotas. Diario Expreso.

<https://www.expreso.ec/actualidad/mediacion-mecanismo-cobrar-alicuotas-81119.html>

Simonova, L. (2025, enero 28). What is a framework? Types and benefits | Kontent.ai.

<https://kontent-ai.translate.goog/blog/what-is->

[framework/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true](https://kontent-ai.translate.goog/blog/what-is-framework/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true)

Valera, F. (2026, abril 24). ¿Qué son los requisitos no funcionales? Tipos, ejemplos y enfoques -

Visure Solutions. <https://visuresolutions.com/es/alm-guide/non-functional-requirements/>

ANEXOS

Anexo 1. Planificación de actividades del proyecto

Id	Modo de tarea	Nombre de tarea	Duración	Comienzo	Fin
1		<i>Prototipo de aplicación móvil para el control de pagos de alicuotas, acceso con QR y reservas de áreas sociales de una urbanización privada</i>			
2		0. Anteproyecto	11 días?	lun 9/3/26	lun 23/3/26
3		0.1 Presentación del anteproyecto	1 día	lun 9/3/26	lun 9/3/26
4		0.2 Aprobación del anteproyecto	10 días	mar 10/3/26	lun 23/3/26
5		1. Planificación del proyecto	3 días?	lun 13/4/26	mié 15/4/26
6		1.1 Elaboración del cronograma	1 día	lun 13/4/26	lun 13/4/26
7		1.2 Definir el alcance del proyecto	1 día	mar 14/4/26	mar 14/4/26
8		1.3 Definir metodología del proyecto	1 día	mié 15/4/26	mié 15/4/26
9		2. Investigación y análisis	4 días?	mar 21/4/26	vie 24/4/26
10		2.1 Definir el planteamiento del problema	1 día	mar 21/4/26	mar 21/4/26
11		2.2 Definir los objetivos del proyecto	1 día	mié 22/4/26	mié 22/4/26
12		2.3 Elaborar justificación e importancia	1 día	jue 23/4/26	jue 23/4/26
13		2.4 Definir el alcance y delimitación	1 día	vie 24/4/26	vie 24/4/26
14		3. Metodología de investigación	10 días?	lun 27/4/26	vie 8/5/26
15		3.1 Definir el tipo de investigación	1 día	lun 27/4/26	lun 27/4/26
16		3.2 Definir la población y muestra	1 día	mar 28/4/26	mar 28/4/26
17		3.3 Diseñar encuestas y entrevistas	1 día	mié 29/4/26	mié 29/4/26
18		3.4 Aplicar los instrumentos de recolección de datos	1 día	mar 5/5/26	mar 5/5/26
19		3.5 Analizar datos estadísticos	1 día	vie 8/5/26	vie 8/5/26
20		4. Diseño del sistema	14 días?	lun 11/5/26	jue 28/5/26
21		4.1 Diseñar prototipo de UX/UI	2 días	lun 11/5/26	mar 12/5/26
22		4.2 Validar diseño de usuario	1 día	mié 13/5/26	mié 13/5/26
Id	Modo de tarea	Nombre de tarea	Duración	Comienzo	Fin
23		4.3 Diseñar arquitectura del sistema	2 días	jue 14/5/26	vie 15/5/26
24		4.4 Diseña base de datos	3 días	lun 18/5/26	mié 20/5/26
25		4.5 Diseñar interfaz de usuario	3 días	jue 21/5/26	lun 25/5/26
26		4.6 Elaborar diagramas del sistema	3 días	mar 26/5/26	jue 28/5/26
27		5. Desarrollo del sistema	15 días?	vie 29/5/26	jue 18/6/26
28		5.1 Configurar el entorno de desarrollo	2 días	vie 29/5/26	lun 1/6/26
29		5.2 Desarrollar módulo de usuario	3 días	mar 2/6/26	jue 4/6/26
30		5.3 Desarrollar módulo de pagos	2 días	vie 5/6/26	lun 8/6/26
31		5.4 Desarrollar módulo de control de acceso	3 días	mar 9/6/26	jue 11/6/26
32		5.5 Desarrollar módulos de reservaciones	2 días	vie 12/6/26	lun 15/6/26
33		5.6 Integración de los módulos	3 días	mar 16/6/26	jue 18/6/26
34		6. Pruebas y validación	6 días?	vie 19/6/26	vie 26/6/26
35		6.1 Realizar pruebas funcionales	2 días	vie 19/6/26	lun 22/6/26
36		6.2 Validación con juicios de expertos	2 días	mar 23/6/26	mié 24/6/26
37		6.3 Corrección de errores	2 días	jue 25/6/26	vie 26/6/26
38		7. Documentación del proyecto	10 días?	lun 29/6/26	vie 10/7/26
39		7.1 Redactar Capítulo I: Planteamiento del problema	2 días	lun 29/6/26	mar 30/6/26
40		7.2 Redactar Capítulo II: Marco teórico	2 días	mié 1/7/26	jue 2/7/26
41		7.3 Redactar Capítulo III: Propuesta tecnológica	2 días	vie 3/7/26	lun 6/7/26
42		7.4 Redactar Capítulo IV: Conclusiones y recomendaciones	2 días	mar 7/7/26	mié 8/7/26
43		7.5 Elaborar manual técnico	1 día	jue 9/7/26	jue 9/7/26
44		7.6 Elaborar manual de usuario	1 día	jue 9/7/26	jue 9/7/26
45		7.7 Revisión final del documento	1 día	vie 10/7/26	vie 10/7/26
Id	Modo de tarea	Nombre de tarea	Duración	Comienzo	Fin
46		8. Cierre del proyecto	28 días	lun 13/7/26	mié 19/8/26
47		8.1 Presentación y finalización del proyecto (tutorías)	5 días	lun 13/7/26	vie 17/7/26
48		8.2 Revisión del trabajo de titulación	5 días	mié 29/7/26	mar 4/8/26
49		8.3 Sustentación del trabajo de titulación	7 días	mar 11/8/26	mié 19/8/26

Elaboración: Investigadores.

Fuente: Elaboración propia en la herramienta Microsoft Project.

Anexo 2. Fundamentación Legal

BASE LEGAL

CONSTITUCIÓN DE LA REPÚBLICA DEL ECUADOR (2008)

La Constitución garantiza el acceso a tecnologías de la información y protege derechos fundamentales

relacionados con privacidad y datos personales.

Art. 16 numeral 2:

Reconoce el derecho al acceso universal a las tecnologías de información y comunicación, lo cual respalda el uso de aplicaciones móviles como herramientas legítimas dentro de procesos administrativos y de seguridad.

Art. 66 numeral 19:

Reconoce el derecho a la protección de datos personales, incluyendo acceso, rectificación y eliminación de información. Este artículo es fundamental, ya que el proyecto procesa datos biométricos (reconocimiento facial) y datos personales (cedula, placa vehicular).

Art. 322:

Reconoce la propiedad intelectual, protegiendo el desarrollo del software creado dentro del proyecto.

Art. 385 y 386:

Promueven el desarrollo científico y tecnológico como parte del sistema nacional de innovación, respaldando el desarrollo de soluciones tecnológicas para mejorar procesos sociales, como el control de acceso vehicular.

LEY ORGÁNICA DE EDUCACIÓN SUPERIOR

Dado que el proyecto se desarrolla como trabajo de integración curricular, se fundamenta en:

Art. 13. Establece que el Sistema de Educación Superior debe promover la creación y desarrollo de ciencia y tecnología.

El proyecto cumple con esta disposición al generar una solución tecnológica aplicada a una problemática real en urbanizaciones privadas.

LEY DE PROPIEDAD INTELECTUAL

El prototipo desarrollado constituye una obra protegida por derechos de autor.

Art. 28. Los programas de ordenador se consideran obras literarias y se protegen como tales, independientemente de su forma (código fuente o código objeto).

Art. 29. Reconoce como titular del software a la persona natural o jurídica que toma la iniciativa y responsabilidad de su creación.

Por tanto, el código fuente del prototipo móvil y del *backend* está protegido por esta normativa.

LEY ORGÁNICA DE PROTECCION DE DATOS PERSONALES

Se presenta la Ley Orgánica de Protección de Datos Personales, publicada en el Registro Oficial Suplemento No. 459 del 26 de mayo de 2021, la cual establece el marco jurídico para la protección y el tratamiento de los datos personales en el Ecuador. Su inclusión tiene como finalidad respaldar el cumplimiento de los principios de licitud, confidencialidad, seguridad y responsabilidad aplicables al desarrollo del prototipo de aplicación móvil, por el manejo de: datos de identificación, datos vehiculares y registros de accesos.

Art. 1. Objeto y finalidad. El objeto y finalidad de la presente ley es garantizar el ejercicio del derecho a la protección de datos personales, que incluye el acceso y decisión sobre información y

datos de este carácter, así como su correspondiente protección, Para dicho efecto regula, prevé y desarrolla principios, derechos, obligaciones y mecanismos de tutela.

Garantiza el derecho a la protección de los datos personales que maneja el sistema (residentes, visitantes, administradores y guardias).

Art. 2. Ámbito de aplicación material. La presente ley se aplicará al tratamiento de datos personales contenidos en cualquier tipo de soporte, automatizados o no, así como a toda modalidad de uso posterior.

Establece que la ley aplica al tratamiento de datos personales en cualquier soporte, incluyendo bases de datos y aplicaciones móviles.

Art. 4. Términos y definiciones. Para los efectos de la aplicación de la presente Ley se establecen las siguientes definiciones: Autoridad de Protección de Datos Personales: Autoridad pública independiente encargada de supervisar la aplicación de la presente ley, reglamento y resoluciones que ella dicte, con el fin de proteger los derechos y libertades fundamentales de las personas naturales, en cuanto al tratamiento de sus datos personales.

Bajo esta definición se incluyen el número de cédula, nombres, dirección, teléfono y, cuando la placa está vinculada a un propietario identificable, también puede constituir un dato personal.

Art. 12. Derecho a la información. El titular de datos personales tiene derecho a ser informado conforme los principios de lealtad y transparente por cualquier medio.

Obliga a informar al titular para qué serán utilizados sus datos personales, por cuánto tiempo se conservarán y quién será responsable del tratamiento.

Art. 13. Derecho de acceso. El titular tiene derecho a conocer y a obtener, gratuitamente, del responsable de tratamiento acceso a todos sus datos personales y a la información detallada en el artículo precedente, sin necesidad de presentar justificación alguna. El responsable del tratamiento

de datos personales deberá establecer métodos razonables que permitan el ejercicio de este derecho, el cual deberá ser atendido dentro del plazo de quince (15) días.

Reconoce el derecho del titular a conocer qué datos personales posee el responsable del tratamiento.

Anexo 3. Validación de expertos

Juicio de expertos

(insertar matriz y constancia)

Anexo 4. Certificado del Docente – Tutor del Trabajo de Integración Curricular

Anexo 5. Certificado Porcentaje de Similitud

Anexo 6. Informe del Docente Revisor

Anexo 7. Manual Técnico



UNIVERSIDAD DE GUAYAQUIL

FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS

CARRERA DE SOFTWARE

**PROTOTIPO DE APLICACIÓN MÓVIL PARA EL CONTROL DE PAGOS
DE ALÍCUOTAS, ACCESO CON QR Y RESERVAS DE ÁREAS
SOCIALES DE UNA URBANIZACIÓN PRIVADA**

MANUAL TÉCNICO

AUTORES:

**LÓPEZ PANTA JHONAS JEREMY
PIEDRA JIMÉNEZ BELKY XIOMARA**

TUTOR(A):

ING. ANGEL CUENCA ORTEGA, M. Sc

GUAYAQUIL – ECUADOR

2026

Introducción

El presente Manual Técnico corresponde al proyecto Prototipo de aplicación móvil para el control de pagos de alícuotas, acceso con QR y reservas de áreas sociales de una urbanización privada, desarrollado con el propósito de documentar la arquitectura, los componentes tecnológicos y los aspectos técnicos que conforman la solución implementada.

Este documento constituye una guía de referencia para desarrolladores, administradores del sistema y personal técnico encargado de la implementación, mantenimiento y futuras mejoras de la aplicación. En él se describen la estructura del sistema, la arquitectura de software, las tecnologías utilizadas, la organización del código fuente, la configuración del entorno de desarrollo, la base de datos, los servicios de la API REST, así como los procedimientos necesarios para la instalación, despliegue y funcionamiento de la solución.

El prototipo fue desarrollado bajo una arquitectura de tres capas, empleando Flutter para el desarrollo de la aplicación móvil, NestJS para la implementación de la API REST, PostgreSQL como sistema gestor de base de datos y Prisma ORM para la gestión del acceso a los datos. Esta combinación de tecnologías permite construir una solución escalable, fácil de mantener y orientada a optimizar la gestión de los pagos de alícuotas, el control de acceso mediante códigos QR y la gestión de reservas de áreas sociales en una urbanización privada.

Finalmente, este manual tiene como finalidad proporcionar la información técnica necesaria para comprender el funcionamiento interno del sistema, facilitar su mantenimiento evolutivo y correctivo, y servir como base para futuras ampliaciones o integraciones, contribuyendo a garantizar la continuidad y sostenibilidad del proyecto.

Objetivo

El objetivo del manual técnico es documentar los aspectos técnicos del prototipo de aplicación móvil, describiendo su arquitectura, tecnologías, componentes, configuración e implementación, con el fin de facilitar su instalación, mantenimiento y futuras mejoras.

Arquitectura del sistema

El sistema fue desarrollado bajo una arquitectura de tres capas, separando la presentación, la lógica del negocio y el acceso a los datos.

Las capas son:

- Presentación: Flutter
- Lógica del negocio: NestJS
- Persistencia de datos: PostgreSQL mediante Prisma ORM

Esta arquitectura permite mantener una adecuada organización del código, mejorar la escalabilidad del sistema y facilitar el mantenimiento.

Requisitos técnicos para ambientes de desarrollo móvil

Para el desarrollo y ejecución del prototipo de aplicación móvil es necesario contar con un entorno de desarrollo que disponga de los recursos de hardware y software adecuados. Estos requisitos garantizan la correcta instalación de las herramientas, la compilación del código fuente, la ejecución de emuladores o dispositivos físicos para pruebas y el funcionamiento eficiente de las tecnologías empleadas durante el desarrollo del sistema.

Tabla 68

Requisitos de hardware

Características	Especificación
Procesador	Intel Core I5 o superior
Memoria RAM	8 GB
Almacenamiento	20 GB libres
Resolución	1366 x 768

Nota: En esta tabla se muestran los requisitos mínimos de hardware para el desarrollo y prueba de la aplicación móvil. Elaboración propia.

Tabla 69

Requisitos de software

Características	Especificación
Windows	10 u 11 de 64 bits
Flutter SDK	3.44
Dart SDK	Incluido en Flutter
Android Studio	Última versión
Visual Studio Code	Última versión
Git	2.x
Node.js	22.x
PostgreSQL	17

Nota: En esta tabla se muestran los requisitos mínimos de software para el desarrollo y prueba de la aplicación móvil. Elaboración propia.

Configuración del entorno de desarrollo

Instalación de Flutter

Para la instalación de Flutter nos dirigimos al sitio oficial. Donde se debe elegir la opción de descargar el archivo para el sistema operativo que tenga el equipo de computo.

Figura 175

Sitio oficial de Flutter

The screenshot displays the Flutter documentation website at `docs.flutter.dev/install/manual`. The page is titled "Install Flutter manually" and provides instructions for setting up the Flutter SDK. A sidebar on the left contains a navigation menu with categories like "Get started", "User interface", and "Beyond UI". The main content area includes a "Tip" suggesting the use of VS Code for a streamlined setup, a "Choose your development platform" section with buttons for Windows, macOS, Linux, and ChromeOS, and a "Download prerequisite software" section. A right-hand sidebar lists "On this page" links.

Nota: Sitio oficial de Flutter. Elaboración propia.

